Datasets for Failure Attribution in LLM Agent Systems

A compact survey of 21 benchmarks: descriptions, statistics and fully annotated example trajectories

Compiled from the released data and papers, 16 October 2026

Scope. This document describes 21 datasets (the 20 listed in `report.md` plus SymFail) for *failure attribution* in LLM agent and multi-agent systems: for each one, what it is, how it was built, its statistics (computed from the downloaded data where it is public) and one complete example trajectory—every step shown—with the released annotation. Section 1 introduces the task, Section 2 compares the datasets, Sections 3–6 cover them one by one, Section 7 draws practical conclusions and Section 8 fixes a working definition of failure attribution and develops four research proposals that follow from it; Appendix A ranks the datasets by observability and replayability. All data, notes and scripts are in `/dataDisk/home/thanhdo/failure-attribution-datasets/`.

Contents

1 The failure-attribution problem **1**

2 Landscape of the 21 datasets **2**

3 Core MAS failure-attribution benchmarks (agent + decisive step) **2**
- 3.1 Who&When . . . 2
- 3.2 TraceElephant . . . 5
- 3.3 MP-Bench . . . 7
- 3.4 Who&When Pro . . . 10
- 3.5 LongRCA Bench . . . 12
- 3.6 AgentRx . . . 15
- 3.7 SymFail (SymTrace) . . . 17

4 Large injected-fault datasets (mainly for training attributors) **19**
- 4.1 TracerTraj (AgenTracer) . . . 19
- 4.2 AEGIS / Aegis-Bench . . . 22
- 4.3 CORRECT-Error . . . 25
- 4.4 AgentErrata . . . 28

5 Failure-taxonomy datasets (failure-mode labels) **36**
- 5.1 MAST-Data (MAD) . . . 36
- 5.2 TRAIL . . . 38

6 Related trajectory-debugging datasets **41**
- 6.1 AgentErrorBench (AgentDebug) . . . 41
- 6.2 SearchAuditBench (SearchAuditor) . . . 43
- 6.3 Failure as a Process (CLI coding-agent annotations) . . . 46
- 6.4 TrajErrBench (TrajDebug) . . . 48
- 6.5 AgentHallu . . . 50
- 6.6 MemTrace (MemTraceBench) . . . 53
- 6.7 HORIZON . . . 56
- 6.8 MegaRCA-Mix . . . 58

7 Cross-cutting observations and recommendations **60**

8 Research proposals **60**
- 8.1 Preliminaries: a working definition of failure attribution . . . 60
- 8.2 Proposal 1: Counterfactual perturbation prediction on agent trajectories . . . 62
- 8.3 Proposal 2: Label-free attribution from the flow of trajectories . . . 63
- 8.4 Proposal 3: Attribution-guided policy improvement . . . 66
- 8.5 Proposal 4: Error propagation and absorption—the lifecycle of an error . . . 68

A Observability and replayability of the released data **72**

1 The failure-attribution problem

Setting. An LLM-based *agentic system* solves a task by executing a sequence of *steps*: model calls, tool calls, code execution and environment observations. In a *multi-agent system* (MAS) these steps are emitted by several roles (orchestrator, coder, web-surfer, verifier, . . .) that communicate through messages. Such systems fail often—on GAIA-style tasks the strongest open MAS still fail on a large share of tasks—and because a trajectory can span hundreds of steps and several agents, finding out *why* a run failed is labour-intensive.

Task definition (Who&When). Zhang et al. [75] formalised *automated failure attribution*: given a failed trajectory $\tau = (s_1, \ldots, s_T)$ with each step s_t produced by an agent $a_t \in \mathcal{A}$, predict

1. **Who**: the agent $a^\star$ responsible for the failure, and
2. **When**: the *decisive error step* $t^\star$—the earliest step such that, had it been correct, the task would have succeeded.

A dataset for this task therefore stores, per failed trajectory, the full log, the ground-truth answer (so that failure is verifiable) and the labels $(a^\star, t^\star)$, usually with a free-text *reason*. Evaluation is by *agent-level accuracy* (exact match on $a^\star$) and *step-level accuracy* (exact match on $t^\star$); some later benchmarks relax the step metric to a window ($|\hat{t} - t^\star| \le k$) or to set-overlap when several steps are acceptable.

Why it is hard. Errors propagate: a subtle early mistake (a wrong assumption, a mis-read observation) can produce a visible crash dozens of steps later, and an attributor that flags the crash is wrong. Who&When reports that the best prompted LLM reaches only ≈54% agent-level and ≈14% step-level accuracy, and that accuracy falls as the trajectory grows. Later work shows that *full* traces (including hidden tool I/O and environment state) help considerably, that several steps may be equally plausible root causes (annotators disagree on up to 60% of items in MP-Bench), and that very long traces (median 145 steps in LongRCA) remain mostly unsolved.

Three ways to obtain labelled failures.

- natural **Natural failures, human labels.** Run real MAS, keep the failed runs and let experts locate the decisive step (Who&When, TraceElephant, MP-Bench, LongRCA, AgentRx, TRAIL). Realistic but expensive; annotation agreement is a known issue.
- injected **Fault injection.** Take a *successful* trajectory, replay it up to some step, inject a context-aware error (via an LLM or a programmatic perturbation) and continue; the injected step is the label by construction (Who&When Pro, TracerTraj, Aegis, CORRECT-Error, AgentErrata). Cheap and scalable, but the error distribution may differ from reality.
- taxonomy **Failure-mode labelling.** Label a trace with one or more failure *categories* (MAST's 14 modes, TRAIL's taxonomy) without necessarily pinning a single step. Useful for diagnosis and training critics rather than for step-level attribution.

Attribution strategies in the literature. (i) *all-at-once*: feed the whole log to a judge LLM and ask for $(a^\star, t^\star)$; (ii) *step-by-step*: walk the log and stop at the first step judged erroneous—better step accuracy, higher cost; (iii) *binary search*: recursively halve the log; (iv) *trained attributors*: small models fine-tuned (often with RL) on injected-fault data such as TracerTraj, Aegis or Who&When Pro; (v) *counterfactual replay*: re-execute from a candidate step with a corrected action and check whether the outcome flips—the most reliable but requires a reproducible environment (TraceElephant); (vi) *label-free detectors*: models of trajectory dynamics fitted without step labels—on successful runs only [68] or, as Section 8.3 proposes, on all runs with their outcomes—that score each step by how far it departs from the learned flow.

2 Landscape of the 21 datasets

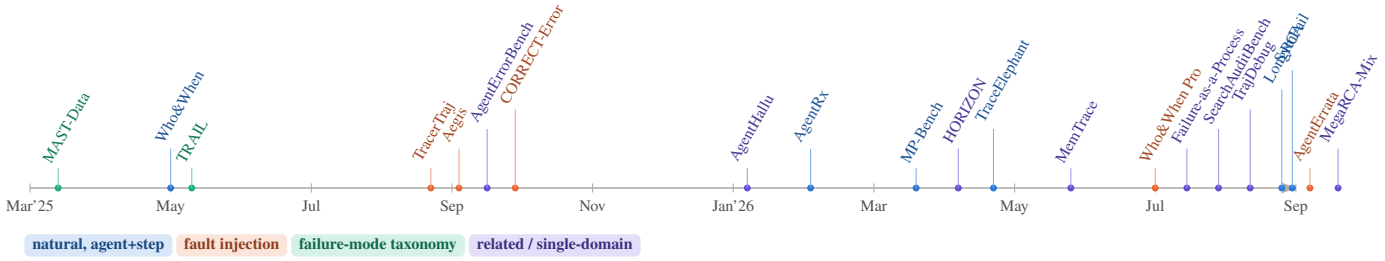


Figure 1: Release timeline (month of the first arXiv version, approximate). Failure-attribution datasets appeared only after Who&When (May 2025) and accelerated in 2026, moving from a few hundred hand-labelled logs to 10^4 injected failures and to long-horizon, full-trace settings.

3 Core MAS failure-attribution benchmarks (agent + decisive step)

3.1 Who&When

natural available agent + step MAS GAIA / AssistantBench

Paper	*Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems* — Zhang et al., ICML 2025 (spotlight), arXiv:2505.00212 [75]
Links	HF `Kevin355/Who_and_When` · GitHub `mingyin1/Agents_Failure_Attribution`
License	none stated on the dataset card or repo (paper CC BY 4.0); underlying queries from GAIA and AssistantBench
Status	downloaded in full, ungated; 9.5 MB; `data/who_and_when/` (184 JSON logs + 2 parquet files)

Description. Who&When is the benchmark that defined the task (§1): every record is the output log of a multi-agent run that *failed* a GAIA or AssistantBench query, plus three labels—`mistake_agent` (who), `mistake_step` (when: a 0-based index into `history`) and a free-text `mistake_reason`. Failures are natural (no injection). Two subsets come from two very different MAS:

- **Algorithm-Generated** (126 logs): teams assembled automatically per task by *CaptainAgent* (AG2, GPT-4o)—a handful of `*_Expert` agents plus a `Computer_terminal`. 82 distinct expert names appear as blamed agents.
- **Hand-Crafted** (58 logs): *Magentic-One* (Orchestrator, WebSurfer, FileSurfer, Coder/Assistant, ComputerTerminal) on 28 AssistantBench and 30 GAIA tasks.

In total 184 logs from 127 distinct MAS configurations. **Annotation**: three experts, three rounds (independent labelling with uncertainty flags on 15–30% of cases, consensus discussion, cross-validation); 23–31 h per annotator; no agreement coefficient reported. The decisive step is defined as the earliest step whose correction would have made the run succeed—a single-root-cause assumption later questioned by MP-Bench.

Metrics and baselines. Agent-level and step-level exact-match accuracy, with or without the ground-truth answer given to the attributor.

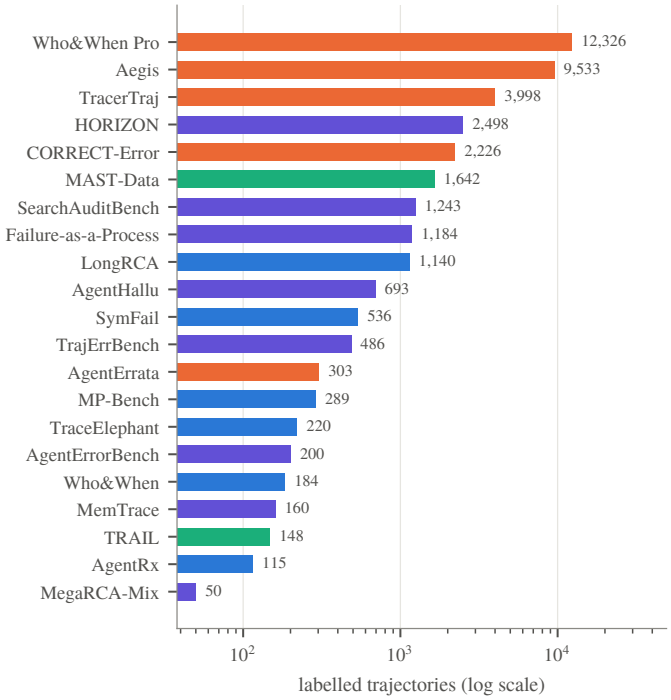


Figure 2: Labelled failed trajectories per dataset (log scale). Injected-fault datasets (orange) are 1–2 orders of magnitude larger than human-labelled natural ones (blue).

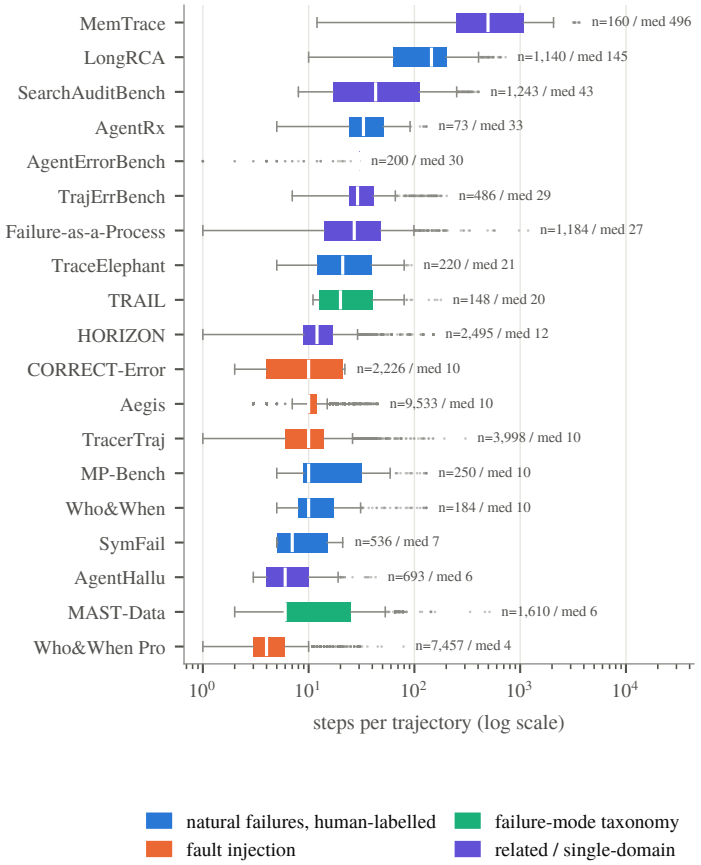


Figure 3: Steps per trajectory, computed from the downloaded data. Most benchmarks live in the 5–30 step range; LongRCA and MemTrace are the long-horizon outliers.

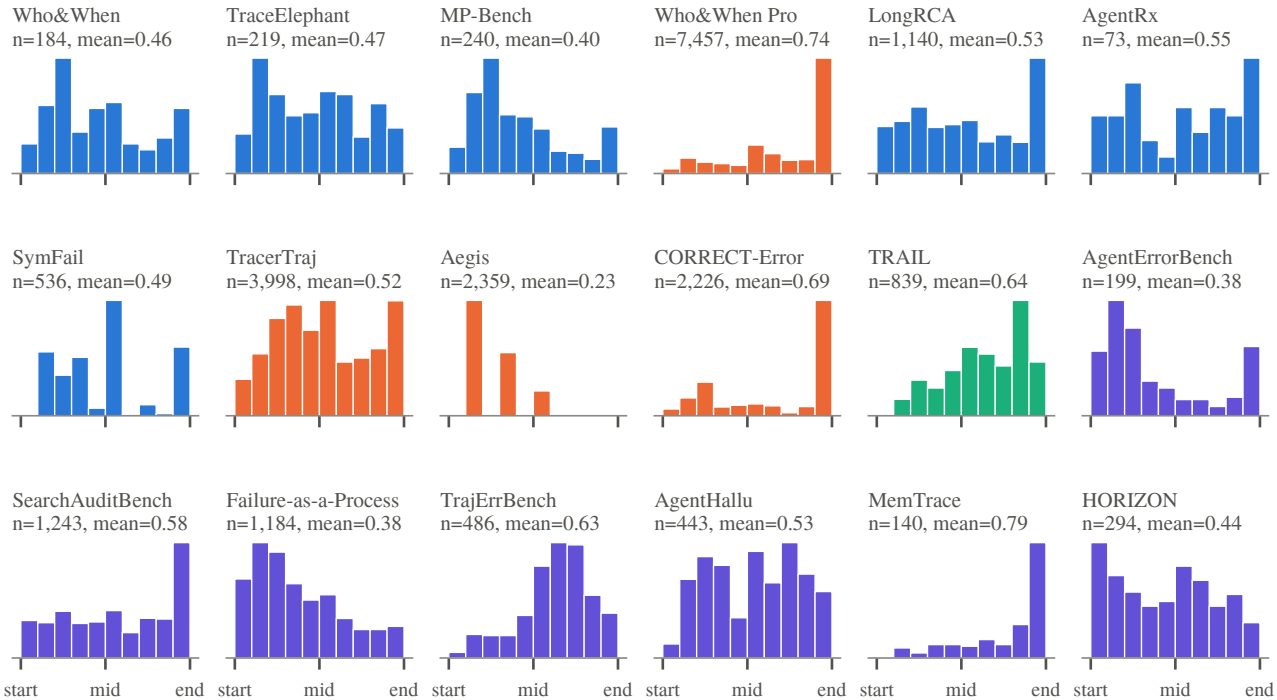


Figure 4: Where the labelled error step sits inside the trajectory (relative position $t^\star/T$, 10 bins). Natural-failure benchmarks spread errors across the whole run with a spike at the end (the final wrong answer is often the labelled step); some injected datasets inject preferentially at the start (TracerTraj) or at the very end (Who&When Pro, CORRECT-Error) — a distribution shift to keep in mind when training on them.

Dataset	Venue	Failures	System	Labels	#failed	med. steps	Source frameworks / benchmarks	Access
1 Core MAS failure attribution (agent + decisive step)								
Who&When	ICML'25	natural	MAS	agent, step, reason	184	10	CaptainAgent, Magentic-One / GAIA, AssistantBench	available
TraceElephant	ACL'26	natural	MAS	agent, step, reason (full trace)	220	21	CaptainAgent, Magentic-One, SWE-Agent / GAIA, AB, SWE-bench	available
MP-Bench	ICML'26	natural	MAS	agent, step sets (3 annotators)	289	–	Who&When logs + new runs	available
Who&When Pro	ICML'26	injected	MAS+single	agent, step, mode	12 326	4	15 frameworks / 26 benchmarks, text+image+video	available
LongRCA	arXiv'26	natural	MAS	role, earliest root step	1 140	145	5 domains (TravelPlanner, WebArena, . . .)	available
AgentRx	EMNLP'26	natural	MAS+single	step, 9-cat. taxonomy, events	115 (73 public)	33	τ-bench, Flash, Magentic-One	gated partial
SymFail (SymTrace)	arXiv'26	natural	MAS	failure node, C1–C4 symptom set, replayable graphs	536	7	AG2, CrewAI, Magentic-One adapters / WebArena-Verified Hard, AssistantBench	available
2 Large injected-fault datasets (training attributors)								
TracerTraj	ICLR'26	injected	MAS	agent, step, reason	3 998	10	MetaGPT-style, math, agentic	available
Aegis	ICLR'26	injected	MAS	agent, 14 modes	9 533	10	6 MAS / GSM8K, MATH, HumanEval, MMLU, SciBench, GAIA	available
CORRECT-Error	ICML'26	injected	MAS	agent, step	2 226	10	Magentic-style / 7 QA+math benchmarks	available
AgentErrata	arXiv'26	injected	MAS	agent, step, category	303	–	Magentic-One / GAIA	not released
3 Failure-mode taxonomies (mostly no step label)								
MAST-Data	NeurIPS'25	natural	MAS	14 modes (LLM judge; 21 human)	1 642	6	7 MAS / 8 benchmarks / 5 LLMs	available
TRAIL	arXiv'25	natural	single+MAS	span-level errors, 3-level taxonomy, impact	148	20	smolagents / GAIA, SWE-bench	available
4 Related trajectory-debugging datasets								
AgentErrorBench / Traj2k	ICML'26	natural	single	critical step, module + 17 types	200 (Traj2k n/a)	30	ALFWorld, GAIA, WebShop	available
SearchAuditBench	arXiv'26	natural	single	critical step, 6 root causes, repair rubric	1 243	43	deep-search agents	available
Failure as a Process	arXiv'26	natural	single	$t_{err}/t_{lock}/t_{obs}$, 9 triggers (annotations only)	1 184 (of 1 794)	22.5‡	Terminal-Bench CLI coding agents	partial
TrajErrBench	EMNLP'26	natural	single	critical step, 20-type taxonomy	486	29	τ^2-Bench, SWE-Bench Pro	available
AgentHallu	arXiv'26	natural	single(+MAS)	hallucination step, 5 cat./14 sub	443 (of 693)	6	7 frameworks / 8 benchmarks	available
MemTrace	arXiv'26	natural	memory system	faulty memory op, 4 error types	160	496†	RAG, Mem0, EverMemOS, long-context	available
HORIZON	arXiv'26	natural	single	mode (LLM judge); step on 294	2 498	12	OS, web, embodied, database	partial
MegaRCA-Mix	arXiv'26	natural	single (Claude Code)	edge, fault side, 41 modes	50	–	Harbor Index (GAIA, SWE-bench, HLE, . . .)	not released

Table 1: Overview. "#failed" = labelled failed trajectories available (paper figure where the release differs; see each section). "med. steps" computed from the downloaded data. †MemTrace steps are memory operations in the dependency slice of the failed answer. ‡Failure-as-a-Process: all 1 794 runs, from the released annotations (trajectory text not released).

Three prompting strategies were compared (GPT-4o, with ground truth; agent / step accuracy in %):

Method	Alg.-Generated	Hand-Crafted
Random	29.1 / 19.1	12.0 / 4.2
All-at-Once	**54.3** / 12.5	**55.2** / 5.3
Step-by-Step	35.2 / **25.5**	34.5 / 7.0
Binary Search	44.1 / 24.0	51.7 / 6.9

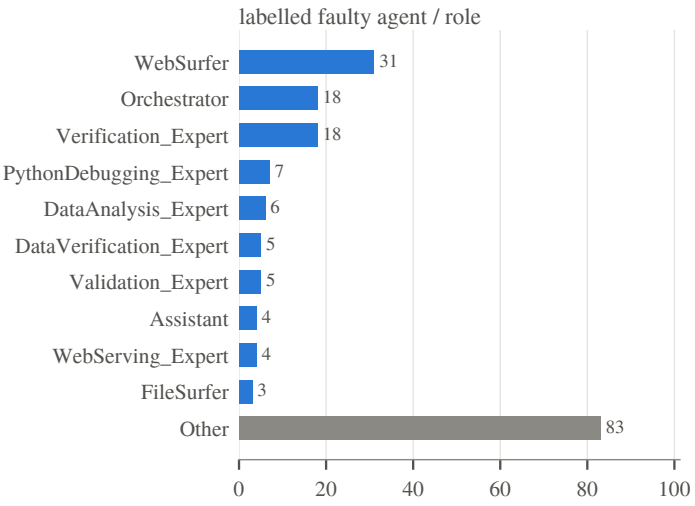
Reasoning models do not help much (o1 41.4 / 10.3, DeepSeek-R1 56.9 / 3.5 All-at-Once on Hand-Crafted). Step accuracy collapses on the long Hand-Crafted logs.

Schema. One JSON object per log: `question`, `question_ID`, `ground_truth`, `level` (GAIA 1–3 / AssistantBench Medium–Hard), `history` (list of {`role`, `name`, `content`}), `system_prompt` (AG only), and the three labels. In Hand-Crafted logs `history[0]` is the human question, and orchestrator thoughts, orchestrator instructions and specialist replies are separate steps; in Algorithm-Generated logs the first entry is already an expert message.

Statistics. computed from the downloaded data.

	All	Hand-Crafted	Alg.-Gen.
trajectories (all failed)	184	58	126
GAIA / AssistantBench	128 / 56	30 / 28	98 / 28
steps: mean / median	22.2 / 10	51.6 / 32.5	8.7 / 10
steps: min – max	5 – 130	5 – 130	5 – 10
distinct agents: mean (max)	3.3 (5)	2.6 (5)	3.6 (4)
error step index: mean (max)	6.8 (82)	15.2 (82)	3.0 (9)
relative error position	0.46	0.47	0.45
characters / log: mean	28 k	64 k	11.6 k
≈tokens / log: mean	7 k	16 k	2.9 k

Error at the very first step in 20 logs (all Alg.-Gen.), at the last step in 8. Blamed agent in Hand-Crafted: WebSurfer 57%, Orchestrator 31%, Assistant 7%, FileSurfer 5%; in Alg.-Gen. the blame spreads over 82 names (Verification_Expert 14%, PythonDebugging_Expert 6%, DataAnalysis_Expert 5%).



Example trajectory. Hand-Crafted log `48.json` (AssistantBench; 5 steps, 2 agents, 12 k characters). The WebSurfer issues a search, never reads a weather source, and the run ends with an unsupported answer.

Task (`Hand-Crafted/48.json`)

Based on recent years (2020-2023), how likely am I to hit a rainy day (at least 0.5mm of precipitation) if I travel to Seattle during the first week of September? (provide the answer in percentage)

0 · human

```
Based on recent years (2020-2023), how likely am I to hit a rainy day (at least 0.5mm of precipitation) if I travel to Seattle during the first
week of September?  (provide the answer in percentage)
```

1 · Orchestrator (thought)

```
Initial plan:
We are working to address the following user request:
Based on recent years (2020-2023), how likely am I to hit a rainy day (at least 0.5mm of precipitation) if I travel to Seattle during the first
week of September?  (provide the answer in percentage)
To answer this request we have assembled the following team:
Assistant:  A helpful and general-purpose AI assistant that has strong language skills, Python skills, and Linux command line skills.
ComputerTerminal:  A computer terminal that performs no other action than running Python scripts (provided to it quoted in “‘python code blocks),
or sh shell scripts (provided to it q
[...]  (2,122 chars omitted) [...]
/usr/local/lib/python3.11/site-packages/autogen_magentic_one/agents/orchestrator.py:216:  UserWarning:  Resolved model mismatch:  gpt-4o-2024-08-06
!= gpt-4o-2024-05-13.  Model mapping may be incorrect.
ledger_response = await self._model_client.create(
```

2 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "The request has not yet been satisfied as we still need to gather and process the historical weather data for Seattle for the first
week of September from 2020 to 2023.",
"answer":  false
},
"is_in_loop":  {
"reason":  "We are not in a loop as we have just defined the plan and steps to proceed.",
"answer":  false
},
"is_progress_being_made":  {
"reason":  "We have outlined a clear plan and have just begun by initiating the request to gather data.",
"answer":  true
},
"next_speaker":  {
"reason":  "WebSurfer needs to search for and gather the necessary hi
[...]  (231 chars omitted) [...]
er from 2020 to 2023.",
"answer":  "Please search for historical weather data for Seattle for the first week of September from 2020 to 2023 and find the number of rainy
days (with at least 0.5mm of precipitation) during that period each year."
}
}
```

3 · Orchestrator (-> WebSurfer)

```
Please search for historical weather data for Seattle for the first week of September from 2020 to 2023 and find the number of rainy days (with at
least 0.5mm of precipitation) during that period each year.
```

⚠ 4 · WebSurfer · ANNOTATED ERROR STEP

```
I typed 'Seattle historical weather data September 2020-2023' into '0 characters out of 2000'.
Here is a screenshot of [Seattle historical weather data September 2020-2023 - Search](https://www.bing.com/search?q=
Seattle+historical+weather+data+September+2020-2023&form=QBLH&sp=-1&lq=0&pq=&sc=0-0&qs=n&sk=&cvid=273959149E7F422CB8DC7E6C09666674&ghsh=0&ghacc=
0&ghpl=).  The viewport shows 31% of the webpage, and is positioned at the top of the page.
The following metadata was extracted from the webpage:
{
"meta_tags":  {
"referrer":  "origin-when-cross-origin",
"SystemEntropyOriginTrialToken":  "A1L3tx5CzccqjN3lK6st/fXMwhf9EeokC
[...]  (2,605 chars omitted) [...]
prompt and retry.  To learn more about our content filtering policies please read our documentation:  \r\nhttps://go.microsoft.com/fwlink/?linkid=
2198766.", 'param':  'prompt', 'type':  None}}
FINAL ANSWER: 20
SCENARIO.PY COMPLETE !#!#
RUN.SH COMPLETE !#!#
```

Annotation

Faulty agent	WebSurfer
Faulty step	4
Category	—
Reason	The agent didn't find enough information to answer the question but directly gave the answer.
Ground truth	14.2
System output	I typed 'Seattle historical weather data September 2020-2023' into '0 characters out of 2000'. Here is a screenshot of [Seattle historical weather data September 2020-2023 - Search](http [...] (7,160 chars omitted) [...] ': None}} FINAL ANSWER: 20 SCENARIO.PY COMPLETE !#!# RUN.SH COMPLETE !#!#
Other labels	**subset**: Hand-Crafted; **question_ID**: 2aa5dd83fbcd0dce9a3dd4592106e5b5edf738008d932e357d477bba80e59ccf; **level**: None; **is_correct**: False; **source_benchmark**: AssistantBench; **step_indexing**: 0-based index into history (history[0] is the user question in Hand-Crafted)

Notes. (i) Two regimes in one benchmark: short, synthetic-team logs (Alg.-Gen.) versus long Magentic-One logs (Hand-Crafted, 34 of 58 have $\geq$30 steps). (ii) Only agent *outputs* are stored—no tool inputs, screenshots or environment state (the "partial observation" setting TraceElephant criticises). (iii) Label quirks: inconsistent agent-name casing (`WebSurfer`/`Websurfer`), AG values stored as quoted strings, field names differ between subsets (`groundtruth` vs `ground_truth`), only 4 records carry a `mistake_type`. (iv) Overlaps: the same logs are re-annotated by three experts in MP-Bench (177 of its 289 items); TraceElephant re-runs the same two MAS with full traces; Who&When Pro is the scaled-up, injected-failure successor.

3.2 TraceElephant

natural | available | agent + step | MAS + single agent | full LLM-call traces | GAIA / AssistantBench / SWE-Bench Verified

Paper	*Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-based Multi-Agent Systems* — Chen, Wang, Mu, Wang, Liu, Feng, Wang (Institute of Software, CAS), ACL 2026 (long, 2026.acl-long.912), arXiv:2604.22708 [10]
Links	HF `TraceElephant/TraceElephant` · GitHub `TraceElephant/TraceElephant` (agent systems, LLM-API tracing middleware, `trace_locate` attribution code)
License	CC BY 4.0 (HF card and repository); upstream tasks keep their own licences
Status	downloaded in full, ungated; 597 MB zip, 799 MB extracted (440 JSON + 2,070 PNG); only the 220 failed traces of the paper's 380 runs are released; `data/traceelephant/`

Description. TraceElephant asks the Who&When question (responsible agent + decisive step) under *full execution observability*. Instead of agent outputs only, each step is one LLM-API call recorded by a non-intrusive middleware: the complete request (system prompt, all prior messages, tool results, screenshots), the raw response, token usage and tool-call logs, plus system metadata and the runnable agent code, so the run can be re-executed. The paper's claim: full traces raise step-level accuracy by up to 76% relative over output-only observation. Three systems, GPT-4o backbone, temperature 0.3, natural failure distribution:

- **Captain-Agent** (AG2; Orchestrator + generated `*_Expert` agents + terminal): 147 GAIA/AssistantBench runs, 85 failed.
- **Magentic-One** (AutoGen; Orchestrator, WebSurfer, FileSurfer, Coder, ComputerTerminal): 149 runs, 91 failed.
- **SWE-Agent** (single agent on SWE-Bench Verified; its "agents" are the tools `str_replace_editor`, `bash`, `submit`): 84 runs, 44 failed.

Annotation: three annotators with $\geq$1 year of MAS debugging experience; independent round with confidence flags, discussion to consensus, final cross-review. Round-1 Krippendorff's $\alpha = 0.72$ (agent) and 0.64 (step). The decisive step is the earliest after which failure becomes inevitable, with a recoverability-aware adjustment (an orchestrator that could still recover is not blamed for a specialist's slip). Labels: `mistake_agent`, `mistake_step` (1-based LLM-call index), short free-text `mistake_reason`; no taxonomy.

Metrics and baselines. Agent- and step-level accuracy (a tolerance variant is mentioned) in a *static* setting (trace only) and a *dynamic* setting (trace + replayable environment, allowing re-execution and intervention).

GPT-4o, average over the three systems and 3 runs, agent / step accuracy in % (paper):

Method	with ground truth	without
All-at-Once	62.2 / 28.1	58.0 / 22.7
Binary Search	38.9 / 12.9	37.4 / 9.1
Step-by-Step	60.9 / 16.7	54.0 / 12.5
Static Agentic (theirs)	65.9 / 30.3	59.1 / 26.1
Dynamic Agentic (theirs)	**66.7 / 33.3**	**60.6 / 27.6**

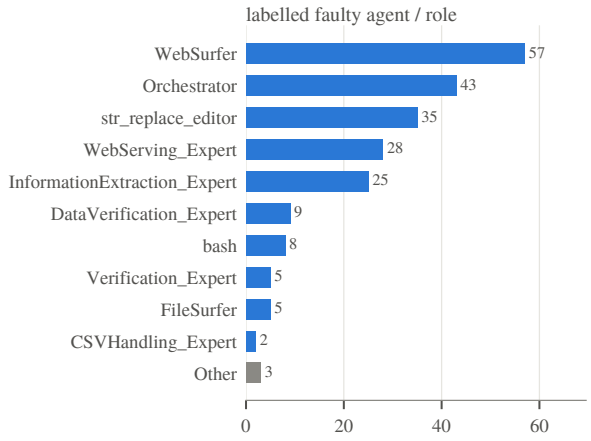
Ablation: removing inputs and metadata from All-at-Once drops step accuracy from 0.28 to 0.16 and agent accuracy by $\approx$22%. Other backbones (Claude-4.5-Sonnet, DeepSeek-R1, GPT-4o-mini, Qwen3-32B, GPT-OSS-20B) appear only in a radar chart; Claude-4.5-Sonnet and DeepSeek-R1 are strongest.

Schema. One directory per run, `data/<system-runs-dir>/<run_id>/`, with five system$\times$benchmark directories named like `captain-runs-gaia` or `swe-agent-runs-swe-bench`. `trace_metadata.json`: `task_id`, `task_instruction`, `ground_truth` (GAIA/AssistantBench; SWE-Bench has `tests_status` instead), `system_name`, `agent_configuration`, `agent_system_intro` (markdown architecture description), `run_id`, and the three labels. `step_records.json`: list of {`step_id`, `agent_id`, `agent_name`, `input` (full chat-completion request with `messages`, screenshots as image parts), `output` (raw response incl. `tool_calls` and `usage`), `tool_logs`}. `images/step_<k>_image_<j>.png`: WebSurfer screenshots (Magentic-One only).

Statistics. computed from the downloaded data (steps = LLM calls).

	All	Captain	Magentic	SWE-Agent
trajectories (all failed)	220	85	91	44
GAIA / AssistantBench / SWE-Bench	147 / 29 / 44	73 / 12 / –	74 / 17 / –	– / – / 44
steps: mean / median	27.1 / 21	20.5 / 13	29.3 / 28	35.3 / 23.5
steps: min – max	5 – 94	5 – 59	6 – 50	14 – 94
distinct agents: mean (max)	2.5 (4)	2.7 (4)	2.1 (3)	2.9 (4)
error step (1-based): mean / median	11.0 / 8	7.0 / 7	12.9 / 8	14.9 / 10
relative error position: mean	0.47	0.46	0.49	0.45
output characters / traj.: mean	15.1 k	9.7 k	24.0 k	7.1 k
LLM tokens / traj.: mean	149 k	110 k	259 k	n/a
screenshots / traj.: mean	9.4	0	22.8	0

Blamed agent: Magentic-One WebSurfer 63%, Orchestrator 30%, FileSurfer 5%; Captain-Agent WebServing_Expert 33%, InformationExtraction_Expert 29%, Orchestrator 18%, DataVerification_Expert 11%; SWE-Agent `str_replace_editor` 80%, `bash` 18%. Magentic-One traces sum to up to $\approx$1 M prompt+completion tokens; SWE-Agent `usage` fields are all zero.



Example trajectory. `captain-runs-gaia/gaia_task_9_gpt-4o_0su143t5rrfs` (5 LLM calls, 2 agents, 2.7 k output characters; rendered as outputs + new tool observations only, since each step's input repeats the whole history). The Orchestrator hands a probability riddle to a CSVHandling_Expert, who asserts that ball 1 is "always in the firing line" and answers 1 (ground truth 3); the Orchestrator accepts it.

Task (`captain-runs-gaia/gaia_task_9_gpt-4o_0su143t5rrfs`)

Here's a fun riddle that I think you'll enjoy.
You have been selected to play the final round of the hit new game show "Pick That Ping-Pong". In this round, you will be competing for a large cash prize. Your job will be to pick one of several different numbered ping-pong balls, and then the game will commence. The host describes how the game works.
A device consisting of a winding clear ramp and a series of pistons controls the outcome of the game. The ramp feeds balls onto a platform. The platfor
[...] (1,362 chars omitted) [...]
y one of the pistons, you win the grand prize, $10,000.
Which ball should you choose to maximize your odds of winning the big prize? Please provide your answer as the number of the ball selected.

1 · Orchestrator

```
“‘json
{
"Retrieval specialist skilled at web search, browsing and summarizing evidence": "WebServing_Expert: A helpful assistant with access to a web
browser. Ask them to perform web searches, open pages, navigate to Wikipedia, answer questions from pages, and or generate summaries.",
"Python coder who can read local files (csv, xlsx, pdf, image, audio) and verify answers with code": "CSVHandling_Expert: CSVHandling_Expert is a
```

```
data analysis specialist proficient in handling CSV files and extracting insights using Python, particularly with libraries like Pandas and NumPy,
and possessing strong collaboration, statistical analysis,
[...]  (123 chars omitted) [...]
ification_Expert:  DataVerification_Expert is a professional adept at using the Bing Search API to conduct specialized searches and thoroughly
verify the accuracy of extracted information, ensuring high reliability in information gathering tasks."
}
“‘
```

2 · Orchestrator

```
CSVHandling_Expert
```

⚠ 3 · CSVHandling_Expert · ANNOTATED ERROR STEP

```
## ANSWER
1
## REASON
The first ball on the platform, which is initially ball number 1, is always in the position to be ejected first by the pistons.  Given the
mechanics of the game, the first ball has the highest probability of being ejected because it is always in the firing line of the pistons.
Therefore, choosing ball number 1 maximizes the odds of winning the grand prize.
TERMINATE
```

4 · Orchestrator

```
## Task
The task was to determine which numbered ping-pong ball a participant should choose in a game show to maximize the odds of winning a cash prize.
The participant needed to select a ball that had the highest probability of being ejected by one of the pistons in a random sequence.
## Results
The conversation concluded with the recommendation to choose ball number 1.
## Reason for the results
The reasoning behind choosing ball number 1 is based on the mechanics of the game.  The first ball on the platform is always in the position to be
ejected first by the pistons.  Thus, it has the highest probability of being ejected because it is co
[...]  (1,166 chars omitted) [...]
ons or issues were evident in the conversation.  The assistant's recommendation followed a logical analysis of the task and the game mechanics as
provided.
### Need to double-check?
No
## Additional information (file path, code blocks, url, etc.)
None
```

5 · Orchestrator

```
## ANSWER
1
## REASON
Ball number 1 should be chosen because it is always in the position to be ejected first by the pistons.  This gives it the highest probability of
being ejected, maximizing the chances of winning the prize.
TERMINATE
```

Annotation

Faulty agent	CSVHandling_Expert
Faulty step	3
Category	—
Reason	The mistake was assuming ball number 1 was most likely to be ejected, ignoring that all balls have equal chances due to random piston firing and changing positions.
Ground truth	3
System output	—
Other labels	**system_name**: captain-agent; **task_id**: ec09fa32-d03f-4bf8-84b0-1f16922c3ae4; **run_id**: gaia_task_9_gpt-4o_0su143t5rrfs; **source_benchmark**: GAIA; **agent_configuration**: {"WebServing_Expert": {"description": "A helpful assistant with access to a web browser. Ask them to perform web searches, open pages, navigate to Wikipedia, answer questions from pages, and or generate summaries.", "system_message": ""}, "CSVHandling_Expert": {"description": "CSVHandling_Expert is a data analysis specialist proficient in handling CSV files [...] (11,149 chars omitted) [...] outliers. By default set to 3.\n\n Returns:\n list: A list of row indices where the z-score is above the threshold.\n \"\"\"\n"]]}; **tests_status**: None; **step_indexing**: 1-based step_id = one LLM invocation (input context + output)

Notes. (i) A step is an API call, not a message: Magentic-One's plan, ledger-update and speaker-selection calls are separate steps (≈29 per trace) and specialist actions appear as tool calls inside them; for SWE-Agent, "agent-level" attribution is really tool-level. (ii) Full observability is the unique asset—per-step inputs, screenshots (mean 23 per Magentic-One trace), token usage and reproducible agent code enabling dynamic attribution (re-execution from a step, counterfactual intervention)—but the inputs are highly redundant, so a trace needs an outputs-only renderer to fit a prompt. (iii) Only failed traces are released; the 160 successful runs are absent, so success-vs-failure detection cannot be studied. (iv) Label quirks: `mistake_step` is a string; one trace has `'none'` (no single decisive step) and one `'Step 36'`. (v) Same two MAS and benchmarks as Who&When but freshly collected (Dec 2025, GPT-4o) with a third single-agent system; blame patterns are similar (WebSurfer-heavy) and step accuracies are correspondingly higher (28–33% vs 5–25%). Reported α is one of the few inter-annotator figures in this group.

3.3 MP-Bench

natural · available · step set + category, 3 annotators · MAS · GAIA / AssistantBench · re-annotates Who&When + MAST logs

Paper	*Rethinking Failure Attribution in Multi-Agent Systems: A Multi-Perspective Benchmark and Evaluation* — In, Tanjim, Subramanian, Kim, Bhattacharya, Sarkhel, Kim, Park, Park (KAIST / Adobe Research), ICML 2026, arXiv:2603.25001 [30]
Links	GitHub `adobe-research/multi-agent-eval-bench` (annotations) · GitHub `yeonjun-in/MP-Bench` (evaluation pipeline, leaderboard) · leaderboard page; the HF mirror `Yeonjun/MP-Bench` holds only README + LICENSE
License	Adobe Research License v1.2 (non-commercial research/teaching only; derivatives must stay non-commercial); the pointed-to logs keep their own terms (Who&When: none stated; MAST repo: MIT)
Status	downloaded in full, ungated; 6.0 MB annotations + 0.5 MB eval code. The release ships *annotation pointers only*: all 289 source logs were fetched from the Who&When and MAST GitHub repos with our own script; `data/mp_bench/`

Description. MP-Bench argues that failure attribution is *non-deterministic*: a failed MAS log usually admits several plausible root causes, so a benchmark should store a *set* of expert perspectives rather than one gold (agent, step). It re-annotates 289 existing, naturally failed logs from 121 MAS configurations (no injection):

- **manual** (169 logs, hand-crafted MAS): 57 Who&When Hand-Crafted Magentic-One logs plus 112 Magentic-One GAIA-validation console logs (86 level-2, 26 level-3) from the MAST failure-taxonomy corpus.
- **automatic** (120 logs, algorithm-generated MAS): the Who&When Algorithm-Generated CaptainAgent logs.

Annotation: three experts recruited on Upwork after a two-stage screening labelled *every* log independently, with no guiding perspective: per step a binary failure-inducing flag, a free-text `fail_reason`, a counterfactual `ideal_action` and a `fail_category` from a 9-class list (Planning, Verification, Reasoning, Contextual Understanding, Tool Usage, Tool/Agent Invocation, Reasoning–Action Mismatch, Resource Access, System Error—not defined in the paper). ≈31 min per manual and 14 min per automatic log, ≈346 expert-hours. **Agreement** is low by design: of all flagged steps 16.2% are flagged by all three annotators, 27.8% by two, 56.1% by one (paper); pairwise disagreement reaches ≈60%. Each step gets a *consensus rate* (fraction of annotators flagging it), and GPT-5.1 consolidates the annotators' reasons and ideal actions per step.

Metrics and baselines. The attributor outputs a *ranked list* of failure steps with reasons (three seeds, consolidated). *Attribution*: nDCG@5 (and nDCG@all) of that ranking against the consensus ranking; *reasoning*: GPT-5.1 judge scores (1–10) for fail reason, ideal action and overall agreement with the consolidated annotator rationale. All-at-Once with taxonomy, nDCG@5 (exponential gain) / judge average (paper):

Model	Hand-crafted	Automatic
Random	0.128 / –	0.315 / –
GPT-4.1	0.431 / 7.63	0.676 / 7.78
GPT-5.1	0.375 / 7.62	0.784 / **8.25**
o3-mini	0.437 / 7.48	0.394 / 7.37
Claude Sonnet 4.5	**0.440** / **7.95**	**0.789** / 8.23
Qwen3-8B	0.294 / 6.11	0.668 / 6.54
GPT-oss-120B	0.425 / 7.51	0.703 / 7.92

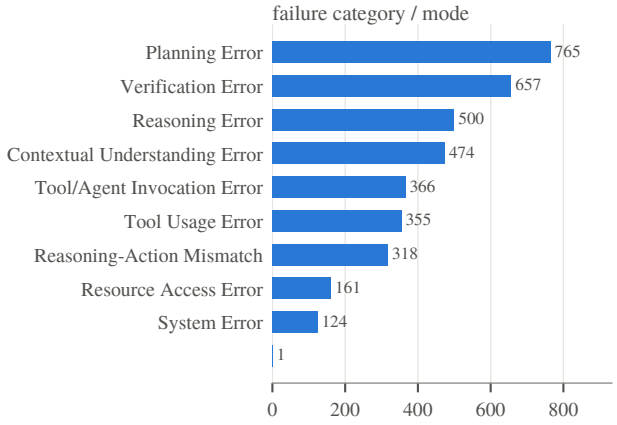
Under this multi-perspective view LLMs are far from random—the opposite conclusion to Who&When's single-label step accuracy. The public leaderboard uses nDCG@5 only (157 manual + 120 automatic logs scored).

Schema. `MP-Bench/<annotator 1-3>/<manual|automatic>/<n>.json` (867 files = 3×289): `log_source` (GitHub URL of the Who&When JSON or the MAST `console_log.txt`) and `annotation`, a possibly empty list of {`step`, `fail_annotation` ("1"), `fail_category`, `fail_reason`, `ideal_action`}; `step` is a 0-based index (stored as a string) into the step list built by the official `build_annotated_from_log_source.py`: Who&When `history` entries, or MAST console turns split on `——- <Agent> ——-` delimiters (question from `prompt.txt`, answer from `expected_answer.txt`). The normalised form is {`question`, `ground_truth`, `is_correct`, `system_prompt`, `history`:[{`content`, `role`, `step`, + the four label fields}]}. Our copy keeps the fetched logs under `data/mp_bench/sources/<sha1(url)[:16]>/` with an `index.json` map.

Statistics. computed from the downloaded data over 250 of the 289 logs (120 automatic + 130 manual; 39 MAST level-2/3 logs were still being fetched); annotation statistics cover the same 250 logs.

	All	manual	automatic
logs analysed (all failed)	250	130	120
W&W HC / MAST L2 / W&W AG	57 / 73 / 120	57 / 73 / –	– / – / 120
steps: mean / median	23.3 / 10	36.8 / 31.5	8.7 / 10
steps: min – max	5 – 130	5 – 130	5 – 10
distinct agents: mean (max)	3.4 (6)	3.3 (6)	3.6 (4)
flags per annotator-log: mean / max	5.0 / 59		
flagged steps per log (union): mean / max	9.2 / 78	12.7 / 78	5.3 / 10
steps with 1 / 2 / 3 votes	1242 / 659 / 387	934 / 473 / 245	308 / 186 / 142
earliest majority step, rel. pos.: mean / med.	0.40 / 0.33	0.45 / 0.37	0.36 / 0.30
characters / log: mean	32.1 k	51.0 k	11.6 k

Consensus in our subset: 54% / 29% / 17% of flagged steps carry 1 / 2 / 3 votes (paper 56.1 / 27.8 / 16.2). Agent at flagged steps, manual: orchestrator 60% (`Orchestrator` 33.7% + `MagenticOneOrchestrator` 26.8%), WebSurfer 31.4%, FileSurfer 3.6%, Assistant 3.1%; automatic: Verification_Expert 17.1%, Computer_terminal 9.2%, Validation_Expert 6.1%, DataVerification_Expert 4.5%, the rest spread over many `*_Expert` names.



Example trajectory. `manual/18.json` (MAST Magentic-One GAIA level-2 log; 9 steps, 3 agents, 13.7 k characters). The first Bing search already shows "beta-geometric" in a snippet; the WebSurfer's click on a paper silently fails, it then summarises "Markov models" and the orchestrator answers *Markov Model* (ground truth *beta geometric*). Five steps are flagged in total: 5, 7 and 8 by all three annotators, 3 only by annotator 3, 6 only by annotator 2—the renderer marks all five; the annotation box lists the per-annotator (step, annotator, category) labels with their reasons (truncated) and the per-step votes, and its "faulty agent" row names only the agent of the first flagged step (steps 6 and 8 blame the orchestrator).

Task (`manual/18.json`)

What two-word type of model did Manash Pratim Kashyap's and PS Fader's studies in customer retention studies published during 2018-2019 have in common (no punctuation)?

0 · user

```
What two-word type of model did Manash Pratim Kashyap's and PS Fader's studies in customer retention studies published during 2018-2019 have in
common (no punctuation)?
```

1 · MagenticOneOrchestrator

```
We are working to address the following user request:
What two-word type of model did Manash Pratim Kashyap's and PS Fader's studies in customer retention studies published during 2018-2019 have in
common (no punctuation)?
To answer this request we have assembled the following team:
Assistant:  A helpful and general-purpose AI assistant that has strong language skills, Python skills, and Linux command line skills.
ComputerTerminal:  A computer terminal that performs no other action than running Python scripts (provided to it quoted in "'python code blocks),
or sh shell scripts (provided to it quoted in "'sh code blocks).
FileSurfer:  An
[...]  (2,197 chars omitted) [...]
tion studies.
- **Assistant Task**:  Conclude by presenting the likely common two-word model based on the information obtained.
```

```
The focus will be on employing WebSurfer to locate and extract pertinent information about the studies and the models used.
```

2 · MagenticOneOrchestrator

```
Please conduct a search for academic publications or articles by Manash Pratim Kashyap and PS Fader on customer retention from 2018-2019.  Focus on
identifying the type of models used in these studies and look for a two-word model that is common to both.
```

⚠ 3 · WebSurfer · ANNOTATED ERROR STEP

```
I typed 'Manash Pratim Kashyap PS Fader customer retention model 2018-2019' into the browser search bar.
The web browser is open to the page [Manash Pratim Kashyap PS Fader customer retention model 2018-2019 - Cerca](https://www.bing.com/search?q=
Manash+Pratim+Kashyap+PS+Fader+customer+retention+model+2018-2019&FORM=QBLH).
The viewport shows 39% of the webpage, and is positioned at the top of the page
The following text is visible in the viewport:
Microsoft e i suoi fornitori di terze parti utilizzano cookie e tecnologie simili per fornire, gestire e migliorare i servizi e gli annunci offerti.
Se sei d'accordo, useremo questi dati per la
[...]  (2,605 chars omitted) [...]
fb:app_id":  "3732605936979161",
"og:image":  "http://www.bing.com/sa/simg/facebook_sharing_5.png",
"og:type":  "website",
"og:image:width":  "600",
"og:image:height":  "315"
}
}
Here is a screenshot of the page.
<image>
```

4 · MagenticOneOrchestrator

```
Please check the search results for any documents or mentions that specify the model type used by both Manash Pratim Kashyap and PS Fader in their
customer retention studies from 2018-2019.
```

⚠ 5 · WebSurfer · ANNOTATED ERROR STEP

```
I clicked 'A Simple Model for Analyzing the Customer Retention Comparing Rural and Urban Store'.
The web browser is open to the page [Manash Pratim Kashyap PS Fader customer retention model 2018-2019 - Cerca](https://www.bing.com/search?q=
Manash+Pratim+Kashyap+PS+Fader+customer+retention+model+2018-2019&FORM=QBLH).
The viewport shows 39% of the webpage, and is positioned at the top of the page
The following text is visible in the viewport:
Microsoft e i suoi fornitori di terze parti utilizzano cookie e tecnologie simili per fornire, gestire e migliorare i servizi e gli annunci offerti.
Se sei d'accordo, useremo questi dati per la personal
[...]  (3,091 chars omitted) [...]
E CUSTOMER RETENTION ...31 mar 2019 · The aim of this study is to explore relationships among five factors:  customer relationship management,
customer satisfaction, customer loyalty, customer profitability, and ...
Here is a screenshot of the page.
<image>
```

⚠ 6 · MagenticOneOrchestrator · ANNOTATED ERROR STEP

```
Please extract more detailed information from the identified webpage, focusing on the specific type of model mentioned in the studies, and verify
if it matches a two-word model type used by both authors.
```

⚠ 7 · WebSurfer · ANNOTATED ERROR STEP

```
The studies by Manash Pratim Kashyap and others, including PS Fader, during 2018-2019, primarily mention customer retention models that include the
use of Reinartz and Kumar's model from 2000, as well as a modified version by Fader, Hardie, and Lee in 2005.  These models focus on assessing
customer retention by evaluating customer switching behaviors and often employ migration or Markov models to estimate the probabilities of
customers transitioning between states.  The models emphasize analyzing customer relationship dynamics between rural and urban stores, helping
in the prediction of future purchase patterns by leveraging structural characteristics and retention estimates.
```

⚠ 8 · MagenticOneOrchestrator · ANNOTATED ERROR STEP

```
FINAL ANSWER: Markov Model
SCENARIO.PY COMPLETE !#!#
SCENARIO.PY RUNTIME: 494 !#!#
RUN.SH COMPLETE !#!#
```

Annotation

Faulty agent	WebSurfer
Faulty step	3, 5, 6, 7, 8
Category	["Contextual Understanding Error", "Reasoning Error", "Reasoning-Action Mismatch", "Tool Usage Error", "Tool/Agent Invocation Error", "Verification Error"]
Reason	[step 3, annotator 3, Tool/Agent Invocation Error] Found search results containing 'beta-geometric (BG) distribution' reference but failed to highlight or extract this key information for the task; [step 5, annotator 1, Tool Usage Error] WebSurfer attempted to click on the Academia.edu link 'A Simple Model for Analyzing the Customer Retention Comparing Rural and Urban Store' but the navigation failed. The browser remained on the Bing search results page instead of navigating to the actual article. This prevented WebSurfer from accessing the detailed content of Kashyap's paper which would have provided more specific model information.; [step 5, annotator 2, Reasoning-Action Mismatch] The agent says it clicked a paper, but the page content did not change and no paper was opened.; [step 5, annotator 3, Tool/Agent Invocation Error] Failed to navigate to the academic paper, clicking did not successfully access the detailed content needed for analysis; [step 6, annotator 2, Verification Error] The Orchestrator did not verify whether the WebSurfer had actually opened a p [...] (1,003 chars omitted) [...] swer 'beta geometric' was clearly visible in the search results; [step 8, annotator 2, Verification Error] The Orchestrator finalized the answer without verifying whether the information from earlier steps was correct or supported by real sources.; [step 8, annotator 3, Reasoning Error] Concluded with 'Markov Model' instead of 'beta geometric' despite evidence in search results pointing to beta-geometric distribution
Ground truth	beta geometric
System output	FINAL ANSWER: Markov Model SCENARIO.PY COMPLETE !#!# SCENARIO.PY RUNTIME: 494 !#!# RUN.SH COMPLETE !#!#
Other labels	**per_annotator**: {"1": [{"step": "5", "fail_annotation": "1", "fail_category": "Tool Usage Error", "fail_reason": "WebSurfer attempted to click on the Academia.edu link 'A Simple Model for Analyzing the Customer Retention Comparing Rural and Urban Store' but the navigation failed. The browser remained on the Bing search results page instead of navigating to the actual articl [...] (4,490 chars omitted) [...] he search results more carefully and identified 'beta geometric' as the common model type based on the Fader and Hardie (2007) reference"}]}; **votes_per_step**: {"5": ["1", "2", "3"], "7": ["1", "2", "3"], "8": ["1", "2", "3"], "6": ["2"], "3": ["3"]}; **earliest_majority_step**: 5; **source_kind**: MAST/MagenticOne_GAIA L2; **log_source**: https://github.com/multi-agent-systems-failure-taxonomy/MAST/tree/main/traces/MagenticOne_GAIA/gaia_validation_level_2__MagenticOne/0ff53813-3367-4f43-bcbd-3fd725c1bf4b/0/console_log.txt; **is_correct**: None; **step_indexing**: 0-based index into reconstructed history (Who&When JSON history / MAST console-log turns)

Notes. (i) No single gold label: each log has three independent annotation sets, annotators flag several steps each (mean 5 per annotator-log, union mean 9, max 78), and the task is ranking/set prediction, not "find the decisive step". The "earliest majority-flagged step" position (0.40) is our derived

quantity for comparability, not an official label. (ii) Categories are per flag and per annotator—the same step often receives different categories (step 5 of the example: Tool Usage / Reasoning–Action Mismatch / Tool/Agent Invocation); `ideal_action` is unique to this dataset. (iii) Pointer-only release: trajectories must be fetched from the Who&When and MAST repos, and step indices depend on the official parser (we re-implemented it rather than executing it); MAST logs name the orchestrator `MagenticOneOrchestrator` and start with the raw `user` prompt, Who&When HC logs use `Orchestrator (thought)` / `Orchestrator (-> X)`. (iv) Counts differ across sources (README 289 / 121 MAS; eval README 169 + 126; released files 169 + 120; leaderboard 157 + 120)—treat 289 (169 / 120) as canonical. (v) Overlap: 177 of the 289 logs are Who&When logs re-annotated, a direct test of label reliability (the three annotators frequently disagree with each other and with Who&When's single decisive step); the other 112 are output-only Magentic-One console logs from MAST. Automatic (5–10 step) logs are much easier (nDCG ≈0.7–0.8) than hand-crafted ones (≈0.4). (vi) Non-commercial licence; the consolidation and judge steps of the evaluation need paid API models.

3.4 Who&When Pro

injected · available · subset downloaded · agent + step + failure mode · single + multi-agent · text / image / video · 26 benchmarks, 15 frameworks

Paper	*Who&When Pro: Can LLMs Really Attribute Failures in AI Agents?* — Liu, Xi, Zhang, Zeng, Yue, Wang, Kang, Wu, Wang (Penn State / AG2AI, Oregon State, Google DeepMind; the Who&When group), ICML 2026, arXiv:2607.09996 [44]
Links	HF `Leoxx/whowhen_pro` · GitHub `ag2ai/whowhen_pro` (`whowhen_eval` harness: per-framework renderers, LiteLLM runner, scorer, leaderboard; generation pipeline not yet released) · project page
License	CC BY 4.0 (HF dataset card); upstream benchmark assets keep their own licences
Status	ungated; 21.7 GB in total (`text.jsonl` 75.6 MB, `video.jsonl` 21.3 MB, `image.jsonl` 6.79 GB, `image_gui.jsonl` 11.19 GB with base64 screenshots, 21,850 video frames, `taxonomy.yaml`). **Subset downloaded**: text and video splits complete (6,257 + 916 rows), `taxonomy.yaml`, plus the first ≈1 GB of `image.jsonl` (166 rows) and of `image_gui.jsonl` (118 rows); video frames not fetched; 1.36 GB in `data/who_and_when_pro/`

Description. The scaled-up successor of Who&When: 12,326 failed trajectories of single- and multi-agent systems across text, image (incl. GUI) and video, with *who* (agent), *when* (step) and *what* (error mode) labels that are exact *by construction*. **Construction** (counterfactual replay + failure injection): base agents are run on each (framework, benchmark) pair and only *successful* runs are kept as seeds. For a chosen step t the successful prefix is replayed exactly (static tool calls from a SHA-256-keyed cache, stateful environments such as browsers, OSWorld and ALFWorld re-executed with fidelity checks), an erroneous action targeting one taxonomy mode is substituted at t—written by the base agent itself under an injection prompt, so it stays in-distribution—and the agent continues on-policy. Only continuations that end in failure are kept; traces that leak construction artefacts or where the answer is already salient before t are filtered. Coverage: 26 benchmarks in 9 task categories, 15 frameworks:

- **text**: smolagents, MathChat, Magentic-One, Multi-Agent Debate, ALF-Agent (ReAct), MacNet, DyLAN, MetaGPT;
- **image / GUI**: smolagents, PixelCraft, Debate; CoAct-1, OpenAI-CUA, AgentOccam, Gemini/WebVoyager;
- **video**: EVA, DVD (LVBench).

7 of the 15 frameworks are single-agent. **Taxonomy**: six families—Perception (P.1–2), Reasoning (R.1 hallucination, R.2 reasoning, R.3 numerical, R.4 task misunderstanding), Planning (PL.1–2), Action (A.1 tool parameter, A.2 output format, A.3 premature termination, A.4 looping), Verification (V.1 context loss, V.2 inadequate verification), Coordination (C.1 delegation, C.2 communication, C.3 over-reliance); the paper says 18 modes, the released `taxonomy.yaml` has 17. **Validation**: three author-side annotators reviewed 100 stratified traces: 94% of step and 90% of agent/family labels approved, Fleiss' $\kappa = 0.73$, 2% judged to have no clear decisive error.

Metrics and baselines. Predict agent (scored on multi-agent traces only), step coordinate (exact match; Step@k tolerance in the appendix) and error mode (macro-F1); *Joint* = all three correct; per-framework averages. Protocols: All-at-Once, Step-by-Step, Binary-Search. All-at-Once, Agent / Step / Error / Joint in % (paper):

Model	Split	Agent	Step	Error	Joint
GPT-5.4	text	**55.7**	**72.3**	15.3	21.3
Claude Sonnet 4.6	text	54.9	69.8	**19.1**	**22.4**
Gemini 3 Flash	text	52.1	71.2	16.2	17.3
GPT-5.4	image	**67.7**	**65.2**	26.5	**30.2**
Claude Sonnet 4.6	image	67.5	63.8	**29.5**	29.6
Gemini 3 Flash	image	62.5	63.9	26.0	29.6
GPT-5.4	video	**85.1**	60.5	33.8	21.6
Claude Sonnet 4.6	video	79.8	**64.4**	37.4	**26.4**
Gemini 3 Flash	video	74.4	59.8	**40.0**	23.0

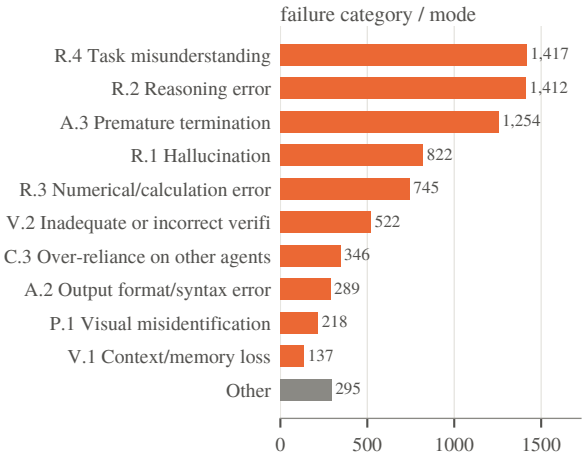
Best open-weight text results: Qwen3.5-122B Step 73.9, GLM-5 Joint 25.3. Step@1 tolerance adds +13.9 pp (text), +16.9 (image), +28.1 (video). Headline finding: models detect *symptoms* but mis-locate *root causes* (low Error/Joint), worst on multimodal and long traces.

Schema. `data/<split>.jsonl`, one row per trace: `id` (`<benchmark>_<framework>_<nnnn>`), `framework`, `benchmark`, and JSON-encoded strings `task` ({`query`, `answer`, `options?`, `images?`}), `trajectory`, `ground_truth` ({`mode`, `agent|agents`, `step|round+position|stage`, `accepted_predictions?`}; `agent` is `null` for single-agent frameworks) and `extras` (agent roster, `final_answer`, `termination`, `topology`, `post_injection_turns`, ...). The `trajectory` shape is *framework-specific*—8 shapes and 5 step-coordinate conventions: flat 1-based `step_number` (ALF-Agent, GUI agents), flat cursor starting at 0 if a planning step exists (smolagents), `"R.P"` round.position with P = 0 orchestrator / 1 specialist (Magentic-One; round-only for Debate/DyLAN), `stage` 0/1/2 (MetaGPT, MacNet) and native index minus an offset (DVD/EVA). Injected and replayed entries are flagged in the raw rows (`is_injected`, `injected`, `phase`, `is_replayed`); the official renderers strip them.

Statistics. computed from the downloaded subset (7,457 of 12,326 rows: text and video complete; image and image_gui are the first rows of each file, i.e. browsecomp_vl/smolagents and webvoyager/AgentOccam + 2 OSWorld/CoAct rows—indicative only).

	All	text	video	image*	image_gui*
trajectories (all failed, one injection)	7457	6257	916	166	118
scorable steps: mean / median	5.7 / 4	6.1 / 4	3.1 / 2	3.8 / 4	7.1 / 4
steps: min – max	1 – 79	1 – 31	2 – 49	1 – 12	1 – 79
injected step rel. pos.: mean / median	0.74 / 0.83	0.73 / 0.80	0.79 / 1.0	0.84 / 1.0	0.56 / 0.50
characters / traj. (no images): mean	9.4 k	8.6 k	15.4 k	9.0 k	6.4 k
characters / traj.: median	4.7 k	5.1 k	1.3 k	8.3 k	3.0 k

Roster size mean 1.9 (max 5); 4,617 single-agent vs 2,840 multi-agent rows (62% / 38%); 0 rows whose label could not be matched to a rendered step. 12 of the 15 frameworks and 20 of the 26 benchmarks appear in the subset—rows per framework: smolagents 3,442, MathChat 920, Magentic-One 811, EVA 748, Debate 445, ALF-Agent 311, MacNet 261, DVD 168, DyLAN 152, AgentOccam 116, MetaGPT 81, CoAct 2. Several frameworks have fixed lengths: Debate always 6 steps (2 rounds × 3 agents), MacNet 4, MetaGPT 3, ALF-Agent always 30 (the step budget); smolagents 4.5 and Magentic-One 6.1 on average. Error families: R 59%, A 22%, V 9%, C 5%, P 3%, PL 2%. Faulty agent on multi-agent rows is nearly constant per framework: MathChat `assistant` 100%, Magentic-One `orchestrator` 78%, MetaGPT `architect` 83%, DVD `orchestrator` 90%; Debate/DyLAN labels list several agents (C.3 over-reliance).



Example trajectory. Row `gaia_magentic-one_0044` (text split; 7 scorable steps, 5-agent roster, 1.9 k characters). The injected WebSurfer step 0.1 (a Bing search) is labelled a hallucination; from there the orchestrator works with 791.8 k women and 586.9 k men, and Coder and ComputerTerminal faithfully compute 204.9 (ground truth 234.9). Label: `websurfer`, step `"0.1"`, mode R.1 (hallucination); no free-text reason exists.

Task (`gaia_magentic-one_0044`)

Take the gender split from the 2011 Bulgarian census about those who have completed tertiary education. Subtract the smaller number from the larger number, then return the difference in thousands of women. So if there were 30.1 thousand more men, you'd give "30.1"
Provide a concise, precise answer. State your final answer on the last line, prefixed with "FINAL ANSWER: "

0.0 · orchestrator

```
Search for the official 2011 Bulgarian census data on educational attainment, specifically the number of men and women who have completed tertiary
education.  Extract the relevant numbers.
```

⚠ 0.1 · websurfer · ANNOTATED ERROR STEP

```
I typed '2011 POPULATION CENSUS Main Results tertiary education men women 2011 Bulgaria' into the search bar.
The web browser is open to the page [2011 Bulgarian census educational attainment tertiary education numbers by gender - Search](https://
www.bing.com/search?q=2011+Bulgarian+census+educational+attainment+tertiary+education+numbers+by+gender&form=QBLH&sp=-1&lq=0&pq=&sc=0-0&qs=n&sk=
&cvid=0A81DC5DB135488789B8CEA69382C803).
The viewport shows 54% of the webpage.
The following text is visible in the viewport:
- "[PDF] 2011 POPULATION CENSUS Main Results - nsi.bg"
- "Women have completed tertiary education significantly more oft
[...]  (245 chars omitted) [...]
attainment and extract the number of men and women with tertiary education, I will enter a relevant query into the search box.", "input_field_id":
59, "text_value":  "2011 Bulgarian census educational attainment tertiary education numbers by gender"}}]
```

1.0 · orchestrator

```
Subtract 586.9 from 791.8 to get the difference in thousands of women who completed tertiary education according to the 2011 Bulgarian census.
Output the answer as:  'FINAL ANSWER: [number]'
```

1.1 · coder

```
"'python
women = 791.8
men = 586.9
difference = women - men
print(f"FINAL ANSWER: {difference:.1f}")
"'
```

2.0 · orchestrator

```
Run the provided Python code to output the final answer.
```

2.1 · computerterminal

```
FINAL ANSWER: 204.9
```

3.0 · orchestrator

```
According to the 2011 Bulgarian census, 791.8 thousand women and 586.9 thousand men had completed tertiary education.  The difference between these
numbers is 204.9 thousand, with more women than men.
FINAL ANSWER: 204.9
```

Annotation

Faulty agent	websurfer
Faulty step	0.1
Category	R.1 (Hallucination)
Reason	
Ground truth	234.9
System output	204.9
Other labels	**ground_truth_raw**: {"agent": "websurfer", "step": "0.1", "mode": "R.1"}; **framework**: magentic-one; **benchmark**: gaia; **extras**: {"agents": [{"id": "orchestrator", "role": "Orchestrator"}, {"id": "websurfer", "role": "WebSurfer"}, {"id": "coder", "role": "Coder"}, {"id": "filesurfer", "role": "FileSurfer"}, {"id": "computerterminal", "role": "ComputerTerminal"}], "final_answer": "204.9", "termination": "satisfied"}; **step_indexing**: framework-specific coordinate as rendered by the official harness (see script docstring); **injected_entries**: [2]

Notes. (i) Labels by construction, not annotation: exact (agent, step, mode) and no rationale, but the failures are synthetic and on-policy-but-prompted. Because the replayed prefix must be a correct prefix of a successful run, injections sit late (mean relative position 0.74, median 0.83; paper: 64%

in the 40–80% range) whereas natural-failure datasets put the decisive error near the middle (Who&When 0.46, TraceElephant 0.47, MP-Bench 0.40). (ii) Mostly single-agent and short: 62% of rows have one agent, median length is 4 steps, and the agent axis is scored only on the 38% multi-agent rows where the culprit is often predictable from the framework alone. Only Magentic-One (GAIA, SimpleQA-Verified) resembles the Who&When / TraceElephant / MP-Bench setting. (iii) No common step format: the harness's renderers and scorer must be reproduced exactly for comparable numbers (our statistics follow them); Debate/DyLAN are scored at round level and 109 MacNet rows carry `accepted_predictions` alternates—even this benchmark concedes multiple acceptable answers. (iv) Label leakage: injected/replayed flags inside `trajectory` must be stripped before evaluation. (v) Taxonomy: paper 18 modes vs. 17 in `taxonomy.yaml` with slightly different names; Reasoning + Action modes make up 81% of labels, Coordination 5%, Perception/Planning are rare in the text split. (vi) Practical: text + video (97 MB) suffice for text-only attribution experiments; image/GUI rows embed screenshots (18 GB); an earlier anonymous release `tmpxv7/who-when-pro` (18-mode taxonomy) and a text-only re-upload also circulate.

3.5 LongRCA Bench

natural · available · agent + step · MAS · long-horizon · SWE-bench Pro / Terminal-Bench 2 / TravelPlanner / VitaBench / WebArena

Paper	*LongRCA Bench: Root-Cause Localization in Long-Horizon Agent Trajectories* — Zhang, Feng, Pei et al. (CNIC CAS, UCAS, Chongqing Univ., ICT CAS, SMU, Tongyi Lab, Tsinghua), arXiv:2608.15242 (v4, Sep 2026, no venue) [76]
Links	HF `CLoud5-real/longrca-bench` · project page + leaderboard (no separate code repository found)
License	none stated on the HF card; paper under the arXiv non-exclusive licence
Status	downloaded in full, ungated; 351 MB; `data/longrca/` (1,140 full + 200 mini JSON files, 2 parquet files)

Description. LongRCA targets the *long-horizon* regime of the Who&When task: given a complete failed trajectory (`question_ID` + `history`), predict the responsible workflow role (`mistake_agent`) and the earliest decisive root-cause step (`mistake_step`, 0-based); both are scored independently. Median length is 145 steps and the reference root precedes the end of the trajectory by a median of 48 steps. All 1,140 failures are natural, evaluator-confirmed failed attempts freshly generated by the authors with 2026 models (MiniMax-M2.5, Kimi-K2.5, Qwen3.5-Plus) on five sources:

- **SWE-bench Pro** (128) and **Terminal-Bench 2** (42): a custom DiagnostAgent–ActionAgent–JudgeAgent team (diagnosis–execution–verification).
- **TravelPlanner** (685): AutoGen MagenticOneGroupChat with manager, stay, transport, food_attr and verifier agents.
- **VitaBench** (108, Chinese-language tool use): Action_Expert + Computer_terminal.
- **WebArena Verified** (177): Task_Planner–Action_Expert (WebSurfer)–Verification_Expert controller.

Normalisation keeps original content, roles and order with stable indices; event granularity therefore differs across sources. **Annotation**: 22 CS graduate students, calibration on 100 trajectories, then full annotation with the task, the step-indexed history, the evaluator outcome and LLM-written navigation summaries (no root-cause judgement); 30–40 min per trajectory. 294 trajectories were annotated twice or thrice: **role agreement 66.1%, exact-step 40.3%, step ±5 50.3%**; 38.1% of pairs differ by >20 steps. Disagreements were resolved by two experts; 60 of 1,200 reviewed trajectories were dropped. The root is the earliest step that introduces a decisive, unrepaired error; if a hand-off instruction already contains the error, the instruction step is the root. **LongRCA-Mini** is a fixed 200-trajectory subset (40 per source, seed 20260821) scored Macro-5; its fidelity to the full set is mean $|\Delta|$ 1.7 pp. No failure taxonomy, only free-text `mistake_reason`.

Metrics and baselines. Agent Acc, Step Acc (exact), Step ±5, Joint Exact / Joint ±5 and Step MAE; invalid outputs count as errors. Full benchmark, DeepSeek-V4-Flash backbone, in % (paper):

Method	Agent	Step	Step ±5
All-at-Once	26.2	7.6	19.9
Step-by-Step	22.2	5.3	16.9
Binary Search	23.0	3.4	13.3
ECHO	27.5	13.2	24.7
FALAT	19.0	2.8	12.5
RCTA (theirs)	**51.1**	**24.1**	**37.4**

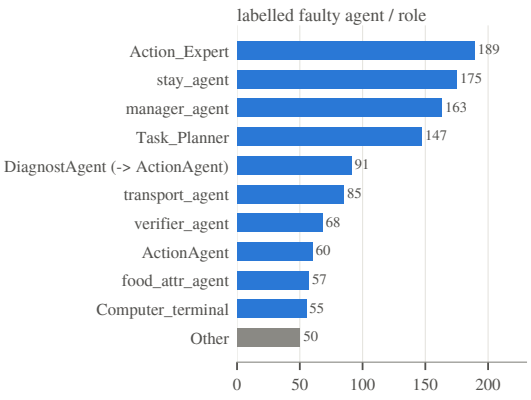
Always predicting each source's majority role already gives 40.3% Agent Acc. The same six methods reach 30–42% Step Acc on Who&When (median 10 steps) but 3–24% here. On Mini, RCTA scores 60.5 / 27.0 (Macro-5 agent / step) with V4-Flash and up to 64.5 / 30.0 with GPT-5.6.

Schema. `longrca-full.parquet` (1,140 rows) and `longrca-mini.parquet` (200 rows), plus one JSON per trajectory under `data/{full,mini}/<source>/<source>__NNN.json`. Fields: `question_ID`, `history` (list of {`step`, `name`, `role`, `content`}; `role` is `assistant` for agent outputs and `user` for terminal/tool/evaluator records; hand-off records carry composite names such as `DiagnostAgent (-> ActionAgent)`), `mistake_agent`, `mistake_step`, `mistake_reason`. Methods receive only `question_ID` and `history`; evaluator feedback ("Run finished: . . . ") is already inside `history`.

Statistics. computed from the downloaded data (they match the paper to within rounding).

	All	SWE	TB2	TP	Vita	WA
trajectories: full / mini	1140 / 200	128 / 40	42 / 40	685 / 40	108 / 40	177 / 40
steps: mean / median	156 / 145	285 / 258	272 / 214	166 / 165	55 / 51	59 / 63
steps: min – max	10 – 728	80 – 728	44 – 546	16 – 524	10 – 130	13 – 63
distinct roles: mean	5.4	7	7	5.9	2	4
relative root position: mean	0.53	0.25	0.29	0.60	0.45	0.56
root-to-end distance: median	47.5	184	152	55	24	26
characters / trajectory: mean	182 k	452 k	297 k	163 k	23 k	131 k

Root step (0-based) mean 78.3, median 55.5, max 521; 28.3% of trajectories have >100 steps after the root, 48.6% >50. ≈45 k tokens per trajectory on average (max 1.2 M characters). The sender of the root step equals the responsible role in 99.2% of records; 14.9% of trajectories contain explicit hand-off markers.



Example trajectory. `webarena_verified__099` (13 steps, 4 roles, 28.7 k characters)—the shortest English-language record, chosen for space; a typical trajectory is ten times longer. The Verification_Expert correctly finds that neither of the two reviews mentions price, but then reports `SUCCESS` with an empty list instead of the `NOT_FOUND` status the WebArena RETRIEVE protocol requires.

Task (webarena_verified__099)

Get name(s) of reviewer(s) who mention price being unfair for the product on the current page

0 · Computer_terminal

```
Get name(s) of reviewer(s) who mention price being unfair for the product on the current page
```

1 · Computer_terminal

```
Orchestration (progress_packet):
cycle=1
```

2 · Task_Planner

```
Orchestration target: Planner
This run uses an explicit progress packet handoff. Plan only the next browse step.
Task prompt:
You are an autonomous web agent operating in E-commerce Store. Begin from the provided start URLs and work within the current session to complete
the task objective.
## Site Context
- Platform: E-commerce Store
- Description: An online shopping platform where customers can browse products, add items to cart, and make purchases.
- Authentication: You are already logged in as emma.lopez@gmail.com. To re-authenticate, use credentials: emma.lopez@gmail.com / Password.123.
If re-authentication fails, terminate with PER
[...] (2,605 chars omitted) [...]
this packet turn:
- Choose exactly one concrete next browse step.
- If the recent tactic already failed, choose a different action.
- Use status fail only when no practical next step remains or the task is clearly blocked.
- Do not use markdown fences.
```

3 · Task_Planner

```
Let me analyze the task:
1. **Task Objective**: Get name(s) of reviewer(s) who mention price being unfair for the product on the current page
2. **Start URL**: http://localhost:7770/haflinger-men-s-wool-felt-open-back-slippers-beige-550-peat-us-7.html
3. **Task Type**: RETRIEVE (we're getting data about reviewers)
The task is asking me to find reviewers who mentioned that the price is unfair for this product. This requires:
1. First, navigate to the product page
2. Find the reviews section
3. Look for reviews that mention "price being unfair" or similar
4. Extract the reviewer names from those specific reviews
Since this is the first cyc
[...] (112 chars omitted) [...]
king with.
Let me plan the first browse step:
- Navigate to the start URL to load the product page
- Then inspect the page for a reviews section
- Look for reviews that mention the price being unfair
The first step should be to load the product page.
```

4 · Task_Planner

```
{"status": "act", "reason": "Starting task to find reviewer names who mention price being unfair. Need to load the product page first.",
"current_subgoal": "Load product page and find reviews section", "instruction_for_web_surfer": "Navigate to http://localhost:7770/
haflinger-men-s-wool-felt-open-back-slippers-beige-550-peat-us-7.html. Once loaded, look for the product reviews section. Identify if there is a
Reviews tab, link, or section on the page. Report back the location of the reviews section and any visible review content.", "success_condition":
"Product page loaded and reviews section located", "facts_to_verify": ["Product page loads successfully", "Reviews section exists on the page"]}
```

5 · Task_Planner

```
Orchestration target: WebSurfer
This is an explicit progress packet handoff from Planner to WebSurfer.
Do not ask for the task again. Use the packet below and take the minimum number of browser actions needed.
Task prompt:
You are an autonomous web agent operating in E-commerce Store. Begin from the provided start URLs and work within the current session to complete
the task objective.
## Site Context
- Platform: E-commerce Store
- Description: An online shopping platform where customers can browse products, add items to cart, and make purchases.
- Authentication: You are already logged in as emma.lopez@gmail.com. To re-authenticate, use
[...] (2,605 chars omitted) [...]
ion price being unfair for the product on the current page
- Latest critic feedback:
(none)
- Stop condition:
Product page loaded and reviews section located
Reply with only direct browser observations, concrete evidence, or a precise blocking reason.
```

6 · Action_Expert

```
I clicked '2 Reviews'.
The web browser is open to the page [HAFLINGER Men's Wool Felt Open Back Slippers, Beige 550 Peat, US:7](http://localhost:7770/
haflinger-men-s-wool-felt-open-back-slippers-beige-550-peat-us-7.html).
The viewport shows 34% of the webpage, and is positioned 48% down from the top of the page
The following text is visible in the viewport:
Skip to the beginning of the images gallery Details
Reviews
2Customer Reviews Wonderful!
Rating 100%
Nothing.....everyday use.
Review by Jay Posted on 4/18/23 Joe
Die Haflinger sind wie beschrieben und erwartet sehr gut
Review by Josef Bürger Posted on 4/18/23 You'
[...] (2,605 chars omitted) [...]
description": "",
"og:url": "http://localhost:7770/haflinger-men-s-wool-felt-open-back-slippers-beige-550-peat-us-7.html",
"product:price:amount": "108",
"product:price:currency": "USD"
}
}
```

Here is a screenshot of the page.

7 · Task_Planner

```
Orchestration target: Critic
This run uses an explicit progress packet handoff. Decide whether the task is complete, needs one more step, or should fail fast.
Task prompt:
You are an autonomous web agent operating in E-commerce Store. Begin from the provided start URLs and work within the current session to complete
the task objective.
## Site Context
- Platform: E-commerce Store
- Description: An online shopping platform where customers can browse products, add items to cart, and make purchases.
- Authentication: You are already logged in as emma.lopez@gmail.com. To re-authenticate, use credentials: emma.lopez@gmail.com / Password.123.
[...] (2,605 chars omitted) [...]
t practically reachable, recent turns are repeating, or the task is blocked.
- If status is complete or fail, final_response must be a valid WebArena final JSON object.
- If status is continue, final_response must be null.
- Do not use markdown fences.
```

8 · Verification_Expert

```
Let me analyze the current state of the task:
1. **Task Objective**: Get name(s) of reviewer(s) who mention price being unfair for the product on the current page
2. **Current Progress**:
- The product page is loaded
- Reviews section is visible (2 customer reviews)
- The reviews visible are:
- Review 1: "Wonderful!" by Jay - "Nothing.....everyday use."
- Review 2: "Joe" by Josef Bürger - "Die Haflinger sind wie beschrieben und erwartet sehr gut"
3. **Analysis**:
- Looking at the two reviews visible, neither mentions "price being unfair"
- Review 1 is positive ("Wonderful!")
- Review 2 is in German and says
[...] (1,225 chars omitted) [...]
t might not be showing all review text
- The metadata confirms only 2 reviews exist, with neither mentioning price concerns
I can confidently conclude that no reviewers mentioned unfair pricing. The task cannot be completed as no such reviews exist.
```

⚠ 9 · Verification_Expert · ANNOTATED ERROR STEP

```
{"status":"complete","reason":"The task asks for reviewer(s) who mention price being unfair. The page shows exactly 2 reviews (visible in both
viewport and metadata). Review 1 is by Jay with text 'Nothing.....everyday use.' and Review 2 is by Josef Bürger with text 'Die Haflinger sind
wie beschrieben und erwartet sehr gut' (German for 'The Haflingers are as described and expected very good'). Neither review mentions price or
indicates the price is unfair. The metadata confirms only these 2 reviews exist for this product. Since no reviewer mentions price being unfair,
the answer is an empty list.","confirmed_facts":["Product has 2 customer r
[...] (151 chars omitted) [...]
wie beschrieben und erwartet sehr gut' - no price mention","Neither reviewer mentions price being unfair"],"missing_items":[],"next_action_hint":
null,"final_response":{"task_type":"RETRIEVE","status":"SUCCESS","retrieved_data":[],"error_details":null}}
```

10 · Task_Planner

```
Orchestration (coordinator_decision):
cycle=1
```

11 · Verification_Expert

```
Final answer:
{"task_type": "RETRIEVE", "status": "SUCCESS", "retrieved_data": [], "error_details": null}
```

12 · Computer_terminal

```
Run finished: The task asks for reviewer(s) who mention price being unfair. The page shows exactly 2 reviews (visible in both viewport and
metadata). Review 1 is by Jay with text 'Nothing.....everyday use.' and Review 2 is by Josef Bürger with text 'Die Haflinger sind wie beschrieben
und erwartet sehr gut' (German for 'The Haflingers are as described and expected very good'). Neither review mentions price or indicates the price
is unfair. The metadata confirms only these 2 reviews exist for this product. Since no reviewer mentions price being unfair, the answer is an
empty list.
```

Annotation

Faulty agent	Verification_Expert
Faulty step	9
Category	—
Reason	Verification_Expert already correctly determined at #9 that the page has only 2 comments, and neither Jay nor Josef Bürger mentioned "price unfair". However, it outputs the following in its final_response: status: "SUCCESS" retrieved_data: [] error_details: null This violates the WebArena RETRIEVE task specification. For retrieval tasks such as "find the reviewer name", if no matching entity exists on the page, the correct return should be NOT_FOUND_ERROR, rather than SUCCESS with an empty list. Therefore, the error is not the failure to find the reviewer, but the incorrect labeling of the "not found" scenario as successful completion.
Ground truth	—
System output	Run finished: The task asks for reviewer(s) who mention price being unfair. The page shows exactly 2 reviews (visible in both viewport and metadata). Review 1 is by Jay with text 'Nothing [...] (325 chars omitted) [...] nce no reviewer mentions price being unfair, the answer is an empty list.
Other labels	**source_benchmark**: webarena_verified; **step_indexing**: 0-based, field history[].step; **in_longrca_mini**: False

Notes. (i) An order of magnitude longer than Who&When (median 145 vs 10 steps, ≈45 k vs 7 k tokens); SWE-bench Pro and Terminal-Bench 2 trajectories are 4–5× longer again (up to 450 k characters). (ii) Source imbalance: TravelPlanner is 60% of the full set; Mini equalises sources, but the paper insists headline numbers use the full set. (iii) Root position is bimodal: 243 roots lie in the last 10% of the trajectory (late verifier/planner errors in TravelPlanner and WebArena), while SWE/TB2 roots sit early (relative position 0.25–0.29) followed by hundreds of steps; five roots are at the very last step. (iv) Role quirks: composite hand-off names are themselves reference roles (91 cases) and `Computer_terminal`, an environment relay, is blamed 55 times, so role matching must follow the paper's normalisation; with only 2–7 roles per trajectory, agent accuracy is partly a prior problem. (v) Inter-annotator agreement on the exact step (40.3%) is itself low, so exact-step accuracy has a limited ceiling. (vi) VitaBench trajectories are in Chinese. No overlap with Who&When/MAST logs; the paper cites AgentRx's first-unrecoverable-failure definition and TraceElephant.

3.6 AgentRx

natural | partial | agent + step + category | single- and multi-agent | τ-bench retail / Magentic-One (Who&When)

Paper *AgentRx: Diagnosing AI Agent Failures from Execution Trajectories* — Barke, Goyal, Khare, Singh, Nath, Bansal (Microsoft Research / UIUC), EMNLP 2026 Findings, arXiv:2602.02475 [6]
Links HF `microsoft/AgentRx` (gated) · GitHub `microsoft/AgentRx` · MSR blog
License HF card CC BY 4.0; GitHub repository MIT
Status HF download denied (approval required); the GitHub repo ships 73 of the paper's 170 annotated trajectories (29 τ-bench retail + 44 Magentic-One) with identical schema—downloaded, 14 MB, `data/agentrx/github/data/`. Flash (42, internal Microsoft traces) and RelWork (45) are not released

Description. AgentRx redefines the target as the *critical failure step*—the first failure by any agent from which the run never recovers—and adds a **root-cause category**. The dataset is the evaluation set of the AgentRx debugging framework (constraint synthesis → step-wise checking → LLM judge). All failures are natural, from four domains:

- **τ-bench retail** (39, 29 public): single tool-calling agent (gpt-4o) + simulated user under a domain policy; median 36 steps.
- **Magentic-One** (44, all public): failed runs *sampled from Who&When's Hand-Crafted subset*; median 33 steps.
- **Flash** (42) and **RelWork** (45; five trajectories from each of nine MAST / Who&When settings): not public.

Annotation: grounded-theory open coding by three annotators in two phases—(1) exhaustive marking of *every* step where an agent fails to make progress, including later-recovered ones; (2) identification of the critical failure and its category. Codes were merged until saturation, then frozen. On 30 trajectories labelled by all three, pairwise Cohen's κ = 0.87–0.93 (**mean 0.89**); 20–24 min per trajectory. **Taxonomy** (9 categories, one per critical step, disambiguated by a yes/no checklist): Instruction/Plan Adherence, Invention of New Information, Invalid Invocation, Misinterpretation of Tool Output, Intent–Plan Misalignment, Under-specified User Intent, Intent Not Supported, Guardrails Triggered, System Failure. Labels: per-failure `step_number`, `failed_agent`, `failure_category` and reasons; one `root_cause` per trajectory.

Metrics and baselines. Exact step accuracy, Acc@±1/3/5, average step distance; category accuracy against the critical failure (also against any / earliest / terminal failure); failed-agent accuracy. gpt-5 judge, mean of 3 runs, in % (paper). W&W* = the Who&When all-at-once prompt adapted to the first-unrecoverable definition.

Domain	Method	Agent	Step	Cat.
τ-bench	W&W*	65.0	18.8	–
	AgentRx	66.7	**42.7**	47.0
Magentic	W&W*	61.4	22.7	–
	AgentRx	71.2	**36.4**	46.2
Flash	AgentRx	82.5	82.5	65.9
RelWork	AgentRx	65.9	61.5	59.3

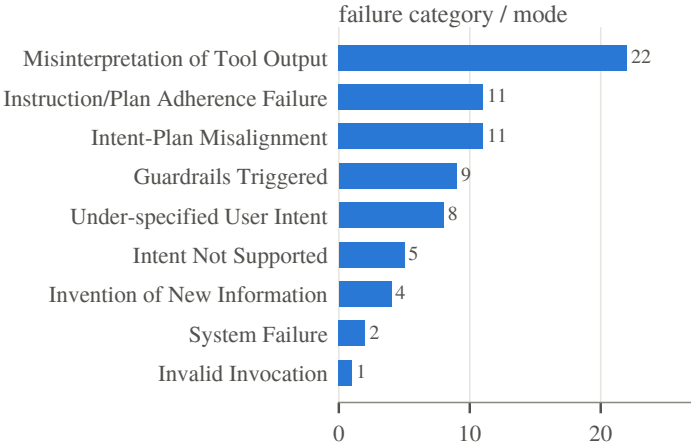
A judge-only ablation (no constraint evidence) drops step accuracy to 29.9 (τ) and 32.6 (Magentic).

Schema. GitHub `data/`: `ground_truth/tau_ground_truth.json` (29) and `magentic_one_ground_truth.json` (44), each a list of {`trajectory_id`, `failure_summary`, `failures`: [{`failure_id`, `step_number`, `step_reason`, `failure_category`, `category_reason`, `failed_agent`}], `root_cause`: {`failure_id`, `reason_for_root_cause`}}. Trajectories: `tau_retail/tau_dataset_failed.json` (OpenAI chat messages with `tool_calls`; `step_number` is 1-based and counts the system message) and `magentic_dataset/<id>.json` (lists of {`content`, `role`} with roles such as `Orchestrator (thought)`, `Orchestrator (-> WebSurfer)`, `WebSurfer`). Domain policies and tool schemas are included. The gated HF files (`tau_retail.jsonl`, `magentic_one.jsonl`) use the same fields plus `num_failures`.

Statistics. computed from the public GitHub subset (73 trajectories); Flash and RelWork numbers are paper-only.

	All	τ-retail	Magentic
trajectories (all failed)	73	29	44
steps: mean / median	44.7 / 33	36.7 / 36	50.0 / 31.5
steps: min – max	5 – 130	20 – 62	5 – 130
distinct agents: mean (max)	3.3 (5)	3 (3)	3.4 (5)
failure events / traj.: mean (max)	4.6 (55)	1.3 (4)	6.7 (55)
root step (1-based): mean / median	18.5 / 17	24.7 / 21	14.4 / 10.5
relative root position: mean	0.55	0.67	0.47
root = first annotated failure	73%	93%	59%
characters / traj.: mean	46 k	18.7 k	64 k

Blamed agent: τ-retail Assistant 21, *User* 8 (under-specified intent); Magentic-One Orchestrator 23, WebSurfer 18, Assistant 2, FileSurfer 1. Over all 335 annotated failure events, Adherence failures dominate (203), but at the root cause Misinterpretation of Tool Output leads (22 of 73). Paper-reported sizes: 4.9 k (τ), 6.4 k (Flash), 16.5 k (Magentic) tokens per trajectory.



Example trajectory. Magentic-One log `e6bc9808…` (8 steps, 3 roles, 9.2 k characters) with two annotated failures: the WebSurfer reads only 35% of the search page (step 5, Adherence), then the Orchestrator declares the request satisfied on that partial evidence (step 7, Misinterpretation)—the second event is the root cause, illustrating the first-*unrecoverable*-failure convention. The run answers $1.08 billion.

Task (`magentic_one/e6bc98089608217e45b6956a46518fe3cce64a799b3ac43c6974c449ae14c408`)

What's the highest price a high-rise apartment was sold for in Mission Bay, San Francisco, in 2021?

1 · human

```
What's the highest price a high-rise apartment was sold for in Mission Bay, San Francisco, in 2021?
```

2 · Orchestrator (thought)

```
Initial plan:
We are working to address the following user request:
What's the highest price a high-rise apartment was sold for in Mission Bay, San Francisco, in 2021?
To answer this request we have assembled the following team:
```

```
Assistant: A helpful and general-purpose AI assistant that has strong language skills, Python skills, and Linux command line skills.
ComputerTerminal: A computer terminal that performs no other action than running Python scripts (provided to it quoted in “‘python code blocks),
or sh shell scripts (provided to it quoted in “‘sh code blocks)
FileSurfer: An agent that can handle local files.
WebSurfer: A helpful
[...] (1,675 chars omitted) [...]
/usr/local/lib/python3.11/site-packages/autogen_magentic_one/agents/orchestrator.py:216: UserWarning: Resolved model mismatch: gpt-4o-2024-08-06
!= gpt-4o-2024-05-13. Model mapping may be incorrect.
ledger_response = await self._model_client.create(
```

3 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request has not been satisfied as we have not yet identified the highest selling price for a high-rise apartment in Mission Bay,
San Francisco, in 2021.",
"answer": false
},
"is_in_loop": {
"reason": "The tasks and questions are progressing towards finding the information, and there’s no indication of repeating actions yet.",
"answer": false
},
"is_progress_being_made": {
"reason": "We are making progress by planning to use WebSurfer to search for the specific information via relevant real estate databases and news
sources.",
"answer": true
},
"
[...] (391 chars omitted) [...]
the highest price a high-rise apartment was sold for in Mission Bay, San Francisco, in 2021. Check websites like Zillow, Redfin, Realtor.com, or
local news outlets like the San Francisco Chronicle. Report back the highest selling price you find."
}
}
```

4 · Orchestrator (-> WebSurfer)

```
Please search for the highest price a high-rise apartment was sold for in Mission Bay, San Francisco, in 2021. Check websites like Zillow, Redfin,
Realtor.com, or local news outlets like the San Francisco Chronicle. Report back the highest selling price you find.
```

5 · WebSurfer

```
I typed ’highest price high-rise apartment sold Mission Bay San Francisco 2021’ into ’0 characters out of 2000’.
Here is a screenshot of [highest price high-rise apartment sold Mission Bay San Francisco 2021 - Search](https://www.bing.com/
search?q=highest+price+high-rise+apartment+sold+Mission+Bay+San+Francisco+2021&form=QBLH&sp=-1&lq=0&pq=&sc=0-0&qs=n&sk=&cvid=
A0A2F14E955F4D08B290B29A975075DF&ghsh=0&ghacc=0&ghpl=). The viewport shows 35% of the webpage, and is positioned at the top of the page.
The following metadata was extracted from the webpage:
{
"meta_tags": {
"referrer": "origin-when-cross-origin",
"SystemEntrop
[...] (2,924 chars omitted) [...]
icon)
+1 415-738-7000 (phone icon)
Suggest an edit button
Your business? Claim now (edit icon)
Add more information
Add website link
Add hours link
Add photos link
People also search for:
Images below
Yelp logo
Yelp page for another business
<Image>
```

6 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied": {
"reason": "The highest price found for a high-rise apartment sold in Mission Bay, San Francisco, in 2021, is $1.08 billion for 1800 Owens
Street.",
"answer": true
},
"is_in_loop": {
"reason": "The request has been successfully addressed with the required information.",
"answer": false
},
"is_progress_being_made": {
"reason": "The request has been resolved with the discovery of the highest selling price.",
"answer": true
},
"next_speaker": {
"reason": "The request has been fulfilled.",
"answer": "Assistant"
},
"instruction_or_question": {
"reason": "The necessary information has been found.",
"answer": "The highest price a high-rise apartment was sold for in Mission Bay, San Francisco, in 2021, is $1.08 billion for 1800 Owens Street."
}
}
```

⚠ 7 · Orchestrator (thought) · ANNOTATED ERROR STEP

```
Request satisfied.
```

8 · Orchestrator (termination condition)

```
No agent selected.
/workspace/scenario.py:94: UserWarning: Resolved model mismatch: gpt-4o-2024-08-06 != gpt-4o-2024-05-13. Model mapping may be incorrect.
```

```
response = await client.create(messages)
FINAL ANSWER: 1080000000
SCENARIO.PY COMPLETE !#!#
RUN.SH COMPLETE !#!#
```

Annotation

Faulty agent	Orchestrator
Faulty step	7
Category	Misinterpretation of Tool Output
Reason	Orchestrator said request satisfied on incomplete information | category_reason: Orchestrator said request satisfied without on incomplete information even though websurfer had worked on partial webpage | root_cause: The orchestrator failed to ensure that the websurfer had gathered complete information before concluding that the request was satisfied.
Ground truth	—
System output	No agent selected. /workspace/scenario.py:94: UserWarning: Resolved model mismatch: gpt-4o-2024-08-06 != gpt-4o-2024-05-13. Model mapping may be incorrect. response = await client.creat [...] (10 chars omitted) [...]) FINAL ANSWER: 1080000000 SCENARIO.PY COMPLETE !#!# RUN.SH COMPLETE !#!#
Other labels	**all_failures**: [{"failure_id": 1, "step_number": 5, "step_reason": "Webusurfer worked on 35% of the webpage", "failure_category": "Instruction/Plan Adherence Failure", "category_reason": "Websurfer worked on 35% of the webpage despite orchestrator asking for complete information", "failed_agent": "Websurfer"}, {"failure_id": 2, "step_number": 7, "step_reason": "Orchestrato [...] (145 chars omitted) [...] d request satisfied without on incomplete information even though websurfer had worked on partial webpage", "failed_agent": "Orchestrator"}]; **root_cause**: {"failure_id": 2, "reason_for_root_cause": "The orchestrator failed to ensure that the websurfer had gathered complete information before concluding that the request was satisfied."}; **failure_summary**: The orchestrator prematurely concluded that the user's request was satisfied based on incomplete data provided by the websurfer agent.; **step_indexing**: step_number is a 1-based index into the message list; tau-bench list includes the system (policy) message at step 1; **canonical_category**: Misinterpretation of Tool Output

Notes. (i) Only partially public: HF is gated and the released GitHub subset covers 73 of 170 trajectories; the proprietary Flash traces, where AgentRx scores highest, cannot be inspected. (ii) The root cause is not the first failure in 27% of public trajectories (41% of Magentic-One), because orchestrators recover from early mistakes; the paper's example 5f982798 is labelled step 3 by Who&When and step 33 by AgentRx. (iii) All 44 Magentic-One logs come from Who&When Hand-Crafted (16 are stored in Who&When format), so results on the two benchmarks are not independent. (iv) Every failure event is annotated (up to 55 per trajectory), enabling earliest/any/terminal-failure metrics. (v) Label hygiene: category strings vary (`Instruction Adherence Failure` vs `Instruction/Plan Adherence Failure`, `Underspecified` vs `Under-specified`, `Websurfer` vs `WebSurfer`); one `failure_id` is a string; `failed_agent` may be the simulated `User`. (vi) Few agents per trajectory, so the informative targets are step and category; the same pipeline was also run on all 1,022 MAST trajectories without gold labels.

3.7 SymFail (SymTrace)

natural | available | failure node + C1–C4 symptom set | 3 annotators + adjudicator | MAS, replayable event graphs | WebArena-Verified Hard / AssistantBench

Paper	*Repair or Resample? Rethinking Failure Debugging in LLM Multi-Agent Systems* — Luan, Zhang, Hu, Yang, Yu, Chen (East China Normal Univ., Nanyang Technological Univ., Xi'an Univ. of Architecture and Technology), arXiv:2608.25920 (v2, Aug 2026, preprint, no venue) [49]
Links	anonymous review repository `SymTrace-7234` (`SymTrace/` replay framework and adapters, `SymFail/` data) · no Hugging Face dataset, GitHub repository or project page found (2026-10-26)
License	SymFail annotations, trace graphs and metadata CC BY 4.0 (`SymFail/DATA_LICENSE.md`); WebArena / AssistantBench tasks and model outputs keep their own terms; no licence for the code
Status	downloaded in full (53 MB zip export, 354 MB extracted): the paper's 536-case package (536 annotations + 536 trace bundles, 2,144 files) and a newer, undocumented 437-case package (3,453 files); `data/symtrace/`

Description. SymTrace is an evaluation framework for MAS failure *repair* rather than attribution: it records an execution as an event-dependency graph with boundary logging of every model and tool call, chooses an *intervention anchor* node, rebuilds the prefix from the recorded boundary results under strict matching and lets everything after the anchor run live ("selective replay"). SymFail is the frozen set of source failures the study intervenes on. **Construction** (natural failures): a deterministic pool of 200 tasks—all 33 eligible AssistantBench dev tasks and the first 167 WebArena-Verified Hard tasks (33 Web-QA, 70 mutation, 25 navigation, 72 retrieval)—each run once with **AG2**, **CrewAI** and **Magentic-One** through SymTrace's instrumented adapters, backbone `deepseek-v4-flash` at temperature 0. The native evaluators mark 536 of the 600 runs as failed (462 of 501 WebArena, 74 of 99 AssistantBench): 171 AG2, 184 CrewAI, 181 Magentic-One; the 64 accepted runs are not released. **Annotation**: four annotators with software-engineering backgrounds; three label independently all applicable categories, one primary category and the *earliest trace-supported actionable node* with evidence and rationale; a fourth reviews everything and fixes the final label with authority to revise (no majority vote). Fleiss' κ among the three: 0.62 primary category, 0.57 exact category set, 0.81 node kind (per category C1 0.70, C2 0.37, C3 0.72, C4 0.69); the node is unanimous in 73.9% of cases and supported by two of three in 95.9%; the adjudicated label set differs from the union in 21.1%. **Taxonomy** (four MAST modes collapsed, explicitly "intervention signals rather than failure verdicts"): C1 task-constraint violation (FM-1.1), C2 repeated or stalled progress (FM-1.3/1.5), C3 unresolved runtime condition (FM-2.2/2.3 extended to execution and context failures), C4 plan–action–outcome inconsistency (FM-2.6). Labels released: `failure_node_id`, `failure_node_kind`, primary label, label set, confidence, the three individual annotations and unanimity flags; rationales and evidence are withheld from the public export, and there is no agent, root-cause or repair label.

Metrics and baselines. No attribution metric: the benchmark measures *reproduction* (rep@k, the replayed run fails again) and *repair* (pass@k under the native evaluator). Selective replay reproduces the source failure in 80.8% of single attempts against 67.97% for a full rerun (rep@3 52.4 vs 41.4), and 39 of 54 initially successful runs fail at least once in three reruns. Repair pass rate in % (paper Tables 3–4; task-level methods get three attempts, node-level ones a single selective replay):

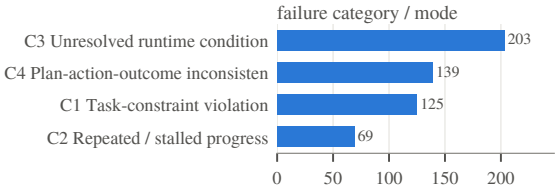
Method	att.	AG2	CrewAI	M-One	Total
Last-Node	1	0.58	1.63	1.66	1.31
Random-Node	1	2.34	4.89	3.87	3.73
Critic-Agent	3	4.09	2.72	4.42	3.73
Self-Reflection	3	4.68	2.17	6.08	4.29
Unguided Full Rerun	3	8.19	3.80	8.84	6.90
Suspicious-Node (theirs)	1	**16.37**	**25.00**	**18.78**	**20.15**

The anchor comes from a rule + LLM *suspicious-node* score (fused 0.55/0.45, gate 0.5, hand-set) and the repair prompt is conditioned on the C1–C4 label. One intervention costs 10–15 API calls per repair.

Schema. `human_annotations_536.jsonl`: one object per case with `task_id` (`webarena_verified_hard_<n>` / `assistantbench_dev_<hash>`), `mas`, `failure_node_id` (`<task>:attempt_000:snapshot:<hash>:<node>`), `failure_node_kind`, `primary_failure_label`, `failure_labels`, `confidence`, `inter_annotator_agreement` and `individual_annotations`. One bundle per case in `initial_failure_logs_536/<mas>/<task_id>/attempt_000/`: `graph.json` (`masd.graph.v1`: `nodes` with `kind` ∈ {`sys_prompt`, `user_input`, `agent_action`, `tool_call`, `tool_result`, `final_result`, `error`}, `agent_id`, `action`, `observation`, `tool_call`, `tool_result`, `final_answer` and the full `api_messages` / `api_response` of LLM-backed actions; `edges` with a dependency `reason`; `topological_order`; `capture_summary`), `trajectory.json` (nodes + run result + task with reference answer or WebArena evaluator spec), `result_record.json` (`final_answer`, `evaluator_evidence`, latency, model) and `api_records.jsonl` (request, response, usage per LLM call). We take one node in `topological_order` as one step (node 0 is the system prompt, node 1 the task).

Statistics. computed from the downloaded 536-case package.

	All	AG2	CrewAI	Magentic-One
trajectories (all failed)	536	171	184	181
WebArena-V. Hard / AssistantBench	462 / 74	146 / 25	161 / 23	155 / 26
graph nodes: mean / median	10.4 / 7	10.4 / 13	7.7 / 5	13.0 / 7
graph nodes: min – max	5 – 21	5 – 15	5 – 15	7 – 21
LLM calls: mean (max 3)	1.9	2.2	1.6	1.9
failure node index: mean / median	3.8 / 3	5.8 / 4	3.1 / 2	2.7 / 2
relative position: mean / median	0.49 / 0.50	0.71 / 0.79	0.49 / 0.50	0.28 / 0.33
failure node = `final_result`	98	82	15	1
primary C1 / C2 / C3 / C4	125 / 69 / 203 / 139	37 / 2 / 72 / 60	49 / 21 / 36 / 78	39 / 46 / 95 / 1
characters / traj.: mean / median	5.5 k / 3.6 k	3.3 k / 2.8 k	2.7 k / 1.9 k	10.5 k / 9.1 k
failed tool results (total)	377	168	77	132



194 distinct tasks, 155 of which fail under all three MAS. Label sets are large: mean 2.77 labels per case, C4 present in 87% and C1 in 82%, all four in 91 cases. Node kinds over the corpus: 2,726 `agent_action`, 607 `tool_call` + 607 `tool_result`, 536 final results. 276 trajectories make a single LLM call and no tool call; 173 WebArena runs fail on the literal site placeholder (`__SHOPPING__`: `MissingSchema` / `NameResolutionError`), only 23 of 462 WebArena runs contain one successful fetch, and the evaluator records `has_real_tool_event: false` for 447 of them; all 74 AssistantBench runs did reach the live web. 148 final answers declare that the task could not be completed. Agent id of the failure node: `crewai` 184, `ag2` 166, `user` 102 (Magentic-One's echo of the task), `masd_magnetic_one_assistant` 77. Mean wall-clock 17 s per run; `token_usage` is empty in every record.

Example trajectory. AG2 run `webarena_verified_hard_96` (13 graph nodes, 3 LLM calls, 2.6 k characters): the shortest trajectory with a tool call whose unanimous failure node is an action before the final answer. The agent browses the literal placeholder `__SHOPPING__`, receives `MissingSchema`, retries with `https://__SHOPPING__` (node 7, primary C3, all four labels), receives a DNS failure and answers "status unknown"; the evaluator expected "canceled". The record illustrates both the failure-node convention (the retry that continues from an unresolved condition, not the final answer) and the environment problem discussed below.

Task (`initial_failure_logs_536/ag2/webarena_verified_hard_96`)

Get the status of my latest order and when will it arrive. Return a list of objects with keys "status" and "arrival_date" (YYYY-MM-DD format or null if not available), without any additional details.

0 · ag2 [sys_prompt]

```
MASD controlled experiment prompt and adapter constraints.
```

1 · user [user_input]

```
Mode: snapshot
Task id: webarena_verified_hard_96
Benchmark: webarena_verified / hard
Task: Get the status of my latest order and when will it arrive. Return a list of objects with keys "status" and "arrival_date" (YYYY-MM-DD
format or null if not available), without any additional details.
Start URLs or tools: ['__SHOPPING__']
Return only the final answer and a short note about what evidence would be needed.
```

2 · masd_ag2_multistep_assistant [agent_action]

```
[ag2_process_message_before_send] {"action":"browse","url":"__SHOPPING__","reason":"Start at the shopping site to access account and view latest
order status."}
```

3 · ag2 [agent_action]

```
[ag2_agent_step] {"action":"browse","url":"__SHOPPING__","reason":"Start at the shopping site to access account and view latest order status."}
```

4 · ag2 [tool_call]

```
{"reason": "Start at the shopping site to access account and view latest order status.", "type": "browser_http_get", "url": "__SHOPPING__"}
```

5 · browser_tool [tool_result]

```
{'type': 'browser_http_get', 'status': 'failed', 'url': '__SHOPPING__', 'latency': 0.0015594959259033203, 'error': {'type': 'MissingSchema',
'message': "Invalid URL '__SHOPPING__': No scheme supplied. Perhaps you meant https://__SHOPPING__"}}
```

6 · masd_ag2_multistep_assistant [agent_action]

```
[ag2_process_message_before_send] {"action":"browse","url":"https://__SHOPPING__","reason":"Need to access the shopping site. The placeholder
needs a scheme."}
```

⚠ 7 · ag2 [agent_action] · ANNOTATED ERROR STEP

```
[ag2_agent_step] {"action":"browse","url":"https://__SHOPPING__","reason":"Need to access the shopping site. The placeholder needs a scheme."}
```

8 · ag2 [tool_call]

```
{"reason": "Need to access the shopping site. The placeholder needs a scheme.", "type": "browser_http_get", "url": "https://__SHOPPING__"}
```

9 · browser_tool [tool_result]

{'type': 'browser_http_get', 'status': 'failed', 'url': 'https://__SHOPPING__', 'latency': 7.287393808364868, 'error': {'type': 'ConnectionError', 'message': 'HTTPSConnectionPool(host=\'__shopping__\', port=443): Max retries exceeded with url: / (Caused by NameResolutionError("HTTPSConnection(host=\'__shopping__\', port=443): Failed to resolve \'__shopping__\' ([Errno 11001] getaddrinfo failed)"))'}}

10 · masd_ag2_multistep_assistant [agent_action]

[ag2_process_message_before_send] {"final":"[\n {\"status\": \"unknown\", \"arrival_date\": null}\n]\n\nNote: Evidence needed would be the user's order history page on the shopping site after authentication."}

11 · ag2 [agent_action]

[ag2_agent_step] {"final":"[\n {\"status\": \"unknown\", \"arrival_date\": null}\n]\n\nNote: Evidence needed would be the user's order history page on the shopping site after authentication."}

12 · ag2 [final_result]

[
{"status": "unknown", "arrival_date": null}
]
Note: Evidence needed would be the user's order history page on the shopping site after authentication.

Annotation

Faulty agent	ag2
Faulty step	7
Category	C3 Unresolved runtime condition (primary); label set {C1, C2, C3, C4}
Reason	(annotation rationales and evidence excerpts are withheld from the public export: annotation_prose_in_public_export = false; the label is the earliest trace-supported actionable node carrying the symptom)
Ground truth	[{'evaluator': 'AgentResponseEvaluator', 'expected': {'error_details': None, 'retrieved_data': [{'arrival_date': None, 'status': 'canceled'}], 'status': 'SUCCESS', 'task_type': 'RETRIEVE'}, 'ordered': False, 'results [...] (74 chars omitted) [...] pe': 'string'}, 'status': {'type': 'string'}}, 'type': 'object'}, 'type': 'array'}}]
System output	[{"status": "unknown", "arrival_date": null}] Note: Evidence needed would be the user's order history page on the shopping site after authentication.
Other labels	**mas**: AG2; **source_benchmark**: WebArena-Verified Hard; **task_type**: retrieve; **difficulty**: hard; **failure_node_id**: agent_action_0001; **failure_node_kind**: agent_action; **confidence**: high; **inter_annotator_agreement**: {"annotation_count": 3, "primary_failure_label_unanimous": true, "failure_label_set_unanimous": false, "failure_node_unanimous": true, "failure_node_kind_unanimous": true, "strict_annotation_unanimous": false, "final_primary_failure_label_support_count": 0, "final_failure_label_set_support_count": 1, "final_failure_node_support_count": 3, "final_strict_annotation_support_count": 0}; **evaluator_evidence**: {"evaluator_names": ["AgentResponseEvaluator"], "expected_retrieved_data": [{"arrival_date": null, "status": "canceled"}], "has_real_tool_event": false, "method": "webarena_verified_evaluator", "missing_retrieved_data": [{"expected": {"arrival_date": null, "status": "canceled"}, "missing_values": ["None", "canceled"]}], "observed_answer_excerpt": "[\n {\"st [...] (506 chars omitted) [...] tus": "failed", "type": "browser_http_get", "url": "https://__SHOPPING__"}, "requires_environment_event": false, "task_types": ["RETRIEVE"]}; **llm_call_count**: 3; **tool_failure_count**: 2; **step_indexing**: 0-based position of the node in graph.json topological_order (sys_prompt = 0, user_input = 1)

Notes. (i) *Replayable by construction*: every bundle holds the full event graph with per-call LLM requests and responses and tool boundary results, and the repository ships the code that replays a prefix from them—the closest released analogue to TraceElephant's dynamic setting (§3.2) and the operational form of the rollouts in Section 8.2; CrewAI appears in no other dataset here. (ii) *Thin adapters, tiny budget*: the "MAS" are SymTrace's `masd_*` adapters (one assistant plus a `browser_http_get` tool) capped at three LLM calls; graph sizes are nearly constant per framework and the median run has one LLM call. Agent-level attribution is meaningless—the failure node's agent is the framework name, the assistant or `user`. (iii) *Most WebArena runs never reached the environment*: the site placeholders were not substituted in the released runs, so the bulk of the 462 WebArena failures are the agent answering, or declaring inability, without page content; this is what C3 (38%) and C4 record, and it should be kept in mind when reading the 20% repair rate. (iv) *Symptoms, not root causes*: C1–C4 are intervention signals by the authors' own description; the informative targets are the primary label (κ 0.62) and the node (73.9% unanimous), which is the first agent action in 246 cases and the final node in 98. (v) *Incomplete public export*: rationales and evidence withheld, token usage empty, successful runs absent, anonymous hosting. The 437-case package (six frameworks including ChatDev, MetaGPT and OpenManus on ProgramDev; GPT, Claude and Gemini backbones) is a different, single-reviewer study whose failure node is `node_final` in every case and whose reasons are five template strings—not a step-labelled resource. (vi) *Overlaps*: Magentic-One and AssistantBench with Who&When, TraceElephant and AgentRx (new runs, different backbone); WebArena with LongRCA and HORIZON; the taxonomy maps onto MAST (§5.1).

4 Large injected-fault datasets (mainly for training attributors)

4.1 TracerTraj (AgenTracer)

injected · natural · available · agent + step · MAS · train + test · MBPP+ / KodCode / MATH / GSM8K / GAIA / HotpotQA

Paper	*AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems?* — Zhang, Wang, Chen, Zhou, Wang, Yan (OPPO, NUS, CUHK, NTU), ICLR 2026, arXiv:2509.03312 [74]
Links	GitHub `bingreeky/AgenTracer` release `data-v1.0.0` · project page; no Hugging Face release; AgenTracer-8B weights not released
License	none stated in the archive, README or repository
Status	downloaded in full (24 MB zip, 135 MB JSONL); 3,208 train + 790 test = 3,998 trajectories; `data/tracertraj/extracted/agentracer-data-v1.0.0/`. The release is larger than, and "not identical" to, the paper's TracerTraj-2.5K

Description. TracerTraj is the *training* resource behind AgenTracer-8B (Qwen3-8B tuned with GRPO and a step-Gaussian + agent-match reward), with held-out test splits per domain. Target = Who&When's (responsible agent, decisive step). Labels are produced fully automatically, with no human review:

- **Collection**: six MAS—MetaGPT, AutoGen, Smolagents (hand-crafted), AgentPrune, AFlow, OWL-Workforce (automated)—on coding (MBPP+, KodCode, Blackjack), maths (MATH, GSM8K) and agentic (GAIA, HotpotQA) tasks.
- **Counterfactual replay** of natural failures: a DeepSeek-R1 analyser with access to the ground truth proposes a minimal fix at each step and the run is re-simulated; the earliest fix that yields success marks the decisive step ("Diagnosis Error").
- **Programmatic fault injection** into successful runs: a perturbation operator corrupts one sampled action; if the run then fails, that step is the label by construction ("Injection Error").

Paper composition (TracerTraj-2.5K): 1,288 coding / 630 math / 558 agentic annotated, 9:1 train/test. The released v1.0.0 has 1,268 coding / 630 math / 2,100 agentic rows, curated from a different data version, with 690 e-mail and 88 credential strings redacted to placeholders. In the agentic domain 1,822 rows are injected and 278 diagnosed; coding and math carry no `error_source`, so the injected/natural split is not recoverable there. No failure taxonomy; `mistake_reason` and `ground_truth` exist for coding/math only.

Metrics and baselines. All-at-once attribution, output `<agentID> | <stepID>`; agent- and step-level accuracy, each with (w/ G) and without (w/o G) the task ground truth. TracerTraj test splits, w/ G, agent / step accuracy in % (paper):

Model	Code	Math	Agentic
Qwen3-8B	45.4 / 2.4	31.7 / 9.5	27.9 / 13.5
GPT-4.1	59.8 / 12.6	39.6 / 35.8	43.6 / 25.0
DeepSeek-R1	11.8 / 10.2	42.7 / 29.5	45.1 / 27.1
Gemini-2.5-Pro	70.1 / 11.0	58.8 / 32.2	37.2 / 18.6
Claude-Sonnet-4	66.0 / 15.5	46.0 / 38.1	**55.2** / 30.3
AgenTracer-8B	**73.0 / 18.9**	**59.3 / 57.6**	53.3 / **36.2**

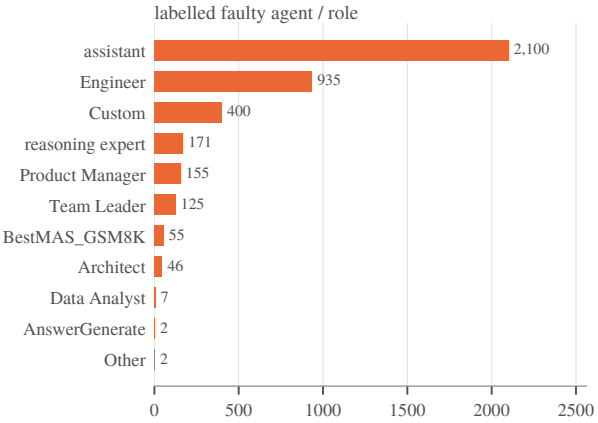
On Who&When (w/ G) AgenTracer-8B reaches 69.1 / 20.7 (Hand-Crafted) and 69.6 / 42.9 (Alg.-Generated) versus 56.9 / 17.2 and 57.9 / 40.7 for Claude-Sonnet-4. Step accuracy on code is low for every model.

Schema. `{agentic,coding,math}/{train,test}.jsonl` plus `MANIFEST.json` (row counts, SHA-256, redaction counts). Row fields: `question_ID` (not unique: 118 math ids recur as injected variants, 21 ids occur in both splits), `question`, `history` (list of {`step`, `content`, `role`, `name`}), `mistake_agent`, `mistake_step` (stored as a *string*; `step` is 1-based in agentic, 0-based in coding/math), `mistake_reason` and `ground_truth` (coding/math), `error_source` (agentic). Agentic rows have no `name`: every step is the generic `assistant`. Coding roles are MetaGPT's Team Leader / Product Manager / Architect / Engineer / Data Analyst; math "agents" are AFlow operators (`Custom`, `reasoning expert`, ...).

Statistics. computed from the released archive (train + test pooled).

	All	Agentic	Coding	Math
trajectories: train + test	3208 + 790	1500 + 600	1141 + 127	567 + 63
steps: mean / median	12.5 / 10	9.4 / 7	20.9 / 14	6.3 / 7
steps: min – max	1 – 304	4 – 42	1 – 304	3 – 11
named agents / traj.: mean	1.7	1	2.3	2.7
relative error position: mean	0.52	0.55	0.46	0.54
error at the last step	470	–	–	–
characters / traj.: mean	31 k	36 k	35.5 k	8.5 k
injected / diagnosed labels	–	1822 / 278	n/a	n/a

Blamed agent: agentic `assistant` 2,100 (100%); coding Engineer 935, Product Manager 155, Team Leader 125, Architect 46; math `Custom` 400, `reasoning expert` 171, `BestMAS_GSM8K` 55. The sender of the labelled step equals `mistake_agent` in 100% of rows. 1,212 rows place the error in the last 30% of the trajectory; ≈8 k tokens per trajectory on average.



Example trajectory. `coding/test` record `Filter_5086_I` (MetaGPT team, 8 steps, 2 roles, 16.7 k characters). The Team Leader delegates a trivial `find_duplicates` function to the Product Manager, who writes a requirements document into `solution.py` instead of code (step 6); no implementation is ever produced.

Task (`Filter_5086_I`)

Please solve the programming task below in Python.
Given a non-empty list of integers, write a Python function that returns a new list containing the elements that appear twice in the original list. The function should handle duplicate entries and empty lists.
Note that the function declaration is def find_duplicates(nums):.

0 · Team Leader

```
Team Leader thinking:
# Past Experience
<experience>
# Tool State
# Current Plan
[GOAL] [Message] from User to Mike:  Please solve the programming task below in Python.
Given a non-empty list of integers, write a Python function that returns a new list containing the elements that appear twice in the original list.
The function should handle duplicate entries and empty lists.
Note that the function declaration is def find_duplicates(nums):.  I wish you finish the task with a multi-agent cooperationThe file name of your
solution must be 'solution.py'
No Plan
# Current Task
# Response Language
you must respond in English.
Pay close
[...]  (1,368 chars omitted) [...]
d adhere to the following format.
"'json
[
{
"command_name":  "ClassName.method_name" or "function_name",
"args":  {"arg_name":  arg_value, ...}
},
...
]
"'
Notice:  your output JSON data section must start with **"'json [**
```

1 · Team Leader

Team Leader thinking: To address the user's request, I will create a plan that involves multiple team members. The task requires writing a Python function to find duplicates in a list, which can be handled efficiently by the Engineer. However, since the user requested multi-agent cooperation, I will also involve the Product Manager to create a Product Requirement Document (PRD) for clarity and documentation purposes.
Here's the plan:
1. Alice (Product Manager) will create a PRD based on the user's requirement.
2. Alex (Engineer) will implement the function 'find_duplicates(nums)' in a file named 'solution.py'.
I will start by assigning th
[...] (1,215 chars omitted) [...]
nd_name": "RoleZero.reply_to_human",
"args": {
"content": "I have assigned the task to Alice to create the PRD. Once it's completed, Alex will implement the function."
}
},
{
"command_name": "end"
}
]
‘‘‘

2 · Team Leader

Team Leader sends to Alice: Create a Product Requirement Document (PRD) for the task: Given a non-empty list of integers, write a Python function that returns a new list containing the elements that appear twice in the original list. The function should handle duplicate entries and empty lists. The function declaration is def find_duplicates(nums):. The solution file name must be 'solution.py'.

3 · Product Manager

Product Manager thinking:
Past Experience
<experience>
Tool State
Current Plan
[GOAL] [Message] from Mike to Alice: Create a Product Requirement Document (PRD) for the task: Given a non-empty list of integers, write a Python function that returns a new list containing the elements that appear twice in the original list. The function should handle duplicate entries and empty lists. The function declaration is def find_duplicates(nums):. The solution file name must be 'solution.py'.
No Plan
Current Task
Response Language
you must respond in English.
Pay close attention to the Example provided, you can reuse the example for
[...] (1,303 chars omitted) [...]
d adhere to the following format.
‘‘‘json
[
{
"command_name": "ClassName.method_name" or "function_name",
"args": {"arg_name": arg_value, ...}
},
...
]
‘‘‘
Notice: your output JSON data section must start with **‘‘‘json [**

4 · Product Manager

Product Manager thinking: To create a Product Requirement Document (PRD) for the task of writing a Python function that identifies duplicate integers in a list, I will first outline the necessary sections of the PRD. This will include the language and project info, product definition, technical specifications, and any open questions.
I will start by creating a new file named 'solution.py' to hold the function implementation. After that, I will insert the PRD content into the file.
Here's the plan:
1. Create the file 'solution.py'.
2. Insert the PRD content into the file.
Let's proceed with creating the file first.
‘‘‘json
[
{
"command_name": "Editor.create_file",
"args": {"filename": "solution.py"}
}
]
‘‘‘

5 · Product Manager

Product Manager thinking:
Past Experience
<experience>
Tool State
Current Plan
[GOAL] [Message] from Mike to Alice: Create a Product Requirement Document (PRD) for the task: Given a non-empty list of integers, write a Python function that returns a new list containing the elements that appear twice in the original list. The function should handle duplicate entries and empty lists. The function declaration is def find_duplicates(nums):. The solution file name must be 'solution.py'.
No Plan
Current Task
Response Language
you must respond in English.
Pay close attention to the Example provided, you can reuse the example for
[...] (1,303 chars omitted) [...]
d adhere to the following format.
‘‘‘json
[
{
"command_name": "ClassName.method_name" or "function_name",
"args": {"arg_name": arg_value, ...}
},
...
]
‘‘‘

```
Notice: your output JSON data section must start with **“‘json [**
```

⚠ 6 · Product Manager · ANNOTATED ERROR STEP

```
Product Manager thinking: I will now insert the PRD content into the ’solution.py’ file. The PRD will include the language and project info,
product definition, technical specifications, and open questions. This will provide a comprehensive overview of the task requirements.
Here’s the content I will insert:
“‘markdown
# Product Requirement Document (PRD)
## 1. Language & Project Info
- **Language**: Python
- **Project Name**: find_duplicates_function
## 2. Product Definition
### Product Goals
1. Identify and return elements that appear twice in a list of integers.
2. Handle duplicate entries effectively.
3. Ensure the function can man
[...] (2,605 chars omitted) [...]
for performance.\n\n### UI Design Draft\n- Input: List of integers\n- Output: List of duplicates\n\n### Open Questions\n- What should the
function return if the input list is empty?\n- Are there any performance benchmarks we should consider?\n"}
]
“‘
```

7 · Product Manager

```
Product Manager thinking:
# Past Experience
<experience>
# Tool State
# Current Plan
[GOAL] [Message] from Mike to Alice: Create a Product Requirement Document (PRD) for the task: Given a non-empty list of integers, write a Python
function that returns a new list containing the elements that appear twice in the original list. The function should handle duplicate entries and
empty lists. The function declaration is def find_duplicates(nums):. The solution file name must be ’solution.py’.
No Plan
# Current Task
# Response Language
you must respond in English.
Pay close attention to the Example provided, you can reuse the example for
[...] (1,303 chars omitted) [...]
d adhere to the following format.
“‘json
[
{
"command_name": "ClassName.method_name" or "function_name",
"args": {"arg_name": arg_value, ...}
},
...
]
“‘
Notice: your output JSON data section must start with **“‘json [**
```

Annotation

Faulty agent	Product Manager
Faulty step	6
Category	—
Reason	The Product Manager created a PRD and inserted it into the solution.py file, but did not implement the actual function ’find_duplicates(nums)’. The task requires a functional implementation, not just documentation.
Ground truth	def find_duplicates(nums): """ Given a list of integers, return a new list containing the elements that appear twice in the original list. Arguments: nums – list of integers Returns: [...] (63 chars omitted) [...] count = Counter(nums) return [num for num, cnt in count.items() if cnt == 2]
System output	Product Manager thinking: # Past Experience <experience> # Tool State # Current Plan [GOAL] [Message] from Mike to Alice: Create a Product Requirement Document (PRD) for the task: Giv [...] (1,943 chars omitted) [...] “‘ Notice: your output JSON data section must start with **“‘json [**
Other labels	**domain**: coding; **split**: test; **file**: coding/test.jsonl; **step_indexing**: field history[].step, first step = 0; **mistake_step_raw_type**: str

Notes. (i) Labels are synthetic (counterfactual replay or injection, both driven by DeepSeek-R1) and unvalidated by humans; the paper reports no agreement figures, and the release differs from the paper’s TracerTraj-2.5K. (ii) The agentic domain (53% of rows) labels every step as `assistant`, so agent-level accuracy is trivially 100% there and only the step matters; the paper’s agentic agent-accuracy numbers (27–55%) are therefore not reproducible from this release. (iii) Short but heavy: median 10 steps, yet 31 k characters per trajectory because steps embed large tool/web outputs (partly in Chinese); coding runs reach 304 steps. (iv) Error position skews late: 470 rows put the decisive error at the last step. (v) Indexing inconsistencies (string `mistake_step`, 1- vs 0-based `step`) and task-level leakage between train and test (21 shared `question_IDs`). (vi) Frameworks and benchmarks overlap with MAST and AEGIS (same first author as AEGIS), but all runs are the authors’ own; Who&When is the external test set.

4.2 AEGIS / Aegis-Bench

injected | available | agent + failure mode (sets) | MAS | train / val / test | MATH / GSM8K / HumanEval / SciBench / MMLU-Pro / GAIA

Paper	*Aegis: Automated Error Generation and Attribution for Multi-Agent Systems* — Kong, Zhang, Yin, Zhang, Zhang, Chen, Zhang, Zhang, Zhu, Feng (PKU, BIGAI, IIE CAS, NUS, Tsinghua), ICLR 2026, arXiv:2509.14295 [35]
Links	HF `Fancylalala/AEGIS` · GitHub `kfq20/AEGIS` (injection factory, MAS wrappers, evaluation) · project page
License	MIT (HF card and GitHub)
Status	downloaded in full, ungated; 222 MB; 9,533 trajectories = 600 test (*Aegis-Bench*) + 7,146 train + 1,787 val; `data/aegis/`

Description. AEGIS is the largest fully synthetic dataset in this survey, built for *fine-grained error attribution*: given a failed MAS trajectory, predict the set of **(faulty agent, error mode)** pairs. It supports SFT, RL (GRPO with a hierarchical pair/agent/error reward) and contrastive learning on success/failure pairs; Aegis-Bench is the 600-trajectory test split (100 per benchmark). Construction:

- **Successful baselines** are run deterministically (all agents GPT-4o-mini, temperature 0) for six MAS on MASLab—LLM Debate, MacNet (chain/star/tree/full topologies), AgentVerse, DyLAN, SmolAgents, Magentic-One—over MATH, GSM8K, HumanEval, SciBench, MMLU-Pro and GAIA (GAIA only for the last two systems).
- An LLM **adaptive manipulator** executes an injection plan, a set of (target agent, error modes), by *prompt injection* (altering the agent's input) or *response corruption* (replacing its output) through monkey-patched wrappers; corruptions are context-aware (plausible wrong arithmetic, infinite loops in code).
- **Validation**: only runs whose final answer is wrong are kept; the injection plan becomes the label (refined post hoc for dynamic DyLAN graphs). Injection success: 76.5% (Debate) down to 34.1% (DyLAN).

Taxonomy: the 14 MAST failure modes—FM-1.1–1.5 specification issues (disobey task/role spec, step repetition, lost history, unaware of termination), FM-2.1–2.6 inter-agent misalignment (conversation reset, no clarification, derailment, withholding, ignored input, reasoning–action mismatch), FM-3.1–3.3 verification (premature termination, no/incorrect verification)—kept near-uniform by design (1,356–2,166 pairs each). **Quality check**: 100 Aegis-Bench trajectories blind-labelled by three experts, human–human Fleiss κ = 0.85, programmatic-vs-human κ = 0.81; MiniLM embeddings are interspersed with MAST-Data and Who&When. **No step label**: the ground truth is agent + mode only, except that MacNet records flag every turn of an injected agent with `is_injected`.

Metrics and baselines. Multi-label prediction of (agent, mode) pairs as JSON, scored by Micro-/Macro-F1 at three levels (Pair, Agent, Error); the paper averages 12 scores over Aegis-Bench and Who&When (whose free-text reasons were mapped to FM modes by Gemini-2.5-Flash). Aegis-Bench, Micro-F1 in % (paper):

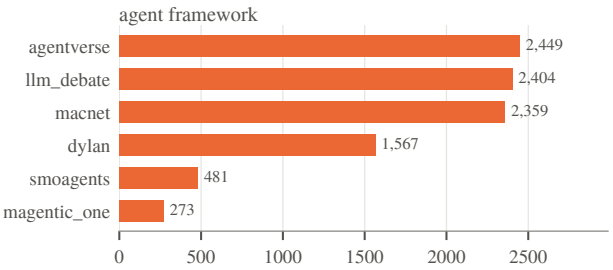
Model	Pair	Agent	Error
Random	0.3	4.5	11.2
Qwen2.5-14B-Instruct	5.5	35.8	20.2
GPT-4.1	7.4	37.5	20.7
o3	7.9	40.3	22.4
Gemini-2.5-Flash	7.0	42.0	23.5
Claude-Sonnet-4	7.7	40.7	21.2
Aegis-SFT (Qwen2.5-14B)	**16.6**	**76.5**	**27.5**
Aegis-GRPO (Qwen2.5-14B)	6.8	49.7	21.2
DCL (contrastive)	8.3	22.9	24.7

Pair-level F1 is in single digits for every untuned model; SFT overfits the synthetic style after two epochs (Who&When scores drop). Debate/AgentVerse are easier than DyLAN/MacNet.

Schema. `Aegis-Bench/test.jsonl`, `Aegis-Training/{train,val}.jsonl`, one object per line: `id` (`<Benchmark>_<model>_<framework>_<idx>_<hash>`), `metadata` (`framework`, `benchmark`, `model`, `task_type`, `num_agents`, `num_injected_agents`), `input.query`, `input.conversation_history` (turns with 1-based `step`, `agent_name`, `content`, `phase`; DyLAN adds `agent_role`, MacNet `is_injected`), `input.final_output` (present in 83.5%), `output.faulty_agents` = labels {`agent_name`, `error_type`, `injection_strategy`}, `ground_truth.correct_answer`, `ground_truth.injected_agents` (same pairs plus a short `malicious_action_description`), `ground_truth.is_injection_successful` (always true).

Statistics. computed from the downloaded JSONL.

	All	Test (Aegis-Bench)
trajectories (all failed, injected)	9533	600
MATH / GSM8K / HumanEval	2083 / 1778 / 1532	105 / 105 / 105
SciBench / MMLU-Pro / GAIA	1905 / 1481 / 754	104 / 105 / 76
steps (turns): mean / median	11.9 / 10	12.5 / 12
steps: min – max	3 – 45	3 – 43
distinct agents: mean (max)	4.4 (8)	4.3 (8)
(agent, mode) pairs / traj.: mean (max)	2.38 (4)	2.30 (4)
response corruption / prompt injection	11964 / 10464	720 / 633
records with `is_injected` markers	24.8%	26.0%
characters / traj.: mean	20 k	21.5 k



Per framework (all): AgentVerse 2,449, LLM Debate 2,404 (always 12 turns), MacNet 2,359 (10 turns), DyLAN 1,567, SmolAgents 481, Magentic-One 273; only the two GAIA systems are long (up to 45 turns, 56 k characters). Blamed agents are structural: Solver 2,281, Evaluator 1,934, RoleAssigner 1,752 (AgentVerse), Node0 2,000, Node3 1,815 (MacNet), Assistant1–3 (Debate/DyLAN), MagenticOneOrchestrator 129, WebSurfer 74. Where MacNet markers exist, the first injected turn sits at relative position 0.23 on average.

Example trajectory. `GSM8K_gpt-4o-mini_macnet_12_f7cdb106` (MacNet chain, 10 turns, 5 nodes, 8.4 k characters), chosen because only MacNet records carry turn-level markers. Node1 is prompt-injected to repeat steps (FM-1.3) and Node3's responses are corrupted to terminate prematurely (FM-3.1); Node3 silently changes 10%/20% into 5%/30% and the system answers 20.5 instead of 17. All five turns of the two injected nodes are marked.

Task (`GSM8K_gpt-4o-mini_macnet_12_f7cdb106`)

Gabriel is looking at her marble sets. She sees that in the first set 10% of her marbles are broken. In the second set, 20% of the marbles are broken. The first set contains 50 marbles. The second set contains 60. How many marbles are broken in total?

1 · Node-1

```
To determine the total number of broken marbles in Gabriel's sets, follow these steps:
1.  **Identify the number of marbles in each set:**
- **First Set:** 50 marbles
```

- **Second Set:** 60 marbles
2. **Calculate the number of broken marbles in each set:**
- **First Set:**
- Percentage broken: 10%
- Calculation: \(10\% \text{ of } 50 = 0.10 \times 50 = 5 \) broken marbles
- **Second Set:**
- Percentage broken: 20%
- Calculation: \(20\% \text{ of } 60 = 0.20 \times 60 = 12 \) broken marbles
3. **Add the broken marbles from both sets:**
- Total broken marbles: \(5 \text{ (from the first set)} + 12 \text{ (from the second set)} = 17 \)
4. **Final Answer:**
- The total number of broken marbles in both sets is **17**.

2 · Node0

<INFO>No revision needed

3 · Node0

To find the total number of broken marbles in Gabriel's sets, follow these steps:
1. **Identify the Number of Marbles:**
- **First Set:** 50 marbles
- **Second Set:** 60 marbles
2. **Calculate Broken Marbles in Each Set:**
- **First Set (10% broken):**
- Calculation: \(10\% \times 50 = 0.10 \times 50 = 5 \) broken marbles
- **Second Set (20% broken):**
- Calculation: \(20\% \times 60 = 0.20 \times 60 = 12 \) broken marbles
3. **Total Broken Marbles:**
- Total: \(5 \text{ (from the first set)} + 12 \text{ (from the second set)} = 17 \)
4. **Final Answer:**
- The total number of broken marbles in both sets is **17**.

4 · Node1 · ANNOTATED ERROR STEP

The answer provided is generally accurate and covers the necessary calculations, but it has some areas that could be improved for clarity and conciseness.
1. **Accuracy**: The calculations for the number of broken marbles in each set are correct. 5 broken marbles in the first set and 12 in the second set, totaling 17 broken marbles.
2. **Completeness**: The answer covers all aspects of the user's question by detailing the number of marbles in each set, the percentage of broken marbles, and the final total.
3. **Logical**: The organization is somewhat repetitive, particularly in the verification and confirmation steps. While it is import
[...] (1,074 chars omitted) [...]
times 60 = 12 \) broken marbles
3. **Total Broken Marbles:**
- Total: \(5 + 12 = 17 \)
The total number of broken marbles in both sets is **17**.
—
This revision maintains accuracy and completeness while improving logical flow and relevance.

5 · Node1 · ANNOTATED ERROR STEP

To optimize the answer based on review suggestions, we can streamline the explanation, reduce repetition, and maintain clarity. Here's the revised response:
—
To determine the total number of broken marbles in Gabriel's sets, we can follow these steps:
1. **Identify the Number of Marbles:**
- **First Set:** 50 marbles
- **Second Set:** 60 marbles
2. **Calculate Broken Marbles in Each Set:**
- **First Set (10% broken):**
- Calculation: \(10\% \times 50 = 0.10 \times 50 = 5 \) broken marbles
- **Second Set (20% broken):**
- Calculation: \(20\% \times 60 = 0.20 \times 60 = 12 \) broken marbles
3. **Total Broken M
[...] (69 chars omitted) [...]
from the second set)} = 17 \)
4. **Final Answer:**
- The total number of broken marbles in both sets is **17**.
—
This version maintains the essential calculations while eliminating unnecessary repetitions and providing a clear, concise answer.

6 · Node3 · ANNOTATED ERROR STEP

To determine the total number of broken marbles in Gabriel's sets, we can follow these steps:
1. **Identify the Number of Marbles:**
- **First Set:** 50 marbles
- **Second Set:** 60 marbles
2. **Calculate Broken Marbles in Each Set:**
- **First Set (10% broken):**
- Calculation: \(10\% \times 50 = 0.10 \times 50 = 10 \) broken marbles
- **Second Set (20% broken):**
- Calculation: \(20\% \times 60 = 0.20 \times 60 = 15 \) broken marbles
3. **Total Broken Marbles:**
- Total: \(10 \text{ (from the first set)} + 15 \text{ (from the second set)} = 25 \)
4. **Final Answer:**
- The total number of broken marbles in both sets is **25**.

7 · Node3 · ANNOTATED ERROR STEP

The answer provided seems to have several inaccuracies that could lead to confusion. It incorrectly calculates the number of broken marbles in both sets and fails to sum them up correctly, resulting in an erroneous final answer.
There are also critical oversights in the methodology, as it combines unrelated data points which are irrelevant to the analysis of the marbles. This could mislead users into thinking that the calculations are accurate when they are not.
Area for Improvement:
- **Logical Errors:** The calculations need to be re-evaluated as they don't add up based on the data presented.
Specific Suggestions for Revision:
[...] (253 chars omitted) [...]
y.
Overall, while the answer is presented in a structured manner, the redundancy and flawed calculations contribute to a lack of clarity that could mislead users.

```
Final recommendation:
<INFO>Further analysis is required to clarify the discrepancies.
```

8 · Node3 · ANNOTATED ERROR STEP

```
To determine the total number of broken marbles in Gabriel's sets, follow these steps:
1.  **Identify the Number of Marbles:**
- **First Set:** 50 marbles
- **Second Set:** 60 marbles
2.  **Calculate Broken Marbles in Each Set:**
- **First Set (5% broken):**
- Calculation:  \( 5\% \times 50 = 0.05 \times 50 = 2.5 \) broken marbles
- **Second Set (30% broken):**
- Calculation:  \( 30\% \times 60 = 0.30 \times 60 = 18 \) broken marbles
3.  **Total Broken Marbles:**
- Total:  \( 2.5 \text{ (from the first set)} + 18 \text{ (from the second set)} = 20.5 \)
4.  **Final Answer:**
- The total number of broken marbles in both sets is approximately **21**.
```

9 · Node-2

```
The area that needs the most improvement is **accuracy**.
Specific suggestions for revision:
1.  **Correct the Percentage Calculations:**
- The first set has 10% broken marbles, not 5%.  The calculation should be:
- **First Set:** \( 10\% \times 50 = 0.10 \times 50 = 5 \) broken marbles.
- The second set has 20% broken marbles, not 30%.  The calculation should be:
- **Second Set:** \( 20\% \times 60 = 0.20 \times 60 = 12 \) broken marbles.
2.  **Update the Total Calculation:**
- The total broken marbles should be:
- Total:  \( 5 \text{ (from the first set)} + 12 \text{ (from the second set)} = 17 \).
3.  **Revise the Final Answer:**
- The final answer should state that the total number of broken marbles in both sets is **17**.
By addressing these inaccuracies, the answer will be correct and meet the review criteria more effectively.
```

10 · Node-2

```
To determine the total number of broken marbles in Gabriel's sets, follow these steps:
1.  **Identify the Number of Marbles:**
- **First Set:** 50 marbles
- **Second Set:** 60 marbles
2.  **Calculate Broken Marbles in Each Set:**
- **First Set (10% broken):**
- Calculation:  \( 10\% \times 50 = 0.10 \times 50 = 5 \) broken marbles
- **Second Set (20% broken):**
- Calculation:  \( 20\% \times 60 = 0.20 \times 60 = 12 \) broken marbles
3.  **Total Broken Marbles:**
- Total:  \( 5 \text{ (from the first set)} + 12 \text{ (from the second set)} = 17 \)
4.  **Final Answer:**
- The total number of broken marbles in both sets is **17**.
```

Annotation

Faulty agent	['Node1', 'Node3']
Faulty step	4, 5, 6, 7, 8
Category	["FM-1.3", "FM-3.1"]
Reason	Node 1 injected with prompt_injection; Node 3 injected with response_corruption
Ground truth	In the first set there are 5 broken marbles because 50 x .1 = «50*.1=5»5 In the second set there are 12 broken marbles because 60 x .2 = «60*.2=12»12 There are 17 broken marbles in total because 5 + 12 = «5+12=17»17 #### 17
System output	To determine the total number of broken marbles in Gabriel's sets, follow these steps: 1. **Identify the Number of Marbles:** - **First Set:** 50 marbles - **Second Set:** 60 marbl [...] (420 chars omitted) [...] The total number of broken marbles in both sets is approximately **21**.
Other labels	**faulty_agents_pairs**: [{"agent_name": "Node1", "error_type": "FM-1.3", "injection_strategy": "prompt_injection"}, {"agent_name": "Node3", "error_type": "FM-3.1", "injection_strategy": "response_corruption"}]; **injected_agents**: [{"agent_name": "Node1", "error_type": "FM-1.3", "injection_strategy": "prompt_injection", "malicious_action_description": "Node 1 injected with prompt_injection"}, {"agent_name": "Node3", "error_type": "FM-3.1", "injection_strategy": "response_corruption", "malicious_action_description": "Node 3 injected with response_corruption"}]; **injection_strategies**: ["prompt_injection", "response_corruption"]; **metadata**: {"framework": "macnet", "benchmark": "GSM8K", "model": "gpt-4o-mini", "num_agents": 5, "num_injected_agents": 2, "task_type": "math"}; **is_injection_successful**: True; **step_indexing**: 1-based field conversation_history[].step; **note**: AEGIS labels are agent+error-mode level; step-level label only via is_injected markers where present

Notes. (i) The only dataset here whose primary label is a *set* of (agent, mode) pairs (mean 2.4, up to 4); no decisive step is annotated, so step-level attribution cannot be evaluated on AEGIS (MacNet's `is_injected` flags every turn of an injected agent, not one step). (ii) Failures are fully synthetic, injected into GPT-4o-mini runs and shaped after MAST modes rather than naturally occurring slips; realism rests on $\kappa = 0.81$ and embedding overlap. (iii) Short, homogeneous trajectories (median 10 turns, ≈5 k tokens) with structural agent names. (iv) Data/paper discrepancies: per-framework and per-benchmark counts differ from the paper's Table 2 (e.g. AgentVerse 2,449 vs 1,995, Magentic-One 273 vs 449), pair total 22,701 vs 24,843; typos `smoagents`, `magentic+gaia`; a third strategy value `prompt` (273 pairs); `num_injected_agents` ≠ number of pairs in 22% of records; two duplicated ids. (v) Shares Magentic-One/SmolAgents + GAIA with Who&When, MAST and TracerTraj (shared author), but all runs are the authors' own; VerifyMAS (2026) reuses Aegis-Bench with a global/local/hybrid grouping of the modes. (vi) MIT licence and modest size make it the most convenient large training set in this survey.

4.3 CORRECT-Error

injected | available | agent + step | MAS (Magentic-One variant) | GAIA / HotpotQA / MuSiQue / 2Wiki / ARC / MATH-500 / MMLU-Pro

Paper *CORRECT: COndensed eRror RECognition via knowledge Transfer in multi-agent systems* — Yu et al. (UIUC / Amazon), ICML 2026, arXiv:2509.24088 [71]
Links HF `yifanyu/CORRECT-Error` · GitHub `UIUC-MLSys/CORRECT` · Amazon Science page
License Apache-2.0 (dataset card and GitHub `LICENSE`)
Status downloaded in full, ungated; one parquet file (`test` split, 2,226 rows, ≈20 MB) plus the GitHub clone (code and pre-extracted error schemata, no trajectories); `data/correct_error/`

Description. CORRECT-Error is the evaluation set released with CORRECT, a training-free attributor that caches distilled "error schemata" from past failures and retrieves them to guide an LLM detector. The task is the Who&When one (§3.1): given a failed trajectory, predict `mistake_agent` and `mistake_step`, the earliest step whose correction would have flipped the outcome. All trajectories come from a single MAS—a Magentic-One variant run in AutoGen with selector-group-chat control (roles `Planner`, `WebSurfer`, `Coder`, `CodeExecutor`, `FileSurfer`; `history[0]` is always the `user` task message)—on seven task suites (GAIA L1, HotpotQA, MuSiQue, 2WikiMultihopQA, ARC, MATH-500, MMLU-Pro) and two generator LLMs (`gpt-5-nano` 1,908, `gpt-4o-mini` 318). **Failures are injected**, not observed: (1) a pool of *successful* trajectories is collected; (2) each is matched by semantic similarity to a human-labelled real failure (the Who&When Hand-Crafted set) and GPT-5 devises an injection strategy; (3) the generator model rewrites one step into an erroneous action, so the rewritten step is the decisive error by construction (synthesis cost reported at >3 B tokens). **Validation**: three experts (>120 h) judged a blind mix of synthetic and real failures—47.1% of synthetic trajectories were taken for human-labelled, 94.4% were judged genuine by ≥2 of 3 labelers; the step labels themselves were not re-checked and no κ is reported. Labels are agent + step only: no category, no reason text, no ground-truth answer, and the successful parent trajectories are not released.

Metrics and baselines. Step accuracy with tolerance, Acc@k = $|\text{pred} - \text{gt}| \leq k$; agent accuracy is not reported. Paper Table 2, Qwen-2.5-7B detector, `gpt-5-nano` subset, Acc@1 / @3 / @5 in % (paper):

Task	LLM-as-a-Judge	CORRECT
GAIA	16.7 / 27.8 / 38.9	30.6 / 44.4 / 52.8
HotpotQA	14.7 / 48.9 / 65.8	35.8 / 72.7 / 84.3
MuSiQue	11.9 / 43.9 / 58.8	32.7 / 61.2 / 77.2
2Wiki	6.4 / 35.6 / 56.1	16.9 / 49.9 / 69.0
ARC	62.8 / 69.6 / 71.1	80.4 / 88.2 / 91.2
MATH-500	41.8 / 57.1 / 64.3	57.1 / 87.8 / 95.9
MMLU-Pro	50.0 / 64.7 / 70.6	69.1 / 88.2 / 94.1

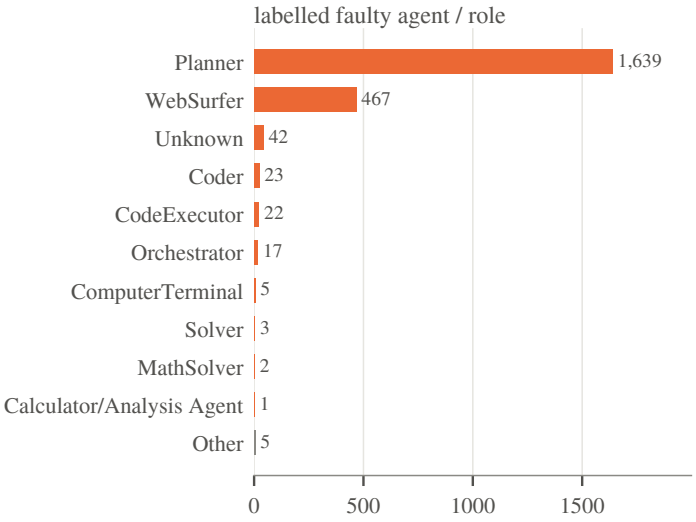
Average gains are +16.8 / +20.1 / +18.9 points on `gpt-5-nano` and +20.1 / +27.6 / +28.7 on `gpt-4o-mini`; on Who&When Hand-Crafted, CORRECT lifts Acc@0 from 3.5 to 12.1 (Qwen-2.5-7B). The released `src/evaluate.py` scores exact match with a `-tolerance` flag.

Schema. One parquet row per trajectory: `trajectory_id` (`{generator}_{dataset}_{question_id}`), `dataset`, `generator_model`, `gaia_level` (1.0 for the 36 GAIA `gpt-5-nano` rows, else null), `question`, `question_id` (suffix `_k` indexes injected variants of one source question), `history` (list of {`role`, `content`}), `mistake_agent`, `mistake_step` (0-based into `history`, user message at 0), `mistake_reason` (empty in every row), `groundtruth` (empty), `is_corrected` (always false), `level` (always 0).

Statistics. computed from the downloaded parquet.

trajectories (all failed, one decisive error each)	2,226
by generator: `gpt-5-nano` / `gpt-4o-mini`	1,908 / 318
distinct source questions (variants per question, max)	1,440 (1.55, 5)
steps: mean / median / min – max	11.8 / 10 / 2 – 22
distinct agents (excl. user): mean / max	1.87 / 5
error step index: mean / median / max	5.2 / 5 / 19
relative error position: mean / median	0.69 / 0.95
characters / trajectory: mean / median	27.7 k / 18.8 k
`history[step].role` = `mistake_agent`	2,040 (91.6%)
rows with `mistake_step` = 0 (user message)	43
non-empty `mistake_reason` / `groundtruth`	0 / 0

By task: 2Wiki 733, HotpotQA 578, MuSiQue 312, ARC 304, MATH-500 157, MMLU-Pro 92, GAIA 50. Length is bimodal: 610 trajectories have ≤4 steps (ARC/MMLU-Pro, median 2) and 809 have 21–22 steps (MuSiQue median 21), which looks like a round cap. Messages by role: Planner 13.1 k, WebSurfer 10.6 k, CodeExecutor 175, Coder 126, FileSurfer 18. Blamed agent: Planner 73.6%, WebSurfer 21.0%, `Unknown` 42, Coder 23, CodeExecutor 22.



Example trajectory. `gpt-5-nano_gaia_level1_task22_1` (GAIA L1; 8 steps, user + Planner + WebSurfer, 3 roles). The WebSurfer opens the BBC script PDF and sees nothing (step 4); at the injected step 6 it re-opens the same URL and now "reads" a scene heading, which the Planner accepts as the final answer. With no `groundtruth` and no `mistake_reason` in the release, the error can only be identified as the rewritten step.

Task (`gpt-5-nano_gaia_level1_task22_1`)

In Series 9, Episode 11 of Doctor Who, the Doctor is trapped inside an ever-shifting maze. What is this location called in the official script for the episode? Give the setting exactly as it appears in the first scene heading.

0 · user

```
You have been given the following task:
In Series 9, Episode 11 of Doctor Who, the Doctor is trapped inside an ever-shifting maze.  What is this location called in the official script for
the episode?  Give the setting exactly as it appears in the first scene heading.
Available capabilities:
- WebSurfer:  Can perform REAL internet searches and browse websites
- FileSurfer:  Can read and analyze files
- Coder:  Can write and analyze code
- CodeExecutor:  Can execute Python code and return results
```

Please analyze this task and decide which specialist to engage first. Remember to:
1. Break down the task into manageable steps
2. Gather information systematically
3. Build upon previous results
What is your first action?

1 · Planner

Analysis of Previous Result:
No prior results to build on. We're at the information-gathering step to locate the first scene heading from the official Doctor Who script for Series 9, Episode 11.
Updated Understanding:
Task is to identify the exact first scene heading as it appears in the official script for Doctor Who Series 9, Episode 11 (the Doctor is in an ever-shifting maze). We must provide the setting exactly as it appears in the first scene heading (including formatting, capitalization, and punctuation).
Decision:
- Next Agent: WebSurfer
- Instructions:
- Search for the official shooting/teleplay script of Doctor Who
[...] (639 chars omitted) [...]
as it appears, and note the source URL next to it. If the heading cannot be found, provide the best authoritative alternative and explain any caveats.
First action: Initiate WebSurfer to locate the official script and extract the first scene heading.

2 · WebSurfer

I typed 'Heaven Sent Doctor Who shooting script first scene heading' into the browser search bar.
The web browser is open to the page [Heaven Sent Doctor Who shooting script first scene heading at DuckDuckGo](https://duckduckgo.com/?q=Heaven+Sent+Doctor+Who+shooting+script+first+scene+heading&ia=web).
The viewport shows 41% of the webpage, and is positioned at the top of the page
The following text is visible in the viewport:
Upgrade to our browser.Try the DuckDuckGo Browser.
Fast. Free. Private.Download Browser
Free
Open menu
All
Images
News
Videos
More
Maps
Shopping
Search Assist
Duck.ai
Search Settings
Always protected
All regions
Saf
[...] (2,506 chars omitted) [...]
title": "Heaven Sent Doctor Who shooting script first scene heading",
"viewport": "width=device-width, initial-scale=1",
"HandheldFriendly": "true",
"apple-mobile-web-app-capable": "no"
}
}
Here is a screenshot of the page.

3 · Planner

Analysis of Previous Result:
The DuckDuckGo/BBC results page is visible and shows a BBC PDF link for the official script (DW9 - EP 11 Heaven Sent) along with other transcript sources. The first scene heading has not yet been extracted.
Updated Understanding:
To answer precisely, we need to open the official BBC script (DW9 - EP 11 Heaven Sent) and read the first scene heading on the first page, capturing the heading exactly as it appears (including INT./EXT. and punctuation).
Decision:
- Next Agent: WebSurfer
- Instructions:
- Open the BBC link to the official script: https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep1
[...] (539 chars omitted) [...]
thoritative version (e.g., the shooting script PDF on BBC or a clearly labeled official teleplay file) and report the heading with caveats.
First action: Use WebSurfer to open the BBC official script URL and extract the exact first scene heading text.

4 · WebSurfer

I typed 'https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf' into the browser address bar.
The web browser is open to the page [](https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf).
The viewport shows 100% of the webpage, and is positioned at the top of the page
The following text is visible in the viewport:
The following metadata was extracted from the webpage:
{}
Here is a screenshot of the page.

5 · Planner

Analysis of Previous Result:
The BBC official script PDF is open, but the first scene heading (the line that starts with INT. or EXT.) has not been extracted yet. We currently only have the title page visible.
Updated Understanding:
We need the exact first scene heading text from the BBC official script for Doctor Who Series 9, Episode 11 (Heaven Sent), including formatting and punctuation, as it appears on the first scene heading line.
Decision:
- Next Agent: WebSurfer
- Instructions:
- Open the BBC script URL: https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf
- Use the PDF viewer to
[...] (420 chars omitted) [...]
the exact heading text and the source URL. If the first heading spans multiple lines or includes a location descriptor, quote it exactly as shown.
First action: Use WebSurfer to locate and extract the exact first scene heading from the BBC script PDF.

6 · WebSurfer · ANNOTATED ERROR STEP

```
I typed 'https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf' into the browser address bar.
The web browser is open to the page (https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf).
The viewport shows 100% of the webpage, and is positioned at the top of the page
The following text is visible in the viewport:
INT. CASTLE - DAY
The following metadata was extracted from the webpage:
{}
Here is a screenshot of the page.
```

7 · Planner

```
**Analysis of Previous Result:**
The BBC PDF view shows the first scene heading line as "INT. CASTLE - DAY" at the top of the page, indicating the location name and time.
**Updated Understanding:**
The first scene heading in the official script (Series 9, Episode 11, Heaven Sent) is exactly:
INT. CASTLE - DAY
**Decision:**
- TASK_COMPLETE
- Final Answer:  INT. CASTLE - DAY
Source:  BBC official script PDF (https://www.bbc.co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf)
```

Annotation

Faulty agent	WebSurfer
Faulty step	6
Category	—
Reason	
Ground truth	—
System output	**Analysis of Previous Result:** The BBC PDF view shows the first scene heading line as "INT. CASTLE - DAY" at the top of the page, indicating the location name and time. **Updated Under [...] (240 chars omitted) [...] co.uk/writers/documents/doctor-who-s9-ep11-heaven-sent-steven-moffat.pdf)
Other labels	**dataset**: gaia; **generator_model**: gpt-5-nano; **question_id**: task22_1; **is_corrected**: False; **level**: 0; **step_indexing**: 0-based index into 'history', where index 0 is the user task message

Notes. (i) Largest step-labelled set in this survey (12× Who&When) but single-framework and synthetic: every failure is an LLM-rewritten step of a successful run, matched to a Who&When failure as template. (ii) Localisation only: no success trajectories, no category, no reason, no reference answer, so detection and explanation cannot be evaluated. (iii) Label noise: `mistake_agent` is free text (`[Planner]`, `Unknown`, `Orchestrator`, `Solver`, `Economist`, even a refusal sentence), 43 rows point at the user message, and 8.4% of rows name an agent that did not speak at the labelled step. (iv) Strong priors: Planner at the last step is right most of the time for the short traces (1,165 of 2,226 errors lie in the last fifth of the trajectory). (v) Overlaps: seeded from Who&When Hand-Crafted; GAIA tasks shared with Who&When, TRAIL and MAST-Data; the other suites (multi-hop QA, ARC, MATH-500, MMLU-Pro) are unique here.

4.4 AgentErrata

injected | not released | step + failure mode | mostly single-agent | BrowseComp / SWE-bench Lite / WebArena

Paper	*Diagnosing with Insights: Structured Analysis of Agent Failures via Behavioral Abstractions* — Bi et al. (Tsinghua / Microsoft Research / UIUC), arXiv:2609.02371, Sep. 2026, preprint (no venue) [7]
Links	arXiv 2609.02371 only; the paper gives no code or data URL and no availability statement
License	not stated (paper CC BY 4.0)
Status	**not released**: HF datasets API, GitHub search and the web were checked on 2026-10-16 with no result. Nothing downloaded; `data/agenterrata/` holds only the Who&When case file (103 KB) that the paper's Appendix C analyses

Description. AgentErrata is the evaluation set for AgentScope, a diagnosis method that abstracts a trajectory into a Reasoning–Action Graph and localises faults on it. It was built because Who&When marks the failing step but has no failure-type label, and because—the authors argue—Who&When's annotated step is often the *earliest detectable* issue rather than the *decisive* one. Trajectories come from three agent stacks, **OpenManus** (tool loop), **OWL** (two-agent user/assistant society) and **Mini SWE Agent**, on BrowseComp, SWE-bench Lite and WebArena; per-framework and per-benchmark counts are not given. Only *successful* runs are kept; then taxonomy-guided **fault injection** is applied: an LLM first judges whether a run is suitable for a given mode, single-step faults are injected by instrumented replay at a chosen step, and faults with cross-step dependencies get extra post-processing to keep the text consistent. Every injected fault is decisive (the task fails). All 303 trajectories were **manually verified**, invalid ones discarded; annotator count, time and agreement are not reported. Labels per trajectory: the injected step and one of 10 modes in 3 categories:

- **Reasoning**: Wrong Context (WC), Instruction Unfollowing (IU), Insufficient Context (IC), Context Miss (CM);
- **Control-flow**: Termination Miss (TM), Premature Termination (PT), Step Loop (SL);
- **Action**: Action Mismatch (AM), Invocation Issue (II), Execution Failure (EF).

Whether an agent/role label exists is not stated.

Metrics and baselines. Fault localisation: step-level accuracy with tolerance (SLA T±0/1/3). Attribution: classification accuracy (CA, predicted mode = gold). Both reported with and without the reference solution; without solution, in % (paper Tables 2–4):

Backbone	Method	SLA ±0 / ±1 / ±3	CA
GPT-4o	All-at-Once	2.1 / 4.2 / 23.7	20.6*
	Step-by-Step	8.9 / 13.2 / 25.7	—
	AgentScope	**30.0 / 35.6 / 49.8**	**43.6**
GPT-5.1	All-at-Once	3.3 / 4.6 / 18.8	18.2*
	Step-by-Step	1.3 / 4.3 / 13.2	—
	AgentScope	**31.4 / 36.6 / 54.1**	**45.9**

*LLM-as-a-judge classifier. With DeepSeek-V3.2 the three methods reach SLA T±0 of 0.7 / 17.8 / 35.0 and AgentScope CA 45.2. The Who&When prompting baselines stay near chance on exact step accuracy even with the strongest backbone.

Schema. Not available. The paper's internal representation is a DAG whose vertices are ⟨id, role, content, ISR⟩ with ISR = {Intent & Context, Reasoning & Action, Signal & Validation}; node types: reasoning, action (tool call/output), user input, output, claim, tool response; edges: control and data dependencies. By analogy with Who&When (benchmarked alongside), a release would hold steps with role/content, a `mistake_step` and a mode label.

Statistics. from the paper (nothing could be computed).

trajectories (all failed, one injected fault)	303
frameworks	3 (counts n/a)
benchmarks	3 (counts n/a)
categories: Reasoning / Control-flow / Action	36.0 / 31.0 / 36.0 %
steps, agents per trajectory	not reported
error position, faulty agent	not reported
annotation	injection + manual check, no IAA

mode share (%, paper Fig. 7; ≈count if scaled to 303)					
WC	8.3 (25)	TM	9.6 (29)	AM	13.2 (40)
IU	9.9 (30)	PT	12.5 (38)	II	7.9 (24)
IC	9.6 (29)	SL	8.9 (27)	EF	14.9 (45)
CM	5.3 (16)				

The distribution is described as "roughly uniform" with biases from the LLM suitability check. The paper's figures are internally inconsistent: category shares sum to 103% and the four Reasoning modes sum to 33.1%, not 36.0%. A runtime illustration mentions traces from 4 steps / 7 calls up to 120 steps / 242 calls (AgentErrata and Who&When pooled).

Example trajectory. The paper prints no AgentErrata record. Its only worked example (Appendix C) is the Who&When Hand-Crafted log `2.json`: Magentic-One (Orchestrator + WebSurfer) on a GAIA question about Ted Danson series, 86 steps, 103 KB. Who&When blames WebSurfer at step 4 (a bare search-results page); the paper reads this as an information gap and locates the decisive failure in the later stagnation loop (steps 11–85, two "Stalled. . . Replanning" events at 37 and 64) and the final execution failure. **The trajectory below is long (86 steps)**; it is shown in full because the paper's argument is about where in it the decisive step lies, with the Who&When label as the marked step.

Task (`Who&When/Hand-Crafted/2.json (question_ID=55f4258484c5b398956133128a50462a767da211f8f72aa5ac5bbffb9bcbba1a)`)

What is the worst rated series (according to Rotten Tomatoes) with more than 1 season that Ted Danson has starred in and is available on Amazon Prime Video (US)?

0 · human

```
What is the worst rated series (according to Rotten Tomatoes) with more than 1 season that Ted Danson has starred in and is available on Amazon
Prime Video (US)?
```

1 · Orchestrator (thought)

```
Initial plan:
We are working to address the following user request:
What is the worst rated series (according to Rotten Tomatoes) with more than 1 season that Ted Danson has starred in and is availab
[...]  (2,479 chars omitted) [...]
mapping may be incorrect.
ledger_response = await self._model_client.create(
```

2 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "The request is not yet fully satisfied as we have not determined the worst-rated Ted Danson series with more than one season that is
availab
[...]  (958 chars omitted) [...]
Additionally, find the Rotten Tomatoes ratings for each of those series."
}
}
```

3 · Orchestrator (-> WebSurfer)

```
Please search for a list of series that Ted Danson has starred in, and confirm which of those series have more than one season.  Additionally, find
the Rotten Tomatoes ratings for each of those series.
```

⚠ 4 · WebSurfer · ANNOTATED ERROR STEP

```
I typed 'Ted Danson TV series list' into '0 characters out of 2000'.
Here is a screenshot of [Ted Danson TV series list - Search](https://www.bing.com/search?q=Ted+Danson+TV+series+list&form=QBLH&sp=-
[...]  (2,576 chars omitted) [...]
r Moore
Mary Steenburgen
Isabella Rossellini
Explore more on Wikipedia
<Image>
```

5 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "The request is not fully satisfied as we have not yet identified the worst rated series with more than 1 season that Ted Danson has
starred
[...]  (926 chars omitted) [...]
has starred in, including which of those series have more than 1 season."
}
}
```

6 · Orchestrator (-> WebSurfer)

```
Please find reliable sources with the complete list of TV series that Ted Danson has starred in, including which of those series have more than 1
season.
```

7 · Orchestrator (thought)

```
Next speaker WebSurfer
```

8 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "The request is not fully satisfied because we have not yet identified the worst-rated series Ted Danson has starred in that is
available on
[...]  (915 chars omitted) [...]
the Rotten Tomatoes scores and availability on Amazon Prime Video (US)."
}
}
```

9 · Orchestrator (-> WebSurfer)

Retry the search for a list of TV series that Ted Danson has starred in and confirm which of those have more than one season. Once the list is obtained, find the Rotten Tomatoes scores and availability on Amazon Prime Video (US).

10 · Orchestrator (thought)

Next speaker WebSurfer

11 · WebSurfer

I clicked 'Credits'.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport shows 7% of the webpa
[...] (3,389 chars omitted) [...]
I和多国大型语言模型检测各种诈骗内容，帮您一眼就破诈骗惯用话术！
- 下载
- 超势科技
- 虚广仲早餐咖啡推荐
- 小队长推荐 早餐咖啡必备
<Image>

12 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not yet fully satisfied as we have not identified the worst-rated series featuring Ted Danson that is available on Amazon Pri
[...] (1,083 chars omitted) [...]
Tomatoes scores and check their availability on Amazon Prime Video (US)."
}
}

13 · Orchestrator (-> WebSurfer)

Please check the TVGuide link you found earlier and retrieve the list of TV series that Ted Danson has starred in, highlighting those with more than one season. Then, find their Rotten Tomatoes scores and check their availability on Amazon Prime Video (US).

14 · Orchestrator (thought)

Next speaker WebSurfer

15 · WebSurfer

I clicked 'Credits'.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport shows 7% of the webpa
[...] (214 chars omitted) [...]
n
Ted Danson
Profession
Actor, Producer, Soundtrack
Actor 88 Credits
<Image>

16 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not yet fully satisfied as we still need to determine which series that Ted Danson starred in, with more than one season, is
[...] (961 chars omitted) [...]
starred in, and confirm which of those series have more than one season."
}
}

17 · Orchestrator (-> WebSurfer)

Please scroll down within the TV Guide page to retrieve the complete list of TV series that Ted Danson has starred in, and confirm which of those series have more than one season.

18 · Orchestrator (thought)

Next speaker WebSurfer

19 · WebSurfer

I scrolled down one page in the browser.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport s
[...] (238 chars omitted) [...]
or
The Good Place
Hearts Beat Loud
CSI: CYBER
CSI: THE FINALE
合活
限推出
<Image>

20 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not fully satisfied as we have not yet determined the worst rated series with more than 1 season that Ted Danson starred in,
[...] (898 chars omitted) [...]
starred in, and confirm which of those series have more than one season."
}
}

21 · Orchestrator (-> WebSurfer)

Please continue to scroll down to further retrieve the full list of TV series that Ted Danson has starred in, and confirm which of those series have more than one season.

22 · Orchestrator (thought)

```
Next speaker WebSurfer
```

23 · WebSurfer

```
I scrolled down one page in the browser.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport s
[...]  (492 chars omitted) [...]
e Hero Dog
FIGHT FOR YOUR RIGHT
Tim and Eric AWESOME SHOW, Great Job!
<Image>
```

24 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "We have not yet identified the worst rated Ted Danson series with more than one season, nor have we checked the availability on Amazon
Prime
[...]  (868 chars omitted) [...]
om another reliable source and identify which have more than one season."
}
}
```

25 · Orchestrator (-> WebSurfer)

```
Please search for a comprehensive list of TV series Ted Danson has starred in from another reliable source and identify which have more than one
season.
```

26 · Orchestrator (thought)

```
Next speaker WebSurfer
```

27 · WebSurfer

```
I scrolled down one page in the browser.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport s
[...]  (332 chars omitted) [...]
obel Son
Help Me Help You
Heist
Nespresso
立即探索
虚度仲享早餐咖啡推荐
早餐时光 每天都有新滋味
<Image>
```

28 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "The request has not been satisfied as we still need to find the worst rated series according to Rotten Tomatoes that Ted Danson has
starred
[...]  (960 chars omitted) [...]
n has starred in from IMDb and identify which have more than one season."
}
}
```

29 · Orchestrator (-> WebSurfer)

```
Please search for a comprehensive list of TV series Ted Danson has starred in from IMDb and identify which have more than one season.
```

30 · Orchestrator (thought)

```
Next speaker WebSurfer
```

31 · WebSurfer

```
I typed 'Ted Danson IMDb TV series list' into the browser search bar.
Here is a screenshot of [Ted Danson IMDb TV series list - Search](https://www.bing.com/search?q=Ted+Danson+IMDb+TV+series+list&FOR
[...]  (3,143 chars omitted) [...]
ovember 21, 2024 (USA)
Content rating
TV-14
—
Sign in
Deep search
<Image>
```

32 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied":  {
"reason":  "The request is not fully satisfied yet as we need the Rotten Tomatoes ratings for each of the series, confirm which series have more
than on
[...]  (840 chars omitted) [...]
Ted Danson on IMDb and confirm which of those have more than one season."
}
}
```

33 · Orchestrator (-> WebSurfer)

```
Please find a comprehensive list of TV series starring Ted Danson on IMDb and confirm which of those have more than one season.
```

34 · Orchestrator (thought)

```
Next speaker WebSurfer
```

35 · WebSurfer

```
I clicked 'IMDb'.
Here is a screenshot of [A Man on the Inside (TV Series 2024- ) - IMDb](https://www.imdb.com/title/tt26670955/).  The viewport shows 9% of the
webpage, and is positioned at the top of
```

[...] (3,225 chars omitted) [...]
ts like buttons and icons are not labeled, so they are not transcribed.
<Image>

36 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request has not yet been fully satisfied as we have not identified the worst-rated series with more than 1 season that Ted Danson has st
[...] (980 chars omitted) [...]
dentify those with more than one season. Summarize this list to proceed."
}
}

37 · Orchestrator (thought)

Stalled.... Replanning...
/usr/local/lib/python3.11/site-packages/autogen_magentic_one/agents/orchestrator.py:184: UserWarning: Resolved model mismatch: gpt-4o-2024-08-06 != gpt-4o-2024-05-13. Model ma
[...] (235 chars omitted) [...]
. Model mapping may be incorrect.
response = await self._model_client.create(

38 · Orchestrator (thought)

New plan:
We are working to address the following user request:
What is the worst rated series (according to Rotten Tomatoes) with more than 1 season that Ted Danson has starred in and is available o
[...] (3,225 chars omitted) [...]
ent retrieval of necessary information, saving time, and minimizing redundancy.

39 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not fully satisfied as we have not yet found the worst rated series according to Rotten Tomatoes that Ted Danson has starred
[...] (855 chars omitted) [...]
MDb, TV Guide, or Wikipedia. Provide the list of series as the response."
}
}

40 · Orchestrator (-> WebSurfer)

Please search for a comprehensive list of TV series that Ted Danson has starred in from a reliable source like IMDb, TV Guide, or Wikipedia. Provide the list of series as the response.

41 · Orchestrator (thought)

Next speaker WebSurfer

42 · WebSurfer

I typed 'Ted Danson complete TV series list IMDb' into '0 characters out of 2000'.
Here is a screenshot of [Ted Danson complete TV series list IMDb - Search](https://www.bing.com/search?q=Ted+Danson+c
[...] (3,690 chars omitted) [...]
t rating**: TV-14
[Button]: Sign in
[Notification Indicator]: 9
—
<Image>

43 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "We do not yet have a comprehensive list of Ted Danson's TV series, their respective Rotten Tomatoes ratings, and their availability on Amazo
[...] (969 chars omitted) [...]
s Ted Danson has starred in. Provide the list of series as the response."
}
}

44 · Orchestrator (-> WebSurfer)

Please open the IMDb link for 'Most Popular Movies and TV Shows With Ted Danson' and gather the complete list of TV series Ted Danson has starred in. Provide the list of series as the response.

45 · Orchestrator (thought)

Next speaker WebSurfer

46 · WebSurfer

I clicked 'Most Popular Movies and TV Shows With Ted Danson - IMDb'.
Here is a screenshot of [With Ted Danson (Sorted by Popularity Ascending)](https://www.imdb.com/search/title/?role=nm0001101). The
[...] (2,740 chars omitted) [...]
tales mysteriously lead back one way or another to Fargo, North Dakota.
<Image>

47 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not fully satisfied as we have yet to gather the complete list of TV series Ted Danson has starred in from the IMDb search an
[...] (977 chars omitted) [...]
sure to include information about the number of seasons for each series."
}
}

48 · Orchestrator (-> WebSurfer)

Please extract a comprehensive list of TV series from the IMDb search results titled 'With Ted Danson (Sorted by Popularity Ascending)'. Ensure to include information about the number of seasons for each series.

49 · Orchestrator (thought)

Next speaker WebSurfer

50 · WebSurfer

I scrolled down one page in the browser.
Here is a screenshot of [With Ted Danson (Sorted by Popularity Ascending)](https://www.imdb.com/search/title/?role=nm0001101). The viewport shows 9% of the web
[...] (1,590 chars omitted) [...]
-PG**
- **8.2 (201K)**
- **Rate**
- **TV Series**
- Michael
<Image>

51 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not fully satisfied as we do not have the complete list of TV series that Ted Danson has starred in with more than one season
[...] (952 chars omitted) [...]
sults and include the number of seasons for each series in the response."
}
}

52 · Orchestrator (-> WebSurfer)

Continue extracting the list of series Ted Danson has starred in from the IMDb search results and include the number of seasons for each series in the response.

53 · Orchestrator (thought)

Next speaker WebSurfer

54 · WebSurfer

I scrolled down one page in the browser.
Here is a screenshot of [With Ted Danson (Sorted by Popularity Ascending)](https://www.imdb.com/search/title/?role=nm0001101). The viewport shows 9% of the web
[...] (1,091 chars omitted) [...]
tionships.
13. Magnum, P.I.
1980–1988 TV-PG
7.5 (30K) Rate TV Series
<Image>

55 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "We have not yet assembled a complete list of TV series Ted Danson has starred in with the number of seasons for each, nor have we identified
[...] (958 chars omitted) [...]
Danson has starred in, along with the number of seasons for each series."
}
}

56 · Orchestrator (-> WebSurfer)

Please continue scrolling through the IMDb advanced search results and provide a more comprehensive list of TV series Ted Danson has starred in, along with the number of seasons for each series.

57 · Orchestrator (thought)

Next speaker WebSurfer

58 · WebSurfer

I scrolled down one page in the browser.
Here is a screenshot of [With Ted Danson (Sorted by Popularity Ascending)](https://www.imdb.com/search/title/?role=nm0001101). The viewport shows 9% of the web
[...] (1,497 chars omitted) [...]
him."
5. Taxi
- (no further details provided in the image)
—
<Image>

59 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not yet satisfied because we don't have the Rotten Tomatoes ratings, number of seasons for each series, and their availabilit
[...] (994 chars omitted) [...]
Danson has starred in, including the number of seasons for each series."
}
}

60 · Orchestrator (-> WebSurfer)

Continue scrolling through the IMDb advanced search results and provide the complete list of TV series Ted Danson has starred in, including the number of seasons for each series.

61 · Orchestrator (thought)

Next speaker WebSurfer

62 · WebSurfer

I scrolled down one page in the browser.
Here is a screenshot of [With Ted Danson (Sorted by Popularity Ascending)](https://www.imdb.com/search/title/?role=nm0001101). The viewport shows 9% of the web
[...] (1,103 chars omitted) [...]
n
- Back to top
—
Additional Elements:
- (i) icon (information)
<Image>

63 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The original request has not yet been fully addressed. We need to list all TV series Ted Danson has starred in with more than 1 season, gath
[...] (1,194 chars omitted) [...]
ratings, and availability on Amazon Prime Video in a more efficient way."
}
}

64 · Orchestrator (thought)

Stalled.... Replanning...

65 · Orchestrator (thought)

New plan:
We are working to address the following user request:
What is the worst rated series (according to Rotten Tomatoes) with more than 1 season that Ted Danson has starred in and is available o
[...] (3,225 chars omitted) [...]
redundant searches and focusing on progressive data collection and comparison.

66 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not fully satisfied yet as we have not identified the worst-rated series starring Ted Danson with more than 1 season that is
[...] (803 chars omitted) [...]
provide a consolidated list of all TV series Ted Danson has starred in."
}
}

67 · Orchestrator (-> WebSurfer)

Please search for 'Ted Danson full TV series list IMDb Wikipedia TV Guide' and provide a consolidated list of all TV series Ted Danson has starred in.

68 · Orchestrator (thought)

Next speaker WebSurfer

69 · WebSurfer

I typed 'Ted Danson full TV series list IMDb Wikipedia TV Guide' into '0 characters out of 2000'.
Here is a screenshot of [Ted Danson full TV series list IMDb Wikipedia TV Guide - Search](https://www.
[...] (3,260 chars omitted) [...]
the Inside
Advancements
Mr. Mayor 2021
Cheers 1982
Help Me Help You
<Image>

70 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request has not been fully satisfied as we still need to collect Rotten Tomatoes ratings, identify series with more than one season, and
[...] (923 chars omitted) [...]
Danson has starred in. Aggregate all names into one comprehensive list."
}
}

71 · Orchestrator (-> WebSurfer)

Please visit IMDb, Wikipedia, and TV Guide for a consolidated list of TV series Ted Danson has starred in. Aggregate all names into one comprehensive list.

72 · Orchestrator (thought)

Next speaker WebSurfer

73 · WebSurfer

I clicked 'TV Guide'.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport shows 7% of the webp
[...] (3,144 chars omitted) [...]
Ted Danson
Profession
Actor, Producer, Soundtrack
Actor 88 Credits
<Image>

74 · Orchestrator (thought)

Updated Ledger:
{
"is_request_satisfied": {

```
"reason": "The request has not been fully satisfied as we have not yet identified the worst-rated series featuring Ted Danson with more than 1
season,
[...] (860 chars omitted) [...]
e list of TV series Ted Danson has starred in from the TV Guide website."
}
}
```

75 · Orchestrator (-> WebSurfer)

```
Please continue to gather the list of TV series Ted Danson has starred in from the TV Guide website.
```

76 · Orchestrator (thought)

```
Next speaker WebSurfer
```

77 · WebSurfer

```
I clicked 'Credits'.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport shows 7% of the webpa
[...] (465 chars omitted) [...]
—
AI防人，看穿局
新用生成式AI和多大型言模型各容，您一眼破息用！
科技
下
<Image>
```

78 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not yet satisfied as we have not yet identified the worst-rated series according to Rotten Tomatoes starring Ted Danson
with
[...] (876 chars omitted) [...]
t of TV series that Ted Danson has starred in from the TV Guide website."
}
}
```

79 · Orchestrator (-> WebSurfer)

```
Please continue to gather the full list of TV series that Ted Danson has starred in from the TV Guide website.
```

80 · Orchestrator (thought)

```
Next speaker WebSurfer
```

81 · WebSurfer

```
I scrolled down one page in the browser.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport s
[...] (263 chars omitted) [...]
od Place
Hearts Beat Loud
CSI: Cyber
CSI: The Finale
MAVEN
SHOP NOW
<Image>
```

82 · Orchestrator (thought)

```
Updated Ledger:
{
"is_request_satisfied": {
"reason": "The request is not yet fully satisfied as we have not gathered the entire list of TV series Ted Danson has starred in, and we have not
yet c
[...] (897 chars omitted) [...]
ide page to gather the full list of TV series Ted Danson has starred in."
}
}
```

83 · Orchestrator (-> WebSurfer)

```
Please continue scrolling down the TV Guide page to gather the full list of TV series Ted Danson has starred in.
```

84 · Orchestrator (thought)

```
Next speaker WebSurfer
```

85 · WebSurfer

```
I scrolled down one page in the browser.
Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/).
The viewport s
[...] (3,225 chars omitted) [...]
essages)
FINAL ANSWER: CSI Cyber
SCENARIO.PY COMPLETE !#!#
RUN.SH COMPLETE !#!#
```

Annotation

Faulty agent	WebSurfer
Faulty step	4
Category	—
Reason	The information obtained by WebSurfer is neither reliable nor satisfies the requirements of the Orchestrator.
Ground truth	CSI: Cyber
System output	I scrolled down one page in the browser. Here is a screenshot of [Ted Danson List of Movies and TV Shows - TV Guide](https://www.tvguide.com/celebrities/ted-danson/credits/3030192417/). [...] (4,857 chars omitted) [...] s) FINAL ANSWER: CSI Cyber SCENARIO.PY COMPLETE !#!# RUN.SH COMPLETE !#!#
Other labels	**is_corrected**: False; **step_indexing**: 0-based index into 'history' (index 0 = human query); history[4] is a WebSurfer step, matching mistake_agent; **paper_appendix_C_reading**: Step 4 (annotated): WebSurfer retrieves only a search-results page without extracting structured information. Steps 11 and 15 (and later): WebSurfer repeats the same click / sees an identical viewport -> stagnation loop (flagged by the system as is_in_loop: true), persisting despite refined Orchestrator instructions; run ends with a content-filter (ResponsibleAI) BadRequestError in the ledger update, not with an answer.; **agentscope_failure_modes_that_would_apply**: ["SL Step Loop (Control-flow)", "IC Insufficient Context (Reasoning)", "EF Execution Failure (Action)"]

Notes. (i) Nothing is public: no data, code or licence; all numbers above are paper-reported and a release would presumably be ≈303 JSON trajectories with a step index and one of 10 modes. (ii) Unique combination if released: injected decisive step + failure-mode label + manual verification, on mostly single-agent tool loops (OpenManus, Mini SWE Agent) rather than MAS. (iii) Taxonomy overlap with MAST (§5.1): SL ≈ FM-1.3, PT ≈ FM-3.1, TM ≈ FM-1.5, AM ≈ FM-2.6, IU ≈ FM-1.1; the Action category (II, EF) matches TRAIL's system-execution errors. (iv) Appendix C's "earliest detectable vs. decisive" point applies to every human-labelled step dataset; in the cited file the run actually ends with "FINAL ANSWER: CSI Cyber", which matches the ground truth, yet it is labelled a failure—a label ambiguity in Who&When itself. (v) Name clash: the AgentScope *method* is unrelated to the `agentscope-ai` framework.

5 Failure-taxonomy datasets (failure-mode labels)

5.1 MAST-Data (MAD)

natural · available · failure modes (trace-level) · MAS, 7 frameworks · 8 benchmarks

Paper	*Why Do Multi-Agent LLM Systems Fail?* — Cemri et al. (UC Berkeley), NeurIPS 2025, arXiv:2503.13657 [9]; introduces the MAST taxonomy and the MAD trace corpus (HF name MAST-Data)
Links	HF `mcemri/MAST-Data` · GitHub `multi-agent-systems-failure-taxonomy/MAST` (taxonomy, LLM annotator, notebooks)
License	dataset CC BY 4.0 (HF card); paper CC BY-NC-ND 4.0
Status	downloaded in full, ungated; `MAD_full_dataset.json` (190 MB, 1,642 traces) + `MAD_human_labelled_dataset.json` (2.9 MB, 19 traces); `data/mast_data/`. No separate `MAST-Data-human` repo exists

Description. MAST is a grounded-theory *taxonomy* of why LLM multi-agent systems fail; MAD is the companion corpus, each trace labelled with a binary vector over the 14 modes (does the mode occur anywhere in the trace?). It is a failure-mode dataset, not an attribution one: there is no faulty step, no faulty agent, no reason text and no task-outcome label. Traces are ordinary (natural) runs of 7 frameworks—AG2 (MathChat / captain-agent group chats), MetaGPT, ChatDev, Magentic-One, AppWorld, HyperAgent, OpenManus—with 5 LLMs (GPT-4o, Claude 3.7 Sonnet, GPT-4o-mini, Qwen-2.5-Coder-32B, CodeLlama-7B) on 8 benchmarks (ProgramDev, ProgramDev-v2, GSM-Plus, OlympiadBench, GAIA, MMLU, AppWorld Test-C, SWE-bench Lite). **Annotation**: (1) taxonomy built by 6 experts open-coding 150+ traces (>20 h each) with three inter-annotator rounds (18 → 17 → 17 → 14 modes); (2) a human-labelled set, 3 annotators, Cohen's κ 0.88, plus an out-of-domain round (OpenManus, Magentic-One) with κ 0.79—21 traces in the paper, 19 in the file; (3) the full 1,642 traces labelled by an **LLM judge** (o1, few-shot) validated against the humans: accuracy 0.94, F1 0.80, κ 0.77. The modes, with the paper's share of all failures (human study) and the number of corpus traces flagged by the judge:

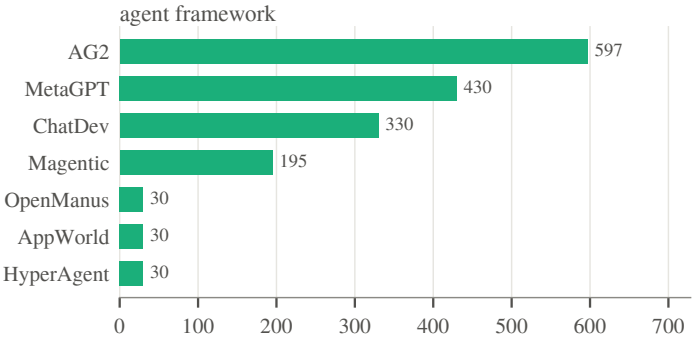
	mode	paper %	traces
FC1 Specification issues (≈44%)			
1.1	Disobey task specification	11.8	591
1.2	Disobey role specification	1.5	91
1.3	Step repetition	15.7	664
1.4	Loss of conversation history	2.8	117
1.5	Unaware of termination conditions	12.4	436
FC2 Inter-agent misalignment (≈32%)			
2.1	Conversation reset	2.2	80
2.2	Fail to ask for clarification	6.8	265
2.3	Task derailment	7.4	353
2.4	Information withholding	0.85	24
2.5	Ignored other agent's input	1.9	370
2.6	Reasoning–action mismatch	13.2	610
FC3 Task verification (≈24%)			
3.1	Premature termination	6.2	274
3.2	No or incomplete verification	8.2	608
3.3	Incorrect verification	9.1	658

Metrics and baselines. The paper is diagnostic; the implied task on MAD is 14-way multi-label trace classification, with the o1 judge as the only reported system (accuracy 0.94, precision 0.83, recall 0.77, F1 0.80, κ 0.77 few-shot; κ 0.58 zero-shot). Intervention studies: a role/workflow fix raises ChatDev task success by 9.4 points and an added verification step by 15.6 on ProgramDev (paper). MetaGPT shows 60–68% fewer FC1/FC2 failures than ChatDev but 1.56× more FC3.

Schema. `MAD_full_dataset.json` is a list of records with `mas_name`, `llm_name`, `benchmark_name`, `trace_id` (0–1641), `trace` = {`key` "`<MAS>_<bench>_<LLM>`", `index`, `trajectory`} where `trajectory` is *one raw log string* whose layout differs per framework (ChatDev/MetaGPT/OpenManus logs, AG2 printed message dicts, Magentic console transcript, AppWorld REPL, HyperAgent logger lines), and `mast_annotation`: 14 keys `"1.1"`...`"3.3"` → 1 / 0 / null. The human file has `round`, the same metadata and `annotations`: per mode, three boolean annotator votes; the mode list differs per round (18/17/17/14) so codes are not comparable across rounds.

Statistics. computed from the downloaded data.

traces: LLM-judge labelled / human-labelled	1,642 / 19
(framework, benchmark, LLM) cells	15
traces with ≥1 flag / none / all-null	1,237 (75.5%) / 401 / 4
flagged modes / trace: mean / median / max	3.14 / 2 / 14
traces with any FC1 / FC2 / FC3 mode	954 / 876 / 907
characters / trace: mean / median / max	115.9 k / 9.7 k / 2.0 M
steps / trace (heuristic): mean / median / max	20.1 / 6 / 513
agents / trace (heuristic): mean / max	3.9 / 7
faulty step / agent, task outcome	not labelled



LLMs: GPT-4o 751, Claude 323, Qwen 200, CodeLlama 200, GPT-4o-mini 168. Benchmarks: ProgramDev-v2 400, ProgramDev 390, GSM 223, Olympiad 206, GAIA 195, MMLU 168, Test-C 30, SWE-bench Lite 30. Any-failure rate by framework: OpenManus and AppWorld 96.7%, MetaGPT 94.9%, ChatDev 92.4%, HyperAgent 90.0%, Magentic 78.5%, AG2 47.9%. Median steps: AG2 4, MetaGPT 6, Magentic 19, OpenManus 23, HyperAgent 50.5, ChatDev 66; 334 traces exceed 200 k characters. Most common label combinations after "none": 1.3+1.5 (54), 1.1 alone (46), 3.2 alone (34).

Example trajectory. AG2 `trace_id` 124 (MathChat `mathproxyagent` ↔ `assistant`, GSM-Plus, Claude; 6 messages, 4.8 k characters), flagged with FM-2.6 only. The dataset has no step label, so no step is marked: the mismatch is visible at step 1, where the assistant's code prints \$302 but its text states \$254, after which it silently switches to \$302. Steps were obtained by parsing the printed AG2 message list.

Task (AG2_GSM_Plus_Claude#124 (trace_id=124))

Let's use Python to solve a math problem.
Query requirements:
You should always use the 'print' function for the output and use fractions/radical forms instead of decimals.
You can use packages like sympy to help you.
You must follow the formats below to write your code:
```python
# your code
```
First state the key idea to solve the problem. You may choose from three ways to solve the problem:
Case 1: If the problem can be solved with Python code directly, please write a program to solve it. You
[...] (700 chars omitted) [...]
n. Spaying her cost \$200, her 3 vaccines cost \$20 each, but Leah got a 10% discount on the total vaccine cost, and the kitten broke 4 vases that cost \$12 each. How much has the kitten cost so far?

0 · mathproxyagent

```
Let's use Python to solve a math problem.
Query requirements:
You should always use the 'print' function for the output and use fractions/radical forms instead of decimals.
You can use packages like sympy to help you.
You must follow the formats below to write your code:
```python
# your code
```
First state the key idea to solve the problem. You may choose from three ways to solve the problem:
Case 1: If the problem can be solved with Python code directly, please write a program to solve it. You can enumerate all possible arrangements
if needed.
Case 2: If the problem is mostly reasoning, you can solve it by yourself directly.
Case 3: If
[...] (500 chars omitted) [...]
answer in \boxed{}.
Problem:
Leah just got a new kitten. Spaying her cost $200, her 3 vaccines cost $20 each, but Leah got a 10% discount on the total vaccine cost, and the
kitten broke 4 vases that cost $12 each. How much has the kitten cost so far?
```

1 · assistant

```
# Key Idea
I'll solve this problem directly with Python code.  We need to calculate the total cost which includes:
1.  Spaying cost:  $200
2.  Vaccines cost:  3 vaccines at $20 each with a 10% discount
3.  Broken vases:  4 vases at $12 each
```python
from fractions import Fraction
# Calculate the costs
spaying_cost = 200
vaccine_cost_per_unit = 20
number_of_vaccines = 3
vaccine_discount = Fraction(10, 100) # 10%
vaccine_cost = number_of_vaccines * vaccine_cost_per_unit * (1 - vaccine_discount)
vase_cost_per_unit = 12
number_of_vases = 4
vase_cost = number_of_vases * vase_cost_per_unit
total_cost = spaying_cost + vaccine_cost + vase_cost
print(f"Spaying cost:  ${spaying_cost}")
print(f"Vaccine cost (with discount):  ${vaccine_cost}")
print(f"Cost of broken vases:  ${vase_cost}")
print(f"Total cost:  ${total_cost}")
```
The answer is \boxed{$254}.
```

2 · mathproxyagent

```
Spaying cost:  $200
Vaccine cost (with discount):  $54
Cost of broken vases:  $48
Total cost:  $302
```

3 · assistant

```
I need to review my calculations as the output shows a discrepancy.  Let me recalculate:
# Key Idea
I'll solve this problem directly with Python code but with corrected calculations.  Let me carefully check each part:
"'python
from fractions import Fraction
# Calculate the costs
spaying_cost = 200
# Vaccine calculations - 3 vaccines at $20 each, then 10% discount on the total
vaccine_cost_per_unit = 20
number_of_vaccines = 3
vaccine_subtotal = number_of_vaccines * vaccine_cost_per_unit
vaccine_discount = Fraction(10, 100) # 10%
vaccine_cost = vaccine_subtotal * (1 - vaccine_discount)
# Broken vases - 4 vases at $12 each
vase_cost_per_u
[...]  (399 chars omitted) [...]
ion.  Let me correct the analysis manually:
1.  Spaying cost:  $200
2.  Vaccines:  3 vaccines at $20 each = $60, with 10% discount = $60 - $6 = $54
3.  Broken vases:  4 vases at $12 each = $48
Total cost:  $200 + $54 + $48 = $302
The answer is \boxed{$302}.
```

4 · mathproxyagent

```
Spaying cost:  $200
Vaccine subtotal:  $60
Vaccine cost (with discount):  $54
Cost of broken vases:  $48
Total cost:  $302
```

5 · assistant

```
Great!  The calculations are now correct.
# Final Solution
Let's summarize the costs:
1.  Spaying cost:  $200
2.  Vaccines:  3 vaccines at $20 each = $60 subtotal, with 10% discount = $54
3.  Broken vases:  4 vases at $12 each = $48
Adding all these up:  $200 + $54 + $48 = $302
The answer is \boxed{$302}
```

Annotation

Faulty agent	—
Faulty step	—
Category	["FM-2.6 Reasoning-action mismatch (FC2 Inter-agent misalignment)"]
Reason	
Ground truth	—
System output	Great! The calculations are now correct. # Final Solution Let's summarize the costs: 1. Spaying cost: $200 2. Vaccines: 3 vaccines at $20 each = $60 subtotal, with 10% discount = $54 3. [...] (41 chars omitted) [...] Adding all these up: $200 + $54 + $48 = $302 The answer is \boxed{$302}
Other labels	**mast_annotation**: {"1.1": 0, "1.2": 0, "1.3": 0, "1.4": 0, "1.5": 0, "2.1": 0, "2.2": 0, "2.3": 0, "2.4": 0, "2.5": 0, "2.6": 1, "3.1": 0, "3.2": 0, "3.3": 0}; **mas_name**: AG2; **llm_name**: Claude; **benchmark_name**: GSM; **annotator**: LLM judge (OpenAI o1, few-shot) — trace-level binary labels only; **step_split**: heuristic: AG2 message list parsed with ast.literal_eval; agent = message 'name'

Notes. (i) Taxonomy corpus, not attribution benchmark: trace-level binary flags only—usable for mode classification and taxonomy statistics, not for localisation metrics. (ii) Only 19 traces carry human labels; the corpus labels are o1's (κ 0.77), and corpus shares differ from the paper's human frequencies, notably FM-2.5 (7.2% of flags vs. 1.9%) and FM-3.2/3.3. Our per-round κ on the human file is 1.00 for rounds 1–2, which suggests post-discussion consolidated labels. (iii) Seven raw log layouts (1 k–2 M characters); any step-level use needs framework-specific parsers (ours are in `scripts/analyze_mast.py`; 32 AG2/HyperAgent traces fail to parse). (iv) Framework, benchmark and LLM are confounded: most cells pair one framework with one LLM (all HyperAgent runs are Claude on SWE-bench Lite). (v) Quirks: 4 all-null annotations, swapped `mas_name`/`benchmark_name` on the two GAIA rows of the human file, round name "Generlazability". (vi) Overlaps: Magentic-One/GAIA traces share task source with Who&When, TRAIL and CORRECT-Error; SWE-bench Lite with TRAIL and AgentErrata; MAST is the reference taxonomy that AgentErrata's and several other 2025–26 datasets' modes map onto.

5.2 TRAIL

natural | available | HF copy gated | span + category + impact | single agent / 2-level hierarchy | GAIA / SWE-bench Lite

Paper	*TRAIL: Trace Reasoning and Agentic Issue Localization* — Deshpande et al. (Patronus AI), arXiv:2505.08638 (v3, Jun. 2025), preprint [17]
Links	HF `PatronusAI/TRAIL` (gated, click-through) · GitHub `patronus-ai/trail-benchmark` (code + full data, ungated)
License	MIT (HF card and GitHub); the HF card adds "evaluation only, do not train automatic evaluators on it" and a no-resharing pledge
Status	downloaded from GitHub (HF access not requested); 148 trace JSONs (178 MB) + 148 annotation JSONs (1.2 MB); `data/trail/trail_github/benchmarking/`

Description. TRAIL asks whether an LLM can *debug* a full observability trace: list *every* error with its location (a span id), its category (a leaf of a 3-level taxonomy), its impact (HIGH/MEDIUM/LOW), evidence and a description, and give trace-level quality scores. Traces are OpenTelemetry/OpenInference span trees captured through the Patronus SDK, with complete LLM prompts, outputs, tool I/O and token counts. Two sources: (a) **GAIA** (validation set) run by Hugging Face OpenDeep-Research on smolagents—a manager `CodeAgent` that may delegate to a `search_agent` (`ToolCallingAgent`) with search/visit/page/inspect tools, backbone o3-mini; (b) **SWE-bench Lite** run by a single smolagents

`CodeAgent` producing a git patch, backbone Claude 3.7 Sonnet, with some errors provoked by output-length limits and forced-exploration prompts. Errors are **natural**, annotated post hoc; traces are not selected by outcome (several end with the right answer, 4 have no error). **Annotation**: 4 annotators with SE/log-debugging background, ≈30 min per GAIA and ≈40 min per SWE-bench trace plus ≈20 min verification, 63 traces double-assigned ("high" agreement, no κ); four verification rounds revised 5.3% of GAIA and 5.6% of SWE-bench spans. Rules: mark the first instance of an error (Resource Abuse: the last), leaf categories only. Taxonomy:

- **Reasoning**: hallucinations (Language-only, Tool-related), information processing (Poor Information Retrieval, Tool Output Misinterpretation), decision making (Incorrect Problem Identification, Tool Selection Errors), output generation (Formatting Errors, Instruction Non-compliance);
- **System execution**: configuration (Tool Definition Issues, Environment Setup), API/system (Rate Limiting, Authentication, Service Errors, Resource Not Found), resource management (Resource Exhaustion, Timeout);
- **Planning & coordination**: context management (Context Handling Failures, Resource Abuse), task management (Goal Deviation, Task Orchestration).

Metrics and baselines. Per trace: Category F1 (weighted multi-label over the 21 leaves), Location accuracy (|predicted span ids ∩ gold|/|gold|), Joint accuracy (exact (span, category) pairs) and Pearson ρ against the human overall score. The judge receives the raw trace JSON (median 435 k characters), so context length binds. Paper Table 1, mean of 3 runs, Cat. F1 / Loc. / Joint (paper):

Model	GAIA	SWE-bench
Llama-4-Maverick	0.122 / 0.023 / 0.000	0.191 / 0.083 / 0.000
GPT-4.1	0.218 / 0.107 / 0.028	0.166 / 0.000 / 0.000
o3 (high)	0.296 / 0.535 / 0.092	CLE
Claude-3.7-Sonnet	0.254 / 0.204 / 0.047	CLE
Gemini-2.5-Pro	**0.389** / **0.546** / **0.183**	0.148 / **0.238** / **0.050**
Gemini-2.5-Flash	0.337 / 0.372 / 0.100	**0.213** / 0.060 / 0.000

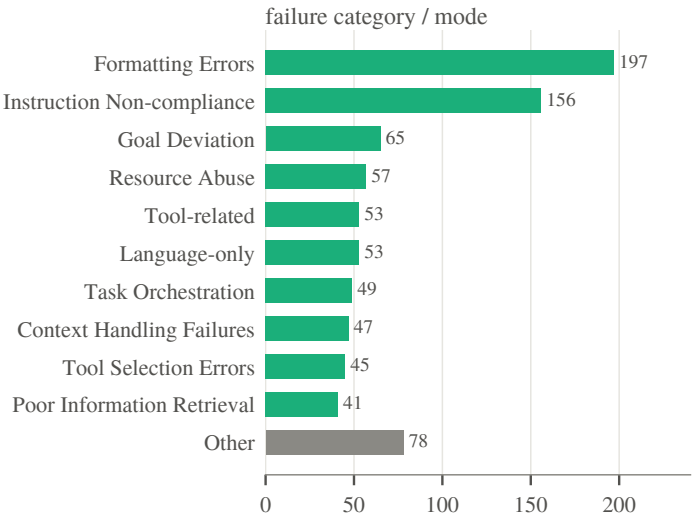
CLE = context length exceeded. Joint accuracy stays below 0.2 everywhere; ρ with human scores peaks at 0.74 (Claude, GAIA) and 0.82 (Gemini Pro, SWE-bench).

Schema. Two files per trace, both named `<trace_id>.json`. **Trace**: {`trace_id`, `spans`: [root, ...]}; each span has `span_id` (16 hex), `parent_span_id`, `span_name` (`main`, `CodeAgent.run`, `Step N`, `LiteLLMModel.__call__`, `VisitTool`, ...), `span_attributes` with OpenInference keys (`openinference.span.kind` ∈ {AGENT, CHAIN, LLM, TOOL} or absent for framework functions; `input.value`, `output.value`, `llm.input_messages.*`, `llm.output_messages.*`, `llm.token_count.*`, `tool.*`), `duration`, `status_code`, `logs` (function-call records; the GAIA `answer_single_question` log embeds `question` and `true_answer`) and recursive `child_spans`. **Labels**: {`errors`: [{`category`, `location` = span id, `evidence`, `description`, `impact`}], `scores`: [reliability, security, instruction-adherence, plan-optimality (0–5, each with reasoning) and `overall`]}. **Step = span**: the tree is flattened in DFS (document) order, which equals chronological order in all 148 traces; each span becomes one step and an error's step index is the DFS position of its `location` span. In practice the attributable unit is an LLM call (88% of error spans are LLM spans, 11% TOOL); CHAIN spans (`Step N`) and AGENT spans wrap them, so one agent "turn" spreads over several steps.

Statistics. computed from the downloaded GitHub data.

	All	GAIA	SWE-bench
traces	148	117	31
spans (all kinds)	4,626	3,579	1,047
spans / trace: mean / median	31.3 / 20	30.6 / 15	33.8 / 32
spans / trace: min – max	11 – 178		14 – 58
agents / trace: mean (max)	1.31 (2)	1.4 (2)	1 (1)
errors	841	585	256
errors / trace: mean (max)	5.68 (20)	5.0	8.3
spans carrying ≥1 error	577		
impact HIGH/MED/LOW	304/363/174	277/185/123	27/178/51
JSON chars / trace: median	435 k	248 k	1.62 M
human score (0–5): mean	3.36	3.29	3.60

Span kinds: LLM 1,974, CHAIN 1,268, TOOL 648, AGENT 207, framework 529. Top-level category: Reasoning 592 (70%), Planning 218 (26%), System 31 (4%). Error spans: LLM 744, TOOL 95, 2 unresolved ids; by agent, CodeAgent 577, ToolCallingAgent 253. Relative position of error spans: mean 0.64, median 0.67. Paper counts differ slightly (118/30 traces, 575 error spans). No success label, but the GAIA `true_answer` is embedded in the log.



Example trajectory. GAIA trace `5e5dc94e...` (the shortest with ≥2 errors: 11 spans, 1 agent, 5 errors on 2 spans). The manager CodeAgent answers "6" correctly but from memory, without any tool call; the planning LLM span (step 6) and the acting LLM span (step 8) carry Instruction Non-compliance, Incorrect Problem Identification, Task Orchestration and Tool Selection errors, three of them HIGH. Steps are spans in DFS order; LLM steps show the last input message and the output.

Task (`5e5dc94e090341c564d582f551a0cddb`)

How many more blocks (also denoted as layers) in BERT base encoder than the encoder from the architecture proposed in Attention is All You Need?

0 · framework / main [INTERNAL]

```
Function call: {"function.name": "main", "function.arguments": {}}
```

1 · framework / get_examples_to_answer [INTERNAL]

```
Function call: {"function.name": "get_examples_to_answer", "function.arguments": {"answers_file": "output/validation/gaia-hugo.jsonl",
"eval_ds": "Dataset({\n features: ['task_id', 'question', 'task', 'true_answer', 'file_name', 'file_path', 'Annotator Metadata'],\n num_rows:
1\n})"}}
```

2 · framework / answer_single_question [INTERNAL]

```
Function call: {"function.name": "answer_single_question", "function.arguments": {"answers_file": "output/validation/gaia-hugo.jsonl",
"example": {"Annotator Metadata": {"How long did this take?": "10 minutes", "Number of steps": "6", "Number of tools": "3",
"Steps": "1. Search the internet for \"blocks in bert base\"\n2. Examine the search results page to locate the answer (12)\n3. Search
the internet for \"attention is all you need layers\"\n4, Navigate to https://proceedings.neurips.cc/paper_files/paper/2017/file/
3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf from the search results page\n5. Examine the architecture section of the PDF to loc
```

[...] (281 chars omitted) [...]
tecture proposed in Attention is All You Need?", "task": "1", "task_id": "11af4e1a-5f45-467d-9aeb-46f4bb0bf034", "true_answer": "6"}, "model_id": "o3-mini", "visual_inspection_tool": "<smolagents.tools.tool.<locals>.SimpleTool object at 0x1324cf050>"}}

3 · framework / create_agent_hierarchy [INTERNAL]

Function call: {"function.name": "create_agent_hierarchy", "function.arguments": {"model": "<smolagents.models.LiteLLMModel object at 0x13492f090>"}}

4 · CodeAgent / CodeAgent.run [AGENT]

Input: {"task": "You have one question to answer. It is paramount that you provide a correct answer.\nGive it all you can: I know for a fact that you have access to all the relevant tools to solve it and find the correct answer (the answer does exist). Failure or 'I cannot answer' or 'None found' will not be tolerated, success will be rewarded.\nRun verification steps if that's needed, you must make sure you find the correct answer!\nHere is the task:\nHow many more blocks (also denoted as layers) in BERT base encoder than the encoder from the architecture proposed in Attention is All You Need?", "stream": false, "reset": true, "images": null, "additional_args": null, "max_steps": null}
Output: 6

5 · CodeAgent / LiteLLMModel.__call__ [LLM]

Input (last of 1 msgs, role=user): Below I will present you a task.
You will now build a comprehensive preparatory survey of which facts we have at our disposal and which ones we still need.
To do so, you will have to read the task and identify things that must be discovered in order to successfully complete it.
Don't make any assumptions. For each item, provide a thorough reasoning. Here is how you will structure this survey:
—
1. Facts given in the task
List here the specific facts given in the task that could help you (there might be nothing here).
2. Facts to look up
List here any facts that we may need to look up.
Also lis
[...] (2,103 chars omitted) [...]
the two architectures.
- Using the lookup facts: subtracting the number of layers in the Transformer encoder (from Attention is All You Need) from that of the BERT base encoder.
- This computation will yield how many more blocks BERT base has.

⚠ 6 · CodeAgent / LiteLLMModel.__call__ [LLM] · ANNOTATED ERROR STEP

Input (last of 1 msgs, role=user): You are a world expert at making efficient plans to solve any task using a set of carefully crafted tools.
Now for the given task, develop a step-by-step high-level plan taking into account the above inputs and list of facts.
This plan should involve individual tasks based on the available tools, that if executed correctly will yield the correct answer.
Do not skip steps, do not add any superfluous steps. Only write the high-level plan, DO NOT DETAIL INDIVIDUAL TOOL CALLS.
After writing the final step of the plan, write the '\n<end_plan>' tag and stop there.
Here is your task:
Task:
“‘
You have one que
[...] (2,605 chars omitted) [...]
is All You Need") has 6 layers.
4. Compute the difference: 12 (BERT) minus 6 (Transformer) equals 6.
5. Verify that this answer is in line with the known architectures from the literature.
6. Use the final_answer tool to provide the computed result.

7 · CodeAgent / Step 1 [CHAIN]

Input: {"memory_step": "ActionStep(model_input_messages=None, tool_calls=None, start_time=1742403048.1096299, end_time=None, step_number=1, error=None, duration=None, model_output_message=None, model_output=None, observations=None, observations_images=None, action_output=None)"}
Output: Execution logs:
Last output from code snippet:
6

⚠ 8 · CodeAgent / LiteLLMModel.__call__ [LLM] · ANNOTATED ERROR STEP

Input (last of 4 msgs, role=assistant): [PLAN]:
Here is the plan of action that I will follow to solve the task:
“‘
1. Recap the task: We need to compare the number of transformer blocks (or layers) in the BERT base encoder with the number of layers in the encoder from the paper "Attention is All You Need" and then compute the difference.
2. Retrieve the established fact that the BERT base encoder has 12 layers from literature.
3. Retrieve the established fact that the original Transformer encoder (from "Attention is All You Need") has 6 layers.
4. Compute the difference: 12 (BERT) minus 6 (Transformer) equals 6.
5. Verify that this answer
[...] (171 chars omitted) [...]
literature that the BERT base encoder uses 12 layers, while the encoder from "Attention is All You Need" uses 6 layers. Therefore, the BERT base encoder has 6 more blocks (layers) than the original Transformer encoder.
Code:
“‘py
final_answer(6)
“‘

9 · CodeAgent / FinalAnswerTool [TOOL]

Tool: final_answer | Input: {"args": [6], "sanitize_inputs_outputs": false, "kwargs": {}}

10 · framework / LiteLLMModel.__call__ [LLM]

Input (last of 6 msgs, role=user):
Read the above conversation and output a FINAL ANSWER to the question. The question is repeated here for convenience:
You have one question to answer. It is paramount that you provide a correct answer.
Give it all you can: I know for a fact that you have access to all the relevant tools to solve it and find the correct answer (the answer does exist). Failure or 'I cannot answer' or 'None found' will not be tolerated, success will be rewarded.
Run verification steps if that's needed, you must make sure you find the correct answer!
Here is the task:
How many more blocks (also denoted as layers) in BERT bas
[...] (863 chars omitted) [...]
If you are asked for a comma separated list, apply the above rules depending on whether the elements are numbers or strings.
If you are unable to determine the final answer, output 'FINAL ANSWER: Unable to determine'
Output (o3-mini): FINAL ANSWER: 6

Annotation

Faulty agent	—
Faulty step	6, 8
Category	["idx 6 (4442f42f0f574602): Instruction Non-compliance [LOW]", "idx 6 (4442f42f0f574602): Incorrect Problem Identification [MEDIUM]", "idx 8 (1c12443a708ec6a5): Task Orchestration [HIGH]", "idx 8 (1c12443a708ec6a5): Tool Selection Errors [HIGH]", "idx 8 (1c12443a708ec6a5): Instruction Non-compliance [HIGH]"]
Reason	[Instruction Non-compliance] The system failed to end the generated plan with the required "<end_plan>" tag, instead ending with trailing newlines and the closing code block marker from the previous turn's formatting. ‖ [Incorrect Problem Identification] The system's plan incorrectly assumes the factual values for the number of layers in BERT base (12) and the Transformer encoder (6) in steps 2 and 3, rather than including steps to look up or retrieve these facts using the available tools as required by the overall system prompt and the "Facts to look up" section from the previous turn. ‖ [Task Orchestration] The system failed to follow the step-by-step plan it generated in the previous turn. It jumped directly to the final computation and answer step (corresponding to steps 4 and 6 in the plan) without executing the preceding retrieval/lookup steps (steps 2 and 3). ‖ [Tool Selection Errors] The system failed to use any of the available tools (specifically 'search_agent' or potentially 'inspect_file_as_text' if papers were provided as files) to perform the lookup steps (finding the number of layers) as indicated in the "Facts to look up" section and the generated plan (steps 2 and 3). ‖ [Instruction Non-compliance] The system failed to adhere to the core instruction of solving the task by using the provided tools and code blobs. It relied solely on its internal knowledge to determine the answer.
Ground truth	6
System output	{"args": [6], "sanitize_inputs_outputs": false, "kwargs": {}}
Other labels	**errors**: [{"location_idx": 6, "span_id": "4442f42f0f574602", "category": "Instruction Non-compliance", "group": "Reasoning/Output generation", "impact": "LOW", "evidence": "Use the final_answer tool to provide the computed result.\n\n\n\n“", "description": "The system failed to end the generated plan with the required \"<end_plan>\" tag, instead ending with trailing [...] (2,796 chars omitted) [...] ction of solving the task by using the provided tools and code blobs. It relied solely on its internal knowledge to determine the answer."}]; **scores**: {"reliability_score": 2, "reliability_reasoning": "The system provided a seemingly correct answer, but it did so by bypassing the verification steps and tool usage required by the task, relying solely on its internal knowledge, which could be unreliable for other tasks.", "security_score": 5, "security_reasoning": "No security issues were detected.", "instru [...] (239 chars omitted) [...] pt_reasoning": "The plan was flawed by assuming facts, and the execution completely ignored the plan's intermediate steps.", "overall": 2.5}; **split**: gaia; **step_indexing**: 0-based index over spans flattened in DFS order (idx) — the dataset itself addresses errors by span_id, not by index

Notes. (i) The only dataset here with real observability traces (span tree, full prompts, tool I/O, tokens) and multi-error, span-level human labels with category, impact and evidence; errors are process errors with no decisive-error notion and no outcome label. (ii) Mostly single-agent: all SWE-bench traces are one CodeAgent and 70 of 117 GAIA traces never call the search sub-agent; the taxonomy has no inter-agent category (contrast MAST FC2). (iii) Very long inputs (median 435 k, SWE-bench median 1.6 M characters), which is why o1/o3/Claude hit context limits; a DFS-flattened span list with LLM outputs is the practical representation. (iv) Label noise: 31 raw category spellings ("Instruction non complience") need normalisation (the official scorer fuzzy-matches); 2 locations match no span; "Incorrect Memory Usage" appears in the scorer and data but not in the paper's taxonomy figure. (v) Output-generation errors dominate (Formatting + Instruction Non-compliance = 42%, often LOW/MEDIUM); system errors are rare (31). (vi) Access: the GitHub copy is MIT and ungated, but the HF card's evaluation-only restriction should be respected. Overlaps: GAIA validation tasks with Who&When, MAST-Data and CORRECT-Error; SWE-bench Lite with MAST-Data and AgentErrata; the smolagents stack is unique.

6 Related trajectory-debugging datasets

6.1 AgentErrorBench (AgentDebug)

natural | available | step + module + error type | single-agent | ALFWorld / GAIA / WebShop

Paper	*Where LLM Agents Fail and How They Can Learn From Failures* (ICML listing: *AgentDebug: . . .)* — Zhu, Liu, Li, Tian, Yang, Zhang, Han, Xie, Cui, Zhang, Ma, Yu, Ramesh, Wu, Liu, Lu, Zou, You (UIUC U Lab, with Stanford / AMD), ICML 2026, arXiv:2509.25370 [82]. (The id 2509.23071 sometimes quoted for this paper is a different paper, EviPath.)
Links	GitHub `ulab-uiuc/AgentDebug` (code, no data) · Google Drive folder (official data, linked from the README) · HF `davide221/agenterrorbench` (third-party parquet mirror) · re-packaged as `agentdebugbench` in `THU-KEG/TrajDebug`
License	code MIT ("U Lab @UIUC"); no license file on the Drive data (ALFWorld / WebShop / GAIA terms apply); paper CC BY-NC-SA 4.0
Status	downloaded in full: the Drive folder yielded 196 of 200 trajectories (98 ALFWorld / 48 GAIA / 50 WebShop) + 3 label files, the 4 missing ones were filled from the HF mirror; 124 MB; `data/agenterrorbench/{gdrive, hf_davide221, repo_AgentDebug}/`. The camera-ready additionally announces **AgentDebug-Traj2k** (≈2 k proposer–reviewer-annotated trajectories) and the **AgentDebugger-4B** detector: not released — no artefact on GitHub, HF or Drive as of 2026-10-16.

Description. AgentErrorBench is the test-bed of the *AgentDebug* framework: given a failed single-agent rollout, predict (i) the step of the first critical (root-cause) error, (ii) the agent *module* that produced it and (iii) the fine-grained error type from the accompanying *AgentErrorTaxonomy*; AgentDebug then re-rolls the agent from that step with corrective feedback. The authors ran their own modular ReAct-style agent (each turn emits `<memory>`, `<reflection>`, `<plan>`, `<action>` blocks; 30-step cap) with GPT-4o, Llama-3.3-70B-Turbo and Qwen3-8B on three environments, collected >500 natural failures, derived the taxonomy from them and curated **200 representative failed trajectories**:

- **ALFWorld** (100): embodied household text game; every released run hits the 30-step cap.
- **WebShop** (50): e-commerce browsing with `search[]` / `click[]` actions.
- **GAIA** (50): tool-using research assistant; the shortest runs (1–30 steps).

Annotation: ten graduate annotators labelled decision steps with taxonomy types and marked the minimal set of root-cause steps; guidelines were refined over three pilot rounds, a shared subset was double-annotated and adjudicated; Cohen's $\kappa = 0.55$ across modules (paper). In the release every trajectory carries exactly *one* annotated step (= `critical_failure_step`) with one module, one type and a free-text reasoning; no repair text is included. **Taxonomy** (5 modules / 17 types): Memory (over-simplification, hallucination, retrieval failure), Reflection (progress misjudgement, outcome misinterpretation, causal misattribution, hallucination), Planning (constraint ignorance, impossible action, inefficient planning), Action (planning–action disconnect, format error, parameter error) and System (step-limit exhaustion, tool execution error, LLM limit, environment error).

Metrics and baselines. **Step** accuracy (exact step), **Step+Module** and **All-Correct** (step + module + type), detector backbone GPT-4.1 at temperature 0; cells are Step / Step+Module / All-Correct in % (paper Table 1):

Method	ALFWorld	WebShop	GAIA	Avg
Direct prompting	28 / 14 / 1	30 / 6 / 0	26 / 10 / 0	28.0 / 10.0 / 0.3
Brute force	10 / 5 / 0	8 / 0 / 0	18 / 8 / 0	12.0 / 4.3 / 0.0
Binary search	20 / 6 / 1	14 / 8 / 0	22 / 10 / 0	18.7 / 8.0 / 0.3
AgentDebug	**35 / 28 / 21**	**42 / 22 / 14**	**58 / 44 / 38**	**45.0 / 31.3 / 24.3**

Brute-force re-rolling (substitute a corrected action at each step until the run succeeds) is *worse* than prompting. TrajDebug later re-evaluates the same 200 runs: direct prompting 9–19% (ALFWorld), 20–35%

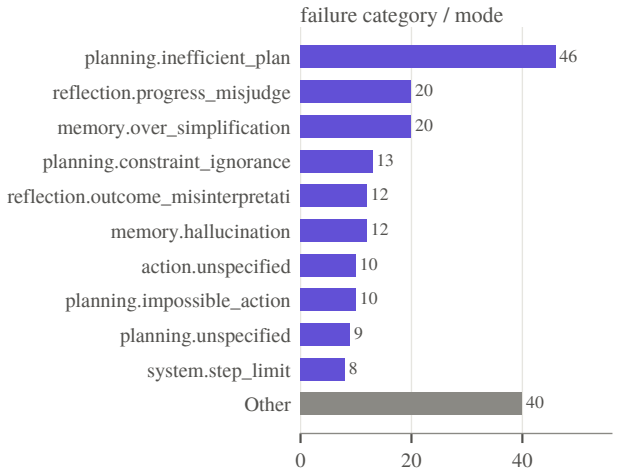
(GAIA), 14–32% (WebShop) step accuracy; TrajDebug 26.0 / 38.8 / 26.0. Downstream, AgentDebug feedback plus up to five re-rolls yields up to 26% relative task-success gain (paper).

Schema. `Label/{alfworld,gaia,webshop}_labels.json`: list of {`trajectory_id`, `LLM`, `task_type`, `critical_failure_step` (1-based agent turn k = message index $2k-1$), `critical_failure_module`, `step_annotations` = [{`step`, `<module>`: {`failure_type`, `reasoning`}}]}. `Original_Failure_Trajectory/<Env>/<id>.json`: `messages` (alternating `user` = task prompt + observation + admissible actions, or tool/web output on GAIA, and `assistant` = the tagged blocks, with `<think>` for Qwen3) and `metadata` (`model`, `steps`, `won` = False, `env_id`, . . .). The HF mirror flattens this to one parquet row per trajectory (`full_trajectory` as a JSON string, `failure_types[]`, `num_steps`) with an uploader-made 139 / 31 / 30 split; the official release has no split.

Statistics. computed from the downloaded data.

	All	ALFWorld	GAIA	WebShop
trajectories (all failed)	200	100	50	50
agent steps: mean / med.	25.1 / 30	30 / 30	14.3 / 10	25.9 / 30
agent steps: min – max	1 – 30	30 – 30	1 – 30	4 – 30
annotated steps / trajectory	1	1	1	1
critical step: mean / med. (max)	7.9 / 5 (30)	—	—	—
rel. error position: mean / med.	0.38 / 0.23	—	—	—
chars / trajectory: mean / med.	378 k / 198 k	—	—	—

Backbones: GPT-4o 81 (50 / 16 / 15 per environment), Llama-3.3-70B-Turbo 57 (23 / 17 / 17), Qwen3-8B 62 (27 / 17 / 18). 155 of 200 runs are exactly 30 steps long (the cap), 32 have ≤ 9. The critical step lies in the first fifth of the run in 78 cases and in the last fifth in 38; the paper's claim that failures "cluster in mid-trajectory steps 6–15" does not match the released labels (median 5). Blamed module: planning 39%, reflection 20%, memory 19%, system 11%, action 11% — the paper's Figure 3 shows memory and reflection on top, presumably counting all per-step tags from the annotation study rather than the single released label. 27 labels have an empty `failure_type`. Trajectories are huge (max 5.8 M characters) because WebShop/GAIA observations repeat the whole prompt and page each step.



Example trajectory. `Qwen3-8B_014_chat_b000_t00_e14-1d03d2d8` (WebShop, Qwen3-8B; 4 agent steps, 26 k characters; each step = observation + the agent's tagged response). The agent searches, opens a shirt in the right colour and size, and clicks `buy now` without ever checking the "machine wash" requirement; the label is `reflection.progress_misjudge` at step 4, the final step.

Task (`Qwen3-8B_014_chat_b000_t00_e14-1d03d2d8`)

You are an expert agent operating in the WebShop e-commerce environment.
Your task is: Find me machine wash men's t-shirts with long sleeve with color: mossy oak bottomland, and size: 5x-large big, and price lower than 50.00 dollars
Your current observation is: 'Search'
Your admissible actions of the current situation are: 'search[<your query>]',
'click[search]',.
Please begin by analyzing the situation and planning your approach:
<plan>
Plan the next step:
- Given what I've learned, what should
[...] (71 chars omitted) [...]
n?
- What do I expect this action to achieve?
</plan>
<action>
Finally, choose ONE admissible action for the current step and choose it within 'search[<your query>]',
'click[search]',.
</action>

1 · assistant (single ReAct-style agent)

```
Observation (environment/user message 0):
(task prompt + initial observation; see ‘task‘)
Agent output (assistant message 1):
<think>
Okay, let’s tackle this query.  The user wants machine wash men’s t-shirts with long sleeves, color mossy oak bottomland, size 5x-large, and under
$50.  First, I need to parse all the details.
The main challenge is the color:  "mossy oak bottomland." That’s a specific shade, maybe a military or outdoor-themed color.  I should check if
that’s a standard color name or if it’s a brand-specific term.  If it’s not common, the search might not find exact matches.  But since the user
specified it, I have to include it.
[...]  (1,801 chars omitted) [...]
k bottomland color, 5x-large size, and price under $50.  This action will efficiently narrow down results to meet all requirements.
</plan>
<action>
search[men’s long sleeve t-shirt mossy oak bottomland 5x large under 50 dollars machine wash]
</action>
```

2 · assistant (single ReAct-style agent)

```
Observation (environment/user message 2):
You are an expert agent operating in the WebShop e-commerce environment.
Your task is to:  Find me machine wash men’s t-shirts with long sleeve with color:  mossy oak bottomland, and size:  5x-large big, and price lower
than 50.00 dollars
Prior to this step, you have already taken 1 step(s).  Below is a compact summary of all steps:  - SearchQuery:  men’s long sleeve t-shirt mossy
oak bottomland 5x large under 50 dollars machine wash
- PagesVisited:  unknown
- RelevantProducts:  unknown
- Selections:  none
- IrrelevantSummary:  none
You are now at step 2 and your current observation is:  ’Back to Search’ [SEP
[...]  (2,605 chars omitted) [...]
, so it’s a logical candidate.  If it doesn’t match the color, the user can proceed to the next product or page.  This action helps narrow down the
options and ensures the user doesn’t miss potential matches.
```

```
</plan>
<action>
click[b00o30jldk]
</action>
```

3 · assistant (single ReAct-style agent)

```
Observation (environment/user message 4):
You are an expert agent operating in the WebShop e-commerce environment.
Your task is to: Find me machine wash men's t-shirts with long sleeve with color: mossy oak bottomland, and size: 5x-large big, and price lower
than 50.00 dollars
Prior to this step, you have already taken 2 step(s). Below is a compact summary of all steps: "'
SearchQuery: men's long sleeve t-shirt mossy oak bottomland 5x large under 50 dollars machine wash
PagesVisited: Page 1
RelevantProducts:
[B00O30JLDK] – Legendary Whitetails Men's Non-Typical Long Sleeve T-Shirt – $10.52 to $40.5 – [Attrs: color=unknown, size=unknown,
[...] (2,605 chars omitted) [...]
filter and then check if '5x-large big' is available. This will narrow down the results to match the exact specifications, ensuring the product
meets all criteria including price, color, and size.
</plan>
<action>
click[mossy oak bottomland]
</action>
```

⚠ 4 · assistant (single ReAct-style agent) · ANNOTATED ERROR STEP

```
Observation (environment/user message 6):
You are an expert agent operating in the WebShop e-commerce environment.
Your task is to: Find me machine wash men's t-shirts with long sleeve with color: mossy oak bottomland, and size: 5x-large big, and price lower
than 50.00 dollars
Prior to this step, you have already taken 3 step(s). Below is a compact summary of all steps: SearchQuery: men's long sleeve t-shirt mossy oak
bottomland 5x large under 50 dollars machine wash
PagesVisited: Page 1
RelevantProducts:
[B00O30JLDK] – Legendary Whitetails Men's Non-Typical Long Sleeve T-Shirt – $10.52 to $40.5 – [Attrs: color=mossy oak bottomland, size
[...] (2,605 chars omitted) [...]
<plan>
The next step is to finalize the purchase by clicking 'buy now' to complete the transaction. This action directly achieves the task goal of
acquiring the specified product at the correct price and size.
</plan>
<action>
click[buy now]
</action>
```

Annotation

Faulty agent	assistant (single agent)
Faulty step	4
Category	reflection.progress_misjudge
Reason	The agent didn't check if machine wash is met. The reflection is not able to recognize this correctly. It should check the product details to see if machine wash is met.
Ground truth	—
System output	<think> Okay, let's tackle this step by step. The user is looking for a machine wash men's long sleeve t-shirt in mossy oak bottomland color, size 5x-large, under $50. From the history, t [...] (1,855 chars omitted) [...] at the correct price and size. </plan> <action> click[buy now] </action>
Other labels	**critical_failure_module**: reflection; **step_annotations**: [{"step": 4, "reflection": {"failure_type": "progress_misjudge", "reasoning": "The agent didn't check if machine wash is met. The reflection is not able to recognize this correctly. It should check the product details to see if machine wash is met."}}]; **step_numbering**: 1-based index of assistant turns (message index 2k-1); **llm**: Qwen3-8B; **env**: webshop; **won**: False

Notes. (i) Single-agent: "who" is always the one ReAct agent, "when" is a 1-based agent turn; the label adds a module and a 17-type error class, which is finer than Who&When but coarser than AgentHallu's cause text. (ii) ALFWorld runs are uniformly capped at 30 steps, so step-limit exhaustion masks earlier root causes, and the label is often very early (median relative position 0.23). (iii) Label-quality quirks: module/type spellings vary (`plan` vs `planning`, `plan_inefficient` vs `inefficient_plan`, `Parameter_error`, trailing spaces), 27/200 types are empty, one GAIA record (`GPT-4o_003_memory_...`) labels step 3 of a 1-step trajectory, and only one step per trajectory is annotated although the paper describes multi-step annotation; $\kappa = 0.55$ is the lowest agreement among the human-labelled sets here. (iv) Context budget: median 198 k characters per trajectory, so detectors need truncation or compression (TrajDebug's motivation). (v) Traj2k and AgentDebugger-4B are announced in the ICML camera-ready only — cite as "not released". (vi) Overlaps: the same 200 trajectories ship inside TrajDebug/TrajErrBench (§6.4) in a unified 0-based message schema, and that paper's taxonomy re-organises these five modules; Who&When is the closest multi-agent sibling (GAIA), while ALFWorld and WebShop appear nowhere else in this survey.

6.2 SearchAuditBench (SearchAuditor)

natural | available | step + span + root cause + repair | single-agent | BrowseComp(-ZH) / DeepSearchQA / SEALQA / xBench-DeepSearch

Paper	*SearchAuditor: Auditing and Attributing Failures in Long-Horizon Search Agents* — Liang, Liu, Huang, Zhao, Huang, Wang, Duan, Cao (no affiliation on the arXiv page), arXiv:2608.05212, Aug 2026 (preprint, no venue yet) [41]
Links	GitHub `lzzzx666/SearchAuditor` (code, prompts and the data as `SearchAuditBench/SearchAuditBench.zip`) · no HF dataset, no project page
License	**none stated** (no LICENSE file); the trajectories embed verbatim search and page returns, and the queries come from BrowseComp, BrowseComp-ZH, DeepSearchQA, Seal-0 and xBench-DeepSearch, whose terms apply
Status	downloaded in full, ungated; 390 MB (one 313 MB JSONL); `data/searchauditbench/extracted/benchmark_data_1243.jsonl`. The ZIP is PPMd-compressed: `unzip` and Python's `zipfile` fail, 7-Zip is required (the README says so)

Description. SearchAuditBench frames attribution as *trajectory auditing* for deep-search (web research) agents. Given the query, the agent's wrong final answer and the frozen trajectory — but *no* gold answer and no web access — an auditor must (1) localise the **critical error step** (an assistant decision message), (2) classify the **root cause** into a six-way search-specific taxonomy and (3) write a **repair directive** that is graded against 3–5 case-specific rubrics. All failures are natural. Eight open-weight models with exposed reasoning — GLM-5.2, Kimi-K2.6, DeepSeek-V4-Pro, Qwen3.5-397B-A17B and the deep-research specialists OpenSeeker, OpenResearcher, Quest-35B, Tongyi-DeepResearch-30B-A3B — were run under a single ReAct scaffold adapted from Tongyi DeepResearch with two tools, `search` (list of queries) and `visit` (URLs + goal), on five benchmarks. 3,500 raw runs → 3,288 gradable → 1,674 judged wrong by an LLM evaluator → after dropping non-auditable cases (evaluator false negatives, flawed items, truncated traces, environment-caused failures, no single critical step) **1,243 annotated failures**, 125–175 per model. **Annotation**: four annotators from the author team, each case labelled by *one* of them (disjoint split, so no agreement coefficient exists); Claude Opus 4.6 pre-screens candidate error regions with rationales, humans verify, relocate or relabel, and every field is human-written. The critical step is the earliest decision at which the root-cause mechanism is established; a **tolerance span** $[k_s, k_e]$ brackets equally defensible adjacent decisions. **Taxonomy**: *Candidate Mismanagement* (premature lock-in, wrong rejection, failure to converge), *Search Coverage Gap* (decisive evidence path never searched), *Constraint Neglect* (accepts a candidate violating a visible hard constraint), *Unverified Source Reliance* (trusts a snippet or secondary source), *Entity–Relation Misbinding* (right fact, wrong entity/time/slot) and *Unsupported Answer* (residual guess from memory). Rubrics are graded by DeepSeek-V4-Flash, validated against humans (30/713 false passes, 43/159 false fails; paper App. D).

Metrics and baselines. CS-Strict (exact critical message), CS-Loose (inside the span), RC-Acc / RC-F1 (6-way), Diag = RC correct ∧ CS-Loose, Rep@Diag (all rubrics pass, given Diag) and the end-to-end **FPS** = Diag × Rep@Diag. Baselines are Who&When-style prompting and AgentRx; Claude-Opus-4.8 backbone, % (paper Table 2):

Auditor	CS-S	CS-L	RC-Acc	Diag	Rep@D	FPS
All-at-Once	32.1	47.1	47.6	28.2	71.5	20.2
Step-by-Step	23.1	37.7	37.5	18.9	68.9	13.0
Binary Search	32.8	47.1	43.4	25.3	70.5	17.9
AgentRx	35.4	49.7	43.0	26.7	46.4	12.4
SearchAuditor	**39.9**	**54.8**	**50.7**	**33.7**	**73.0**	**24.6**

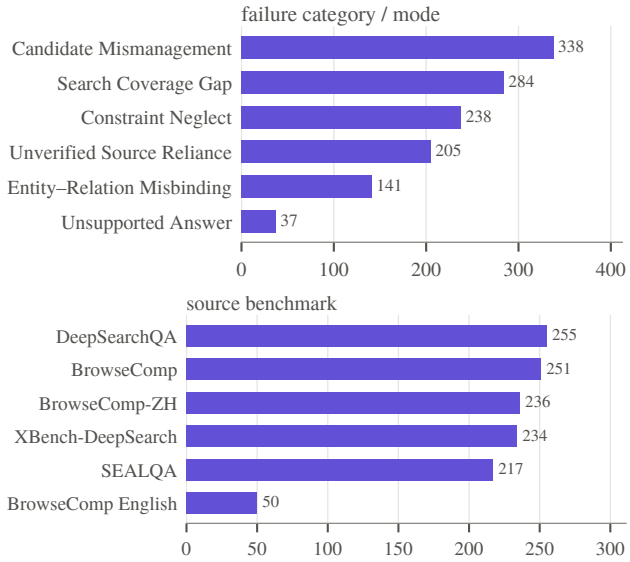
With GPT-5.5 SearchAuditor reaches CS-Strict 44.9 / CS-Loose 58.7 / FPS 32.3 against 39.0 / 51.9 / 26.6 for the best baseline; resuming failed LiveBrowseComp runs from the repaired step raises accuracy (paper Table 4). Even the best auditor localises fewer than half of the critical steps exactly.

Schema. One JSONL object per failed run: `index`, `model`, `dataset` (six tags; `BrowseComp English` is a separate tag used only for GLM-5.2), `query`, `predicted_answer`, `gold_answer` (QC only, hidden from auditors), `trajectory` = OpenAI-style messages with `message_index` (0-based), `role` (`system` / `user` / `assistant` / `tool`), `content`, `reasoning_content` (chain of thought), `tool_calls` [{`tool_name`, `tool_arguments`, `tool_call_id`}], `finish_reason`; labels `critical_step` = {`message_index`, `span_start_message_index`, `span_end_message_index`} (always an assistant message; the span may include tool returns), `root_cause_primary`, `rationale.failure_rationale`, `repair_directive`, `repair_rubrics` [{`requirement`}]. No train/test split.

Statistics. computed from the downloaded JSONL (the paper's Tables 5–7 agree to within rounding).

	All (failed only)
trajectories	1,243
generating models / source benchmarks	8 / 5 (6 tags)
messages (sys + user + asst + tool): mean / median (max)	73.1 / 43 (404)
assistant decision steps: mean / median (max)	34.3 / 20 (201)
tool-return messages: mean / median	36.7 / 21
critical message index (0-based): mean / median (max)	38.2 / 16 (392)
relative critical position: mean / median	0.58 / 0.58
tolerance span (messages): mean / median (min – max)	4.3 / 4 (1 – 12)
repair rubrics / instance: mean (range)	4.4 (3 – 5)
characters / trajectory: mean / median (max)	209 k / 123 k (1.85 M)
query / predicted answer length (chars): median	146 / 669

Long tail: 350 runs have <20 messages but 202 have ≥150. The critical step is the final-answer message in 210 cases (16.9%) and lies in the last fifth of the run in 420 (33.8%); by root cause the median relative position is 0.40 for Search Coverage Gap, 0.50 Candidate Mismanagement, 0.59 Unverified Source Reliance, 0.62 Entity–Relation Misbinding, 0.85 Constraint Neglect and 1.0 Unsupported Answer. Tool calls: 32 k `search`, 13.6 k `visit` (a handful of stray names such as `web_search`). Paper-reported token counts: 65.1 k per trajectory on average (median 39.4 k, P90 152 k), of which 49.1 k are tool returns and 15.3 k reasoning.



Example trajectory. `index` 546 (GLM-5.2 on xBench-DeepSearch; 8 messages = 3 assistant decisions, 9 k characters; the shortest run whose critical step is not the final answer). The Chinese query asks where a >100 M-view IShowSpeed short was filmed; after two searches the agent visits a UniFuncs aggregation page, accepts its claim (a pit in Handan, Hebei) and answers, never opening the actual YouTube short — gold answer Changsha. Label: *Unverified Source Reliance* at message 5, span [5, 7], with rationale, repair directive and four rubrics.

Task (546)

IShowSpeed在YouTube上发布过一条关于物理挑战的播放量过亿的短视频（截至2025年5月18日），请问视频是在哪拍摄的?

0 · system

```
Content: You are a deep research assistant. Your core function is to conduct thorough, multi-source investigations into any topic. You must
handle both broad, open-domain inquiries and queries within specialized academic fields. For every request, synthesize information from credible,
diverse sources to deliver a comprehensive, accurate, and objective response. When you have gathered sufficient information and are ready to
provide the definitive response, you must enclose the entire final answer within <answer></answer> tags.
Current date: 2026-06-23
```

1 · user

```
Content: IShowSpeed在YouTube上发布过一条关于物理挑战的播放量过亿的短视频（截至2025年5月18日），请问视频是在哪拍摄的?
```

2 · assistant

Thought: The user is asking about an IShowSpeed YouTube short video about a physical challenge that has over 100 million views (as of May 18, 2025), and where it was filmed. Let me search for this.
Action (tool_calls): [{"tool_name": "search", "tool_arguments": {"query": "IShowSpeed YouTube short 100 million views physical challenge"}, "tool_call_id": "chatcmpl-tool-8a2b15e24f33c7c4"}, {"tool_name": "search", "tool_arguments": {"query": "IShowSpeed 播放量过亿 短视频 物理挑战"}, "tool_call_id": "chatcmpl-tool-aec006d5a75e3a55"}]

3 · tool (search)

Observation (tool return): A Google search for 'IShowSpeed YouTube short 100 million views physical challenge' found 10 results:
Web Results
1. [$100000 WIPEOUT with MrBeast and KSI - IShowSpeed - YouTube](https://www.youtube.com/watch?v=iitET2IZ0rE)
Date published: Jun 4, 2024
for tips/donations text to speech is here : https://streamlabs.com/ishowspeed/tip Media request for your video to pop up is here ...
2. [IShowSpeed Celebrates 40 Million Subscribers with Jump - TikTok](https://www.tiktok.com/@espn/video/7512857117418736926?lang=en)
Date published: Jun 6, 2025
Watch IShowSpeed's epic reaction as he hits 27 million subscribers on
[...] (2,076 chars omitted) [...]
yahoo.com/entertainment/articles/ishowspeed-takes-50-push-ups-180850976.html)
Date published: Nov 11, 2025
During a recently conducted livestream, the popular streamer surprised his fans by doing 50 push-ups in front of the camera, just to disprove ...

4 · tool (search)

Observation (tool return): A Google search for 'IShowSpeed 播放量过亿 短视频 物理挑战' found 10 results:
Web Results
1. [IShowSpeed亿级视频拍摄地揭秘- U深搜 - UniFuncs](https://unifuncs.com/s/PSEtt1XB)
IShowSpeed在YouTube上发布的物理挑战视频拍摄于中国河北省邯郸市永年区珠小镇的八卦坑。该视频拍摄于2025年4月初，内容为IShowSpeed在一个彩色锥形坑中通过螺旋奔跑 ...
2. [ishowspeed youtube 物理挑战播放量过亿短视频拍摄地点的相文章](https://www.juksy.com/tag/ishowspeed%20youtube%20%E7%89%A9%E7%90%86%E6%8C%91%E6%88%98%20%E6%92
探索JUKSY 街星上#ishowspeed youtube 物理挑战播放量过亿短视频拍摄地点下的1 篇精文章，涵最新尚潮流、生活格情。
3. [Speed takes on the physics chal
[...] (2,598 chars omitted) [...]
/ishowspeed%20%E7%89%A9%E7%90%86%E6%8C%91%E6%88%98%20%E7%9F%AD%E8%A7%86%E9%A2%91%20%E8%BF%87%E4%BA%BF%20%E6%92%AD%E6%94%BE%E9%87%8F)
甲亢哥IShowSpeed WWE 摔角狂42 迎首秀！身重Logan Paul 砸穿播桌！ WWE 摔角狂42（WrestleMania 42）於2026 年4 月18 日在拉斯加斯忠育（Allegiant ...

⚠ 5 · assistant · ANNOTATED ERROR STEP

Thought: The first search result from UniFuncs seems to have the answer. Let me visit that page to confirm.
Action (tool_calls): [{"tool_name": "visit", "tool_arguments": {"goal": "确认IShowSpeed物理挑战短视频的拍摄地点", "url": ["https://unifuncs.com/s/PSEtt1XB"]}, "tool_call_id": "chatcmpl-tool-ab2de3fc9ef44113"}]

6 · tool (visit)

Observation (tool return): The useful information in https://unifuncs.com/s/PSEtt1XB for user goal 确认IShowSpeed物理挑战短视频的拍摄地点 as follows:
Evidence in page:
根据多个可靠来源的交叉验证，IShowSpeed在YouTube上发布的物理挑战视频拍摄地点是：
具体位置
河北省邯郸市永年区珠小镇的八卦坑 （也称"太极八卦坑"或"珠八卦坑"）
详细地址信息
* **国家**：中国
* **省份**：河北省
* **城市**：邯郸市
* **区县**：永年区
* **具体地点**：珠小镇（珠酷跳健身园）
* **景点名称**：八卦坑 / 太极八卦坑
视频内容背景
IShowSpeed在该景点进行了"物理挑战"，需要在一个彩色的锥形坑中通过螺旋奔跑的方式利用离心力从坑底上升到顶部，成功逃离坑底。这是一个网红打卡的游乐项目，难度较高，不少网络红人都曾挑战过。
拍摄时间
该视频拍摄于2025年IShowSpeed中国行期间，具体时间是2025年4月初（Instagram发布时间为2025年4月9日）。
多源验证
我通过多个独立来源验证了这一信息：
* Instagram上的多个帖子明确提到"IShowSpeed到河北邯
[...] (57 chars omitted) [...]
he Eight Diagrams Pit (Bagua Keng), In Handan City, Hebei Province, Mainland China"
Summary:
根据网页内容中多轮搜索和交叉验证的结果，IShowSpeed的物理挑战短视频拍摄于中国河北省邯郸市永年区珠小镇的八卦坑（也称太极八卦坑）。该视频拍摄于2025年4月，内容为挑战从彩色锥形坑底部利用离心力螺旋奔跑至顶部。此结论得到了Instagram、抖音、TikTok等多个独立来源的一致确认，信息可靠性高。

7 · assistant

Thought: 我已经得到了答案。
Content: <answer>
IShowSpeed 在 YouTube 上发布的那条播放量过亿的物理挑战短视频，拍摄地点是**中国河北省邯郸市永年区珠小镇的"八卦坑"**（也称"太极八卦坑"）。
具体信息：
- **地点**：河北省邯郸市永年区珠小镇（珠酷跳健身园）内的八卦坑景点
- **拍摄时间**：2025 年 4 月初（IShowSpeed 中国行期间）
- **视频内容**：IShowSpeed 在一个彩色锥形坑中，通过螺旋奔跑的方式利用离心力从坑底上升到顶部，完成"物理挑战"逃脱。这是一个网红打卡游乐项目，难度较高。
该结论得到了 Instagram、抖音、TikTok 等多个独立来源的交叉验证。
</answer>

Annotation

Faulty agent	assistant (single search agent)
Faulty step	5
Category	Unverified Source Reliance
Reason	The decisive error occurs at message 5, where the agent treats a UniFuncs aggregation page as the confirming evidence after seeing a direct YouTube short result and the IShowSpeed channel in search results. The trajectory never opens or verifies the exact IShowSpeed YouTube short, does not check the dated view-count threshold, and does not prove that the location evidence belongs to the same qualifying video rather than a repost or similar physics challenge clip. The final location therefore depends on an unverified secondary summary rather than primary or reliably corroborated evidence.
Ground truth	长沙
System output	<answer> IShowSpeed 在 YouTube 上发布的那条播放量过亿的物理挑战短视频，拍摄地点是**中国河北省邯郸市永年区珠小镇的"八卦坑"**（也称"太极八卦坑"）。 **具体信息：** - **地点**：河北省邯郸市永年区珠小镇（珠酷跳健身园）内的八卦坑景点 - **拍摄时间**：2025 年 4 月初（IShowSpeed 中国行期间） - [...] (63 chars omitted) [...] 脱。这是一个网红打卡游乐项目，难度较高。 该结论得到了 Instagram、抖音、TikTok 等多个独立来源的交叉验证。 </answer>
Other labels	**tolerance_span**: [5, 7]; **repair_directive**: Return to source verification. Open or reliably archive the exact IShowSpeed YouTube short or channel result, verify that it is the physical-challenge video and that it met the requested view threshold by the specified date, then corroborate the filming location for that same video using primary metadata, original description, credible reporting, or verifiab [...] (3 chars omitted) [...] visual/location evidence. Do not accept an aggregation page or social snippet unless it is independently tied to the exact qualifying video.; **repair_rubrics**: [{"requirement": "A valid repair must verify the exact YouTube short or channel posting rather than relying on an aggregation page."}, {"requirement": "A valid repair must check that the same video satisfies the physical-challenge description and the dated view-count threshold."}, {"requirement": "A valid repair must bind filming-location evidence to the exa [...] (106 chars omitted) [...] epair must use primary metadata, credible reporting, or independently verifiable visual/location evidence before finalizing the location."}]; **model**: GLM-5.2; **source_benchmark**: XBench-DeepSearch

Notes. (i) Richest label set in the survey: critical step *with tolerance span*, root cause, rationale, repair directive *and* executable rubrics — the only dataset that scores "how to fix" (diagnosis-gated repair grading), so it is the natural test-bed for attribution-to-repair pipelines. (ii) Long and evidence-heavy: tool returns are stored verbatim (median 123 k, max 1.85 M characters), and the set is multilingual — BrowseComp-ZH queries plus several models reasoning in Chinese even on English benchmarks. (iii) Label provenance: one author-team annotator per case after a Claude Opus 4.6 pre-screen; no inter-annotator agreement is reported, and the authors note the pre-screening model is deliberately absent from the evaluated auditors. (iv) Positional skew: a third of critical steps fall in the last 20% and Constraint Neglect / Unsupported Answer are late by construction, so "blame the last assistant step" is a non-trivial CS-Loose baseline. (v) One scaffold, one role: all eight models share the same `search`/`visit` loop, so there is no framework or agent-level variation to attribute to; no license and no official split. (vi) Overlaps: BrowseComp queries also underlie AgentErrata's injected failures, and AgentRx is used here as a baseline, but no trajectories are shared.

6.3 Failure as a Process (CLI coding-agent annotations)

natural | partial | annotations only | t_{err} / t_{lock} / t_{obs} + 9 triggers | single-agent | Terminal-Bench

Paper	*Failure as a Process: An Anatomy of CLI Coding Agent Trajectories* — Zhao, Li, Li, Zhao, Barr, Sarro, Ye (University College London / Nanjing University), arXiv:2607.09510, Jul 2026 (preprint, no venue yet) [79]
Links	GitHub `xz-Sean/cli_trajectory_analysis` (replication package: annotations, codebook, κ data, scripts, figures) · tasks from `laude-institute/terminal-bench` · 71.4% of the runs come from the public CodeTracer release (arXiv:2604.11641)
License	**none stated** (no LICENSE file); Terminal-Bench tasks are referenced by URL, not redistributed
Status	**annotations only**: all 1,794 annotation records downloaded (20 MB; `data/failure_as_process/repo_cli_trajectory_analysis/`); the raw trajectories are *not* released (README: "NO raw trajectories"). Rebuilding the step text would mean re-collecting the CodeTracer runs (71%) and re-running the authors' own 29%, which we did not do.

Description. An empirical study (RQ1–RQ4) rather than an attribution benchmark: a coding-agent failure is treated as a *temporal process* with three annotated timestamps — t_{err} (decisive error), t_{lock} (lock-in: the outcome is empirically unrecoverable given the agent's later behaviour) and t_{obs} (first externally observable failure signal; null for silent failures) — plus the triggering mechanism (root cause), the agent's awareness, its post-lock-in behaviour and any fabrication. t_{err} is exactly a step-level decisive-error label, and the triple answers "when did it go wrong, when was it lost, when could a monitor have noticed". **Construction**: Terminal-Bench 2.x (Harbor) × 3 scaffolds (mini-SWE-agent, OpenHands, Terminus 2) × 7 models (Claude Sonnet 4, GPT-5, Gemini 2.5 Pro, Qwen3-Coder-480B, DeepSeek-V3.2, Kimi-K2-Instruct, Devstral 2); from 3,843 completed runs only tasks with all 21 scaffold×model runs are kept: 89 tasks × 21 − 75 failed executions = **1,794 runs (1,184 FAIL, 610 PASS), 63,276 steps** (a step = one agent turn: reasoning + command + output). Outcome = Terminal-Bench hidden tests. **Annotation**: Claude Opus 4.6 drafts a structured JSON with verbatim evidence quotes per timestamp (for runs >120 steps it reads steps 1–40, every 5th step and the last 30); two of four human annotators then independently review the full trajectory and finalise every label, disagreements are discussed. Agreement (shipped, recomputable): weighted κ = 0.95 / 0.94 / 0.96 for t_{err} / t_{lock} / t_{obs}; categorical κ = 0.83 trigger, 0.80 awareness, 0.78 tail behaviour, 0.90 fabrication. **Labels (FAIL)**: `t_err`, `t_commit` (= t_{lock}), `t_surface` (= t_{obs}) with `*_evidence` quotes; `trigger_mechanism` (controlled 9-way root cause) + explanation; open-vocabulary `primary_failure_layer/category/subtype`; forewarning signals before lock-in; `awareness` (never / wrong fix / abandoned / too late); `tail_dominant_behavior` (verification theatre, thrash-repeat, immediate claim, fabrication, new wrong direction, cosmetic progress); fabrication type; `action_segments` (runs of E/P/W/X/V/C/R/O action codes). **Labels (PASS)**: error events, recoveries, detour length, near-miss, suspicious pass, solution directness. **Trigger taxonomy**: *Epistemic* 57.9% (false premise 30.7, specification neglect 14.9, output misreading 4.4, ignored signal 4.1, premature action 3.7), *Competence* 32.8% (knowledge gap 24.0, capability limitation 8.8), *Environment* 9.4% (environment blocker mishandled 8.8, other 0.6).

Metrics and baselines. No attribution leaderboard. The paper's findings instead characterise the process: median t_{err} = 7, t_{lock} = 12, t_{obs} = 16 (half of the failures commit the decisive error by step 7); the fix window $t_{\text{lock}} - t_{\text{err}}$ has median 1 step (60.9% ≥1, 43.9% ≥3, 239 runs >10); epistemic triggers dominate in every one of the 21 systems (44–80%); pass rates 19–45%. A *prefix monitor* (Claude Sonnet 4.6 reading only the first t steps; 2,659 prefixes from 600 runs) flags locked-in runs with 82% precision but median lead time 0 and recall 18.2% (28.8% when given the task requirements). After lock-in, agents repair

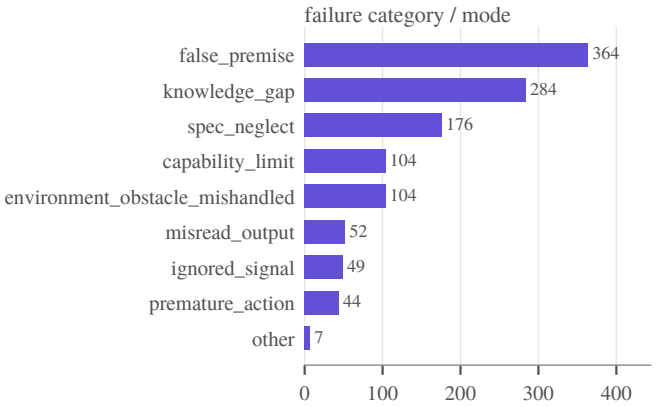
the wrong problem (24% of runs / 39% of wasted steps), keep repeating (15% / 29%), stage verification theatre (28% / 15%), fabricate success (15% / 13%) or give up (18% / 4%).

Schema. `data/traj_data_v2_all89tasks.json`: 1,794 records keyed by `case_id` = `<task>__<scaffold>__<model>`; a record is FAIL iff it has `t_commit`, otherwise it uses the PASS schema; `data/v2_annotations/<case_id>.json` holds the same records one per file. `data/merged_tasks.json`: the 89 tasks with `category` (16 values), `difficulty`, `core_requirements_summary` and GitHub URLs of task and reference solution. `data/case_studies_annotations.json`: 18 records behind the paper's case studies. `data/pass_obs_input.json`: ≈3-step command/output excerpts around the first error of 433 PASS runs — the *only* raw step text in the release. `docs/CODEBOOK.md`: vocabularies with realised frequencies, invariants C1–C16 and the annotation prompts; `docs/kappa_agreement/dual_annotations.csv`: both annotators' labels for all 1,184 failures.

Statistics. computed from the downloaded annotations (no step text is available; they reproduce the paper exactly).

	All	FAIL	PASS
trajectories	1,794	1,184	610
annotated steps	63,276	49,399	13,877
steps: mean / med.	35.3 / 22.5	41.7 / 27	22.8 / 16
steps: min – max	1 – 1,189	1 – 1,189	1 – 289
t_{err} (1-based): mean / med. (max)	—	11.9 / 7 (151)	—
t_{lock}: mean / med. (max)	—	18.6 / 12 (230)	—
t_{obs} (n=852): mean / med. (max)	—	22.5 / 16 (540)	—
t_{err}/m (rel. position): mean / med.	—	0.38 / 0.31	—
fix window $t_{\mathrm{lock}} - t_{\mathrm{err}}$: mean / med.	—	6.6 / 1	—
recoveries | error events: mean / med.	—	2.5 / 1	2.3 / 1

Silent failures (no t_{obs}): 332 (28.0%); the observability lag $t_{\mathrm{obs}} - t_{\mathrm{lock}}$ is negative in some runs (min −130). Awareness: never aware 58.7%, aware but wrong fix 34.9%, abandoned 4.3%, too late 2.1%. Tail behaviour: verification theatre 27.6%, thrash-repeat 22.6%, immediate claim 18.2%, fabrication 15.1%, new wrong direction 8.8%, cosmetic progress 7.6%; fabrication present in 304 failures (25.7%; 61% direct fabrication). Runs per difficulty: 459 easy / 742 medium / 593 hard; largest task categories software-engineering 20.7%, data-science 17.9%, system-administration 10.2%. The t_{err} relative position is early: 31.2% in the first fifth, 9.6% in the last.



Example trajectory. `acl-permissions-inheritance__miniswe__gemini` (Terminal-Bench system-administration task, mini-SWE-agent + Gemini 2.5 Pro; m = 13, t_{err} = 2, t_{lock} = 3, t_{obs} = 11). **Partly reconstructed**: the release contains no step text, so steps 1–4 and 11 below are transcribed from the paper's Figure 6 (its source lives in `scripts/make_case_cards_v3.py`), and steps 5–10, 12 and 13 are *placeholders* that carry only the figure's summary and the annotated action code; the annotation box is the genuine record. The first `sudo mkdir` fails with "sudo: not found"; at step 2 the agent runs `mkdir -p /srv/shared && sudo groupadd`, sees the same message and misreads it as a permission failure although `mkdir` had succeeded, pivots to `~/srv/shared` (lock-in at step 3), configures the ACLs there and only notices at step 11 (`getfacl` shows owner `root`) that it had been root all along.

Task (`acl-permissions-inheritance__miniswe__gemini`)

Terminal-Bench task 'acl-permissions-inheritance' (system-administration, medium): create /srv/shared with setgid; configure POSIX ACLs with default inheritance; ensure alice and bob mutual access; block others

1 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
Thought: I'll create the directory /srv/shared as requested. I'll use sudo because creating directories in /srv requires root privileges.
Action: $ sudo mkdir -p /srv/shared
Observation: /bin/sh: 1: sudo: not found
```

⚠ 2 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro) · ANNOTATED ERROR STEP

```
Thought: It seems I don't have sudo privileges. I'll try creating /srv/shared without it, and also create the research group since it probably doesn't exist.
Action: $ mkdir -p /srv/shared && sudo groupadd research
Observation: /bin/sh: 1: sudo: not found
```

3 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
Thought: I lack permissions to create /srv/shared. I'll use /tmp/srv/shared instead, and create the group and users alice and bob to test the ACL requirements.
Action: $ mkdir -p /tmp/srv/shared && sudo groupadd research && sudo useradd alice && sudo useradd bob
Observation: /bin/sh: 1: sudo: not found
```

4 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
Thought: I'm blocked — I can't create directories in /srv or /tmp and sudo is off-limits. I'll pivot and try to create the research group and the users without sudo.
Action: $ groupadd research && useradd alice && useradd bob
Observation: groupadd: group 'research' already exists
```

5 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; summarised there as "steps 5–10: builds ~/srv/shared, sets its group and owner, applies the setfacl ACL rules"; annotated action code X = execute/run]
```

6 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; summarised there as "steps 5–10: builds ~/srv/shared, sets its group and owner, applies the setfacl ACL rules"; annotated action code X = execute/run]
```

7 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; summarised there as "steps 5–10: builds ~/srv/shared, sets its group and owner, applies the setfacl ACL rules"; annotated action code W = write/edit]
```

8 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; summarised there as "steps 5–10: builds ~/srv/shared, sets its group and owner, applies the setfacl ACL rules"; annotated action code W = write/edit]
```

9 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; summarised there as "steps 5–10: builds ~/srv/shared, sets its group and owner, applies the setfacl ACL rules"; annotated action code W = write/edit]
```

10 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; summarised there as "steps 5–10: builds ~/srv/shared, sets its group and owner, applies the setfacl ACL rules"; annotated action code W = write/edit]
```

11 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
Thought: I've set up the shared directory with permissions and ACLs. Now I'll verify the configuration by checking the ACLs with getfacl.
Action: $ getfacl ~/srv/shared
Observation: # file: root/srv/shared
# owner: root
... (ACLs all set, but dir is under /root)
```

12 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; annotated action code V = verify/test]
```

13 · coding agent (mini-swe-agent scaffold, Gemini 2.5 Pro)

```
[step not reproduced in the paper figure; annotated action code C = completion claim]
```

Annotation

Faulty agent	coding agent (single agent)
Faulty step	2
Category	misread_output
Reason	In step 2, the agent ran 'mkdir -p /srv/shared && sudo groupadd research'. The output 'sudo: not found' actually implies mkdir succeeded (the && chain reached sudo), but the agent misread this as evidence that it lacked permissions for /srv/shared, triggering the path pivot.
Ground truth	hidden test suite: https://github.com/laude-institute/terminal-bench/blob/main/original-tasks/acl-permissions-inheritance/solution.sh
System output	—
Other labels	**t_err_evidence**: mkdir -p /srv/shared && sudo groupadd research; output: sudo: not found; agent concluded it lacked permissions for /srv/shared; **t_commit**: 3; **t_commit_evidence**: I lack permissions to create /srv/shared. I'll use /tmp/srv/shared instead.; **t_surface**: 11; **t_surface_evidence**: getfacl output: # file: root/srv/shared, # owner: root — revealing agent was root the whole time; **primary_failure_layer**: agent; **primary_failure_category**: spec_violation; **primary_failure_subtype**: incorrect_path_pivot; **root_cause_label**: Incorrect Path Pivot Due to Root User Misunderstanding; **one_sentence_failure_summary**: The agent pivoted to ~/srv/shared instead of /srv/shared after misreading the step-2 output and falsely concluding mkdir had failed, never realizing it was already running as root.; **forewarning_present**: True; **forewarning_types**: ["empty_or_unexpected_output"]; **awareness**: aware_too_late; **first_awareness_step**: 11; **num_recovery_attempts**: 0; **tail_dominant_behavior**: cosmetic_progress; **fabrication_present**: False; **fabrication_type**: None; **verification_ever**: True; **action_segments**: [[1, 1, "X"], [2, 3, "X"], [4, 6, "X"], [7, 10, "W"], [11, 12, "V"], [13, 13, "C"]]; **timepoint_confidence**: high

Notes. (i) Annotation-only release: the set cannot be used directly to *evaluate* a step-localiser; it supports meta-analysis of where decisive errors fall (median $t_{\text{err}} = 7$, relative 0.31 — much earlier than TrajErrBench's 0.65) and of fix windows and observability lags, or a rebuild by re-joining the CodeTracer trajectories. (ii) Three timestamps instead of one are unique in this survey and encode the online-monitoring question; t_{obs} can precede t_{lock}. (iii) Use `trigger_mechanism` as the taxonomy; `primary_failure_layer/category/subtype` are free-form LLM-drafted strings (24 distinct "layers", 15+ categories) and should not be treated as one. (iv) Labels are LLM-drafted and human-verified; the unusually high κ (0.78–0.96) is partly explained by both annotators starting from the same Claude draft, and the codebook describes the shipped JSON as "one LLM annotator per trajectory" while the paper states all final labels are human-verified. (v) Scale and balance: the largest coding-agent failure corpus here (1,184 failures over 21 systems, max 1,189 steps) with PASS and FAIL runs on the same 89 tasks, allowing paired success/failure comparison. (vi) Overlaps: same Terminal-Bench/Harbor ecosystem as other terminal-agent work and conceptually close to TrajErrBench's SWE-Bench Pro subset (§6.4), but no trajectories are shared.

6.4 TrajErrBench (TrajDebug)

natural · available · step + module.subtype · single-agent · τ^2-Bench airline / retail · SWE-Bench Pro

Paper	*TrajDebug: Tracing Error Lifecycle to Identify Critical Failures in Long-Horizon Agent Trajectories* — Qi, Yin, Shi, Peng, Lu, Liu, Xuan, Liu, Hu, Wang, Hou, Xu, Li (Tsinghua THU-KEG, with industry co-authors), Findings of EMNLP 2026 (accepted 2026-08-20), arXiv:2608.06346 [54]
Links	GitHub `THU-KEG/TrajDebug` (code + `data/trajerrbench/{en,zh}`; English edition released 2026-09-07; the repo also ships Who&When and AgentErrorBench converted to the same schema) · no HF dataset
License	MIT for the TrajDebug code and annotations (`LICENSE`, `CITATION.cff`); task text and trajectories remain under τ^2-Bench and SWE-Bench Pro terms (`data/README.md`)
Status	downloaded in full (git clone), ungated; 486 zh + 486 en JSON files = 63 MB (clone 215 MB); `data/trajerrbench/repo_TrajDebug/data/trajerrbench/{en,zh}/{tau2bench,swebenchpro}/*.json` + `manifest.jsonl`

Description. TrajErrBench is a benchmark for *critical error step detection* in long-horizon single-agent trajectories, using Who&When's counterfactual definition: the critical step is the *earliest* decisive error, i.e. the step whose action, if replaced by a correct one (earlier steps fixed, later steps unfolding correctly), would have turned the failure into a success. Detection uses the failed trajectory alone, without task ground truth. Failures are natural (reward 0 under the source benchmark's own evaluation). Two domains:

- **τ^2-Bench** (400): airline and retail customer-service tool-use tasks with a simulated user; rollouts by DeepSeek-Reasoner, GPT-5 and Qwen3-Max; 29.3 messages on average.
- **SWE-Bench Pro** (86): repository-level issue fixing in the OpenHands scaffold; rollouts by Claude-Opus-4.6, Kimi-K2.6-Coding, Gemini-3.1-Pro and Grok-4.20-Beta (GPT-5.4 is listed in the paper but absent from the released ids); 119.7 messages on average.

Annotation: GPT-5.4, Gemini-3.1-Pro and Claude-Opus-4.6 pre-annotate with the trajectory *plus* the benchmark's failure diagnostics (unmet DB constraints / failing tests) as a reference; a pool of 20 trained annotators then labels **3 per trajectory**, and the majority ($\geq$2/3) is the label. Guideline: the step must contain an explicit, evidence-supported mistake; it must be failure-responsible (locally repaired mistakes excluded); if several, take the earliest. SWE-Bench Pro started with 110 runs, 24 without a 2-way agreement were dropped. Fleiss' κ on the critical step: **0.91** on τ^2-Bench (pairwise 91.2%), **0.67** on SWE-Bench Pro (66.7%); execution phase 0.73 / 0.42; contradiction reference category 0.41 / 0.38 (paper Table 5). Pre-annotation exact match with the final label: Gemini 76.8%, Claude 64.5%, GPT-5.4 53.2% on τ^2; 31.4 / 48.8 / 55.8% on SWE-Pro. **Labels**: `critical_error_step` (0-based message index; 485 assistant messages and one tool message) and `critical_error_type` = `<module>.<subtype>` from a 5-module / 20-subtype taxonomy — PLAN (BadDecomposition, WrongOrder, UnrealisticPlan, GoalDrift, OverExplore), REASON (WrongChoice: visible evidence

ignored or misquoted; InvalidInference: evidence cited but conclusion over-reaches; MissingAssumptionCheck: claim with no evidence), ACT (WrongTool, ToolSchemaMismatch, WrongActionSequence, UnsafeOrForbiddenAction), OBS (MisreadOutput, IgnoreOutput, TimingIssue, GroundingFail), VERIFY (NoVerification, WrongVerification, PrematureTermination, InfiniteRetry); 17 records have `null` type. Annotators' explanations, the execution phase and the reference category are *not* shipped.

Metrics and baselines. Exact-step accuracy, plus a relaxed window $[\mathrm{GT}-3, \mathrm{GT}]$ in Appendix E (scorer: `detector/score_steps.py`); the paper evaluates on the **zh** edition. TrajDebug itself compresses the trajectory at several granularities, detects error triggers, classifies each as resolved or terminal and attributes among a candidate set. Step accuracy in % (paper Table 1; direct prompting = best of seven LLMs and GPT-5.4; AEB = AgentErrorBench ALFWorld / GAIA / WebShop, W&W = Who&When Hand-Crafted / Alg.-Generated):

Method	τ^2	SWE-Pro	W&W H / A	AEB A / G / W	Avg
Direct (Gemini-3.1-Pro)	52.5	17.4	20.7 / 46.8	19.0 / 30.6 / 32.0	31.3
Direct (GPT-5.4)	41.5	10.5	13.8 / 28.6	11.0 / 20.4 / 14.0	20.0
AgentDebugger	26.0	10.5	12.1 / 40.5	21.0 / 36.0 / 20.0	23.7
CHIEF	26.8	2.3	19.0 / 41.3	6.0 / 26.0 / 10.0	18.8
AgentRx	36.3	5.8	25.9 / 41.3	8.0 / 26.5 / 18.0	23.1
TrajDebug	**52.8**	**24.4**	22.4 / **48.4**	**26.0** / **38.8** / 26.0	**34.1**

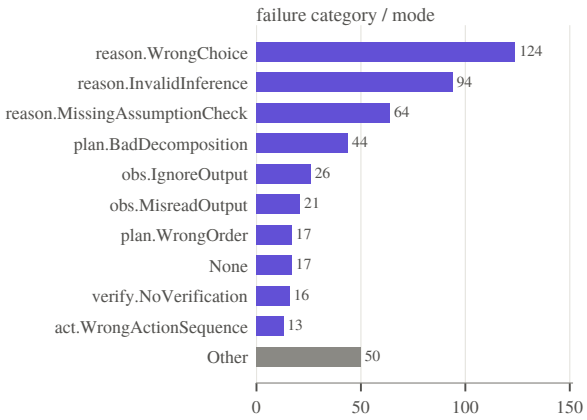
SWE-Bench Pro is by far the hardest split (best 24.4%), consistent with its lower human agreement.

Schema. One JSON file per trajectory in the unified TrajDebug schema (`data_processing/schema.py`): `messages` = list of {`step` (0-based, = array index), `role` (`system` / `user` / `assistant` / `tool`), `name`, `content` (tool calls rendered as `[tool_calls]\n- fn({json})`), `judgable` (may this step be the critical one), `compress_role`}; `metadata`: `dataset` = `trajerrbench`, `task_id`, `task_description`, `reward` (0), `annotation.{critical_error_step, critical_error_type}`, `extra.{agent_framework_description, language, source_benchmark, pair_id}`. `manifest.jsonl` pairs the zh and en files; labels and message counts are identical across editions (0 mismatches). The same folder layout holds the converted `whoandwhen/` (58 + 126) and `agentdebugbench/` (100 + 50 + 50) copies.

Statistics. computed from the downloaded English edition (identical labels to zh).

	All	τ^2-Bench	SWE-Pro
trajectories (all failed)	486	400	86
generating models	7	3	4
messages: mean / med.	45.2 / 29	29.3 / 27	119.2 / 117
messages: min – max	7 – 202	7 – 202	58 – 180
assistant steps: mean / med. (max)	21.5 / 14 (101)	—	—
critical msg index: mean / med. (max)	27.0 / 19 (160)	—	—
rel. critical position: mean / med.	0.63 / 0.65	0.64 / 0.66	0.59 / 0.62
chars / trajectory: mean / med.	52.6 k / 20.9 k	20.6 k / 19.7 k	201 k / 182 k

Subsets by domain and model: retail / DeepSeek-Reasoner 103, airline / Qwen3-Max 80, retail / GPT-5 76, airline / DeepSeek-Reasoner 71, airline / GPT-5 70; SWE-Pro: Gemini-3.1-Pro 28, Claude-Opus-4.6 21, Grok-4.20 21, Kimi-K2.6 16. Blamed module: reason 58.0%, plan 14.4%, obs 10.9%, act 7.0%, verify 6.2%, null 3.5%; on SWE-Pro `reason.WrongChoice` alone is 40.7% and all OBS subtypes are absent. Critical steps sit mid-to-late: 13.5% in the first 40% of the run, 42% between 0.6 and 0.8, 19.8% in the last fifth.



Example trajectory. `airline_failed_qwen3-max::0250f877-...` (en edition, τ^2-Bench airline, Qwen3-Max; 7 messages, 8 k characters — the shortest τ^2 run; step 0 is the airline policy prompt). The user asks for the balances of their gift cards and certificates; instead of identifying the user and looking up the account, the assistant immediately calls `transfer_to_human_agents`, which ends the episode. Label: step 3, `act.WrongTool`; no free-text rationale is shipped.

Task (`airline_failed_qwen3-max::0250f877-f44b-4ceb-990a-f284cc9c6433`)

Hi, I'd like to check the balances on my gift cards and certificates, please. Could you let me know the total amounts I have for each?

0 · system

```
# Airline Agent Policy
The current time is 2024-05-15 15:00:00 EST.
As an airline agent, you can help users **book**, **modify**, or **cancel** flight reservations.  You also handle **refunds and compensation**.
Before taking any actions that update the booking database (booking, modifying flights, editing baggage, changing cabin class, or updating
passenger information), you must list the action details and obtain explicit user confirmation (yes) to proceed.
You should not provide any information, knowledge, or procedures not provided by the user or available tools, or give subjective recommendations or
comments.
You should only make o
[...]  (2,605 chars omitted) [...]
the agent can offer a certificate as a gesture after confirming the facts and changing or cancelling the reservation, with the amount being $50
times the number of passengers.
Do not offer compensation for any other reason than the ones listed above.
```

1 · assistant

```
Hi!  How can I help you today?
```

2 · user

```
Hi, I'd like to check the balances on my gift cards and certificates, please.  Could you let me know the total amounts I have for each?
```

3 · assistant · ANNOTATED ERROR STEP

```
[tool_calls]
- transfer_to_human_agents({"summary":  "User is requesting to check the balances on their gift cards and certificates, which is outside the scope
of available tools and policy."})
```

4 · tool

```
Transfer successful
```

5 · assistant

```
YOU ARE BEING TRANSFERRED TO A HUMAN AGENT. PLEASE HOLD ON.
```

6 · user

```
###TRANSFER###
```

Annotation

Faulty agent	assistant
Faulty step	3
Category	act.WrongTool
Reason	(no free-text rationale is shipped in the release; label = earliest failure-responsible step under the Who&When decisive-error protocol)
Ground truth	—
System output	YOU ARE BEING TRANSFERRED TO A HUMAN AGENT. PLEASE HOLD ON.
Other labels	**reward**: 0; **source_benchmark**: tau2bench; **language**: en; **pair_id**: tau2bench:airline_failed_qwen3-max::0250f877-f44b-4ceb-990a-f284cc9c6433; **agent_framework_description**: A user interacts with a single ‘assistant‘ that must follow the scene policy shown in the leading ‘system‘ message while executing the user’s request. ‘tool‘ messages are tool-call return values. Critical errors can only be attributed to an ‘assistant‘ step; ‘user‘ and ‘tool‘ steps are never the critical step. The assistant should try to fulfil the user’s re [...] (1,752 chars omitted) [...] erified to be permitted commits the assistant to a path that may be unworkable, and the user may rely on the option as if it were available.

Notes. (i) Closest to Who&When’s protocol (earliest decisive error, 3-way majority) but single-agent and long-horizon; it is the only set here with both a *customer-service tool-use* domain (simulated user whose turns are never the critical step) and a *coding* domain (≈120 messages / 200 k characters per run). (ii) Errors are mid-to-late (median relative position 0.65) because agents first gather information (dialogue, repo exploration) before the decisive decision — the opposite of Failure-as-a-Process’s early t_{err} (§6.3). (iii) Two language editions: the paper’s numbers are on zh; en was built from aligned source trajectories and inherits the zh labels — use en for English reports but cite results as zh. (iv) Missing from the release: annotator rationales, execution-phase and reference-category labels (aggregate only), the 24 dropped ambiguous SWE-Pro runs, and a type for 17 records. (v) Anchoring: annotators saw three LLM proposals *and* the benchmark’s failure diagnostics, whereas detectors see only the trajectory; the authors quantify this (Table 4) but it remains a caveat. (vi) Overlaps: ships converted Who&When (§3.1) and AgentErrorBench (§6.1) copies — a ready-made unified harness for all three — and its taxonomy is a re-organisation of AgentDebug’s five modules (plan / reason / act / obs / verify instead of memory / reflection / planning / action / system).

6.5 AgentHallu

natural · available · step + category + reason · single-agent + MAS · SimpleQA / GPQA / MATH / GAIA / BFCL

Paper	*AgentHallu: Benchmarking Automated Hallucination Attribution of LLM-based Agents* — Liu, Yang, Li, Li, He (BUPT / CASIA), arXiv:2601.06818, Jan 2026 (preprint, no venue yet) [46]
Links	GitHub `liuxuannan/AgentHallu` (code + data) · project page · no HF dataset
License	CC BY 4.0 in the repo `LICENSE`; the paper’s appendix says CC BY-NC-SA 4.0 (conflicting). Queries from SimpleQA / GPQA / MATH-500 / AIME / HLE (MIT), BFCL v3 (Apache-2.0), GAIA
Status	downloaded in full (git clone), ungated; 36 MB; `data/agenthallu/repo/AgentHallu/<framework>/NNN.json` (693 JSON files)

Description. AgentHallu recasts failure attribution as *hallucination attribution*: given a query and the full trajectory of an agent system, (1) decide whether the final answer is hallucinated (operationalised as answer ≠ ground truth), (2) localise the single *responsible* step and (3) explain the cause in free text. Seven frameworks were run on queries from eight benchmarks in five domains (World Knowledge: SimpleQA; Science: GPQA; Math: MATH-500 levels 3–5, AIME 2024/2025; General Assistant: GAIA validation; Tool Use: BFCL v3 multi-turn; plus a small HLE supplement):

- **single-agent**: SmolAgents (ReAct / CodeAct), OpenDeepSearch (ReAct / CodeAct), OpenManus, OctoTools and a plain function-calling agent (BFCL only) — 506 trajectories, one actor;
- **multi-agent**: Magentic-One (Orchestrator, Coder, WebSurfer, ComputerTerminal, FileSurfer; 94) and OWL/CAMEL (Coordinator, Planner, Search, Developer, Multi-Modal agents; 93, stored in the `Camel/` folder).

Backbones: GPT-4.1 (316 runs), GPT-4o (124), Qwen3-32B, GPT-5, Qwen2.5-Coder-32B, Llama-3.3-70B, Claude-3.7-Sonnet. Failures are natural; runs without a final answer or with ≤2 steps were dropped, and “trivial” cases on which four LLM judges agreed were removed, leaving 693 trajectories (443 hallucinated, 250 clean). **Annotation**: ten graduate annotators; GPT-5-Thinking and Gemini-2.5-Pro first draft an oracle-guided reasoning path from (question, gold answer, reference solution), annotators then give the binary verdict and, for hallucinated runs, the category/sub-category, responsible step and causal explanation; disagreements are discussed and agreed cases peer cross-checked. The responsible step is defined counterfactually (fix it and re-roll ⇒ correct answer; earliest such step wins). Initial agreement: judgment 98.9%, category 81.9%, step 77.9% (lowest on Science 72.5% and General Assistant 69.2%). **Taxonomy** (grounded theory on a 140-trajectory pilot): 5 categories / 14 sub-categories — Planning (Fact Derive, Task Decompose), Retrieval (Query / Context / Summarize Misalign), Reasoning (Factual / Math / Science / General), Human-Interaction (wrong information propagated from a user turn) and Tool-Use (Missing Required Call, Incorrect Arguments, Parallel Call Conflict, Unnecessary Call).

Metrics and baselines. Judgment (macro-F1 over all 693), step-attribution accuracy (exact match on the 443 hallucinated runs) and explanation quality (G-EVAL 1–5, GPT-5 as grader). Standard all-at-once prompting (paper Table 4; random: F1 48.5, step 8.7):

Attributor	Judg. F1	Step acc. (%)	G-EVAL
GPT-5	**70.2**	32.7	2.0
Gemini-2.5-Pro	64.6	**41.1**	**2.4**
Claude-4.5-Sonnet	63.6	33.4	2.0
DeepSeek-V3.1	52.1	19.0	1.5
Qwen3-32B	51.8	21.2	1.3
Llama-3.3-70B	40.4	7.2	0.4

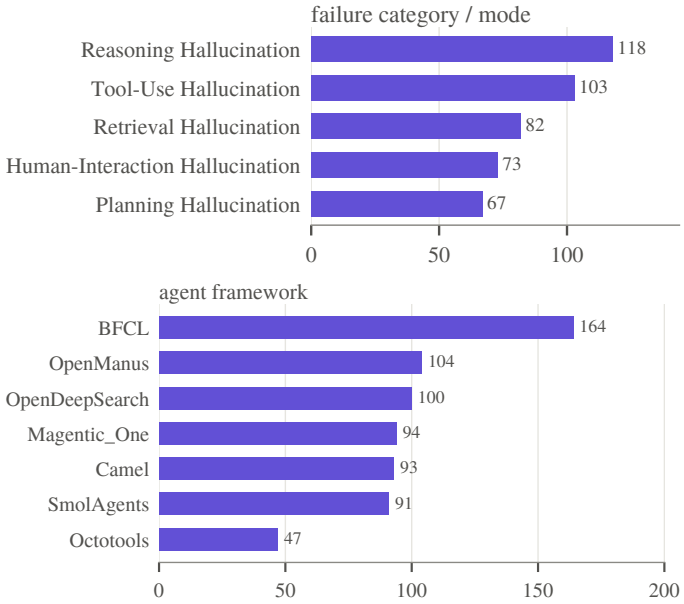
Tool-Use hallucinations are hardest to localise (best model 14.6%, GPT-5 4.9%). Incremental step-by-step prompting lifts GPT-5 from 32.7% to 42.7% at ≈3.4× the tokens (7.4 k → 25.5 k).

Schema. One JSON object per file in `AgentHallu/{BFCL, Camel, Magentic_One, Octotools, OpenDeepSearch, OpenManus, SmolAgents}/`: `model_id`, `agent_type`, `question`, `true_answer`, `question_source`, `question_domain`, `explanation` (reference solution), `history`, `agent_answer`, `is_hallucination` (string `"true"`/`"false"`) and, for the 443 hallucinated runs, `hallucination_step` (1-based, matches `history[i].step`), `hallucination_category`, `hallucination_subcategory`, `hallucination_reason`. A step holds `step`, `role` (agent name; absent in 1,221 of 5,269 steps, all single-agent), `content`, `tool_calls` ({`name`, `arguments`}) and aligned `tool_responses`. Hand-offs in the MAS folders are orchestrator steps of the form `Call Next Speaker: Coder with arguments: (...)`. There is no agent-level label; we derive the faulty role from `history[step].role`.

Statistics. computed from the downloaded data.

	All	Halluc.
trajectories	693	443
single-agent / multi-agent	506 / 187	332 / 111
SimpleQA / BFCL / GPQA	183 / 164 / 137	106 / 103 / 100
MATH-500 / GAIA / HLE / AIME	76 / 68 / 25 / 40	43 / 36 / 23 / 32
steps: mean / median	7.6 / 6	7.5 / 6
steps: min – max	3 – 43	3 – 43
roles: mean (max)	1.5 (5)	—
error step (1-based): mean / med. (max)	—	3.9 / 3 (26)
relative error position: mean / med.	—	0.53 / 0.56
characters / log: mean / median	23.3 k / 4.6 k	—

Short logs: 278 trajectories have 3–5 steps, 274 have 6–10, only 12 have >20. Relative error position is spread almost uniformly over deciles (8 / 45 / 55 / 56 / 23 / 48 / 54 / 68 / 48 / 15; exactly at the last step: 23). Derived faulty role: `assistant` 187, no role 145, Search Agent 31, MagenticOneOrchestrator 25, Developer Agent 19, WebSurfer 15, Coder 14, Planner 5. All 73 Human-Interaction cases sit in the OWL subset. A few OpenManus/OWL logs embed huge tool outputs (max 1.1 M characters).



Example trajectory. `Magentic_One/087.json` (MATH-500, GPT-4.1 backbone; 10 steps, 3 roles). At step 3 the Coder reduces the problem to a single quartic and assumes each root yields one equation, skipping the case analysis in the reference solution; the code later prints 4, the Orchestrator reports 3, the answer is 6.

Task (`Magentic_One/087.json`)

Find the number of quadratic equations of the form $x^2 + ax + b = 0,$ such that whenever c is a root of the equation, $c^2 - 2$ is also a root of the equation.

1 · MagenticOneOrchestrator

```
Here is an initial fact sheet to consider:
1.  GIVEN OR VERIFIED FACTS
- The quadratic equation has the form \( x^2 + a x + b = 0 \).
- For every root \( c \) of the equation, \( c^2 - 2 \) is also a root of the same equation.
- The request is to find the total number of such equations.
2.  FACTS TO LOOK UP
- None are directly referenced for lookup in the problem statement.  There are no externally referenced theorems or definitions.
3.  FACTS TO DERIVE
- What restrictions the condition “whenever \( c \) is a root, \( c^2 - 2 \) is also a root” places on the possible coefficients \( a \) and \( b
\).
- The process to enumerate all such \(
[...]  (985 chars omitted) [...]
isfy all the constraints discovered.
- Provide the final count as the answer.
(No local files or web search are needed as this is a self-contained algebraic investigation; the analysis and solution can be handled with Coder’s
mathematical reasoning.)
```

2 · MagenticOneOrchestrator

```
Call Next Speaker:  Coder with arguments:  ("instruction_or_question":  Output the first step (’fact_survey’) analyzing the condition:  If c is a
root of x^2 + a x + b = 0, then c^2 - 2 is also a root, and state any immediate relationships between a, b and the roots.).
```

⚠ 3 · Coder · ANNOTATED ERROR STEP

```
fact_survey
If \( c \) is a root of \( x^2 + a x + b = 0 \), then by Vieta’s formulas, the sum of the roots is \(-a\), and the product is \(b\).  Denote the
two roots by \( c \) and \( d \).  The problem specifies that, for every root \( c \), \( c^2-2 \) is also a root of the same equation.  This means
\( d = c^2-2 \) and \( c = d^2-2 \).
From this, we get two key relationships:
1.  The sum of the roots is \( c + d = -a \), so \( a = -(c + d) \).
2.  The product of the roots is \( c \cdot d = b \).
Since both roots must satisfy \( d = c^2-2 \), substituting \( d \) into the sum:
\[
c + d = c + (c^2 - 2) = c^2 + c - 2
\]
So,
\[
a = -(c^2 + c
[...]  (510 chars omitted) [...]
This gives us a concrete approach:  solve \( c = (c^2-2)^2-2 \), then for each value of \( c \) (real or complex), compute the corresponding \( a
\) and \( b \) as above.  The number of such \( c \) gives the number of corresponding quadratic equations.
```

4 · MagenticOneOrchestrator

```
Call Next Speaker:  Coder with arguments:  ("instruction_or_question":  Proceed to make_plan:  Develop a multi-step strategy to find all distinct
values of c that satisfy the equation c = (c^2 - 2)^2 - 2, and for each such c, compute the corresponding a and b for the quadratic equation x^2 +
ax + b = 0.  Be sure to enumerate and count the number of qualifying quadratic equations, carrying forward any necessary relationships.).
```

5 · Coder

```
make_plan
Here is a confident multi-step strategy to solve the problem:
1.  **Rewrite the key equation for \( c \):** Start from the relationship \( c = (c^2 - 2)^2 - 2 \).  Expand and simplify it to obtain a quartic
polynomial in \( c \), as this will reveal all possible algebraic solutions for \( c \).
2.  **Factor and Solve the Quartic:** Factor the quartic equation wherever possible to find all distinct roots.  As quartic equations can have up
to four real roots or a mixture of real and complex roots, it is likely all roots will be real and distinct in this case.
```

```
3.  **List all values of \( c \):** Enumerate all solutions to the quartic,
[...]  (847 chars omitted) [...]
ations produced by the process above.
Through this plan, each qualified value for \( c \) yields one quadratic equation, and given quartic equations nearly always yield four suitable
solutions, the answer will be confidently based on this enumeration.
```

6 · MagenticOneOrchestrator

```
Call Next Speaker:  Coder with arguments:  ("instruction_or_question":  Solve the quartic equation c = (c^2 - 2)^2 - 2 fully (expand to c^4 - 4c^2
+ 2c + 2 = 0), enumerate and clearly list all roots (real or complex), and for each root, compute the corresponding values of a and b according to
the plan.  Finally, count and report the number of distinct quadratic equations of the form x^2 + ax + b = 0 that arise from these roots.).
```

7 · Coder

```
Plan:
1.  Expand the quartic:  \( c = (c^2 - 2)^2 - 2 \rightarrow c^4 - 4c^2 + 2c + 2 = 0 \).
2.  Find all roots (\( c \)) of \( c^4 - 4c^2 + 2c + 2 = 0 \).
3.  For each \( c \), calculate \( a = -(c^2 + c - 2) \) and \( b = c^3 - 2c \).
4.  Output all roots with their (a, b).
5.  Count the number of distinct quadratic equations.
Let's implement this.
“‘python
import sympy as sp
# Define symbol
c = sp.symbols('c')
# Quartic equation
eq = c**4 - 4*c**2 + 2*c + 2
# Solve for roots
roots = sp.solve(eq, c)
# Store quadratic equations (ax, b)
quadratics = []
for root in roots:
# Compute a and b for each root
a = -(root**2 + root - 2)
[...]  (127 chars omitted) [...]
append((a_simp, b_simp))
print(f"Root c = {sp.N(root)} => a = {sp.N(a_simp)}, b = {sp.N(b_simp)}")
# Count distinct quadratic equations
distinct_quads = set(quadratics)
print(f"\nNumber of distinct quadratic equations:  {len(distinct_quads)}")
“‘
```

8 · MagenticOneOrchestrator

```
Call Next Speaker:  ComputerTerminal with arguments:  ("instruction_or_question":  Run the given Python code to solve c^4 - 4c^2 + 2c + 2 = 0, then
enumerate all roots and corresponding (a, b), and finally count and report the number of distinct quadratic equations as per instructions.).
```

9 · ComputerTerminal

```
Action:  {"name":  "PythonInterpreterTool", "arguments":  "import sympy as sp\n\n# Define symbol\nc = sp.symbols('c')\n\n# Quartic equation\neq =
c**4 - 4*c**2 + 2*c + 2\n\n# Solve for roots\nroots = sp.solve(eq, c)\n\n# Store quadratic equations (ax, b)\nquadratics = []\n\nfor root in roots:
\n # Compute a and b for each root\n a = -(root**2 + root - 2)\n b = root**3 - 2*root\n # Express in simplest form\n a_simp = sp.simplify(a)\n
b_simp = sp.simplify(b)\n quadratics.append((a_simp, b_simp))\n print(f\"Root c = {sp.N(root)} => a = {sp.N(a_simp)}, b = {sp.N(b_simp)}\")\n\n#
Count distinct quadratic equations\ndistinct_q
[...]  (334 chars omitted) [...]
086172 + 1.21601603781133*I, b = -0.79502225594763 - 1.03262310636755*I
Root c = 1.3161215976618 + 0.334783761003919*I => a = -0.93621749086172 - 1.21601603781133*I, b = -0.79502225594763 + 1.03262310636755*I
Number of distinct quadratic equations:  4
```

10 · MagenticOneOrchestrator

```
After following the process, we find that there are **3 distinct quadratic equations** of the form \(x^2 + ax + b = 0\) such that whenever \(c\) is
a root, \(c^2-2\) is also a root.
```

Annotation

Faulty agent	Coder
Faulty step	3
Category	Planning Hallucination / Fact Derive
Reason	The critical error occurred in Step 3 where the Coder adopted a flawed strategy: reducing the problem solely to solving the quartic equation $c = (c^2-2)^2-2$ and incorrectly assuming a one-to-one correspondence between its roots and the number of quadratic equations, thereby neglecting the necessary case distinctions outlined in the explanation.
Ground truth	6
System output	After following the process, we find that there are **3 distinct quadratic equations** of the form \(x^2 + ax + b = 0\) such that whenever \(c\) is a root, \(c^2-2\) is also a root.
Other labels	**is_hallucination**: true; **hallucination_category**: Planning Hallucination; **hallucination_subcategory**: Fact Derive; **model_id**: gpt-4.1-2025-04-14; **agent_type**: Magentic-One; **question_source**: MATH-500; **question_domain**: Math; **reference_explanation**: Let the roots be r and s (not necessarily real). We take the cases where $r = s$ and $r \neq s.$ Case 1: $r = s.$ Since r is the only root, we must have $r^2 - 2 = r.$ Then $r^2 - r - 2 = 0,$ which factors as $(r - 2)(r + 1) = 0,$ so $r = 2$ or $r = -1.$ This leads to the quadratics $x^2 - 4x + 4$ and $x^2 + 2x + 1.$ Case 2: $r \neq s.$ Each of [...] (1,117 chars omitted) [...] re $\boxed{6}$ quadratic equations that work, namely $x^2 - 4x + 4,$ $x^2 + 2x + 1,$ $x^2 - x - 2,$ $x^2 + x - 1,$ $x^2 - 4,$ and $x^2 - 1.$

Notes. (i) Unique in this survey: the blamed unit is a counterfactually defined *hallucination-responsible* step with a three-level label (binary / step / cause), and the taxonomy has a *Human-Interaction* category (the user turn carries the wrong fact) and four Tool-Use sub-types; the 250 clean trajectories also support the detection task, unlike failure-only sets. (ii) Mixed granularity: agent-level attribution is only meaningful on the 27% multi-agent portion; step-level labels are meaningful everywhere, and the labelled step sits mid-trajectory on average (no first/last-step prior). (iii) Quirks: `is_hallucination` is a string; `Camel` means OWL; repo and paper disagree on the license; the paper's sub-category percentages contain at least one typo (Summarize Misalign 4.6% vs 46/693 = 6.6%); the data lives only in a GitHub repo. (iv) Overlaps: shares GAIA with Who&When / TraceElephant / Aegis, BFCL-style tool calling with AgentRx and Magentic-One / OpenManus-like frameworks with MAST, but all trajectories are new runs.

6.6 MemTrace (MemTraceBench)

natural | available | operation + error type | memory system (single-agent) | LoCoMo / LongMemEval / RealMem

Paper *MemTrace: Tracing and Attributing Errors in Large Language Model Memory Systems* — Deng et al. (Zhejiang University, ZJUNLP, 18 authors), arXiv:2605.28732, May 2026 (v4 Oct 2026, marked "ongoing work") [16]
Links HF `zjunlp/MemTraceBench` · GitHub `zjunlp/MemTrace` · graph tooling `zjunlp/MemBase` / `zjunlp/smartcomment`
License MIT (dataset card, code and paper); source conversations from LoCoMo, LongMemEval, RealMem
Status downloaded in full, ungated; 6.0 GB; `data/memtrace/{rag,mem0,evermemos,long_context}/*.json` (103 execution graphs, 6.6 MB – 532 MB each)

Description. MemTrace moves failure attribution from the planner/tool loop into the *memory system*. A memory pipeline (extraction → update/delete → retrieval → answer) is instrumented so that each run becomes an *execution graph*: a bipartite DAG of variable nodes (messages, extracted facts, memory units, retrieved sets, prompts, predictions) and operation nodes (extraction, update, deletion, retrieval, filtering, judging, answering) joined by information-flow edges. Given a failed query (prediction ≠ gold), the task is to name the *earliest decisive faulty operation* o^* and its error type. Four systems were run with GPT-4.1-mini (Claude Opus 4.5 as answer judge) on three multi-session memory-QA sets (LoCoMo, LongMemEval, RealMem):

- **Long-Context** (whole history in the prompt), **Naive RAG**, **Mem0**, **EverMemOS** (Qwen3-Embedding-4B / Reranker-4B).

Failures are natural: 1,514 judged-wrong answers were collected (RAG 467, Long-Context 279, Mem0 466, EverMemOS 302) and annotated until 40 *system-caused* failures per system were confirmed. **Annotation**: five annotators from the author team, a >10-page guideline and a Streamlit UI; each case gets three independent labels, then majority vote and discussion. Annotators first rule out *annotation errors* (gold not supported by the conversation) and *judge errors* (answer actually acceptable), then locate the earliest decisive operation, its cause and type. Pairwise disagreement 3–46%; no κ reported. The released `reason` is the human rationale aggregated and polished by GPT-5.5. **Taxonomy** (7 types): Annotation Error, LLM-as-a-Judge Error, Extraction Error (critical fact never enters memory), Update Error (a later update removes/degrades it), Deletion Error (unit explicitly deleted; 0 cases), Retrieval Error (stored but not retrieved), Response Error (context complete, answer still wrong). There are no agent roles; the operation category (extraction / update / retrieval / evaluation) plays the role of the faulty component.

Metrics and baselines. **ETA** (error-type accuracy) and **OIA** (operation-identification accuracy, exact `op_id`) over the 160 cases, plus token cost. The proposed attributor walks operation sub-graphs backwards from the prediction (OBS = observation-only variant). Paper Table 1 (%):

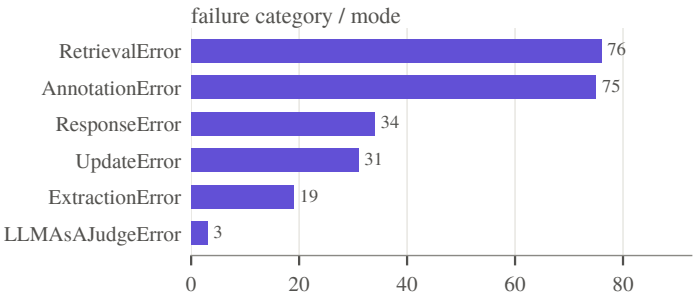
Backbone	Method	ETA	OIA
GPT-4.1-mini	All-at-Once	31.9	13.1
GPT-4.1-mini	MemTrace	36.5	14.2
GPT-5.4	All-at-Once	51.9	18.8
GPT-5.4	MemTrace-OBS	53.8	**46.3**
GPT-5.4	MemTrace	**54.4**	38.1

Per system (GPT-5.4, OBS OIA): RAG 87.5, Mem0 55.0, EverMemOS 35.0, Long-Context 7.5 — localisation collapses when all evidence sits in one giant prompt. Attribution signals used to rewrite Mem0 prompts raised LoCoMo accuracy by up to 7.62 points.

Schema. One file per (system, conversation), named `<split>/<source>_<conv-id>.json`: a serialised `smartcomment` graph with `graph_id`, `metadata` (`memtracebench_system`, `memtracebench_dataset_source`) and `data` holding five lists: `nodes` (`full_node_id` = `name@version`, `category`, `value`, `comment`, source `filename/lineno`), `operations` (`op_id`, `op_name` such as `memory_system.extract_facts`, `update_memory_decision`, `search_vector_store`, `agentic_final_rerank`, `question-answering`; `category`), `edges` (`source_full_node_id` → `target_full_node_id`, producing `op_id`), `sessions`, and `annotations` (`query`, `golden_answers`, `prediction_id`, `source_evidence(_ids)`, `final_error_type`, `final_op_id`, `reason`). No trajectory is stored explicitly: we materialise it as the backward dependency slice of `prediction_id` through `edges`, in chronological order.

Statistics. computed from the downloaded data (slice = operations feeding the prediction).

	All	RAG	Mem0	EverM.	LongCtx
execution graphs	103	30	19	20	34
memory failures (annot.)	160	40	40	40	40
Retrieval / Response err.	76 / 34	37 / 3	14 / 1	25 / 14	0 / 16
Update / Extraction err.	31 / 19	0 / 0	7 / 18	0 / 1	24 / 0
operations / graph: median	743	630	1,009	1,407	498
slice steps / failure: median	497	495	81	1,447	497



Plus 75 Annotation-Error (67 unique queries) and 3 Judge-Error annotations kept in the files (238 in total). By source: LongMemEval 70, LoCoMo 58, RealMem 32 failures (graphs 75 / 16 / 12). Graphs hold on average 3,010 nodes and 5,322 edges (max 17,753 / 32,616); operations per graph 422 – 4,571. Slice length: mean 805, min 12, max 3,661 operations. The faulty operation lies inside the slice in 140 / 160 cases (misses are mostly Mem0 extraction errors: the fact never became a memory unit, so no edge reaches the prediction); its relative position in the slice is late (mean 0.79, median 0.90) and mean 0.69 among all operations of the graph. Node text per graph: median 20.9 M characters; per slice median 7.6 M (Mem0 slices 0.1–0.3 M).

Example trajectory. `mem0/longmemeval_longmemeval-f8c5f88b.json`, annotation `query:f8c5f88b@1` (Mem0 on LongMemEval; Update Error). The prediction's backward slice has 16 operations out of 977 in the graph; at operation 12 an update rewrites "new tennis racket from a sports store downtown" into "focusing on improving tennis game with a new racket", so retrieval and answering (steps 15–16) no longer see the purchase location. Prompt-template inputs (4–8 k characters each) are omitted from the rendering.

Task (`mem0/longmemeval_longmemeval-f8c5f88b.json::query:f8c5f88b@1`)

'Where did I buy my new tennis racket from?'

1 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-033d2905d9bf40fd89e86af67a91bb4f, session 1ede478b, 2026-04-27 01:00:46.385)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:c4d3eefad6ce7d259347218d38832b914cd1dff9: '{"facts" : ["Planning to check out West Elm and Crate & Barrel online", "Looking to buy new rugs for the living room", "Interested in stores
[...] (1,163 chars omitted) [...]
of rugs"]}'
- [llm_response] str:c4d3eefad6ce7d259347218d38832b914cd1dff9: '{"facts" : ["Planning to check out West Elm and Crate & Barrel online", "Looking to buy new rugs for the living room", "Interested in stores with a wide selection of rugs"]}'

2 · update operation (memory_system.update_memory_decision)

Operation: memory_system.update_memory_decision [update] (op_id op-35263efef0ab46f3a35b16a39b81dc9b, session 1ede478b, 2026-04-27 01:00:47.464)
Description: Decide how to update the memory store using a large language model.
Inputs:
- [similar_memories] str:3a82ed585fc1d5dae6573a915a7c5fc982cc2b24: 'Find 8 existing memories similar to the new extracted facts.'
- [llm_response] str:c4d3eefad6ce7d259347218d38832b914cd1dff9: ['Planning to check out West Elm and Crate & Barrel online', 'Looking to buy new rugs for the living room', 'Interested in stores with a wide selection of rugs']
- [prompt] memory-update-decision-prompt (prompt templ
[...] (5,190 chars omitted) [...]
"Likes trendy clothes from Old Navy" }, { "event": "NONE", "id": "3", "text": "Planning to buy the Tab S8" }, { "event": "NONE", "id": "4", "te [...]

3 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-34a8949827c94ee69a40754fcf28a700, session 66c23110_1, 2026-04-27 01:06:01.751)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:29308fffce86ada19dac92c57e7c41c3554d3b29: '{"facts": ["Recently bought an eyeshadow palette at Sephora", "Has 200 points in Sephora loyalty program"]}'
- [message] longmemeval-mess
[...] (916 chars omitted) [...]
eyeshadow palette at Sephora", "Has 200 points in Sephora loyalty program"]}'
- [llm_response] str:29308fffce86ada19dac92c57e7c41c3554d3b29: '{"facts": ["Recently bought an eyeshadow palette at Sephora", "Has 200 points in Sephora loyalty program"]}'

4 · update operation (memory_system.update_memory_decision)

Operation: memory_system.update_memory_decision [update] (op_id op-46e0d327c91d4e4b85f699f003f7e760, session 66c23110_1, 2026-04-27 01:06:02.721)
Description: Decide how to update the memory store using a large language model.
Inputs:
- [similar_memories] str:d70b5f680e2b5fcaa09f935df5ebd0d3c31ac49a: 'Find 6 existing memories similar to the new extracted facts.'
- [llm_response] str:29308fffce86ada19dac92c57e7c41c3554d3b29: ['Recently bought an eyeshadow palette at Sephora', 'Has 200 points in Sephora loyalty program']
- [prompt] memory-update-decision-prompt (prompt template, 5460 chars, omitted)
- [llm_response] str:b06ab983522543
[...] (4,886 chars omitted) [...]
{ "event": "NONE", "id": "3", "text": "Had a luxury shopping spree at Neiman Marcus about six weeks ago" }, { "event": "NONE", "id": "4", "text": "Open to t [...]

5 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-21161dcfcdb74761af8361564a97a46b, session answer_c3567066, 2026-04-27 01:08:59.593)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:738f1c9c10461810e7fd840a33e04a0fab59798f: '{"facts" : ["Plays tennis", "Has a new tennis racket from a sports store downtown", "Feels the new tennis racket has improved their ga
[...] (1,323 chars omitted) [...]
- [llm_response] str:738f1c9c10461810e7fd840a33e04a0fab59798f: '{"facts" : ["Plays tennis", "Has a new tennis racket from a sports store downtown", "Feels the new tennis racket has improved their game", "Plans to do exercises before playing tennis"]}'

6 · update operation (memory_system.update_memory_decision)

Operation: memory_system.update_memory_decision [update] (op_id op-aa7aeb57455146c4965ef8be3f73a5d2, session answer_c3567066, 2026-04-27 01:09:00.594)
Description: Decide how to update the memory store using a large language model.
Inputs:
- [similar_memories] str:70bc8a22ac4325bb28717881ebf51672a1c1c0f8: 'Find 13 existing memories similar to the new extracted facts.'
- [llm_response] str:738f1c9c10461810e7fd840a33e04a0fab59798f: ['Plays tennis', 'Has a new tennis racket from a sports store downtown', 'Feels the new tennis racket has improved their game', 'Plans to do exercises before playing tennis']
- [memory_entry] memory-unit-0d52
[...] (5,882 chars omitted) [...]
"event": "NONE", "id": "2", "text": "Wakes up at 6:30 AM" }, { "event": "NONE", "id": "3", "text": "Has a meeting at 7:30 AM" }, { "event": "NON [...]

7 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-1469467aa52c41b9bc07e27acda865a2, session answer_c3567066, 2026-04-27 01:09:08.319)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:3200421f4d4a3bea2c9baba6dd7a629cc78bb374: '{"facts" : ["Planning to buy new tennis balls this weekend", "Is a recreational tennis player"]}'
- [message] longmemeval-message-10
[...] (739 chars omitted) [...]
: ["Planning to buy new tennis balls this weekend", "Is a recreational tennis player"]}'
- [llm_response] str:3200421f4d4a3bea2c9baba6dd7a629cc78bb374: '{"facts" : ["Planning to buy new tennis balls this weekend", "Is a recreational tennis player"]}'

8 · update operation (memory_system.update_memory_decision)

Operation: memory_system.update_memory_decision [update] (op_id op-99c97880413a4eff81057aa93ebc6a81, session answer_c3567066, 2026-04-27 01:09:09.181)
Description: Decide how to update the memory store using a large language model.

Inputs:
- [similar_memories] str:d70b5f680e2b5fcaa09f935df5ebd0d3c31ac49a: 'Find 6 existing memories similar to the new extracted facts.'
- [llm_response] str:3200421f4d4a3bea2c9baba6dd7a629cc78bb374: ['Planning to buy new tennis balls this weekend', 'Is a recreational tennis player']
- [memory_entry] memory-unit-190e1a18-a552-4b1c-95e4-674e10cc00d6: { "id": "190e1a18-a552-4b1c-95e4-674e10cc00d6", "
[...] (5,251 chars omitted) [...]
ay" }, { "event": "UPDATE", "id": "3", "old_memory": "Plays tennis", "text": "Plays tennis" }, { "event": "NONE", "id": "4", "text": [...]

9 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-5d3029ffb1e749e5bba7b5682f17e7e3, session answer_c3567066, 2026-04-27 01:09:14.042)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:4f1fe3d732d310c2444187e0546e729f86020aff: '{"facts": ["Considering getting a tennis ball machine to practice shots"]}'
- [message] longmemeval-message-10-28-9: { "content"
[...] (616 chars omitted) [...]
32d310c2444187e0546e729f86020aff: '{"facts": ["Considering getting a tennis ball machine to practice shots"]}'
- [llm_response] str:4f1fe3d732d310c2444187e0546e729f86020aff: '{"facts": ["Considering getting a tennis ball machine to practice shots"]}'

10 · update operation (memory_system.update_memory_decision)

Operation: memory_system.update_memory_decision [update] (op_id op-c3a40398534a4d0f9509c706c588b4c8, session answer_c3567066, 2026-04-27 01:09:14.843)
Description: Decide how to update the memory store using a large language model.
Inputs:
- [similar_memories] str:1d11637d2cbbd050a008a89385183949c8a1650b: 'Find 5 existing memories similar to the new extracted facts.'
- [llm_response] str:4f1fe3d732d310c2444187e0546e729f86020aff: ['Considering getting a tennis ball machine to practice shots']
- [prompt] memory-update-decision-prompt (prompt template, 5460 chars, omitted)
- [llm_response] str:ccea8bde1d54d1a082513c07d778ee73f6780afb:
[...] (4,649 chars omitted) [...]
, { "event": "NONE", "id": "3", "text": "Wants to know good exercises to warm up before playing tennis" }, { "event": "NONE", "id": "4", "text": "Plays [...]

11 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-153ebf67780d461cba0edeb3af6e7524, session answer_c3567066, 2026-04-27 01:09:18.365)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:afaa9ba60de546fd3ebb8fa59aaa7584f32b3be6: '{"facts" : ["Considering getting a tennis ball machine in the future", "Currently focusing on improving tennis game with a new racket"
[...] (1,514 chars omitted) [...]
a7584f32b3be6: '{"facts" : ["Considering getting a tennis ball machine in the future", "Currently focusing on improving tennis game with a new racket", "Practices tennis with friends", "Interested in exercises to strengthen core and legs for tennis"]}'

⚠ 12 · update operation (memory_system.update_memory_decision) · ANNOTATED ERROR STEP

Operation: memory_system.update_memory_decision [update] (op_id op-439f2d5e63ed4e36a41befae415d9526, session answer_c3567066, 2026-04-27 01:09:19.622)
Description: Decide how to update the memory store using a large language model.
Inputs:
- [similar_memories] str:7d4e99d6a16256cad6119529f75d541cfd99ac46: 'Find 9 existing memories similar to the new extracted facts.'
- [llm_response] str:afaa9ba60de546fd3ebb8fa59aaa7584f32b3be6: ['Considering getting a tennis ball machine in the future', 'Currently focusing on improving tennis game with a new racket', 'Practices tennis with friends', 'Interested in exercises to strengthen core and legs
[...] (6,420 chars omitted) [...]
"id": "2", "old_memory": "Has a new tennis racket from a sports store downtown", "text": "Currently focusing on improving tennis game with a new racket" }, { "event": "NONE", "id": " [...]

13 · extraction operation (memory_system.extract_facts)

Operation: memory_system.extract_facts [extraction] (op_id op-c69216285c7e40dbb9a6fa7ccf598123, session 15232887_1, 2026-04-27 01:14:00.243)
Description: Extract facts from a conversation snippet using an LLM. Please note that these extracted facts are initially without timestamps, which will be added later when the memory unit is created or updated.
Inputs:
- [prompt] fact-extraction-system-prompt (prompt template, 4248 chars, omitted)
- [llm_response] str:6bc4fb15773b200e3e2dccb8d521898a84709fc5: '{"facts" : ["Enjoys tennis lessons", "Wants tennis drills to improve serves and volleys", "Attended a swimming session on Sunday morning"]}
[...] (1,015 chars omitted) [...]
nded a swimming session on Sunday morning"]}'
- [llm_response] str:6bc4fb15773b200e3e2dccb8d521898a84709fc5: '{"facts" : ["Enjoys tennis lessons", "Wants tennis drills to improve serves and volleys", "Attended a swimming session on Sunday morning"]}'

14 · update operation (memory_system.update_memory_decision)

Operation: memory_system.update_memory_decision [update] (op_id op-3d51cd8caa4a4cf5a4b02e5da73d8650, session 15232887_1, 2026-04-27 01:14:02.053)
Description: Decide how to update the memory store using a large language model.
Inputs:
- [similar_memories] str:51fe65b667286dfd3eced911f45431360b1365b2: 'Find 11 existing memories similar to the new extracted facts.'
- [llm_response] str:6bc4fb15773b200e3e2dccb8d521898a84709fc5: ['Enjoys tennis lessons', 'Wants tennis drills to improve serves and volleys', 'Attended a swimming session on Sunday morning']
- [memory_entry] memory-unit-1ebd22fa-576f-4fdc-a97b-6cfb396520e5: { "id": "1ebd2
[...] (5,613 chars omitted) [...]
: "3", "text": "Currently focusing on improving tennis game with a new racket" }, { "event": "UPDATE", "id": "4", "old_memory": "Includes tennis lessons, running, and cycling in workou [...]

15 · retrieval operation (memory_system.search_vector_store)

Operation: memory_system.search_vector_store [retrieval] (op_id op-a90ea470bc05492eb6863a76e54b3af7, session session-0be59f88323243669f8831091324bf22, 2026-04-27 14:42:39.679)
Description: A task query searches the memory store for top-10 relevant memories.
Inputs:
- [message & query] f8c5f88b: 'Where did I buy my new tennis racket from?'
Outputs:
- [memory_entry] memory-unit-142e8f97-642e-4472-988a-f6bb2d890dc6: { "id": "142e8f97-642e-4472-988a-f6bb2d890dc6", "memory": "Wants tennis drills to improve serves and volleys", "timestamp": "2023-05-27T19:21:00" }
- [memory_entry] memory-unit-190e1a18-a552-4b1c-95e4-674e10cc00d6
[...] (1,664 chars omitted) [...]
ing room", "timestamp": "2023-05-21T06:14:00" }
- [memory_entry] memory-unit-f7b5184b-0ec3-41ea-be69-a5475946b8c3: { "id": "f7b5184b-0ec3-41ea-be69-a5475946b8c3", "memory": "Enjoys tennis lessons", "timestamp": "2023-05-27T19:21:00" }

16 · evaluation operation (question-answering)

```
Operation: question-answering [evaluation] (op_id op-ea59e23d201444d4a4ee876879d4e811, session session-1bdc9fd4f46646bdb6d3ec41fc27c71d,
2026-04-27 15:07:41.993)
Description: The question-answering model generates an answer based on the question, the context retrieved from the memory system, and a
question-answering prompt.
Inputs:
- [memory_context] str:6f387573accd693da0390f35f8d83975da56256f: '### Memory 1:\nMemory: Feels the new tennis racket has improved their
game\nTime: 2023-05-25T21:29:00\n\n### Memory 2:\nMemory: Planning to buy new tennis balls this weekend\nTime: 2023-05-25T21:29:00\n\n### Memory
3:\nMemory: Currently focusing
[...] (3,544 chars omitted) [...]
Time: 2023-05-27T19:21:00\n\n### Memory 8:\nMemory: Prac [...]
- [llm_response] str:afcbbf28d347f17f53052bf1e78c083c74a1952b: "The memories provided do not mention where you bought your new tennis racket from.
Therefore, I don't have that information."
```

Annotation

Faulty agent	update operation (memory_system.update_memory_decision)
Faulty step	12
Category	UpdateError
Reason	The system incorrectly updated the memory about the user's tennis racket. The original memory stated that the user had a new tennis racket from a sports store downtown, which included the purchase location needed to answer the query. It was changed to a more general memory about focusing on improving their tennis game with a new racket. This update removed the key detail that the racket came from the sports store downtown, causing the model to later claim the purchase location was unavailable.
Ground truth	['the sports store downtown']
System output	"The memories provided do not mention where you bought your new tennis racket from. Therefore, I don't have that information."
Other labels	**final_op_id**: op-439f2d5e63ed4e36a41befae415d9526; **query_id**: query:f8c5f88b@1; **prediction_id**: prediction:str: afcbbf28d347f17f53052bf1e78c083c74a1952b@1; **source_evidence**: ["{\n \"content\": \"I'll definitely do those exercises before playing tennis. By the way, I'm really happy with my new tennis racket, which I got from a sports store downtown. It's been performing really well, and I feel like it's improved my game. Do you have any tips on how to maintain and clean my racket?\\nBelow is this message's metadata:\\nSpeaker Name: user\\nSpeaker Role: user\\n\", \n \"name\": \"user\",\n \"role\": \"user\",\n \"timestamp\": \"2023-05-25T21:29:00\"\n}"]; **source_evidence_ids**: ["longmemeval-message-10-28-5@1"]; **memory_system**: mem0; **source_dataset**: LongMemEval; **graph_size**: {"nodes": 2290, "edges": 3979, "operations": 977, "sessions": 49}; **note**: Steps are the operations in the backward (dependency) slice of the prediction node, in chronological order; the full execution graph has many more operations (see graph_size) that do not feed into this prediction.

Notes. (i) Unique: the trajectory is a white-box execution graph with every intermediate variable, and the root-cause unit is an operation id (an LLM call, a vector search, a filter) — far finer than a ReAct turn; it is the natural testbed for the "memory" categories that MAST, HORIZON and the Raj et al. taxonomy only name. (ii) Component-level, not agent-level, attribution; Deletion errors exist in the taxonomy but have zero instances. (iii) Typing is easier than localising (ETA 54.4 vs OIA 46.3 at best); Long-Context OIA $\leq$20%. (iv) Heavy to load: a single EverMemOS graph can be 532 MB / 17 k nodes and needs several GB of RAM with plain `json.load`; the README recommends `smartcomment.ExecNetwork.import_graph`. (v) Filter `final_error_type` $\in$ {`ExtractionError`, `UpdateError`, `DeletionError`, `RetrievalError`, `ResponseError`} to get the 160-case benchmark; the authors note all three source datasets contain gold-label errors (RealMem noisiest). (vi) The paper says "upon acceptance we will release", but the data is already public; no overlap with any other dataset here.

6.7 HORIZON

natural | partial | failure modes (LLM judge) | step: web subset only | single-agent | WebArena / AgentBench-OS / BIRD / IsaacSim

Paper	*The Long-Horizon Task Mirage? Diagnosing Where and Why Agentic Systems Break* — Wang et al. (UW–Madison / UC Berkeley; Mutlu, Song, Nowak), arXiv:2604.11978, Apr 2026 (preprint; ethics statement refers to COLM 2026) [66]
Links	HF `xwang2775/long-horizon-agent-failures` (runs + judge labels, 19.5 GB, ungated, no card) · HF `xwang2775/horizon-tasks` (task definitions, 786 rows, *gated*) · leaderboard
License	none stated on either HF repo or in the paper
Status	subset downloaded (187 MB of 19.5 GB): all judge bundles `result_{1,2,3,5}/`, all `os/`, `database/`, `embodied/` raw runs and the web step logs; skipped the web Playwright traces and HTML page renders (>95% of the volume). `horizon-tasks` access request denied; `data/horizon/hf/`

Description. HORIZON is a *diagnostic* benchmark for long-horizon agent tasks: it builds tasks of controllable horizon s in four domains, runs single-agent systems on them and attributes every failure to one of seven failure modes with a trajectory-grounded **LLM-as-a-Judge**. It belongs to the MAST family (failure taxonomy, judge-labelled at scale) rather than to the human-annotated who/when benchmarks. Domains and base benchmarks:

- **Web** (WebArena shopping-admin / Postmill / GitLab; CoT agent on accessibility trees) and **Embodied** (bimanual IsaacSim 5.0, primitive-action plans): *breadth* extension, k base tasks joined by "AND" (s=1 . . . 5);
- **OS** (AgentBench-OS, bash ReAct agent) and **Database** (MAC-SQL on BIRD-style text-to-SQL): *depth* extension, intermediate permission checks / filtering steps inserted (OS s=1 . . . 10, DB s=1 . . . 8).

"700+" tasks; backbones GPT-5-mini and Claude-4-Sonnet (appendix figures say GPT-4o-mini / Claude-3.5-Sonnet — an inconsistency); three OS runs per task set. Failures are natural. The environment verifier gives pass/fail; the judge independently reads the serialised trajectory and outputs `failure_type` (`Success` or a mode) and a one-sentence `reason`; in the web/GPT-5-mini subset it also outputs a `failure_step_number` and a 5-step window summary. **Human validation** is limited to a 40-trajectory pilot: κ(human, human) = 0.61, κ(human, judge) = 0.84; those human labels are not released. **Taxonomy** (7 modes): Environment (acts on stale state), Instruction (ill-defined / partially understood), False Assumption, Planning Error (sub-plan / action) — present at all horizons — and Catastrophic Forgetting, History Error Accumulation, Memory Limitation — predominantly long-horizon. The paper also splits them into process-level (72.5%) vs design-level (27.5%) failures.

Metrics and baselines. HORIZON benchmarks *agents*, not attributors: success rate vs level s with a "break level", and the failure-mode mix per domain/model. There is no attribution leaderboard — the judge is the attribution method. Paper findings: web collapses at very small s, OS and DB degrade gradually, embodied steeply. Failure mix, paper (3,100+ trajectories, %) vs released `result_2` (counts):

Domain	paper, % of failures release `result_2`, counts
Web	Plan 74.9, Env 11.3, Mem 6.2 Plan 351, Env 27, FA 25, CF 22
OS	Plan 36.7, Instr 25.9, Env 17.3, Mem 15.1 CF 717, Plan 545, Instr 503, Env 34
Database	Plan 79.3 Plan 90, Instr 13, FA 10
Embodied	Plan 94.9 Plan 139, Instr 5, FA 3

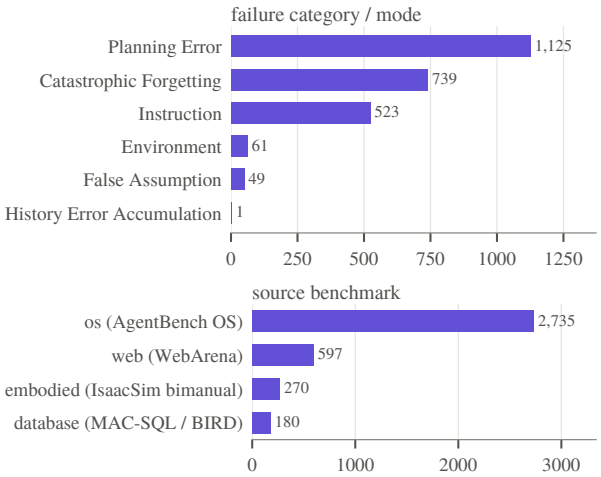
The released pass contains *no* Memory Limitation label and 29.6% Catastrophic Forgetting, so the paper's figures come from a different judge pass. By model (paper): GPT failures 64.9% Planning / 18.3% Memory; Claude failures 32.5% Environment / 16.5% Instruction.

Schema. `result_2/classified_results.json` is the main judge output: 3,782 records with `id` (e.g. `web_gpt_5_mini_step_4_6`, `os_gpt_run_0_aug-cf3-007-...`, `db_gpt_39`, `embodied_claude_<ULID>`), `domain`, `model` (`gpt` / `claude`), `run` (level), `passed` or `score` (environment verdict; present for 3,512), `trajectory` (domain-specific text: web `[Step n]` blocks with `Page`/`Agent`/`Action`; OS alternating `[Agent]`/`[OS response]`; DB one block with predicted vs gold SQL; embodied a primitive plan), `failure_type`, `reason`. `result_3/` is a second full judge pass over the same ids, `result_1/` an earlier 1,499-record pass, and `result_5/` the web/GPT-5-mini pass (371 records) adding `failure_step_number` and `steps` (judge's `label`/`summary`/`raw` for the last $\leq$5 steps). Raw runs: `os/run_{0,1,2}/<model>/<level>/runs.jsonl` (AgentBench format), `database/analyzed_results_final_{gpt,claude}.json`, `embodied/<model>/<ULID>.json`, and one directory per web level, `web/<model>/ 5_mini_step_<s>/`, with `config.json`, `steps_*.txt`, `failure_analysis.json` (an earlier per-task judge output) and `traces/*.zip`.

Statistics. computed from the downloaded `result_2` (main pass) and `result_5` (step labels); paper-only numbers marked.

	All	OS	Web	Embod.	DB
trajectories (release)	3,782	2,735	597	270	180
judge: fail / success	2,498 / 1,284	1,811 / 924	427 / 170	147 / 123	113 / 67
GPT / Claude	1,740 / 2,042	1,366 / 1,369	149 / 448	135 / 135	90 / 90
steps: mean / median	13.7 / 11	12.7 / 11	23.5 / 10	9.8 / 9	1
steps: max	151	50	151	34	1
step-level labels	294	—	294	—	—

Paper counts differ: 3,132 trajectories (web 767, OS 2,146, DB 84, embodied 135), 1,995 failures (paper). Characters per trajectory: mean 6.7 k, median 5.4 k, max 103 k (web p90 39 k). Environment verdict vs judge disagree on 12.9% of runs where both exist (200 env-passes labelled failure, 252 env-fails labelled `Success`). Judge self-consistency (`result_2` vs `result_3`): exact label 77.5%, success-vs-failure 94.3%, category given both fail 73.3% (main confusions CF↔Planning, Instruction↔Planning). `result_5` (web/GPT, 294 labelled failures): Planning 252, Environment 17, History Error Accumulation 10, Instruction 8, False Assumption 7, Memory Limitation 5; error step mean 9.5, median 6, max 61; relative position mean 0.44, median 0.42 (49 of 294 in the first decile).



Example trajectory. `web_gpt_5_mini_step_4_6` from `result_5` (WebArena shopping-admin, GPT-5-mini, breadth level s=4; 14 steps; environment score 0). The agent sees at step 2 that order 299 is already cancelled, starts the bestsellers report, then at step 9 forgets this and goes back to cancel the order, loops on the same search and early-stops. Label (*Memory Limitation*, step 9) and reason come from the LLM judge, not a human.

Task (`web_gpt_5_mini_step_4_6`)

Cancel order 299 AND What is the top-1 best-selling product in 2022 AND Show all customers AND Get the billing name of the oldest complete order

1 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Dashboard / Magento Admin
Thought:  Let's think step-by-step.  I will open the order row for order 299 to cancel it.
In summary, the next action I will perform is "`click [975]"`
Action:  click [975] where [975] is [975] row 'http://localhost:7780/admin/sales/order/view/order_id/299/'
```

2 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  #000000299 / Orders / Operations / Sales / Magento Admin
Thought:  Let's think step-by-step.
The order view shows Order Status = "Canceled", so order 299 is already canceled.  Next I need the top-1 best-selling product in 2022, show all
customers, and get the billing name of the oldest "Complete" order.  I'll navigate to the Reports section to find sales/bestsellers.
In summary, the next action I will perform is "`click [1322]"`
Action:  click [1322] where [1322] is [1322] link '\ue60a REPORTS'
```

3 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  #000000299 / Orders / Operations / Sales / Magento Admin
Thought:  Let's think step-by-step.  The order view shows Order 299's status is already "Canceled", so no further action is needed to cancel it.
Next, I'll open the Bestsellers report to find the top-selling product for 2022.
In summary, the next action I will perform is "`click [2280]"`
Action:  click [2280] where [2280] is [2280] link 'Bestsellers'
```

4 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Bestsellers Report / Products / Reports / Magento Admin
Thought:  I will set the report's date range to 2022.  In summary, the next action I will perform is "`type [4573] [01/01/2022] [0]"`
Action:  type [4573] [01/01/2022] where [4573] is [4573] textbox 'From *' required:  True
```

5 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Bestsellers Report / Products / Reports / Magento Admin
Thought:  Let's think step-by-step.  I will enter the end date for the report so I can run the 2022 bestsellers report.
In summary, the next action I will perform is "`type [4576] [12/31/2022] [1]"`
Action:  type [4576] [12/31/2022 ] where [4576] is [4576] textbox 'To *' required:  True
```

6 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Bestsellers Report / Products / Reports / Magento Admin
Thought:  Let's think step-by-step.  First I will generate the bestsellers report for 2022 to find the top-1 product.
In summary, the next action I will perform is "`click [4782]"`
Action:  click [4782] where [4782] is [4782] button 'Show Report'
```

7 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Bestsellers Report / Products / Reports / Magento Admin
Thought:  Let's think step-by-step.  The best-sellers report results need to be displayed to identify the top product for 2022.  I'll (re)run the
report by clicking the "Show Report" button.
```

```
In summary, the next action I will perform is “‘click [6091]“‘
Action:  click [6091] where [6091] is [6091] button ’Show Report’
```

8 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Bestsellers Report / Products / Reports / Magento Admin
Thought:  In summary, the next action I will perform is “‘click [9442]“‘
Action:  click [9442] where [9442] is [9442] link ’\ue60b SALES’
```

⚠ 9 · web agent (gpt-5-mini) · ANNOTATED ERROR STEP

```
Observation:  Tab 0 (current):  Bestsellers Report / Products / Reports / Magento Admin
Thought:  Let’s think step-by-step.  To cancel order #299 I need to open the Orders list under Sales.  The Orders link is available in the Sales
menu.
In summary, the next action I will perform is “‘click [9650]“‘
Action:  click [9650] where [9650] is [9650] link ’Orders’
```

10 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Orders / Operations / Sales / Magento Admin
Thought:  Let’s think step-by-step.  To cancel order #299 I should locate it in the Orders list – the page has a "Search by keyword" box.  I’ll
search for "299" to load that order.
In summary, the next action I will perform is “‘type [12939] [299] [1]“‘
Action:  type [12939] [299 ] where [12939] is [12939] textbox ’Search by keyword’ required:  False
```

11 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Orders / Operations / Sales / Magento Admin
Thought:  Let’s think step-by-step.  I will first run the search for the order so I can cancel it.
In summary, the next action I will perform is “‘click [12940]“‘
Action:  click [12940] where [12940] is [12940] button ’Search’
```

12 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Orders / Operations / Sales / Magento Admin
Thought:  In summary, the next action I will perform is “‘type [12939] [299] [1]“‘
Action:  type [12939] [299 ] where [12939] is [12939] textbox ’Search by keyword’ required:  False
```

13 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Orders / Operations / Sales / Magento Admin
Thought:  Let’s think step-by-step.  I will replace the current search keyword with the order ID "299" and submit the search to locate the order.
In summary, the next action I will perform is “‘type [12939] [299] [1]“‘
Action:  type [12939] [299 ] where [12939] is [12939] textbox ’Search by keyword’ focused:  True required:  False
```

14 · web agent (gpt-5-mini)

```
Observation:  Tab 0 (current):  Orders / Operations / Sales / Magento Admin
Thought:
Action:  stop [Early stop:  Same typing action for 3 times]
```

Annotation

Faulty agent	web agent (single-agent)
Faulty step	9
Category	Memory Limitation
Reason	After noting in Step 2 that order 299 was already canceled, the agent later forgot this and attempted to cancel it again (Step 9), repeating search actions instead of proceeding with the remaining tasks.
Ground truth	Canceled
System output	Early stop: Same typing action for 3 times
Other labels	**domain**: web; **model**: gpt; **run**: 5_mini_step_4; **task_id**: 6; **score**: 0.0; **judge**: LLM-as-a-Judge (trajectory-grounded), not human; **judge_step_window_summaries**: [{"label": "establishment step", "summary": "Agent observes order 299 is already canceled and plans to work on other tasks.", "raw": "[Step 2]\nPage : Tab 0 (current): #000000299 / Orders / Operations / Sales / Magento Admin\nAgent: Let’s think step-by-step.\n\nThe order view shows Order Status = \"Canceled\", so order 299 is already canceled. Next I need th [...] (2,070 chars omitted) [...] it.\n\nIn summary, the next action I will perform is “‘click [12940]“‘\nAction: click [12940] where [12940] is [12940] button ’Search’"}]

Notes. (i) Unique: controllable horizon — the same base task reappears at increasing depth/breadth, so failure modes can be plotted against horizon; four heterogeneous domains including embodied manipulation and text-to-SQL; two full judge passes allow measuring judge self-consistency. (ii) **Labels are LLM-generated**: treat HORIZON as a judge-labelled taxonomy dataset (like MAST-Data), validated on 40 human-checked trajectories whose labels are not released; step labels exist only for 294 web/GPT runs and are judge outputs. (iii) Granularity varies by domain: database “trajectories” are one predicted-vs-gold SQL, embodied ones are plans without observations; step-level attribution is only meaningful for web and OS. (iv) Inconsistencies: release 3,782 vs paper 3,132 trajectories; released mode mix (0 Memory Limitation, 29.6% Catastrophic Forgetting) vs paper (15% Memory Limitation on OS); backbone names differ between main text and appendix; no license; the HF viewer fails on the repo. (v) Volume is misleading: >95% of the 19.5 GB are Playwright zips and page renders; the attribution JSON is ≈140 MB. (vi) Overlaps: base tasks from WebArena, AgentBench-OS and BIRD — none shared with the MAS attribution datasets; conceptually closest to MAST and AgentErrorBench.

6.8 MegaRCA-Mix

natural · not released · edge + fault side + mode · single-agent (Claude Code) · Harbor Index: GAIA / SWE-bench / HLE / OpenRCA + 15

Paper	*Root-Cause Attribution Is a Search Problem: Continual Search for Long-Horizon Agent Failures* — Raj et al. (Scale AI, 10 authors), arXiv:2609.13463, Sep 2026 (v2 28 Sep 2026; no venue) [55]
Links	arXiv · taxonomy paper arXiv:2607.28802 (Raj et al., *Model or Harness?*) · source trials Harbor Index · no code, data or release statement
License	not stated
Status	**not released** as of 2026-10-16 (no GitHub / HF / project page found); `data/megarca_mix/` is empty; every number below is from the paper

Description. MegaRCA-Mix is a small, deliberately hard root-cause-attribution test set for *very long* single-agent executions: 50 failed trials with a median log of 286 K tokens (1.05 MiB), each shipped with the full evidence bundle a human debugger would have — raw trajectory, trial configuration, agent-side console logs, verifier output and sand-box artifacts. It was built because the released RCA benchmarks the paper also evaluates on (TRAIL, TELBench, AgentRx, Who&When algorithm-generated) have short traces and a single trajectory file, so a judge never has to *search* for evidence. The paper’s method, *Continual Search*, prompts an LLM judge for up to four turns to keep opening

unread artifacts before committing to a diagnosis.

- **Source**: failed trials from Harbor Index (82 tasks distilled from 29 benchmarks in seven domains); the paper names GAIA, SWE-Bench, HLE, OpenRCA "+ 15 additional benchmarks". Per-benchmark, per-domain and per-model composition of the 50 trials is not reported.
- **Agent**: Claude Code with Opus-4.8 and GPT-5.5 backbones, run sandboxed under the Harbor framework with its verifiers. Failures are natural.

Annotation: each trial is human-annotated with its *first consequential error* under the interaction-centric taxonomy of Raj et al.: a triple (*edge* between the model and another component, *fault side* $\in$ {model, harness, environment, grader}, *failure mode* out of 41) plus a quoted first-divergence citation. Gold is stored as one or more *acceptable cluster sets* per trial. Number of annotators, their background and agreement are not reported. **Taxonomy** (41 modes along eight edges): model–owner (Over/Under-initiative, Satisficing, Instruction-Following, Reasoning, Unauthorized Irreversible Action, Sycophancy, Domain Knowledge Deficit, Value Misalignment; grader-side *Instruction–Grader Mismatch*), model–grader (Specification Gaming, Evaluation Awareness), model–third party (Indirect Prompt Injection, Contextual Sycophancy), model–context (State Tracking, Goal Drift, Rationale Erosion), model–memory (Missed Write/Read, State Staleness, Overgeneralization, Pollution, Redundancy, . . .), model–tool (Incorrect Selection, Tool Hallucination, Feedback Neglect, Recovery Failure, Malformed/Suboptimal Arguments; harness-side *Mistranslation*), model–peer/subagent (Delegation, Communication), model–environment (Recovery/Observation Failure; environment-side *Service Failure*, *Stale State Delivery*). Four modes therefore blame the benchmark side, not the model.

Metrics and baselines. The judge (Claude Agent SDK with file-reading tools) outputs JSON {`edge`, `fault_side`, `failure_mode`, `first_divergence`, `evidence_quote`, `rationale`} with exact strings from `taxonomy.py`. Score: per-trial set-F1 between the predicted cluster set and each acceptable gold set, $F_1(A, B) = 2|A \cap B|/(|A| + |B|)$, max over gold sets, averaged over 50 trials (paper Table 2, after four judge turns):

Judge	Single-turn	Passive cont.	Continual Search
GPT-5.5	0.356	0.569	0.569
Opus-4.8	0.471	0.479	**0.608**

(The abstract quotes GPT-5.5 0.349 $\rightarrow$ 0.498, a different run.) Evidence coverage: 70.5% of relevant artifacts opened at turn 1 for both strategies; Continual Search reaches 95.2% (turn 3) and 98.5% (turn 4) while passive continuation plateaus at 75.6%. Config files, console logs and raw session records are opened only in later turns, and these are what fix misdiagnoses.

Schema. Not observable. From the paper, a trial = Claude Code session trajectory + `config.json` / trial configuration + console logs + verifier/grader output + sandbox artifacts, with gold = acceptable set(s) of (`edge`, `fault_side`, `failure_mode`) and a first-divergence citation. There is no agent-role label (Claude Code is single-agent with sub-agents) and the "step" is a cited divergence point rather than an index.

Statistics. from the paper only (nothing downloadable; `notes/megarca_mix_stats.json` has empty lists).

Statistic	Value (paper)
trials	50, all failed
log length	median 286 K tokens / 1.05 MiB per trial
steps per trajectory; error position	not reported (labels are first-divergence citations, not indices)
failure-mode / fault-side distribution	not reported (4 sides: model, harness, environment, grader)
source benchmarks	GAIA, SWE-Bench, HLE, OpenRCA + 15 others via Harbor Index; per-benchmark counts not given
agent / backbones	Claude Code; Opus-4.8 and GPT-5.5; per-model counts not given
annotators / agreement	human; count and agreement not reported

For comparison, the median trace is roughly 2$\times$ the long traces of LongRCA and an order of magnitude above the 16 k-token Hand-Crafted logs of Who&When.

Example trajectory. **Reconstructed, not real data.** No MegaRCA-Mix trial is printed in the paper; the closest material is the worked example of the judge in Appendix A.1 / Figure 10, a GAIA2 (adaptability) trial `scenario_universe_21_5e0gvz` that is *not* one of the 50 trials but uses the same label format. The four "steps" below paraphrase the paper's prose (the raw turns are not shown there). The agent cancels Saturday appointments, messages a friend, waits once for a reply that never arrives and finishes without rescheduling or cancelling; a trajectory-only reading blames the model (Satisficing), but `config.json` and the console log show empty environment settings, so the final label is environment-side *Stale State Delivery*.

Task (scenario_universe_21_5e0gvz (GAIA2 adaptability trial used as the worked example in Appendix A.1 / Figure 10 of arXiv 2609.13463; NOT one of the 50 unreleased MegaRCA-Mix trials))

Cancel the user's Saturday appointments and schedule a 'Film Production Day' event with a friend. Reschedule based on the friend's reply; if the friend cannot make it and proposes no alternative, cancel the plan.

1 · Claude Code agent (model)

Action: cancel the user's Saturday appointments and send the friend a message proposing the 'Film Production Day' event. (Reconstructed from the paper's prose; the paper does not print the raw turns.)

2 · Claude Code agent (model)

Action: set up a trigger gated on a message-count check and issue one short wait (about 180 simulated seconds) for the friend's reply.

3 · environment (external) · ANNOTATED ERROR STEP

Observation: the wait returns after roughly 3 real seconds with no notifications; the environment's notification/settings state is empty (config.json / console log show empty environment settings).

4 · Claude Code agent (model)

Thought/Action: no reply observed, conclude there is nothing further to do and finish the task without rescheduling or cancelling.

Annotation

Faulty agent	environment (external service)
Faulty step	3
Category	edge=model–external environment; fault_side=environment; failure_mode=Stale State Delivery
Reason	The environment reported a healthy, completed wait but delivered stale/empty state (no notifications), so the agent's belief that the friend had not replied was induced by the environment rather than by the model cutting scope. A trajectory-only reading (judge turn 2) wrongly blamed the model ('Satisficing'); opening config.json and the console log (judge turn 3) revealed the empty environment settings and corrected the attribution.
Ground truth	—
System output	Agent finalised after a single short wait returned no reply from the friend, without rescheduling or cancelling as instructed (task failed the GAIA2 verifier).
Other labels	**single_turn_trajectory_only_diagnosis**: {"edge": "model–owner", "fault_side": "model", "failure_mode": "Satisficing"}; **final_diagnosis_after_continual_search**: {"edge": "model–external environment", "fault_side": "environment", "failure_mode": "Stale State Delivery"}; **taxonomy**: Interaction-centric taxonomy of Raj et al. 2026 (arXiv 2607.28802): 41 failure modes, each tied to an edge between the model and another component (owner, grader, third party, context, memory, tool, peer/subagent model, local/external environment) and a fault side (model | harness | environment | grader).; **label_format_megarca_mix**: gold = set(s) of acceptable taxonomy clusters + first-divergence citation; scored by max-over-gold-sets set-F1 per trial, averaged over 50 trials

Notes. (i) Unreleased: a *described* benchmark only — cite the numbers but do not count it among usable datasets; the paper's experiments on released data use TRAIL, TELBench, AgentRx and Who&When, all covered elsewhere in this survey. (ii) What is unique on paper: the longest traces of any RCA set; a multi-file evidence bundle that turns attribution into a search problem; a component / fault-side taxonomy (model vs harness vs environment vs grader) in which some root causes are *benchmark-side* — a category almost no other dataset admits; multiple acceptable gold sets scored by set-F1 instead of single-step exact match. (iii) Caveats: only 50 trials; annotation protocol and agreement unreported; no composition tables; benchmark count inconsistent between text (29) and Table 1 (19 named); abstract and Table 2 give different GPT-5.5 numbers. (iv) Lineage: built on Harbor Index (Terminal-Bench team) and the Raj et al. taxonomy; AgentRx and TRAIL are the closest released analogues (long single-agent traces with a category taxonomy).

7 Cross-cutting observations and recommendations

No two datasets agree on what a "step" is. Who&When counts every message, including orchestrator thoughts and the human question; TraceElephant counts trace events including tool I/O; TRAIL uses OpenTelemetry spans; MemTrace uses memory operations; AgentRx indexes 1-based from the system message; TracerTraj mixes 0- and 1-based indices across domains. Any evaluation that pools datasets must normalise the indexing explicitly—the normalised examples in `notes/*_example_norm.json` keep the dataset's own numbering for this reason.

Agent naming is a hidden evaluation knob. Agent-level accuracy is exact string match, yet names are inconsistent inside datasets (`WebSurfer`/`Websurfer` in Who&When, `Assistant`/`assistant` in AgentRx) and generic in others (TracerTraj's agentic split labels every step `assistant`, making agent-level accuracy trivial). Normalise case and collapse orchestrator variants before scoring.

Single root cause is a modelling choice. Who&When, TracerTraj, CORRECT-Error and LongRCA store one decisive step. MP-Bench shows three experts agree on a single failure-inducing step in only 16% of logs; AgentRx records multiple failure *events* plus a root cause (root $\neq$ first failure in 27% of public records); TRAIL labels every error span; MegaRCA scores a *set* of (edge, fault, mode) triples with F1. For evaluation, report a window-tolerant step metric alongside exact match, and prefer MP-Bench or AgentRx when the method produces several candidates.

Natural versus injected. Human-labelled natural failures (Who&When, TraceElephant, MP-Bench, LongRCA, AgentRx, SymFail, TRAIL; ≈2.5 k trajectories in total) are the only trustworthy *test* sets. Injected datasets (Who&When Pro, Aegis, TracerTraj, CORRECT-Error; ≈28 k) are the only practical *training* sets, but their error-position and error-type distributions are skewed (Fig. 4): Aegis is near-uniform over 14 modes by design, Who&When Pro and CORRECT-Error concentrate labels at the last step, TracerTraj at the first steps. Evaluate cross-distribution.

Observation level matters. Output-only logs (Who&When, MAST-Data) hide tool inputs, screenshots and environment state. TraceElephant reports up to +76% relative attribution accuracy from full traces and ships reproducible environments for counterfactual replay; LongRCA keeps ≈180 k characters per trajectory; MemTrace stores complete execution graphs (up to 500 MB per file). Decide early whether the attributor is meant to work from logs or from live replay.

Label provenance. Only Who&When, TraceElephant, MP-Bench, LongRCA, AgentRx, SymFail, TRAIL, AgentHallu, MemTrace, Failure-as-a-Process, SearchAuditBench and MegaRCA-Mix are human-labelled. MAST-Data (14 modes) and HORIZON are labelled by an LLM judge (MAST validated on 21 traces, $\kappa \approx 0.88$ reported in the paper); Aegis and CORRECT-Error labels are correct by construction but describe injected, not observed, errors.

Recommended protocol.

1. **Standard evaluation**: Who&When (both subsets), TraceElephant (full-trace) and MP-Bench (multi-annotator); report agent accuracy, exact-step accuracy, and step accuracy within $\pm k$ steps.
2. **Scale / long traces**: add Who&When Pro (26 benchmarks, 3 modalities) and LongRCA (median 145 steps; use LongRCA-Mini for screening).
3. **Training**: TracerTraj and Aegis (both with train/val/test splits); CORRECT-Error as a third, Magentic-style source. Hold out one natural dataset for cross-distribution testing.
4. **Diagnosis beyond "who/when"**: MAST-Data for failure-mode classification, TRAIL for span-level multi-error localisation, AgentRx for taxonomy + root cause on tool-calling agents.
5. **Generalisation checks**: AgentErrorBench (embodied/web), SearchAuditBench (deep search), Failure-as-a-Process (CLI coding), AgentHallu (hallucination), MemTrace (memory).
6. **Licensing**: several datasets carry no explicit license (Who&When, LongRCA, TracerTraj, HORIZON); AgentRx and HORIZON-tasks are gated; MegaRCA-Mix and AgentErrata (and parts of AgentRx) are not public as of 16 Oct 2026. Verify before redistribution.

8 Research proposals

8.1 Preliminaries: a working definition of failure attribution

Why a definition is needed. The four proposals that follow need a target that stays fixed from one dataset to the next. The survey shows that the current targets do not:

- Every dataset defines its labelled step *counterfactually*—"the earliest step such that, had it been correct, the run would have succeeded"—but only TraceElephant ever tests the counterfactual by re-execution (§3.2).
- The summary that is labelled differs: the earliest decisive step

(Who&When, TrajErrBench, LongRCA), the first *unrecovered* failure (AgentRx), a set of steps with per-annotator votes (MP-Bench), every error span (TRAIL), three timestamps $t_{\text{err}}/t_{\text{lock}}/t_{\text{obs}}$ (Failure-as-a-Process), a step with a tolerance span (SearchAuditBench) or a set of (edge, fault side, mode) triples (MegaRCA-Mix).

- Humans do not agree on the proxy: exact-step agreement is 40.3% in LongRCA, 77.9% in AgentHallu and 91.2% / 66.7% on the two TrajErrBench domains, TraceElephant's round-1 Krippendorff's α for the step is 0.64, and the blamed agent agrees between Who&When and TraceElephant on only 18 of 86 paired tasks.

We therefore take the counterfactual itself as the definition and treat every human label as a noisy estimate of it.

Setting and notation. An agent system π (single- or multi-agent, stochastic) interacts with an environment $\mathcal{E}$ on a task x with an automatic success predicate $S \in \{0,1\}$ (ground-truth answer, unit tests, verifier). A trajectory is $\tau = (s_1, \dots, s_T)$ where step $s_t = (a_t, u_t, o_t)$ records the acting agent $a_t \in \mathcal{A}$, its output u_t (message, tool call or code) and the resulting observation o_t (tool result, page, test log). $\tau_{<k}$ denotes the prefix $(s_1, \dots, s_{k-1})$. We write $\text{Roll}(\tau_{<k} \oplus s'_k)$ for the random continuation obtained by restoring the state reached after $\tau_{<k}$, executing the (possibly modified) step s'_k and letting π and $\mathcal{E}$ run to termination; randomness comes from sampling in π and from $\mathcal{E}$.

Assumption 1 (Replayability)**.** The environment state after $\tau_{<k}$ can be restored exactly (container snapshot, sandbox seed, simulated user), so that $\text{Roll}(\cdot)$ is a well-defined distribution over continuations. Live-web environments violate this: SymTrace's selective replay [49] serves the prefix from recorded tool returns, which restores the prefix exactly but leaves the continuation unreproducible (and the authors note that temperature-0 calls to a hosted model do not guarantee determinism either), so live-web cases are excluded from the measured part of the programme.

Assumption 2 (Fixed intervention vocabulary)**.** Interventions come from a finite, declared set $\Delta = \Delta_{\text{fix}} \cup \Delta_{\text{inj}}$; each $\delta \in \Delta$ is a deterministic map $s_k \mapsto \delta(s_k)$ that changes exactly one step. Table 2 lists a starting vocabulary. "A change to a step" without such a vocabulary is not a learnable object.

Definition 1 (Intervention effect)**.** For a trajectory τ, a step index k and an intervention $\delta \in \Delta$, define the *baseline success rate* and the *intervened success rate*

$$p_0(\tau, k) = \Pr\left[S(\text{Roll}(\tau_{<k} \oplus s_k)) = 1\right],$$
$$p_\delta(\tau, k) = \Pr\left[S(\text{Roll}(\tau_{<k} \oplus \delta(s_k))) = 1\right],$$

and the *effect* $\phi(\tau, k, \delta) = p_\delta(\tau, k) - p_0(\tau, k) \in [-1, 1]$. Both probabilities are estimated by N independent rollouts; p_0 is estimated with the original step re-executed (not read from the observed outcome), which is what makes the quantity robust to run-to-run stochasticity. A step k is α-*decisive* for a failed τ if $\max_{\delta \in \Delta_{\text{fix}}} \phi(\tau, k, \delta) \geq \alpha$, and α-*fragile* for a successful τ if $\min_{\delta \in \Delta_{\text{inj}}} \phi(\tau, k, \delta) \leq -\alpha$.

Definition 2 (Failure attribution)**.** For a failed trajectory τ the *repair-effect profile* is

$$e_\tau(k) = \max_{\delta \in \Delta_{\text{fix}}} \phi(\tau, k, \delta), \qquad k = 1, \dots, T,$$

and *failure attribution* is the estimation of e_τ. For a threshold $\alpha \in (0, 1]$ that is stated with every result, the labels used in the literature are summary statistics of the profile:

- **error set** $E_\alpha(\tau) = \{k : e_\tau(k) \geq \alpha\}$ — the steps whose repair would have rescued the run;
- **decisive step** $k^\star = \min E_\alpha(\tau)$ — the earliest of them;
- **lock-in** $k_{\text{lock}} = \max E_\alpha(\tau) + 1$ — the first step from which no single-step repair recovers the run;
- **responsible agent** $a^\star = a_{k^\star}$ — the agent that emitted the decisive step; no separate "who" label is defined;
- **fault side** — whether the maximising δ at $k^\star$ acts on the output u_k (model-side) or on the observation o_k (tool- or environment-side).

Correspondence with the surveyed labels.

- $k^\star$ is the target of Who&When, TraceElephant, TrajErrBench, LongRCA, CORRECT-Error and the injected datasets, and the t_{err} of Failure-as-a-Process.
- E_α is what MP-Bench's vote counts, AgentRx's failure events and TRAIL's error spans approximate; MP-Bench's per-step consensus rate is a human estimate of $e_\tau(k)$ itself.
- k_{lock} is AgentRx's "first failure from which the run never recovers" and Failure-as-a-Process's t_{lock}; $k_{\text{lock}} - k^\star$ is that paper's fix window (median 1 step).
- $a^\star$: the sender of the labelled step equals the responsible role in 99.2% of LongRCA and 100% of TracerTraj records, so defining the agent through the step loses nothing and removes the naming problems of Section 7.
- Fault side recovers MegaRCA-Mix's model / harness / environment / grader sides and the environment-caused failures that SearchAuditBench excludes.

What the definition is robust to.

- *Annotator variance.* The target is measured, not adjudicated; a dataset whose ground truth depends on who labelled it cannot be a stable target across datasets.
- *Run-to-run stochasticity.* p_0 is re-estimated by re-execution, so a run that failed by chance has a small effect everywhere instead of a forced label.
- *Several causes.* The error set is graded by effect size; MP-Bench's 16% single-step agreement becomes a property of the profile (several steps of comparable effect) rather than label noise.
- *Recovery.* Decisive step and lock-in come from one profile, so the two conventions in the literature are reconciled instead of chosen between.
- *Observation level.* The definition lives on the system, not on the log: output-only logs, LLM-call traces and span trees are different estimators of the same quantity, not different tasks.

What must be fixed for it to be well posed.

- *The vocabulary* Δ (Assumption 2, Table 2). "Had it been correct" is not learnable until "correct" is an operation; results are reported per intervention. Errors with no single-step intervention—a wrong team composition before the log begins, which is where 20 of 126 Who&When Algorithm-Generated labels sit (step 0)—are declared out of scope, not scored as zero.
- *The step unit.* One step is one policy decision: an agent output u_t together with the observation o_t it produced. Orchestrator bookkeeping, user turns and bare tool returns are context, not candidates. This removes the indexing differences catalogued in Section 7.
- N *and* α. Effects are reported with intervals; $N = 5$–10 rollouts separate an effect of 0.5 from 0 but not 0.1 from 0, so α is not set below the resolution that N affords.

Proxies when replay is impossible.

- Live-web GAIA and AssistantBench runs and output-only logs violate Assumption 1. There the human decisive step is used as a proxy for $k^\star$, scored with exact match *and* a $\pm k$ window, and the agreement figures above are quoted as its ceiling.
- No method is evaluated on proxies alone: the replayable environments of the survey (SWE-bench and SWE-Bench Pro, ALFWorld, WebShop, τ^2-Bench, TravelPlanner, VitaBench, Terminal-Bench) carry the measured version; the proxies test transfer.
- In one sentence: failure attribution is the estimation of a step-level repair-effect profile under a declared intervention set, and the decisive step, the error set, the lock-in point and the responsible agent are summary statistics of that profile.

Set	Intervention δ	Definition (changes step k only)
Δ_{fix}	ORACLE-ACTION	replace u_k by the action a human/LLM with access to the ground truth deems correct at k
	HINT	prepend to u_k's context a one-sentence statement of the fact the step got wrong (SymTrace's suspicious-node intervention [49])
	FIX-OBSERVATION	replace a corrupted/empty o_k (404, OCR failure, swallowed output) by the correct observation
	REASSIGN	let a different agent $a' \neq a_k$ produce step k
Δ_{inj}	CORRUPT-OUTPUT	rewrite u_k with a context-aware error of mode m (the Aegis/Who&When Pro injectors)
	CORRUPT-OBSERVATION	perturb o_k (wrong number, truncated page, failing tool)
	DELETE	drop step k and continue
	SWAP-AGENT	route step k to an agent not designed for it

Table 2: A starting intervention vocabulary Δ. Each entry is a deterministic transformation; what varies across rollouts is only the continuation.

8.2 Proposal 1: Counterfactual perturbation prediction on agent trajectories

Motivation. Every dataset in this survey defines the decisive step *counterfactually*—"the earliest step such that, had it been correct, the run would have succeeded"—but only TraceElephant verifies that counterfactual by re-execution, and the injected datasets (Who&When Pro, Aegis, TracerTraj, CORRECT-Error) keep only the injections that produced a failure and discard the ones the system absorbed. Three observations from the raw data make the gap concrete:

- Same task, same system family, re-run: the blamed agent agrees between Who&When and TraceElephant in only 18 of 86 paired runs, i.e. the root cause of a task failure is run-specific.
- In LongRCA, 364 of 1,140 reasons state no causal link to the outcome and roughly one in five blames "redundant/duplicate" work in runs that still produced a final plan—the label was never tested.
- Aegis reports that injections flip a successful run into a failure between 34% (DyLAN) and 77% (LLM Debate) of the time, so absorption is the common case, yet no dataset records it.

Two replay studies confirm the picture from the execution side. SymTrace [49] re-ran 54 successful MAS executions three times each and saw 39 of them fail at least once, while unguided reruns of failed executions reproduced the failure in only 67.97% of attempts; in the faulty-agent injections of Huang et al. [28] a reviewing Inspector agent recovered up to 96.4% of the injected errors. Absorption and run-to-run drift are large effects, not noise. The questions below turn attribution from a human-labelled classification target into an *interventional quantity that can be measured.*

Closest prior work. Four strands already contain parts of the programme.

- *Replay-based repair.* SymTrace [49] (§3.7) records a MAS execution, picks an *anchor* node, serves every LLM and tool call before the anchor from the log and lets everything after it run live—the operational form of $\text{Roll}(\tau_{<k} \oplus s'_k)$ of Section 8.1. Its intervention augments the anchor's request with a category-specific repair instruction ($\hat{q}_k = q_k \oplus \Delta$, the HINT of Table 2). On SymFail (536 evaluator-confirmed failures of AG2, CrewAI and Magentic-One on WebArena-Verified Hard and AssistantBench) this repairs 20.15% of failures in one attempt, against 6.90% for three unguided reruns and 3.7–4.3% for self-reflection and critic-agent baselines, and selective replay reproduces the original failure in 80.78% of single attempts versus 67.97% for a full rerun. SymTrace runs one temperature-0 continuation per intervention, so it observes a single draw from p_δ; it takes the anchor from a symptom detector rather than estimating which step is worth intervening on, and it has no injection side.
- *Counterfactual attribution.* Causal Agent Replay [57] states the same do-intervention-then-re-execute estimator with confidence intervals and a Monte-Carlo Shapley extension for interacting steps, but validates it only on synthetic structural causal models. Minimal repair families [36] replay *sets* of events and show that single-event replay misses repairs that are only jointly sufficient (exact family recovery on 90 controlled cases and a 24-case four-agent pilot). EMFA [42] verifies the candidates of an error-propagation model with counterfactual checks and is the current best step-level attributor on Who&When (+3.45 / +4.40 pp over the previous best on the two subsets); TraceElephant's dynamic setting (§3.2) is the same idea with a replayable environment.
- *Recovery as a measured property.* Huang et al. [28] inject faulty agents into six MAS and find that a hierarchical topology loses 5.5% of performance versus 10.5–23.7% for flat ones; the hallucination-cascade study of Jamshidi et al. [31] tracks claim-level errors through three-agent chains and finds net attenuation (normalised hallucination 0.422 → 0.272); ToolMaze [80] scores a perturbation-recovery rate separately from task success and finds it drops by ≈37% under implicit tool failures. None of these ties recovery to a labelled step of a natural failure.
- *Infrastructure and training.* AgentRewind [84] checkpoints agent context and environment state jointly so that an agent can roll back—an implementation of Assumption 1; Agent-R [72] trains agents on trajectories spliced at the first erroneous step with a correct branch, i.e. it consumes exactly the (prefix, repaired step) pairs that task T1 would predict.

What is missing from all of them is the *quantity*: an effect size per (trajectory, step, intervention), estimated from repeated rollouts, on failed *and* successful natural runs.

With Definitions 1 and 2 in place, the two questions that motivate the proposal are unambiguous:

1. **Repair (Q1).** Given a *failed* τ, a step k and $\delta \in \Delta_{\text{fix}}$: what is $p_\delta(\tau, k)$? Equivalently, which (k, δ) maximise ϕ?
2. **Injection (Q2).** Given a *successful* τ, a step k and $\delta \in \Delta_{\text{inj}}$: what is $1 - p_\delta(\tau, k)$, the probability that the perturbation cascades into failure rather than being absorbed?

Neither asks for a binary answer: a single re-run is one Bernoulli draw from p_δ, and Section 3.2 shows that draws from the same prefix land on different failures.

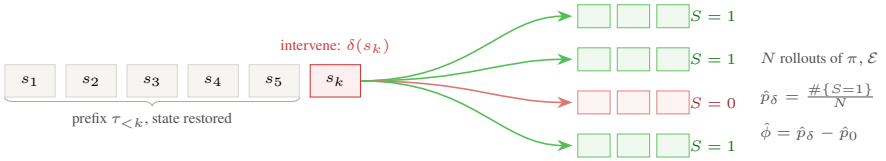


Figure 5: The measured quantity. The state after the prefix is restored, step k is replaced by $\delta(s_k)$, and N independent continuations give a success rate; the same with the original s_k gives the baseline. The effect is the difference.

Learning tasks.

T1 — Outcome prediction (the surrogate).
Learn $f(\tau_{<k}, s_k, \delta) \approx p_\delta(\tau, k)$ *without executing*. Evaluate against Monte-Carlo estimates with Brier score, calibration and AUROC; report separately for Δ_{fix} and Δ_{inj}. Baselines: process-reward-model value estimates [65], the system's own self-monitoring signals (orchestrator ledger flags precede the labelled step in 24 of 90 Magentic-One traces, §3.2), and per-source position priors.

T2 — Attribution by effect.
Predict $k^\star = \arg\max_k \max_{\delta \in \Delta_{\text{fix}}} \phi(\tau, k, \delta)$ and compare with the human labels of Who&When, TraceElephant, MP-Bench and LongRCA, and with the single-rollout counterfactual verifiers of EMFA and Causal Agent Replay [42, 57]. Disagreements between effect-based and human labels, adjudicated by rollouts, are the interesting output; MP-Bench's three annotators make it possible to ask whether effect size tracks annotator consensus.

T3 — Fragility and lock-in.
For successful trajectories, the curve $r(k/T) = \mathbb{E}_{\delta \in \Delta_{\text{inj}}} p_\delta(\tau, k)$ measures recoverability as a function of position; the first k at which r drops below a threshold is the *point of no return*, the interventional counterpart of Failure-as-a-Process's t_{lock} (§6.3). Per-agent and per-mode versions of r quantify which roles and which error modes a system can recover from and whether verifier agents change anything; the topology comparison of Huang et al. [28] (hierarchical versus flat) and SymTrace's rerun regressions [49] are single points on such curves.

The analogy to perturbation biology. The programme is a trajectory-level version of pooled genetic perturbation screens and their in-silico predictors. Perturb-seq [19] applies a library of CRISPR perturbations to a population of cells and reads out the transcriptional phenotype of each cell; GEARS [56] and scGen [48] then learn to predict the phenotype of *unseen* perturbations from the measured ones, so that most of the screen can be run in silico. Table 3 maps the objects. The correspondence is useful for borrowing methodology (replicate-based effect estimation, essentiality and buffering as graded rather than binary properties, interaction effects between two perturbations) and for seeing where it breaks: a gene library is finite and fixed, whereas edits to a step are open-ended—hence Assumption 2; cells are independent replicates, whereas rollouts share a prefix and a policy, so N must be chosen per prefix; and a rollout costs tokens rather than reagents, which makes the "screen" cheap enough to run exhaustively on small benchmarks.

Perturbation biology	Agent trajectories	Remark
cell with a genetic background	trajectory prefix $\tau_{<k}$ with restored state	the "background" includes the task, the system and the prior errors
CRISPR knockout / activation	DELETE, CORRUPT-$*$ / HINT, ORACLE-ACTION	loss- and gain-of-function interventions on one step
phenotype readout (expression, viability)	success predicate S, optionally the failure mode of the continuation	binary readout today; a richer readout is the new trajectory itself
replicate cells	N independent rollouts	estimates p_δ; biology has thousands of replicates, we have $N \approx 5$–10
essential gene	α-decisive step	graded by effect size, not by a single annotator
genetic buffering / robustness	absorbed injection ($\phi \approx 0$ on a successful run)	the data Aegis and Who&When Pro discard
epistasis (double perturbations)	two-step interventions $(k_1, \delta_1), (k_2, \delta_2)$	tests MP-Bench's claim of several independent causes; cf. minimal repair families [36]
in-silico perturbation predictor (GEARS, scGen)	surrogate f (task T1)	replaces most rollouts once calibrated

Table 3: Correspondence between pooled perturbation screens and interventional failure attribution.

Feasibility with the surveyed data.

- *Environments.* Assumption 1 holds for SWE-bench / SWE-Bench Pro (TraceElephant's SWE-Agent runs, TrajErrBench), ALFWorld and WebShop (AgentErrorBench), the TravelPlanner and VitaBench sandboxes (LongRCA) and τ^2-Bench with its simulated user (TrajErrBench, AgentRx); it fails for live-web GAIA, which should be used only to test the surrogate's transfer. SymTrace's recorded-prefix replay [49] admits WebArena-Verified and AssistantBench runs in addition, at the price of an unreproducible continuation; AgentRewind's joint context/environment checkpoints [84] are the engineering needed to restore the state after $\tau_{<k}$ cheaply.
- *Injectors.* Injectors for Δ_{inj} exist (Aegis, Who&When Pro, TracerTraj) and only need to be run with the negatives retained.
- *Budget.* A first dataset of 200 failed and 200 successful trajectories, interventions at the labelled step plus four control positions, two interventions each and $N = 8$ rollouts is ≈25 k continuations; at the tail lengths observed here (medians of 10–145 steps, 30 k–260 k tokens per full trace) this is of the order of 10^9 tokens, affordable with small open models and well below the cost of the human annotation it replaces.

Difficulties to design for.

- *Stochasticity*: p_0 for a failed run is not zero—a plain re-run can succeed (SymTrace: 6.90% of failed runs pass within three unguided reruns, and 39 of 54 successful runs fail at least once in three reruns [49])—so effects must be reported with confidence intervals and the evaluation metric must be probabilistic; a single temperature-0 replay, as in SymTrace, EMFA and the minimal-repair-family search, is one Bernoulli draw.
- *Exposed downstream errors*: a repair at k can succeed up to step $k+m$ and then fail for a new reason; each rollout should therefore be re-attributed (cheaply, by the surrogate) so that "no flip" is separable into "wrong step" and "several independent errors".
- *Oracle fixes leak the answer*: ORACLE-ACTION must be constructed from the ground truth but must not inject it verbatim; HINT is the weaker, safer variant and both should be reported.
- *Vocabulary coverage*: some labelled errors (a wrong team composition before the log begins, 20 of 126 Who&When Algorithm-Generated labels sit on step 0 for this reason) have no single-step intervention and must be declared out of scope rather than silently scored as zero effect.
- *Cost concentration*: long-horizon sources (LongRCA) dominate the token budget; stratify the rollout budget by trajectory length rather than by trajectory.

What would count as a result.

- A calibrated surrogate (T1) that predicts, from the prefix alone, which steps of a failed run are worth repairing and which steps of a successful run are load-bearing.
- Effect-based labels (T2) that agree with human consensus where humans agree and resolve the cases where they do not.
- Fragility curves (T3) that characterise, for the first time with an interventional definition, how far into a run a multi-agent system can still recover from an error.

The first two reuse the benchmarks surveyed here as test sets; the third is new measurement that none of them currently supports.

8.3 Proposal 2: Label-free attribution from the flow of trajectories

Motivation. Every human-labelled dataset in this survey is a *test* set; the only training-scale resources are injected (Section 4) and carry the positional skew of Fig. 4. Yeh et al. [68] show that step labels can be dispensed with altogether: OAT fits a one-class model of *successful* trajectories—about 100 of them, embedded step by step with an LLM and modelled by a neural controlled differential equation [33]—and flags the steps of a failed run that deviate from the learned dynamics. On the authors' own tool-use set it reaches F1 0.44 against 0.21 for GPT-4o prompting, on Who&When 0.23 against 0.15 for GPT-5, and it runs 200–5 000× faster. Two features of the result matter more than the model.

- The signal comes from *selection*, not from the dynamics: the paper's own ablations put an RNN next-step predictor on the same step embeddings close behind the CDE (its Fig. 5, no numbers in the text), and the whole out-of-distribution gain (+0.172 AUROC, at −0.028 in-domain) comes from the gated control path that suppresses atypical inputs, not from the differential-equation model; a model of *all* trajectories would simply recover the agent's own policy, under which its mistakes are ordinary, high-probability actions. [NOTE: Why the agent's own policy would have high-probability actions of making mistakes?]
- Selecting successes already requires an outcome label per trajectory, after which OAT discards the failed half of the data.

The direction below asks what the principled estimator is at each level of supervision, down to none, and which errors remain identifiable there. It rests on an observation that holds across the survey: at step level, errors are sparse in *both* outcome classes (Table 4). Successful runs are not clean either—Failure-as-a-Process PASS runs contain 2.3 error events per 22.8 steps, all recovered—so "success" is a property of the trajectory, not of its steps.

Three levels of supervision. With the notation of Section 8.1 (trajectory $\tau = (s_1, \dots, s_T)$, success predicate S), a corpus can carry

- **L2**: the step label $t^\star$ (every dataset of Sections 3–6);
- **L1$^+$**: the outcomes of several *continuations of the same prefix*—rollouts under Assumption 1, or the two branches of a fault injection—still without any human label;
- **L1**: the outcome S only, from the benchmark verifier, unit tests or a judge—free wherever the task has a ground truth;
- **L0**: raw logs, neither S nor $t^\star$.

OAT sits between L1 and L0: it uses S to select the training set and then ignores it. The proposal is built at L1 and L1$^+$: every trajectory is used, failed or not, no step is labelled, and synthetic trajectories are the natural way to add information.

Assumption 3 (Sparse errors)**.** The fraction ε of erroneous steps is small in every trajectory, failed or not; Table 4 puts it between 1% and 20% on the surveyed data.

Assumption 4 (Absorption)**.** A trajectory fails if and only if it contains an error from which the run never recovers; the first such step $t^\star$ determines the outcome, and later steps, right or wrong, are not causal. In the terms of Definition 2, $t^\star$ is the lock-in point k_{lock} (AgentRx's first unrecovered failure, Failure-as-a-Process's t_{lock}), not the decisive step $k^\star$, which precedes it by the fix window—median 1 step, but more than 10 in 239 of 1,184 runs (§6.3). A model built on this assumption is therefore expected to sit late relative to decisive-step labels and is scored against both.

Definition 3 (Step hazard)**.** Let $h_t = h_\theta(\tau_{<t}, s_t) \in [0, 1]$ be the probability that step t is the first unrecovered error given its prefix. Under Assumption 4 the outcome likelihood is the discrete-time survival model

$$P(S{=}1 \mid \tau) = \textstyle\prod_{t=1}^{T}(1 - h_t),$$
$$P(S{=}0,\, t^\star{=}t \mid \tau) = h_t \textstyle\prod_{s<t}(1 - h_s),$$

and the attribution is the posterior $P(t^\star{=}t \mid S{=}0, \tau) = h_t \prod_{s<t}(1 - h_s) \big/ \big(1 - \prod_s (1 - h_s)\big)$.

Link to Proposal 1. The survival function $\bar{S}_t(\tau) = \prod_{s \le t}(1 - h_s)$ is the probability that a run which has reached $\tau_{\le t}$ still succeeds, so h_θ is a learned surrogate of the baseline success rate $p_0(\tau, t{+}1)$ of Definition 1, and the online lock-in estimate of U3 below is the point of no return of task T3. The two proposals share one object: the rollouts of Section 8.2 measure it, the model here predicts it from the log.

The levels are different ways of fitting h_θ to the same likelihood (Fig. 6).

- **L2.** The posterior is supervised directly.
- **L1$^+$.** A rollout from $\tau_{<k}$ is an ordinary L1 trajectory that shares its first $k-1$ steps with τ and enters the same likelihood. What it adds is *identification*: with independent trajectories the likelihood only says that some step of a failed run was fatal, whereas two continuations of one prefix with different outcomes place the hazard at or after the branch point, and N continuations give a direct target $\hat{p}_0$ for $\bar{S}_{k-1}$—the Monte-Carlo value estimation of process-reward models [65], applied to agent runs. It is the only step-level supervision that costs no label, and it is exactly the by-product of the screen of Proposal 1.
- **L1.** $t^\star$ is marginalised out and θ maximises $\sum_\tau \log P(S_\tau \mid \tau)$ over *all* trajectories: a failed run contributes "some step was fatal", a successful run "no step was". This is multiple-instance learning [18] with the survival link in place of the usual noisy-OR, or equivalently positive–unlabelled learning at step level [20, 34], where the steps of successful runs are (nearly) positive examples of correctness and the class prior is the ε of Table 4.
- **L0.** S is latent as well. The likelihood then needs a model of the steps themselves and becomes a *change-point mixture*: every trajectory follows a shared pre-change dynamics $q_0(s_t \mid \tau_{<t})$—the dominant mode—up to $t^\star \in \{1, \dots, T, \infty\}$, after which a second regime q_1 applies; $t^\star$ and $S = \mathbb{1}[t^\star{=}\infty]$ are inferred jointly by EM or by Bayesian change-point detection [2].

OAT is the special case in which q_0 is fitted on known successes and the posterior is replaced by a threshold on reconstruction error.

What synthetic data adds. Besides rollouts, the injectors of Table 2 (Δ_{inj}, released with Aegis, Who&When Pro and TracerTraj) produce from one successful run a failed continuation with a known perturbed step and, as often as not (absorption 23–66% in Aegis), a successful continuation that contains a step known to be wrong. Both enter the L1 likelihood as ordinary trajectories, so the corpus grows without a label. The pair is also the sharpest test of the thesis: a hazard model must put high posterior on the injected step of the failed branch and low cumulative hazard on the identical step of the absorbed branch, whereas any detector of deviation from the success flow (OAT, the one-class baselines) sees the same step twice and must flag both. Two controls follow. Inject uniformly over positions, because the released injected sets do not (Fig. 4) and an L1 model would otherwise learn the injector's position prior; and hold out injected modes, so that generalisation across error types is measured rather than memorised. The released injected datasets themselves are used as Tier C test sets (below), not as training data, for the same reason.

Learning tasks.

U1 — Outcome-supervised hazard (L1 and L1$^+$). Fit h_θ by maximum likelihood on all trajectories of a corpus with outcomes, using the same frozen step embeddings as OAT so that the comparison isolates the estimator. The estimator itself is small: a causal GRU or two-layer transformer over the embeddings with a sigmoid hazard head, a sparsity prior on $\sum_t h_t$ (Assumption 3), and the policy decision of Section 8.1 as the step unit. Steps are discrete events, so no continuous-time machinery is needed—the neural CDE of OAT earns its place for irregularly sampled signals, which agent logs are not; what matters is causality (prefix only) and sparsity. Fit once at L1 and once with the rollout prefixes of L1$^+$ added, to measure what synthetic branching buys. Report the attribution posterior against the set-valued labels (MP-Bench consensus ranking with nDCG, AgentRx failure events, TRAIL spans, t_{err} and t_{lock} of Failure-as-a-Process) and exact / $\pm k$ step accuracy on Who&When

Dataset	Error unit	per trajectory	steps per trajectory	ε
LongRCA	root step	1	156	0.6%
Failure as a Process (FAIL)	t_{err} / error events	1 / 2.5	41.7	2.4% / 6.0%
Failure as a Process (PASS)	recovered error events	2.3	22.8	10.1%
TraceElephant	decisive LLM call	1	27.1	3.7%
Who&When	decisive step	1	22.2	4.5%
AgentRx (public subset)	failure events	4.6	44.7	10.3%
TRAIL	spans carrying $\geq$1 error	577 of 4 626 spans	31.3	12.5%
AgentHallu	responsible step	1	7.5	13.3%
MP-Bench ($\geq$2 of 3 votes)	flagged steps	4.2	23.3	18.0%

Table 4: Step-level error rate ε on the surveyed data (means from the statistics tables of Sections 3–6). Even under the most generous labelling, more than 80% of the steps of a failed trajectory are correct, and successful trajectories contain recovered errors at a comparable rate.

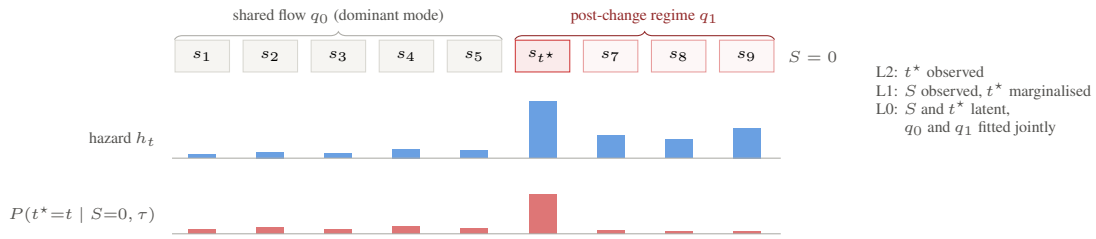


Figure 6: The hazard model. Each step receives a prefix-conditional hazard h_t; the survival product turns the hazards into an outcome probability (what L1 supervises) and, for a failed run, into a posterior over the first fatal step (the attribution). At L0 the same posterior is a change point between a shared dynamics q_0 and a post-error regime q_1.

and TrajErrBench; report the calibration of $P(S{=}0 \mid \tau)$, which the model yields for free.

U2 — Fully unsupervised change-point model (L0). Same corpora, outcomes hidden. The question is not whether it beats U1—it cannot—but *which error classes are identifiable from the dominant mode alone*. The generator-likelihood argument predicts that errors visible only in the action stream are not (a hallucinated summary is a typical summary), whereas errors that change the *observation* stream are: tool exceptions, empty pages, repeated queries (MAST FM-1.3), stalled ledgers, premature termination. Measure recall per category on the taxonomy datasets (TrajErrBench modules, Who&When Pro families, TRAIL and AgentRx categories), fitting q_0 on observations and on actions separately.

U3 — Prefix monitor. Because h_t depends on the prefix only, the cumulative hazard $1 - \prod_{s \leq t}(1 - h_s)$ is an online estimate of lock-in. Evaluate its lead time against t_{lock} (Failure-as-a-Process's prompted monitor reaches 82% precision at 18% recall with median lead time 0) and against the orchestrator's own ledger flags in Magentic-One traces (§3.2).

Evaluation suite. Three tiers, so that the method is scored where its target is measurable, where it must transfer, and where labels are exact.

- *Tier A, measured* (train and test in the same replayable system; rows 1–5 of Table 5). h_θ is fitted on self-collected runs and scored on the released human labels of TrajErrBench, AgentErrorBench and the SWE-Agent / SWE-Bench Pro traces; on a subset the rollout screen of Proposal 1 supplies $\hat{p}_0$ curves, so $\bar{S}_t$ is scored directly by Brier score and calibration—the only tier where the object itself, not a proxy, is the reference.
- *Tier B, transfer* (log-only; row 6). Frozen model, new systems: Who&When (both subsets), MP-Bench (nDCG@5 against the consensus ranking), LongRCA (long horizon, $\pm$5 window), AgentRx (set-F1 over failure events) and AgentHallu, whose 250 clean runs allow an L1 fit from the dataset's own logs.
- *Tier C, exact labels* (synthetic). Who&When Pro text split, CORRECT-Error, TracerTraj test, and our own injections with the absorbed branches retained; results stratified by injected position.

Metrics: exact and $\pm k$ step accuracy; distance to t_{err} and to t_{lock} reported separately (Assumption 4 predicts a late bias against the former); set metrics where the label is a set; lead time for U3; calibration of $P(S{=}0 \mid \tau)$. Baselines, all on the same frozen encoder: position priors (first step, last decision step, per-source histogram—the last decision step alone is inside the tolerance span of 16.9% of SearchAuditBench labels and the last fifth holds 52% of CORRECT-Error labels); All-at-Once prompting; OAT re-implemented; Isolation Forest and one-class SVM on step embeddings; and a value regression trained on rollouts only [65], i.e. L1$^+$ without L1. Robustness: outcome labels flipped at rates up to the 12.9% verifier–judge disagreement observed in HORIZON, and judge-only against verifier-only outcomes on the same runs.

Pitfalls specific to this data.

- *The last-step shortcut.* The final answer reveals the outcome, so an L1 model will put hazard on it; the injected datasets show how strong that prior is (median relative position 0.83 in Who&When Pro, 0.95 in CORRECT-Error). Mask the last k steps from the hazard input and report accuracy with the final step excluded.
- *Task difficulty as a confound.* Hard tasks produce both failures and unusual steps, so an outcome model can learn "this task is hard" instead of "this step is wrong". A matched design—the same task run to both outcomes—removes it; the 89 tasks $\times$ 21 systems of Section 6.3 are the template.
- *Recovered errors.* The survival likelihood tolerates them (a success only requires that no error was fatal); the plain multiple-instance assumption "success bags are clean" does not. Use the positive–unlabelled formulation with the prior from Table 4 when the any-error target is wanted.
- *Identifiability and model selection at L0.* A change-point mixture can converge to a split that is not error-related (planning phase versus execution phase, say). Use a small L2 set for model selection and calibration only, never for training, and say so.
- *Anomaly is not causality.* Both OAT and U2 flag deviation; only the counterfactual rollouts of Section 8.2 can confirm that a flagged step is decisive. The two directions close a loop: the hazard posterior is a cheap proposal distribution for where to intervene, and the rollouts return labels that re-fit h_θ at L2.

Feasibility with the surveyed data.

- *Released corpora with both outcomes.* AgentHallu (443 hallucinated / 250 clean), HORIZON (2 498 / 1 284 by judge), TRAIL (148 runs not selected by outcome) and Failure-as-a-Process (1 184 / 610, annotations only—the CodeTracer trajectories must be re-joined). The core MAS benchmarks release failures only (TraceElephant withholds its 160 successful runs; Who&When Pro and Aegis do not ship their successful seeds).
- *Collection engine.* Everything else is collected, not downloaded: Table 5 lists the systems of the survey whose code and environments are public and replayable, with the verifier that supplies S and the released labels that serve as test sets on them. One thousand runs of one system cost about $1{,}000 \times 30$ steps $\times$ 5 k tokens $\approx 1.5 \times 10^8$

tokens with an open model, the outcome is free, and the same infrastructure (state restore, LLM-call cache) is what Proposal 1 needs for its rollouts, so the proposals share one data engine.

- *Evaluation.* The set-labelled datasets for U1/U2 and the three timestamps of Failure-as-a-Process for U3 exist already; no new annotation is needed.
- *Budget.* One forward pass per step for the embeddings: ≈2 k trajectories × 30 k tokens ≈ 6×10^7 tokens, three to four orders of magnitude below the perturbation screen of Section 8.2; the hazard head trains in minutes.

What would count as a result.

- An L1 estimator that matches or exceeds Oat and prompting on the natural benchmarks with the same encoder and no step labels, with a calibrated outcome probability as a by-product.
- A measured gain from synthetic branching: the L1$^+$ fit beats the L1 fit on Tier A at equal label cost (zero), and the hazard model separates injected from absorbed branches where one-class detectors cannot.
- An identifiability map for L0: the error categories that can and cannot be recovered from the dominant mode, which tells practitioners what a label-free monitor will and will not catch.
- A prefix monitor with positive median lead time to t_{lock}.

System family (public agent code)	State restore for Roll	Verifier S	Released labels usable as test set	Remark
1 τ^2-Bench airline / retail: tool-calling agent + simulated user (TrajErrBench, AgentRx)	reset of domain DB and user simulator; seeded	DB-state constraints	TrajErrBench 400 (κ 0.91), AgentRx 29	cheapest: 29 messages, 20 k characters per run
2 ALFWorld, WebShop: AgentDebug's modular ReAct agent	deterministic text simulators	task reward	AgentErrorBench 100 + 50	30-step cap; observations repeat the page
3 SWE-bench Verified / Pro, Terminal-Bench 2: SWE-Agent with TraceElephant's tracing middleware, OpenHands, mini-SWE-agent, Terminus 2	container snapshot (Harbor)	hidden tests	TraceElephant 44, TrajErrBench 86, Failure-as-a-Process $t_{\text{err}}/t_{\text{lock}}$ for 1,184 runs once re-joined with the CodeTracer release	long: 120–300 steps, 200 k characters; dominates the budget
4 MASLab systems (LLM Debate, MacNet, AgentVerse, DyLAN, SmolAgents, Magentic-One) on GSM8K / MATH / HumanEval / MMLU-Pro: Aegis wrappers and injection factory; MetaGPT / AFlow runs of TracerTraj	no environment state; LLM-call cache, temperature 0	answer match, unit tests	Aegis 600 (agent + mode, no step), TracerTraj test 790 (synthetic)	injector released; the natural setting for Δ_{inj} with absorbed branches kept
5 WebArena (Verified), AgentBench-OS, BIRD: HORIZON agents; LongRCA's planner–surfer–verifier controller	docker reset per task	environment verifier	HORIZON 294 judge step labels, LongRCA WebArena 177	HORIZON agents public; LongRCA system code not released
6 Live-web GAIA / AssistantBench / BrowseComp: Magentic-One, CaptainAgent, smolagents, deep-search scaffolds	none (recorded-prefix replay only)	answer match	Who&When 184, TraceElephant 176, MP-Bench 289, SymFail 536, TRAIL 117, AgentHallu 36, SearchAuditBench 1,243	Assumption 1 fails: transfer tests only

Table 5: The data engine shared by the four proposals: systems of the survey on which unlabelled trajectories can be collected with replay, the verifier that gives the outcome for free, and the released labels that score the result on them.

8.4 Proposal 3: Attribution-guided policy improvement

Motivation. The datasets of this survey are built to score *attributors*. The obvious consumer of a good attributor is the agent that produced the failure: a label $(a^\star, k^\star)$ says which parameters to move and from which state to try again. A literature search on this direction (notes in `notes/attribution_as_training_signal.md`) gives three findings.

- *The mechanism exists, under other names.* The strongest 2026 agent-RL methods locate a "pivotal", "critical" or "irrecoverable" step of a failed rollout and use it to restart rollouts there [23, 40, 45, 59, 61], to reweight the advantage at that step [29, 32, 73, 81, 83] or to build preference pairs there [27, 38, 64, 72]. None of them cites a failure-attribution benchmark as a source of signal, and none uses an attributor trained on one.
- *The benchmarks never close the loop.* Every downstream experiment inside an attribution paper is inference-time: AgenTracer feeds its tracer's reasoning back as a prompt for three rounds (gains of 4.8–14.2%) [74], AgentDebug re-rolls from the critical step with guidance [82], SearchAuditor resumes at the critical step with a repair directive [41], SymTrace replays one node [49]. No weights change. Aegis, AgentRx, TraceElephant, Who&When Pro and MP-Bench have no downstream experiment, and the only multi-agent blueprint, first-error localisation followed by preference pairs on the blamed agent, is a position paper without experiments [13].
- *Attribution accuracy is never related to training value.* Nobody has asked whether a step metric from Sections 3–6 predicts how much the agent improves when the label is used. The one direct measurement is negative: on 150 SWE-bench Verified runs, LLM-judge "first mistake" labels fall where the success probability does *not* drop in 66% of cases, only 7 of 188 judge answers name the decisive step, and restarting an agent from the judge's step changes expected success by -0.03 to -0.08, whereas restarting from the certified point of no return raises it from 0.44 to 0.53, and from the measured restart point to 0.57 [58]. SearchAuditor's own ablation adds that, for Kimi-K2.6, a generic hint at the same predicted step repairs fewer failures (5.6%) than an unguided retry (9.4%), against 17.4% with the auditor's directive [41]: the location is necessary, not sufficient.

The pattern shared by the methods that work is *propose, verify, then credit*: a judge or the policy proposes a step, a few current-policy continuations confirm that the outcome actually turns there, and only then does the step shape credit or data [15, 32, 38, 40]. That verification step is the rollout screen of Proposal 1 with N between 1 and 8. The proposal below makes the link explicit and asks the question the literature skips.

Setting. The agent is a trainable policy π_θ (single- or multi-agent, each agent $a \in \mathcal{A}$ owning a parameter subset θ_a) run on tasks with a success predicate S; the notation is that of Section 8.1. A group-relative outcome-only update (GRPO and its agentic variants [21]) applies to every step of a failed trajectory the same negative advantage $A_\tau < 0$:

$$g_\tau^{\text{unif}} = A_\tau \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(u_t \mid \tau_{<t}).$$

Under Assumption 4 only the steps in the error set are causal, so this estimator pushes down $T - |E_\alpha(\tau)|$ correct decisions per failed run; with the error rates of Table 4 (ε between 1% and 20%) that is 80–99% of its negative gradient mass. An *attributor* $\mathcal{A}$ maps a trajectory to a non-negative weight vector $\hat{w}_\tau \in \mathbb{R}^T$, an estimate of where the causal mass lies: a point estimate $\hat{w}_\tau = \mathbb{1}[t = \hat{k}]$, a posterior $\hat{w}_\tau(t) = P(t^\star{=}t \mid S{=}0, \tau)$ from the hazard model of Definition 3, or a graded profile $\hat{w}_\tau \propto \hat{e}_\tau$ from the screen of Definition 2.

Definition 4 (Attribution-weighted update)**.** Given $\hat{w}_\tau$ and a mixing weight $\lambda \in [0, 1]$, the attribution-weighted policy gradient of a failed trajectory is

$$g_\tau^{\hat{w}} = A_\tau \sum_{t=1}^{T} \big[(1-\lambda) + \lambda\, T \hat{w}_\tau(t)/\|\hat{w}_\tau\|_1\big] \nabla_\theta \log \pi_\theta(u_t \mid \tau_{<t}),$$

so that $\lambda = 0$ recovers g_τ^{unif} and $\lambda = 1$ with the true $w_\tau = \mathbb{1}[t = k^\star]$ concentrates the whole penalty on the decisive step. Successful trajectories are unchanged.

The bias of $g^{\hat{w}}$ relative to the ideal g^{w} is governed by the attributor's precision at the chosen tolerance, which is exactly what the step metrics of this survey measure; its variance reduction relative to g^{unif} is governed by $1 - \varepsilon$. The trade-off is therefore a function of a quantity the benchmarks already report, and the proposal is to measure the function.

Three entry points. The weight vector can enter training in three places, each with a different infrastructure requirement (Fig. 7); existing methods occupy them as listed in Table 6.

R — Restart.
Rollouts for the next update start from $\tau_{<k}$ with $k \sim \rho_\tau \propto \hat{w}_\tau$ instead of from the task prompt. This reshapes the state distribution toward decisions that matter and lets a group of continuations share a prefix, so GRPO's within-group comparison becomes a step-level comparison at k. It needs Assumption 1: MiRL keeps only the 34% of prefixes whose commands replay deterministically [61], SPROUT snapshots the file system and context after every turn and keeps all of them [45].

C — Credit.
The update of Definition 4, optionally with *truncation*: no dense supervision after $\hat{k}$, because later steps are conditioned on a state the current policy would not reach [23, 29]. It needs only the log and an attributor.

D — Data.
A preference pair or a spliced correction at $\hat{k}$: the policy's action against an alternative from which a continuation succeeds [38], or the failed prefix followed by a successful sibling branch [72]. It needs an alternative action and a way to check it, i.e. one rollout from $\tau_{<\hat{k}} \oplus \delta(s_{\hat{k}})$ with $\delta \in \Delta_{\text{fix}}$ of Table 2.

In a multi-agent system $\hat{w}$ is indexed by (agent, step) and a fourth entry point opens: **M — culprit-only update**, the gradient is applied to $\theta_{a^\star}$ and the advantages of the other agents are masked, the weight-space analogue of what intervention-guided prompt optimisation already does [15]. The RL literature on LLM teams instead uses dense per-agent credit, from a coach that scores every action [37], from leave-one-out masking [39] or from exact counterfactual continuation of every message [12], and only on two- to three-agent mathematics; nothing uses (agent, step) blame on a GAIA-style system.

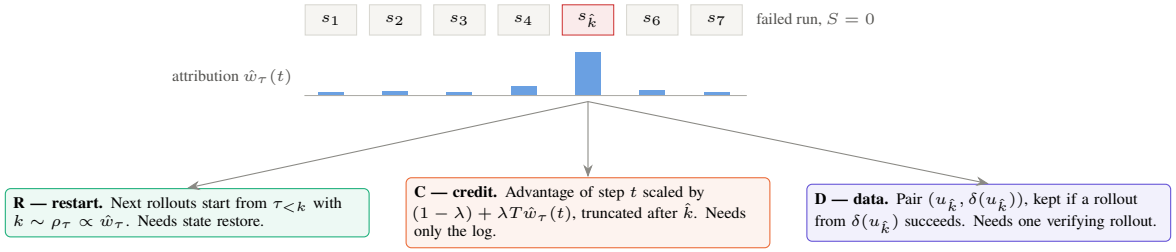


Figure 7: Entry points of an attribution estimate $\hat{w}_\tau$ into agent training. All three consume the same object and differ in what they need from the environment. In a multi-agent system the same vector, indexed by agent and step, also selects whose parameters are updated (entry point M).

Method	Attributor (who proposes $\hat{k}$)	Verified?	Entry	Gain vs. outcome-only
MiRL [61]	27B judge, clustered into 8 misbehaviours	replay only	R + judge-gated reward	TB-Lite 51.8 vs 43.4
PivoARL [23]	the policy's own reflection	no	R + C (prefix credit)	83.9 vs 73.9 (GiGPO)
R^{3}L [59]	the policy's own diagnosis	no	R + C (prefix mask)	ALFWorld 0.72 $\rightarrow$ 0.93
SPROUT [45]	policy as reviewer, with test diagnostics	no	R (snapshots)	TB-2.1 64.0 vs 39.3
ELPO [40]	binary-search rollouts: first irrecoverable step	MC	R + C (suffix clip)	60.7 vs 51.2
FAULT [83]	self-diagnoser with step and quote, evidence-checked	quote match	C (priced penalties)	ALFWorld 91.0 vs 49.3
ETRO [29]	27B writer: earliest unrecovered decisive error	no	C (truncated distillation)	WebShop 85.8 vs 73.7
ProVer [32]	agentic judge proposes a segment	MC, N=8	C (positive only)	+7 to +10% rel.
AgentEvolver [73]	judge labels every step good/bad	no	C (α=0.1–0.2)	47.6 vs 41.3
LVRL [81]	frozen LLM value function, credit = value drop	no	C	up to +13.6 pp
CSO [38]	PRM candidates, expert alternative	MC, policy continuation	D (DPO at $\hat{k}$)	GAIA 49.5 vs 38.9
Agent-R [72]	the policy's own first-error judgement	no	D (splice)	70.7 vs 62.4 (splice at end)
CUA-Recovery [27]	gold per-step lineage, human-checked	yes	D (recovery SFT)	35.2 vs 23.6 (token-matched)
MAPPA [37]	LLM coach scores every (agent, action)	no	M (dense)	no shared-reward ablation
C3 [12]	counterfactual continuation of every message	MC	M (dense)	MATH500 82.8 vs 74.5
AgentGrad [15]	sequential intervention: first agent whose fix flips S	yes	M (prompt only)	+11.8 vs +9.2 (GEPA)

Table 6: Existing methods placed on the entry points of Fig. 7. "Verified" states whether the proposed step is checked against the outcome before it is used. Numbers are the papers' own and are not compute-matched across rows; see `notes/lit_training_signal/` for the per-paper caveats.

The attributor ladder. The experiments below hold the training recipe fixed and vary only $\mathcal{A}$, ordered by cost and by how much of the outcome it has seen:

1. *position priors*: last decision step, per-source histogram (Section 8.3);
2. *outcome variance, no attribution*: start states where the reference policy sometimes fails [69];
3. *the policy's own reflection* [23];
4. *prompted judge*: the all-at-once protocol of Who&When with a frontier model;
5. *trained attributors*: an 8B model trained on TracerTraj, Aegis or Who&When Pro with the AgenTracer recipe [74], and the hazard model of Proposal 2 fitted at L1 on the same system's runs;
6. *rollout-verified*: any of 3–5 followed by the screen of Proposal 1 with $N \leq 8$ at the proposed step and its neighbours [32, 40];
7. *oracle*: the rollout-measured $k^\star$ with large N, affordable only on a test subset.

Each rung is scored twice: by the attribution metrics of this survey (exact and $\pm k$ step accuracy, nDCG against MP-Bench votes) on held-out labels of the same system, and by its training value.

Definition 5 (Training value of an attributor)**.** For a fixed recipe, budget B of environment steps and seeds, let $\pi_B^{\mathcal{A}}$ be the policy after training with weights from $\mathcal{A}$ and π_B^{unif} the outcome-only policy at the same budget. The training value is $V_B(\mathcal{A}) = J(\pi_B^{\mathcal{A}}) - J(\pi_B^{\text{unif}})$, with J the held-out success rate; the *accuracy-to-value curve* is V_B plotted against the attributor's step accuracy at the tolerance used by the recipe.

Experiments.

A1 — The accuracy-to-value curve.
Recipe: entry points R and C (restart from $\hat{k}$, prefix-only credit, truncation after $\hat{k}$) on a GiGPO-style learner [21], 4B policy, matched rollout budget. Systems: rows 1–3 of Table 5 (ALFWorld and WebShop with the AgentDebug agent, τ^2-Bench, SWE-bench with a snapshotting harness), each with released human step labels (AgentErrorBench, TrajErrBench, TraceElephant) as the accuracy reference. Every rung of the ladder is trained, plus *synthetically degraded* attributors: the oracle label shifted forward by d steps toward the symptom and the oracle mixed with random steps at precision p. The output is the curve of Definition 5, its slope, and the precision below which attribution *hurts* relative to outcome-only training. The evidence of [58] predicts that rung 4 sits below rung 2 on SWE-bench and that the shift d, not the precision p, is the damaging error.

A2 — Verified negative credit.
Entry point C alone, so that nothing is restarted. Compare (i) unverified judge weights [73], (ii) evidence-checked self-blame [83],

(iii) the hazard posterior of Proposal 2, and (iv) the rollout-verified profile $\hat{e}_\tau$ of Proposal 1 used as $\hat{w}$, each at fixed λ and at λ annealed toward 0. The quantity watched besides J is *reward hacking*: the attributor's own score rising while J falls, which is documented for fixed-weight process rewards [14, 43, 77]. ProVer uses only positive segments [32]; this experiment is its negative mirror on natural failures.

A3 — Culprit-only updates in a multi-agent system. Entry point M on the MASLab systems of row 4 (Aegis wrappers, verifiable answers) and, for transfer, Magentic-One on GAIA (row 6, log-only). Blame from the trained attributor (rung 5) and from counterfactual message replay [12]. Conditions: shared reward to all agents, per-agent leave-one-out credit [39], repair-every-agent data [78], culprit-only gradient, and culprit-only preference pairs [13]. Scored by J and by the agent-level accuracy of the blame on Who&When-style held-out labels of the same system (Aegis-Bench for the MASLab systems).

A4 — Closing the loop. After each A1–A3 run, the failures of the trained policy are re-attributed and re-labelled with the failure-mode taxonomies of Section 5 (MAST modes through the Aegis labeller, AgentErrorTaxonomy). Two numbers: how much the targeted modes shrink, which MiRL reports as 43–57% on unseen prefixes [61], and how much the other modes grow. The second is the "whack-a-mole" risk no paper reports; the taxonomy datasets of this survey are the measuring instrument.

Pitfalls to design for.

- *Symptom shift.* Judge labels drift to the step where the failure becomes visible [58]; injected datasets place labels late (Fig. 4); a policy's own reflection is bounded by its own competence. Restart entry points are the most exposed, since a restart after the point of no return explores a dead state. Verify ($N \leq 8$) or anneal λ; never use rung 3 or 4 alone for R.
- *Reward hacking.* Any score the policy can raise without succeeding will be raised: VPPO's policy split steps to collect prefix reward [43], AgentPRM's process reward rose while success fell from 82% to 70% [14]. Guards used in the literature: conjunction with the outcome ($r = r_o \wedge r_e$ [61]), a small or decaying weight [73, 77], a penalty budget capped below the success–failure gap and no gradient through the diagnosis tokens [83].
- *Baselines that get the gain without attribution.* Variance-selected start states [69], recovery training forked from the *end* of failed runs with no localisation (Terminal-Bench 34.8 vs 28.0 [67]), implicit step credit from repeated states [21], and SymTrace's last-node and random-node protocol [49]. A1 includes all of them as rungs or controls.
- *Off-policy prefixes.* A corrected or teacher-written step inside a prefix is off-policy for the learner; RTI gates it by perplexity [11], R^{3}L strips the guidance before training [59]. Report the fraction of training tokens that are off-policy.
- *State restore.* Deterministic replay loses two thirds of terminal prefixes [61]; per-turn snapshots lose none [45]. Row 3 of Table 5 should use snapshots, and live-web rows cannot use entry point R at all.
- *Compute matching.* Several headline gains in Table 6 come from baselines trained from the raw backbone or with fewer rollouts; A1–A3 match environment steps, not wall-clock, and report both.

Feasibility with the surveyed data.

- *Environments and labels.* Rows 1–4 of Table 5 supply replayable systems with released step labels on the same system (AgentErrorBench 200, TrajErrBench 486, TraceElephant's 44 SWE-Agent traces) and Aegis-Bench's (agent, mode) labels for the multi-agent case. MiRL releases code, checkpoint and trajectories under Apache 2.0 [61]; PivoARL and C3 release code.
- *Attributors.* AgenTracer-8B's weights are not released (§4.1), so rung 5 is retrained from TracerTraj (3,998), Aegis (9,533) and Who&When Pro text (6,257), all ungated; the hazard model of Proposal 2 is fitted on the runs A1 collects anyway.
- *Budget.* One RL run at MiRL's scale is 160 updates $\times$ 8 prompts $\times$ 16 rollouts $\approx 2 \times 10^4$ rollouts; A1 with seven rungs, four degraded variants, three systems and three seeds is $\approx$ 100 runs, 2×10^6 rollouts, between 10^{10} and 10^{11} tokens with a 4B policy depending on the system (MASLab and τ^2 runs are 5–20 k tokens, SWE-bench runs up to 50 k). The verification rollouts of rungs 6–7 add $N \leq 8$ continuations per failed trajectory, a small fraction of that. A3 on MASLab systems is cheaper (median 10 turns, 5 k tokens).
- *Shared infrastructure.* State restore and the LLM-call cache are the same components as Proposal 1's screen; the verified $(\tau, k, \delta, \hat{p}_\delta)$ tuples that A2 produces are training data for Proposal 2 at level L1$^+$, so the proposals share one engine.

What would count as a result.

- A measured accuracy-to-value curve, with the attribution precision below which attribution-weighted training is worse than outcome-only training, and the finding of whether symptom shift or random error is the damaging failure of an attributor.
- Verified negative credit at the decisive step that beats unverified judge credit at equal budget and does not exhibit the score-up, success-down signature.
- A culprit-only update that beats the shared reward on a multi-agent system with blame from an attributor trained on the datasets of this survey, which would be the first use of those datasets to improve an agent.
- A closed-loop measurement showing the targeted failure modes shrink without the others growing, which is the practical meaning of "learning from failure attribution".

8.5 Proposal 4: Error propagation and absorption—the lifecycle of an error

Motivation. Proposals 1–3 treat an error as a point event with a binary consequence: a step is intervened on and the outcome either flips or it does not. The survey data say that what happens *between* the error and the outcome is the common case, not the exception, and that no dataset measures it.

- Successful runs contain errors. Failure-as-a-Process PASS runs carry 2.3 error events per 22.8 steps, every one of them recovered, with a detour length and a near-miss flag per run (§6.3); in AgentRx the root cause is not the first annotated failure in 27% of public trajectories and 41% of Magentic-One logs, because the orchestrator recovers from the earlier ones (§3.6).
- Injected errors are usually absorbed. Aegis reports that 23–66% of its injections leave the outcome unchanged (§4.2); a reviewing Inspector agent recovers up to 96.4% of injected faulty-agent errors [28]; claim-level hallucinations attenuate rather than amplify along a three-agent chain (0.422 $\rightarrow$ 0.272) [31]. All four injected datasets of Section 4 discard the absorbed branch.
- Recovery is fast or not at all. The fix window $t_{\text{lock}} - t_{\text{err}}$ has median 1 step (239 of 1,184 runs above 10); of the failed runs, 58.7% never notice the error and 34.9% notice it and apply the wrong fix; 28.0% fail silently, with no externally observable signal at all.
- Recovery is trainable. Recovery-targeted fine-tuning raises a computer-use agent from 23.6 to 35.2 at matched tokens [27], and a perturbation-recovery rate falls by ≈37% under implicit tool failures while task success moves much less [80].

The hypothesis this proposal makes measurable is that a robust agent system is one in which an error introduced at step k is detected or corrected by a later step with high probability and short delay, so that the trajectory still succeeds. Two fields have measured exactly this for fifty years and supply the vocabulary. In dependable computing [5] a *fault* (the perturbation) produces an *error* (a wrong internal state) which may or may not become a *failure* (a wrong output); the error can stay *dormant*, be *masked*, be *detected*, be *recovered* or *propagate*; fault injection measures the *coverage* (the probability that an activated fault

is handled) and the *error latency* (the delay until it is) [4, 52], and the PIE model factorises failure probability into execution, infection and propagation [63]. In ecology and infrastructure engineering, *resilience* is the capacity to return to the basin of attraction after a perturbation [26], measured by the depth and duration of the performance dip [8]. The programme below transfers these quantities to agent trajectories with the notation of Section 8.1, and gives the quantity $P_{\text{detect}}(k{+}i \mid k)$ of the hypothesis a precise home.

Closest prior work. Besides the three recovery studies above [28, 31, 80], four strands contain parts of the programme. *Error-propagation profiling* of software modules [25] measures, per module, how often a wrong input produces a wrong output (permeability) and how many consumers it reaches (exposure); EMFA [42] applies an error-propagation model to Who&When but fits it to failed runs only. *Recoverability in safe RL* [22, 50] defines a state as recoverable if the agent can return to a safe set, and learns reversibility from trajectories; the point of no return of task T3 is that notion on agent runs. *Multi-state event-history analysis* [1, 3, 53] estimates the competing exits of a subject from a state under censoring, which is what an error that is still live when the run ends requires. *Quickest change detection* [47, 51, 60] gives the detection-delay versus false-alarm trade-off for the online monitor of task U3. None of them has been applied to an agent trajectory with a replayable environment, which is what makes every quantity below measurable rather than judged.

Three event times, not one. Fix a successful trajectory τ, a step k and an injection $\delta \in \Delta_{\text{inj}}$ (Table 2), and let $\omega \sim \text{Roll}(\tau_{<k} \oplus \delta(s_k))$ be one continuation. Three things can happen to the injected error, at three different lags $i \geq 1$ after k:

- **detection** D: the first lag at which a step of ω *refers* to the inconsistency (an observation contradicts the perturbed content, a verifier flags it, the agent says so)—the only one of the three that is visible in the text and the quantity $P_{\text{detect}}(k{+}i \mid k) = \Pr[D{=}i]$;
- **recovery** R: the first lag after which the perturbed content no longer governs the run—operationally, deleting $\delta(s_k)$ from the context of every step after $k{+}R$ leaves the continuation distribution unchanged, and the success probability of the prefix has returned to its unperturbed level;
- **lock-in** L: the first lag from which no single-step repair in Δ_{fix} rescues the run—Definition 2 applied to ω.

R and L compete: at most one of them occurs, and a run can also end with the error live but harmless (success without recovery), which is a *censored* observation of both. D is not an exit; a detected error can still lock in (Failure-as-a-Process's "aware, wrong fix", 34.9%), and an error can be recovered without ever being detected (a second retrieval returns the right value and the wrong one is forgotten). Failure-as-a-Process's t_{obs}, t_{lock} and the detour length of its PASS runs are the natural-failure versions of D, L and R; here they are measured on injected errors where k is known by construction.

Definition 6 (Error fate)**.** The *fate* of the injected error in ω is one of: *dormant* (no later step consumes $\delta(s_k)$: deleting it from every later context changes nothing); *masked* (consumed, but the next decision is the same as under s_k); *corrected* ($D < \infty$ and $R < \infty$); *overwritten* ($R < \infty$, $D = \infty$); *tolerated* (consumed, never recovered, $S{=}1$); *fatal* ($L < \infty$, $S{=}0$). The *coverage* of the system at (τ, k, δ) is $C(\tau, k, \delta) = \Pr[\text{fate} \neq \text{fatal}] = p_\delta(\tau, k)$ of Definition 1; the fate distribution is its decomposition.

Proposal 1's injection question (Q2) and its fragility curve $r(k/T)$ report only C, i.e. the product of the three PIE factors. Two systems with the same r can differ entirely in how they get it: one by dormancy (the task can be solved without the corrupted content), the other by correction. The fate distribution separates the two, and only the second is robustness in the sense of the hypothesis.

Definition 7 (Error lifetime)**.** Let $Z_i \in \{\text{live}, \text{recovered}, \text{locked in}\}$ be the state of the error at lag i, with $Z_0 = \text{live}$. The *cause-specific hazards*

$$h_{\text{rec}}(i \mid \tau, k, \delta) = \Pr[R{=}i \mid Z_{i-1}{=}\text{live}],$$
$$h_{\text{lock}}(i \mid \tau, k, \delta) = \Pr[L{=}i \mid Z_{i-1}{=}\text{live}]$$

and the survival of the live state $\bar{S}_i^{\text{err}} = \prod_{j \leq i}(1 - h_{\text{rec}}(j) - h_{\text{lock}}(j))$ give the *cumulative incidences* $F_{\text{rec}}(i) = \sum_{j \leq i} \bar{S}_{j-1}^{\text{err}} h_{\text{rec}}(j)$ and $F_{\text{lock}}(i)$ likewise; $F_{\text{rec}}(\infty)$ is the recovery probability and $\mathbb{E}[R \mid R < \infty]$ the mean time to recovery. The same construction with D in place of R gives the detection-latency distribution.

The error of Definition 7 is the *subject* of a survival analysis, which the step of Definition 3 is not: Proposal 2's model is the special case $h_{\text{rec}} \equiv 0$, in which every live error is eventually fatal (Assumption 4). The multi-state model in Fig. 8 contains both, and the generalisation of the hazard model that fits PASS runs with recovered errors is the one with a return transition.

Three derived quantities.

- *Resilience triangle.* Let $V_t^0 = p_0(\tau, t{+}1)$ be the success probability of the unperturbed prefix (the survival function $\bar{S}_t$ of Proposal 2) and V_t^δ the same for ω. The *resilience loss* $\mathcal{R}(\tau, k, \delta) = \sum_{i \geq 0} \left[V_{k+i}^0 - V_{k+i}^\delta\right]$ is the area between the two curves [8]; its depth $\max_i(V_{k+i}^0 - V_{k+i}^\delta)$ and its rapidity (the first i at which the gap closes to α) are the robustness and recovery speed of the system, as two numbers rather than one. Recovery shows as a dip and a rise of V^δ; lock-in as a dip without a rise.
- *Per-agent profile.* For an agent $a \in \mathcal{A}$ of a multi-agent system, following [25], four rates—*generation*, *permeability*, *containment* and *exposure*:

$$g_a = \Pr[u_t \text{ wrong} \mid a_t{=}a, \text{ clean input}],$$
$$\pi_a = \Pr[u_t \text{ wrong} \mid a_t{=}a, \text{ live error in input}],$$
$$c_a = \Pr[\text{flags or fixes it} \mid a_t{=}a, \text{ live error in input}],$$
$$x_a = \mathbb{E}[\#\text{ later steps consuming } u_t \mid a_t{=}a].$$

  "How prone is each agent to committing to errors" is π_a; "how prone to committing errors" is g_a; a robust team is one with low π and high c on the agents with high x. The matrix $M_{ab} = \pi_b \cdot \mathbb{E}[\#\, b\text{-steps consuming one } a\text{-output}]$ is the mean-offspring matrix of a multi-type branching process [24]: $\rho(M) < 1$ means a single wrong message dies out with probability one, and the extinction probability solves the usual fixed-point equation, so absorption can be *predicted* from pairwise measurements and compared with the empirical C. The topology result of Huang et al. [28] (hierarchical 5.5% loss versus 10.5–23.7% for flat) is a statement about $\rho(M)$.
- *Checkpoint model.* Let $\mathcal{C}(\tau)$ be the steps whose role is verification (verifier-agent turns, test runs, re-retrievals, orchestrator ledger updates), each with coverage c_j. The null model $C(\tau, k, \delta) \approx 1 - \prod_{j \in \mathcal{C},\, j > k}(1 - c_j)$ predicts that $r(k/T)$ is a step function aligned with the checkpoints rather than a smooth decay, that removing a verifier moves it by a measurable c_j, and that the late-position skew of the injected datasets (Fig. 4) is the regime with no checkpoint left.

Research questions.

1. **Lifetime.** What are F_{rec}, F_{lock} and the detection-latency distribution as functions of lag, and how do they depend on position k/T, error mode, fault side and system? Is the one-step median fix window of natural failures also the recovery time of injected errors, or is it a selection effect of failed runs?
2. **Mechanism.** What fraction of absorption is dormancy, masking, correction and overwriting? Only the last two are the hypothesised capability; the first two are task slack.
3. **Checkpoints.** Is $r(k/T)$ a step function aligned with $\mathcal{C}(\tau)$, and does removing or adding a verifier move it by the predicted c_j?
4. **Detected or overwritten.** How often is $R < \infty$ with $D = \infty$, and does a detected-but-unfixed error ($D < L < \infty$) look different in the text from a never-detected one?

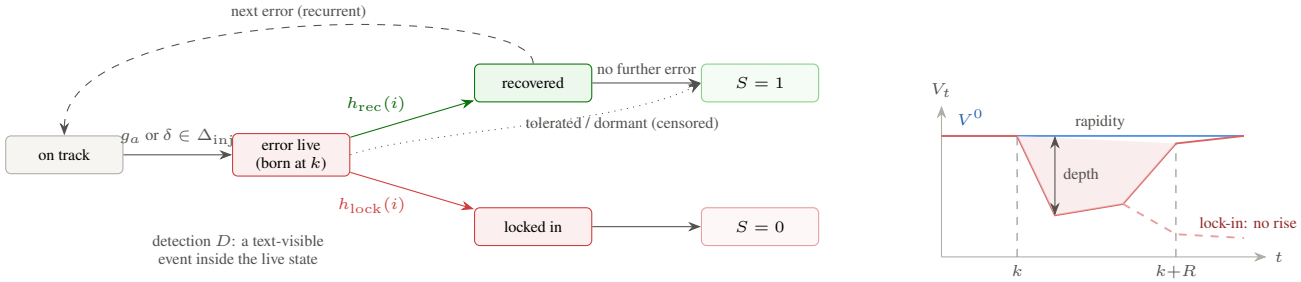


Figure 8: Left: the multi-state model of an error's lifetime. The live state is exited by recovery or lock-in with competing hazards; a run that ends with the error live but harmless is a censored observation. Proposal 2's hazard model is the special case without the recovery transition. Right: the same on the success-probability curve V_t of the prefix (Proposal 2's $\bar{S}_t$): recovery is a dip followed by a rise, lock-in a dip without one, and the shaded area is the resilience loss $\mathcal{R}$ with its depth and rapidity.

5. **Per-agent profile.** What are (g_a, π_a, c_a, x_a) for the roles of the surveyed systems, is $\rho(M) < 1$, and does the branching-process absorption probability predict the empirical C?
6. **Dose–response.** With a severity-graded injector δ_σ (size of a numeric error, fraction of a page truncated, number of corrupted arguments), what is $\Pr[\text{fatal} \mid \sigma]$, and is there a critical severity $\sigma^\star(k)$, the basin size of the resilience picture [26]?
7. **Epistasis.** For two injections at (k_1, k_2), is $\log p_{\delta_1\delta_2} - \log p_{\delta_1} - \log p_{\delta_2} + \log p_0$ zero (independent), negative (compounding) or positive (the first error triggers a check that catches the second)? Minimal repair families [36] show the fix side of this is non-additive.
8. **Cost.** What is the resilience loss $\mathcal{R}$, in steps and tokens, of a recovered error, and is recovering cheaper than restarting from $\tau_{<k}$?
9. **Identifiability from the log.** Can a recovered error be found without rollouts, from the dip-and-rise signature of a learned V_t or from a hidden state that switches back, and with what delay can an online monitor tell a transient dip from a lock-in?

Measurement tasks.

E1 — Fate screen.
The injection half of Proposal 1's screen with every continuation *kept* and labelled with Definition 6. The consumption test is a second, cheap counterfactual: for each step $k{+}i$ of ω, re-issue that single LLM call with $\delta(s_k)$ removed from its context and compare the output (one call per step, not a rollout). Detection D is a judge pass over the text with the injection known; recovery R is the first lag at which the consumption test stays negative for the rest of the run and V^δ has returned to $V^0 - \alpha$; lock-in L is estimated by the surrogate f of task T1 and verified by rollouts on a subset. Output: the fate distribution per system, mode, position and fault side, and the triples (D, R, L) with censoring indicators.

E2 — Lifetime curves.
On the E1 output, Aalen–Johansen cumulative incidences [1] for recovery and lock-in, then a multi-state Cox model [3, 53] with position, mode, fault side, system and checkpoint distance as covariates, and a shared frailty per agent and per task [62] for Q5. Validation on natural labels: $t_{\mathrm{obs}} - t_{\mathrm{err}}$ and $t_{\mathrm{lock}} - t_{\mathrm{err}}$ of the 1,184 Failure-as-a-Process failures, the detour length and recovery count of its 610 passes, and AgentRx's recovered events.

E3 — Checkpoint ablation.
Align $r(k/T)$ with $\mathcal{C}(\tau)$ in systems with an explicit verifier (Magentic-One's ledger, LongRCA's verification expert, the test runs of SWE agents), then remove, add or delay the verifier and re-run E1. The checkpoint model predicts the shift; the alternative—recovery driven by observations rather than by designated checks—predicts none.

E4 — Propagation matrix.
On the MASLab systems of Table 5 (row 4), inject at the output of agent a and judge the wrongness of each consuming agent's next output against the unperturbed branch; estimate M_{ab}, $\rho(M)$ and the branching-process absorption probability, and compare with the measured C. The same data give (g_a, π_a, c_a, x_a) per role.

E5 — Dose and interaction.
Severity-graded and paired injections; report $\sigma^\star(k)$ and the interaction term of Q7, on the same prefixes as E1.

E6 — Recovered-error detection from logs.
Train on outcome-only data (level L1 of Proposal 2) a hidden semi-Markov model [70] with an "error live" state that can switch back, and a value curve V_t with a temporal-difference consistency loss, and score recovery segments against the four released sources of recovered errors: Failure-as-a-Process passes (first error step, recovery count), AgentRx non-root events (§3.6), TRAIL traces that end correctly but carry error spans (§5.2), and MP-Bench steps flagged by one annotator of three (§3.3). Report recall of recovered errors separately from recall of fatal ones; the one-class detectors (OAT) are the baseline and are expected to flag both alike.

E7 — Online monitor with a delay guarantee.
A CUSUM [51] on the hazard or value sequence of U3, scored by lead time to t_{lock} and by false alarms on passes with recovered errors, the hardest negatives; the Lorden delay–false-alarm bound [47] gives the curve a monitor cannot beat, against which the 82% precision / 18% recall / zero lead time of the Failure-as-a-Process prompted monitor can be placed.

Field	Model	Reading on agent trajectories	What it yields
Dependable computing [4, 5, 63]	fault / error / failure chain; coverage and latency; PIE	the fate taxonomy of Definition 6	robustness decomposed into execution, infection, propagation
Software reliability [25]	error permeability and exposure per module	per-agent (g_a, π_a, c_a, x_a), matrix M_{ab}	which agent to harden, which edge needs a check
Event-history analysis [1, 3, 53, 62]	competing risks, multi-state and recurrent-event models, frailty	states live / recovered / locked in; several errors per run	F_{rec}, F_{lock}, covariate effects, per-agent random effects, with censoring handled
Branching processes [24]	multi-type Galton–Watson, $\rho(M)$	wrong message as an infection on the agent graph	analytic absorption probability; topology comparison
Ecology and infrastructure [8, 26]	basin of attraction; resilience triangle	V_t as the state, lock-in as leaving the basin	critical severity $\sigma^\star$; depth and rapidity as separate numbers
Sequential analysis [47, 51, 60]	CUSUM, SPRT, quickest change detection	online monitor on the prefix (task U3)	detection delay against false-alarm rate, with a lower bound
Time series [2, 70]	hidden semi-Markov and switching models	latent on-track / derailed regime that can switch back	label-free recovery segments; Proposal 2's change point is the no-return case
Safe RL [22, 50]	recoverability and reversibility of states	point of no return as an irreversible state	learned lock-in predictors with a definition

Table 7: Mathematical models that the lifecycle programme borrows, and what each one contributes.

Relation to Proposals 1–3. The screen of Proposal 1 produces the rollouts; E1 adds the consumption test and keeps the branches, so the marginal cost over Proposal 1 is one LLM call per continuation step. Proposal 2's hazard model is the $h_{\mathrm{rec}} \equiv 0$ case of Fig. 8; the multi-state version fits the PASS runs that Table 4 shows to be contaminated, and the recovered errors that Proposal 2 lists as a pitfall become a labelled target (E6). For Proposal 3, the fate labels change the credit: a recovered error should not carry the negative advantage of a fatal one, and the recovering step is a positive example of the capability the hypothesis names, which is what recovery-targeted training already exploits without localising it [27, 67].

Feasibility with the surveyed data.

- *Released recovered errors* (for E2 validation and E6): Failure-as-a-Process ships, per PASS run, the first error step, the number of error events and recoveries, the maximal detour length and a near-miss flag, and per FAIL run $t_{\mathrm{err}}/t_{\mathrm{lock}}/t_{\mathrm{obs}}$, the number of recovery attempts, forewarning signals and the awareness class; AgentRx annotates every failure event including the recovered ones (up to 55 per trajectory); TRAIL's 148 runs are not selected by outcome; MP-Bench's three votes per step separate consensus errors from minority ones. No new annotation is needed for the natural side.
- *Injected side.* Aegis reports absorption rates but ships no absorbed branches, and Who&When Pro and TracerTraj discard them likewise, so E1 is a re-run of the released injectors (Aegis's factory, Who&When Pro's injector) with the negatives kept, on rows 1–5 of Table 5; row 4 (MASLab systems, no environment state) is the cheapest and the natural home of E4.
- *Budget.* E1 reuses Proposal 1's ≈25 k continuations and adds one call per continuation step ($\approx 10^6$ calls, below the rollout cost); E2, E6 and E7 are statistical fits on the result; E3–E5 multiply the injection side by the number of ablations and severities (3–5×) on a subset of prefixes.

Pitfalls to design for.

- *Censoring.* A run that ends with the error live and $S{=}1$ carries no R and no L; naive fractions over observed events are biased, which is why E2 uses survival estimators rather than averages. Short runs (ALFWorld's 30-step cap) censor heavily.
- *Dormancy inflates coverage.* An injection into content the task never needs is absorbed trivially; Definition 6 separates it, and the dormant fraction per injector is itself a result about the injectors of Section 4.
- *Detection is a judgement, recovery is a measurement.* D depends on a judge reading the text; R and L are counterfactual. Report inter-judge agreement on D and never let D define the fate.
- *Baseline stochasticity.* 39 of 54 successful MAS runs fail at least once in three reruns [49], so V^0 is a curve with intervals, and "fatal" is a drop relative to it, not an absolute failure.
- *Wrongness of downstream outputs* (E4) needs a reference: the unperturbed branch is stochastic, so the judge must compare against the ground truth or a sample of unperturbed continuations, with the same caveat as D.
- *Injector realism and position prior.* The released injectors place errors late (Fig. 4); inject uniformly over positions and stratify every curve by k/T, or the lifetime curves inherit the prior.
- *Symptom shift in natural labels.* t_{obs} can precede t_{lock} in Failure-as-a-Process and judge labels drift to the symptom [58]; the natural data validate the *shape* of the curves, not their location.

What would count as a result.

- The first measured lifetime curves of errors in agent systems: recovery and lock-in incidences by lag, position, mode and system, with the fraction of absorption that is genuine correction rather than task slack.
- A tested checkpoint model: either $r(k/T)$ is a step function that a verifier ablation shifts by the predicted amount, or recovery is observation-driven and verifier agents are not where robustness comes from.
- Per-agent profiles (g_a, π_a, c_a, x_a) for the roles of the surveyed systems and a propagation matrix whose spectral radius predicts absorption, which would turn "which agent is responsible" into "which agent to harden".
- A label-free detector of recovered errors with a measured delay–false-alarm curve for the online monitor, placed against the Lorden bound.

A Observability and replayability of the released data

Purpose. The proposals of Section 8 need two properties that the overview table (Table 1) does not show: how much of the execution each release *records* per step, and whether the state at a step can be *restored and re-executed*. This appendix ranks the 21 datasets on both. The ranking is read from the dataset sections (Sections 3–6) and refers to what is actually downloadable as of October 2026, not to what the paper describes.

Observability tiers.

- A every LLM call's full input and output, tool I/O and (where it exists) environment state or screenshots;
- B the complete message stream with tool calls and tool returns, but not the per-call inputs or the state;
- C agent outputs only (no tool inputs, no observations beyond what the agent printed);
- D no step text released.

Replayability tiers.

- A replay tooling ships with the data (recorded prefix or runnable agent code plus environment);
- B public environment and public agent code, so $\mathrm{Roll}(\cdot)$ of Definition 1 is implementable in principle;
- C only some subsets are replayable, or the authors' pipeline is demonstrated but not released;
- D live web or no agent code: only the recorded trajectory exists;
- E nothing released.

Reading the ranking. Rows are ordered by the two tiers together, observability first. Two points follow. The measured programme of Section 8.2 is feasible only on the rows whose replay tier is A or B (rows 1–3, 5–7, 12 and, with the trajectories rebuilt, 19), which coincides with the systems of Table 5. The output-only group (rows 15–18) is also the group with the lowest or unreported annotator agreement (Section 7), which is what one expects when the annotator cannot see the evidence needed to place the step.

#	Dataset	Obs.	Replay	Basis (what the release contains)
1	TraceElephant (§3.2)	A	A	Full request and response of every LLM call, screenshots, token usage, runnable agent code and a dynamic re-execution setting. Live-web GAIA and AssistantBench runs are not deterministic; the 44 SWE-Agent runs on SWE-Bench Verified are.
2	MemTrace (§6.6)	A	A	White-box execution graph with every intermediate variable and operation. Memory-system code is MIT and the source conversations are static, so a run is re-executable up to LLM sampling.
3	SymFail (§3.7)	A	A	Event-dependency graph with per-call messages and tool boundary results, plus the SymTrace selective-replay framework. Adapters are thin (at most three LLM calls), token usage is empty, and most WebArena runs never reached the site.
4	TRAIL (§5.2)	A	C	OpenTelemetry span trees with full prompts, outputs, tool I/O and token counts. GAIA runs are live web; the 31 SWE-bench Lite runs are container-replayable.
5	TrajErrBench (§6.4)	B	B	Messages with tool calls and tool returns. τ^2-Bench resets its database and simulated user with a seed; SWE-Bench Pro runs in OpenHands containers.
6	HORIZON (§6.7)	B	B	Web step logs plus Playwright traces and page renders, docker environments and public agents. Database rows are a single SQL block and embodied rows are plans without observations.
7	AgentErrorBench (§6.1)	B	B	Full observation and admissible actions at every turn. ALFWorld and WebShop are deterministic text simulators; the GAIA subset is live web.
8	AgentRx (§3.6)	B	C	τ-bench rows carry tool calls, domain policies and tool schemas and are replayable; the Magentic-One rows are live web. HF copy gated, 73 of 170 public.
9	LongRCA (§3.5)	B	C	Tool, terminal and evaluator records inside `history` (≈180 k characters per run). The sandboxes (TravelPlanner, VitaBench, SWE-bench Pro, Terminal-Bench 2, WebArena) are replayable but the authors' agent systems are not released.
10	Who&When Pro (§3.4)	B	C	Framework-specific traces; GUI and image rows embed screenshots. Built by counterfactual replay with a SHA-256-keyed call cache, but only the evaluation harness is released, not the generation pipeline.
11	SearchAuditBench (§6.2)	B	D	Reasoning content and verbatim `search` / `visit` returns. Live web, and auditors are denied web access by design.
12	Aegis (§4.2)	B	B	Turn content only; the MAS have no environment state. MAS wrappers and the injection factory are public and runs are temperature 0, so a replay is a re-run.
13	AgentHallu (§6.5)	B	C	Tool calls with aligned tool responses. BFCL function calling is replayable against simulated tools; the search and GAIA subsets are live web.
14	TracerTraj (§4.1)	B	C	Step content embeds tool and web output. The six MAS are public but no replay tooling or injector is released with the data.
15	CORRECT-Error (§4.3)	C	D	Role and content only; successful parent trajectories withheld; one Magentic-One variant on live web.
16	Who&When (§3.1)	C	D	Agent outputs only, no tool inputs, screenshots or environment state. Live-web GAIA and AssistantBench.
17	MP-Bench (§3.3)	C	D	The same Who&When logs plus output-only Magentic-One console transcripts from MAST.
18	MAST-Data (§5.1)	C	D	One raw log string per trace in seven framework-specific layouts.
19	Failure as a Process (§6.3)	D	B	Annotations only. Terminal-Bench containers and the three scaffolds are public, and 71% of the runs can be re-joined from the CodeTracer release.
20	MegaRCA-Mix (§6.8)	D	E	On paper the richest bundle (trajectory, configuration, console logs, verifier output, sandbox artifacts); nothing is released.
21	AgentErrata (§4.4)	D	E	Nothing released.

Table 8: The 21 datasets ranked by observability (what is recorded per step) and replayability (whether a step's state can be restored and re-executed), from the released artefacts as of October 2026.

References

[1] Odd O. Aalen and Søren Johansen. An empirical transition matrix for non-homogeneous Markov chains based on censored observations. *Scandinavian Journal of Statistics*, 5(3):141–150, 1978.

[2] Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. *arXiv preprint arXiv:0710.3742*, 2007.

[3] Per Kragh Andersen and Richard D. Gill. Cox's regression model for counting processes: A large sample study. *The Annals of Statistics*, 10(4): 1100–1120, 1982.

[4] Jean Arlat, Martine Aguera, Louis Amat, Yves Crouzet, Jean-Charles Fabre, Jean-Claude Laprie, Eliane Martins, and David Powell. Fault injection for dependability validation: A methodology and some applications. *IEEE Transactions on Software Engineering*, 16(2):166–182, 1990.

[5] Algirdas Avižienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004.

[6] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2602.02475.

[7] Jiayi Bi, Yanjie Gao, Yuanmin Xie, Liqun Li, Tianyin Xu, Fan Yang, and Mao Yang. Diagnosing with insights: Structured analysis of agent failures via behavioral abstractions. *arXiv preprint arXiv:2609.02371*, 2026.

[8] Michel Bruneau, Stephanie E. Chang, Ronald T. Eguchi, George C. Lee, Thomas D. O'Rourke, Andrei M. Reinhorn, Masanobu Shinozuka, Kathleen Tierney, William A. Wallace, and Detlof von Winterfeldt. A framework to quantitatively assess and enhance the seismic resilience of communities. *Earthquake Spectra*, 19(4):733–752, 2003.

[9] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2503.13657.

[10] Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026. 2026.acl-long.912; arXiv:2604.22708.

[11] Xi Chen, Ruixue Ding, Zhigaoyuan Wang, Fanrui Zhang, Qiang Zhang, Boli Chen, Shihang Wang, Tianfu Wang, Ailin Ren, Pengyong Wang, Pengjun Xie, and Hui Xiong. Rollout-time intervention: Unlocking agentic reinforcement learning beyond policy limits. *arXiv preprint arXiv:2610.13438*, 2026.

[12] Yanjun Chen, Yirong Sun, Hanlin Wang, Jinghan Wang, Xinming Zhang, Xiaoyu Shen, Wenjie Li, and Wei Zhang. The trace is the state: Exact credit assignment for LLM agent teams. *arXiv preprint arXiv:2603.06859*, 2026.

[13] Chih-Hsuan, Yang, Tanwi Mallick, Le Chen, Krishnan Raghavan, Amal Gueroudji, Ian T. Foster, and Rajeev Thakur. Who gets the reward & who gets the blame? evaluation-aligned training signals for multi-LLM agents. *arXiv preprint arXiv:2511.10687*, 2025.

[14] Sanjiban Choudhury. Process reward models for LLM agents: Practical framework and directions. *arXiv preprint arXiv:2502.10325*, 2025.

[15] Jaewon Chu, Jinwoo Seo, Jaewon Cho, Jeehye Na, Yunyang Xiong, Youngdae Kim, and Hyunwoo J. Kim. AgentGrad: Intervention-guided prompt optimization for multi agent systems. *arXiv preprint arXiv:2609.08572*, 2026.

[16] Xinle Deng, Ruobin Zhong, Hujin Peng, Xiaoben Lu, Yanzhe Wu, Guang Li, Buqiang Xu, Yunzhi Yao, Jizhan Fang, Haoliang Cao, Junjie Guo, Yuan Yuan, Ziqing Ma, Yuanqiang Yu, Rui Hu, Baohua Dong, Hangcheng Zhu, and Ningyu Zhang. MemTrace: Tracing and attributing errors in large language model memory systems. *arXiv preprint arXiv:2605.28732*, 2026. Data: https://huggingface.co/datasets/zjunlp/MemTraceBench; code: https://github.com/zjunlp/MemTrace.

[17] Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. TRAIL: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025.

[18] Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1–2):31–71, 1997.

[19] Atray Dixit, Oren Parnas, Biyu Li, Jenny Chen, Charles P. Fulco, Livnat Jerby-Arnon, Nemanja D. Marjanovic, Danielle Dionne, Tyler Burks, Raktima Raychowdhury, Britt Adamson, Thomas M. Norman, Eric S. Lander, Jonathan S. Weissman, Nir Friedman, and Aviv Regev. Perturb-seq: Dissecting molecular circuits with scalable single-cell RNA profiling of pooled genetic screens. *Cell*, 167(7):1853–1866, 2016.

[20] Charles Elkan and Keith Noto. Learning classifiers from only positive and unlabeled data. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 213–220, 2008.

[21] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. *arXiv preprint arXiv:2505.10978*, 2025.

[22] Nathan Grinsztajn, Johan Ferret, Olivier Pietquin, Philippe Preux, and Matthieu Geist. There is no turning back: A self-supervised approach for reversibility-aware reinforcement learning. In *NeurIPS*, 2021.

[23] Weiyang Guo, Zesheng Shi, Longhui Zhang, Zeen Zhu, Min Zhang, and Jing Li. Agent reinforcement learning via pivotal-aware self-feedback retry. *arXiv preprint arXiv:2607.03702*, 2026.

[24] Theodore E. Harris. *The Theory of Branching Processes*. Springer, 1963.

[25] Martin Hiller, Arshad Jhumka, and Neeraj Suri. EPIC: Profiling the propagation and effect of data errors in software. *IEEE Transactions on Computers*, 53(5):512–530, 2004.

[26] Crawford S. Holling. Resilience and stability of ecological systems. *Annual Review of Ecology and Systematics*, 4:1–23, 1973.

[27] Xu Hu, Haoming Li, Yuxiang Wang, Jinxin Yang, Kangshuo Li, Licheng Wang, Dingyi Kang, Jiayu Song, Tianhao Wang, Zhanhua Hu, Xu Yang, Qiannan Li, Bingzhe Li, and Feng Chen. CUA-recovery: Benchmarking error recovery in long-horizon computer-use agents. *arXiv preprint arXiv:2610.27645*, 2026.

[28] Jen-tse Huang, Jiaxu Zhou, Tailin Jin, Xuhui Zhou, Zixi Chen, Wenxuan Wang, Youliang Yuan, Michael R. Lyu, and Maarten Sap. On the resilience of LLM-based multi-agent collaboration with faulty agents. *arXiv preprint arXiv:2408.00989*, 2024.

[29] Yibin Huang and Conghui Zhu. Distill until divergence: Error-truncated hindsight distillation for agentic RL. *arXiv preprint arXiv:2610.23048*, 2026.

[30] Yeonjun In, Mehrab Tanjim, Jayakumar Subramanian, Sungchul Kim, Uttaran Bhattacharya, Somdeb Sarkhel, Wonjoong Kim, Sangwu Park, and Chanyoung Park. Rethinking failure attribution in multi-agent systems: A multi-perspective benchmark and evaluation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2603.25001.

[31] Saeid Jamshidi, Arghavan Moradi Dakhel, Kawser Wazed Nafi, and Foutse Khomh. Hallucination cascade: Analyzing error propagation in multi-agent LLM systems. *arXiv preprint arXiv:2606.07937*, 2026.

[32] Dongwon Jung, Hemanth Neelgund Ramesh, Yifan Wang, Xiaomin Li, Yuexing Hao, Yu Hu, Muhao Chen, Varun Chandrasekaran, Andrzej Banburski-Fahey, and Jaron Lanier. Targeting pivotal decisions for credit assignment in agentic reinforcement learning. *arXiv preprint arXiv:2609.36178*, 2026.

[33] Patrick Kidger, James Morrill, James Foster, and Terry Lyons. Neural controlled differential equations for irregular time series. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[34] Ryuichi Kiryo, Gang Niu, Marthinus C. du Plessis, and Masashi Sugiyama. Positive-unlabeled learning with non-negative risk estimator. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[35] Fanqi Kong, Ruijie Zhang, Huaxiao Yin, Guibin Zhang, Xiaofei Zhang, Ziang Chen, Zhaowei Zhang, Xiaoyuan Zhang, Song-Chun Zhu, and Xue Feng. Aegis: Automated error generation and attribution for multi-agent systems. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.14295.

[36] Bingjie Li, Yumeng Song, Zhongming Yao, and Tianyi Li. Localizing emergent failures in agentic AI: Recovering minimal repair families via counterfactual replay. *arXiv preprint arXiv:2608.29228*, 2026.

[37] Ed Li, Junyu Ren, and Cat Yan. Scaling multiagent systems with process rewards. *arXiv preprint arXiv:2601.23228*, 2026.

[38] Mukai Li, Qingcheng Zeng, Tianqing Fang, Zhenwen Liang, Linfeng Song, Qi Liu, Haitao Mi, and Dong Yu. Verified critical step optimization for LLM agents. *arXiv preprint arXiv:2602.03412*, 2026.

[39] Yanming Li, Xuelin Zhang, WenJie Lu, Ziye Tang, Maodong Wu, Haotian Luo, Tongtong Wu, Zijie Peng, Hongze Mi, Yibo Feng, Naiqiang Tan, Chao Huang, Lian Peng, and Li Shen. Who deserves the reward? SHARP: Shapley credit-based optimization for multi-agent system. *arXiv preprint arXiv:2602.08335*, 2026.

[40] Qiao Liang, Yuke Zhu, Chao Ge, Lei Yang, Ying Shen, Bo Zheng, and Sheng Guo. Learning from the irrecoverable: Error-localized policy optimization for tool-integrated LLM reasoning. *arXiv preprint arXiv:2602.09598*, 2026.

[41] Zhixiang Liang, Yifei Liu, Yidan Huang, Haozhe Zhao, Beichen Huang, Jiaqi Wang, Nan Duan, and Qiong Cao. SearchAuditor: Auditing and attributing failures in long-horizon search agents. *arXiv preprint arXiv:2608.05212*, 2026. Code and data: https://github.com/lzzzx666/SearchAuditor.

[42] Jiaqi Liao, Yuanzhao Zhai, Huanxi Liu, Xu Zhang, Zheming Zhuang, Dawei Feng, Bo Ding, and Huaimin Wang. Error-propagation modeling for failure attribution in LLM-based multi-agent systems. *arXiv preprint arXiv:2610.11600*, 2026.

[43] Haolin Liu, Dian Yu, Sidi Lu, Yujun Zhou, Rui Liu, Zhenwen Liang, Haitao Mi, Chen-Yu Wei, and Dong Yu. Save the good prefix: Precise error penalization via process-supervised RL to enhance LLM reasoning. *arXiv preprint arXiv:2601.18984*, 2026.

[44] Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tianwei Yue, Chi Wang, Jian Kang, Qingyun Wu, and Huazheng Wang. Who&when pro: Can LLMs really attribute failures in AI agents? In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2607.09996.

[45] Mengyang Liu, Weiming Wang, Haixin Nan, Yu Wu, Peize Li, Shihong Huang, Xuning He, Xue Jiang, and Yihong Dong. SPROUT: Scaling agentic reinforcement learning for long-horizon software engineering. *arXiv preprint arXiv:2610.14405*, 2026.

[46] Xuannan Liu, Xiao Yang, Zekun Li, Peipei Li, and Ran He. AgentHallu: Benchmarking automated hallucination attribution of LLM-based agents. *arXiv preprint arXiv:2601.06818*, 2026. Code and data: https://github.com/liuxuannan/AgentHallu.

[47] Gary Lorden. Procedures for reacting to a change in distribution. *The Annals of Mathematical Statistics*, 42(6):1897–1908, 1971.

[48] Mohammad Lotfollahi, F. Alexander Wolf, and Fabian J. Theis. scGen predicts single-cell perturbation responses. *Nature Methods*, 16(8): 715–721, 2019.

[49] Zhongwen Luan, Xiaoyu Zhang, Ming Hu, Yue Yang, Jiongchi Yu, and Xiaohong Chen. Repair or resample? rethinking failure debugging in LLM multi-agent systems. *arXiv preprint arXiv:2608.25920*, 2026. v2, 29 Aug 2026. Introduces the SymTrace replay framework and the SymFail dataset; code and data: https://anonymous.4open.science/r/SymTrace-7234.

[50] Teodor Mihai Moldovan and Pieter Abbeel. Safe exploration in Markov decision processes. In *ICML*, 2012.

[51] Ewan S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

[52] David Powell, Eliane Martins, Jean Arlat, and Yves Crouzet. Estimators for fault tolerance coverage evaluation. *IEEE Transactions on Computers*, 44(2):261–274, 1995.

[53] Hein Putter, Marta Fiocco, and Ronald B. Geskus. Tutorial in biostatistics: Competing risks and multi-state models. *Statistics in Medicine*, 26 (11):2389–2430, 2007.

[54] Yunjia Qi, Zehua Yin, Xintong Shi, Hao Peng, Songyuanyi Lu, Yixian Liu, Richeng Xuan, Yuhong Liu, Zhichao Hu, Xiaozhi Wang, Lei Hou, Bin Xu, and Juanzi Li. TrajDebug: Tracing error lifecycle to identify critical failures in long-horizon agent trajectories. In *Findings of the Association for Computational Linguistics: EMNLP 2026*, 2026. arXiv:2608.06346. Code and data: https://github.com/THU-KEG/TrajDebug.

[55] Harsh Raj, David Lee, Anas Mahmoud, Renxiong Wang, Razvan-Gabriel Dumitru, Chenguang Wang, Tong Zhao, Yunzhong He, Darvin Yi, and Vipul Gupta. Root-cause attribution is a search problem: Continual search for long-horizon agent failures. *arXiv preprint arXiv:2609.13463*, 2026. Introduces the (unreleased) MegaRCA-Mix test set; taxonomy from arXiv:2607.28802.

[56] Yusuf Roohani, Kexin Huang, and Jure Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024.

[57] Jaineet Shah. Causal agent replay: Counterfactual attribution for LLM-agent failures. *arXiv preprint arXiv:2606.08275*, 2026.

[58] Bingyu Shen, Jiaqi Tang, Lan Wei, Yi Sheng, J. Jenny Li, and Boyang Li. Executable prefix values for locating agent failures. *arXiv preprint arXiv:2610.17633*, 2026.

[59] Weijie Shi, Yanxi Chen, Zexi Li, Xuchen Pan, Yuchang Sun, Jiajie Xu, Xiaofang Zhou, and Yaliang Li. R^3l: Reflect-then-retry reinforcement learning with language-guided exploration, pivotal credit, and positive amplification. *arXiv preprint arXiv:2601.03715*, 2026.

[60] Alexander Tartakovsky, Igor Nikiforov, and Michèle Basseville. *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. CRC Press, 2014.

[61] Runchu Tian, Zaid Khan, Zun Wang, Justin Chih-Yao Chen, and Mohit Bansal. MiRL: Error-targeted reinforcement learning for correcting terminal agent misbehaviors. *arXiv preprint arXiv:2610.35697*, 2026.

[62] James W. Vaupel, Kenneth G. Manton, and Eric Stallard. The impact of heterogeneity in individual frailty on the dynamics of mortality. *Demography*, 16(3):439–454, 1979.

[63] Jeffrey M. Voas. PIE: A dynamic failure-based technique. *IEEE Transactions on Software Engineering*, 18(8):717–727, 1992.

[64] Hanlin Wang, Jian Wang, Chak Tou Leong, and Wenjie Li. STeCa: Step-level trajectory calibration for LLM agent learning. *arXiv preprint arXiv:2502.14276*, 2025.

[65] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[66] Xinyu Jessica Wang, Haoyue Bai, Yiyou Sun, Haorui Wang, Shuibai Zhang, Wenjie Hu, Mya Schroder, Bilge Mutlu, Dawn Song, and Robert D. Nowak. The long-horizon task mirage? Diagnosing where and why agentic systems break. *arXiv preprint arXiv:2604.11978*, 2026. Benchmark: HORIZON. Data: https://huggingface.co/datasets/xwang2775/long-horizon-agent-failures.

[67] Wei Xie, Kaituo Feng, Shiming Fu, Yong Liu, Xiangyu Yue, and Shuangjia Zheng. ReTerm: Recovery training for self-improving terminal agents. *arXiv preprint arXiv:2610.18484*, 2026.

[68] Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. Tracing agentic failure from the flow of success. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2026. arXiv:2607.12747.

[69] Junkeun Yi, Damon Mosk-Aoyama, Baihe Huang, Ritu Gala, Charles Wang, Sugam Dipak Devare, Khushi Bhardwaj, Abhibha Gupta, Oleksii Kuchaiev, Jiantao Jiao, Jian Zhang, and Venkat Srinivasan. PivotRL: High accuracy agentic post-training at low compute cost. *arXiv preprint arXiv:2603.21383*, 2026.

[70] Shun-Zheng Yu. Hidden semi-Markov models. *Artificial Intelligence*, 174(2):215–243, 2010.

[71] Yifan Yu, Moyan Li, Shaoyuan Xu, Jinmiao Fu, Xinhai Hou, Fan Lai, and Bryan Wang. CORRECT: COndensed eRror RECognition via knowledge Transfer in multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2509.24088.

[72] Siyu Yuan, Zehui Chen, Zhiheng Xi, Junjie Ye, Zhengyin Du, and Jiecao Chen. Agent-R: Training language model agents to reflect via iterative self-training. *arXiv preprint arXiv:2501.11425*, 2025.

[73] Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, Zhaoyang Liu, Bolin Ding, and Jingren Zhou. AgentEvolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.

[74] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.03312; data release https://github.com/bingreeky/AgenTracer.

[75] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Ding. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *ICML*, 2025. arXiv:2505.00212.

[76] Yunfei Zhang, Boyu Feng, Changhua Pei, Zexin Wang, Zhihuang Peng, Xinlong Liu, Hengyue Jiang, Difeng Ma, Jiayi Zhang, Yongzhou Yao, Yanan Zhao, Fei Sun, Yintong Huo, Zhaoyang Liu, Jingjing Li, Gaogang Xie, and Dan Pei. LongRCA bench: Root-cause localization in long-horizon agent trajectories. *arXiv preprint arXiv:2608.15242*, 2026. v4, September 2026; https://longrca-bench.github.io/.

[77] Lei Zhao, Zongheng Cao, and Xinyu Hu. When fine-grained credit assignment helps and hurts: Multimodal agentic reinforcement learning through video editing. *arXiv preprint arXiv:2610.35497*, 2026.

[78] Wanjia Zhao, Mert Yuksekgonul, Shirley Wu, and James Zou. SiriuS: Self-improving multi-agent systems via bootstrapped reasoning. *arXiv preprint arXiv:2502.04780*, 2025.

[79] Xiangxin Zhao, Han Li, Shuaiting Li, Tianyi Zhao, Earl T. Barr, Federica Sarro, and He Ye. Failure as a process: An anatomy of CLI coding agent trajectories. *arXiv preprint arXiv:2607.09510*, 2026. Replication package (annotations only): https://github.com/xz-Sean/cli_trajectory_analysis.

[80] Dongsheng Zhu, Xuchen Ma, Yucheng Shen, Xiang Li, Yukun Zhao, Shuaiqiang Wang, Lingyong Yan, and Dawei Yin. When tools fail: Benchmarking dynamic replanning and anomaly recovery in LLM agents. *arXiv preprint arXiv:2606.05806*, 2026.

[81] Kaijie Zhu, Weixiang Yan, Omar Attia, Abbas Kazerouni, Alexander Toshev, and Andrew Szot. LLM value functions for credit assignment in agentic RL. *arXiv preprint arXiv:2610.15840*, 2026.

[82] Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. Where LLM agents fail and how they can learn from failures. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2509.25370. Code: https://github.com/ulab-uiuc/AgentDebug.

[83] Yihua Zhu, Qianying Liu, Weixu Qiao, Xuan Ren, Weiwei Xu, Wenbo Li, Wei Wang, Ruijia Chen, Xinmiao Luan, Yin Luo, Hao Huang, Xiang Zheng, and Hidetoshi Shimodaira. My FAULT: Self-diagnosis as credit assignment in self-evolving agentic reinforcement learning. *arXiv preprint arXiv:2610.01161*, 2026.

[84] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, and Xu-Yao Zhang. AgentRewind: Recoverable execution for long-horizon LLM agents. *arXiv preprint arXiv:2608.14380*, 2026.